

Digital Electronics & Systems

Lecture 1-9 Introduction

19ECE204

Ms. Bhavana V
Assistant Professor
Dept. of ECE
ASE, Bengaluru

Digital Electronics & Systems

- **TEXT BOOK:** Stephen Brown, Zvonko Vranesic, “Fundamentals of Digital logic with Verilog Design”, Tata McGraw Hill Publishing Company Limited, Special Indian Edition, 2007.

Course Objectives

- To understand the fundamentals of Boolean Logic and the building blocks of digital circuits
- To introduce the abstraction of simple practical problems into Boolean Logic and their efficient implementation and to introduce the fundamentals of design with combinational and sequential subsystems

Course Outcomes

- **CO1:** Able to frame Boolean equations for solving a simple real-life problems and realize them using gate-level building blocks
- **CO2:** Able to apply minimization techniques for efficient Boolean logic implementation
- **CO3:** Able to realize digital blocks using combinational and sequential subsystems
- **CO4:** Able to design using state machine descriptions for practical real-life engineering problems

Syllabus

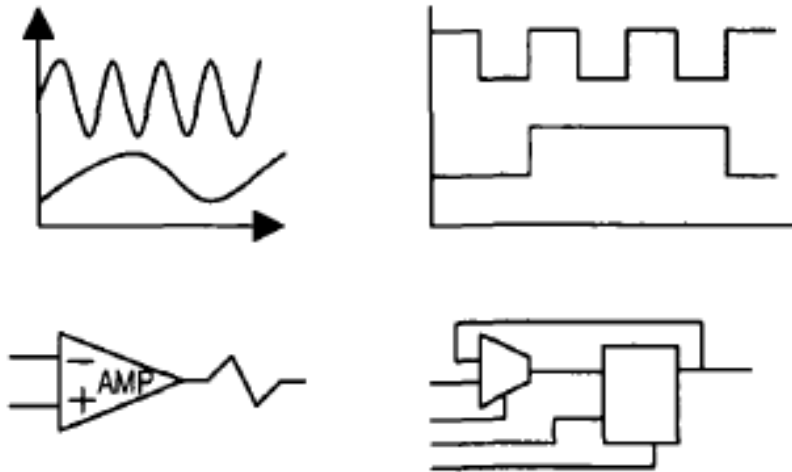
Unit 1 Introduction to logic circuits - Variables and functions, inversion - Truth tables - Logic gates and Networks - Boolean algebra - Synthesis using gates - Design examples - Optimized implementation of logic functions - Karnaugh map - Strategy for minimization - Minimization of product of sums forms - Incompletely specified functions - Multiple output circuits - Tabular method for minimization - Number representation and arithmetic circuits: Addition of unsigned numbers - Signed numbers - Fast adders.

Unit 2 Combinational circuit building blocks - Multiplexers - Decoders - Encoders - Code converters - Arithmetic comparison circuits - Sequential circuit building blocks - Basic latch - Gated SR latch - Gated D latch - Master slave and edge triggered - D flip-flops - T flip-flop - JK flip-flop - Registers - Counters - Reset synchronization - Other types of counters.

Unit 3 Synchronous sequential circuits - Basic design steps - State assignment problem - Mealy state model - Serial adders - State minimization - Introduction to Asynchronous sequential circuits – Introduction to CMOS logic.

ANALOG VS DIGITAL

- Analog devices and systems: Process analog signals (time-varying signals that can take any value across a continuous range known as dynamic range)
- Digital devices and systems: Process digital signals (analog signals that are modeled as having at any time one of two discrete values)

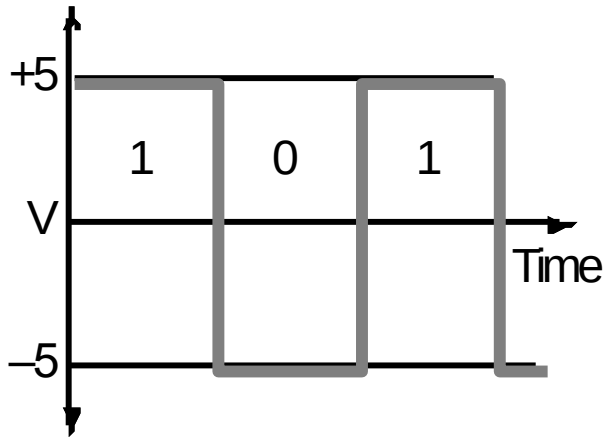


a. Analog Circuit.

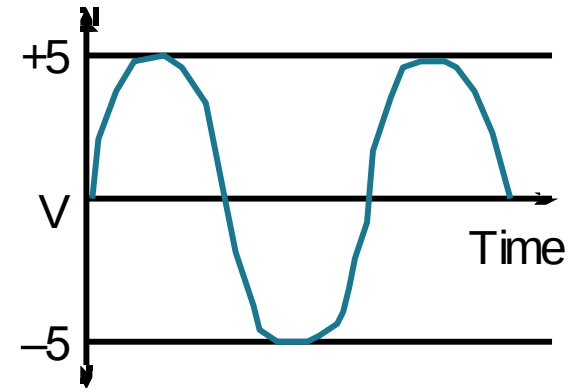
b. Digital Circuit.

Digital Systems

Digital vs. Analog Waveforms



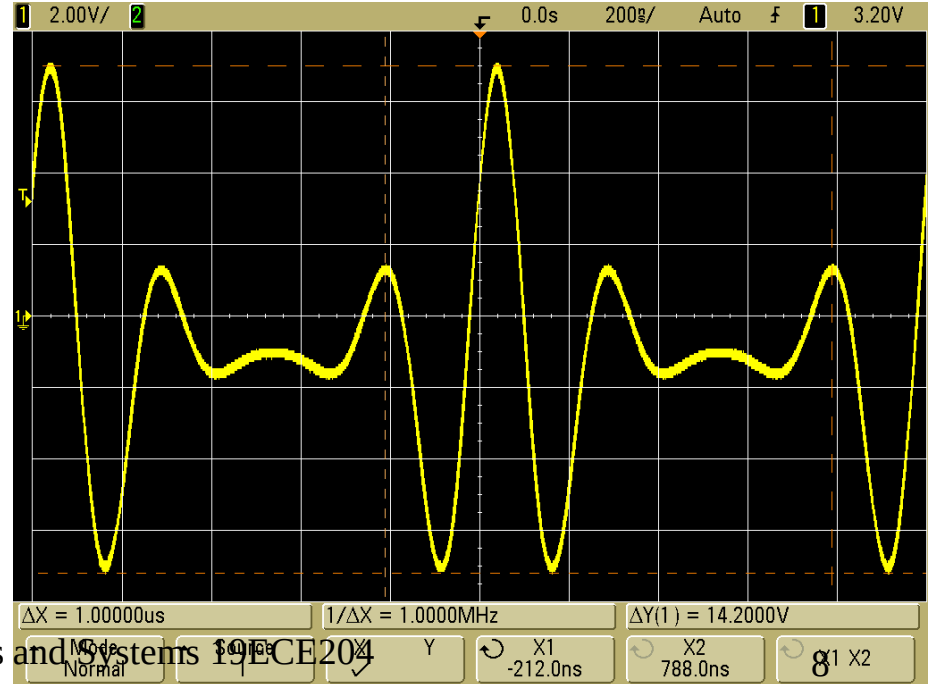
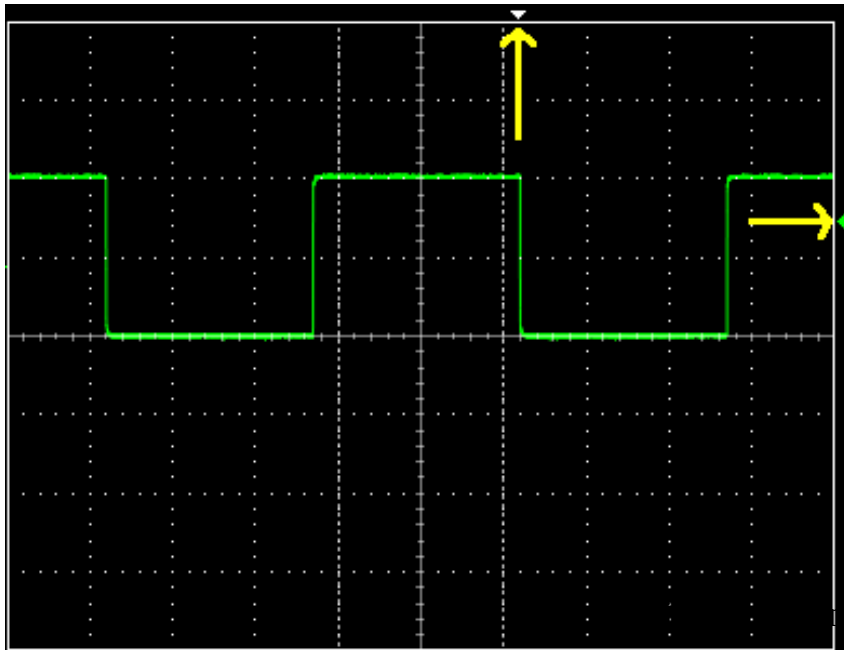
Digital:
only assumes discrete values



Analog:
values vary over a broad range
continuously

Digital Systems cntd..

- A digital **signal** is a signal that can only have discrete values in time
 - Most common are **binary** digital signals, where only two values are allowed often designated as 0 and 1
- The opposite is analog signals that can take infinite values



Digital Hardware Systems

Boolean Algebra and Logical Operators

Algebra: variables, values, operations

In Boolean algebra, the values are the symbols 0 and 1

If a logic statement is false, it has value 0

If a logic statement is true, it has value 1

Operations: AND, OR, NOT

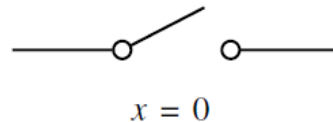
X	Y	X AND Y
0	0	0
0	1	0
1	0	0
1	1	1

X	Y	X OR Y
0	0	0
0	1	1
1	0	1
1	1	1

X	NOT X
0	1
1	0

Variables & Functions

- Logic circuits form the foundation of digital systems
- Binary logic circuits perform operations of binary signals
- Let's consider the simplest element logic circuit: switch.
 - A switch can be in two states: “open” and “close”.
 - Graphical symbol for a switch



State of the switch can be given as a binary variable x

Representations of Digital Design: Switches

Normally Open

A switch connects two points under control signal.

- when the control signal is 0 (false), the switch is open
- when it is 1 (true), the switch is closed

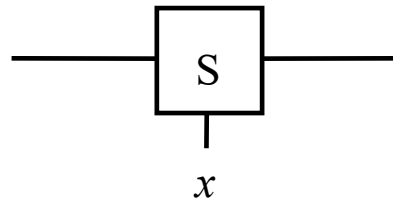
We will use Normally open switch in our discussion

A switch connects two points under control signal.

- when the control signal x is 0 (false), the switch is open
- when it is 1 (true), the switch is closed



(a) Two states of a switch

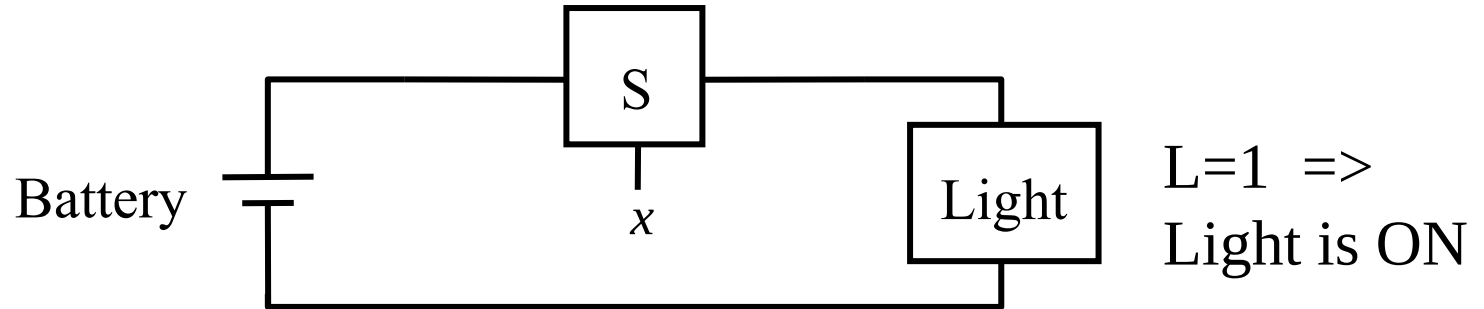


(b) Symbol for a switch

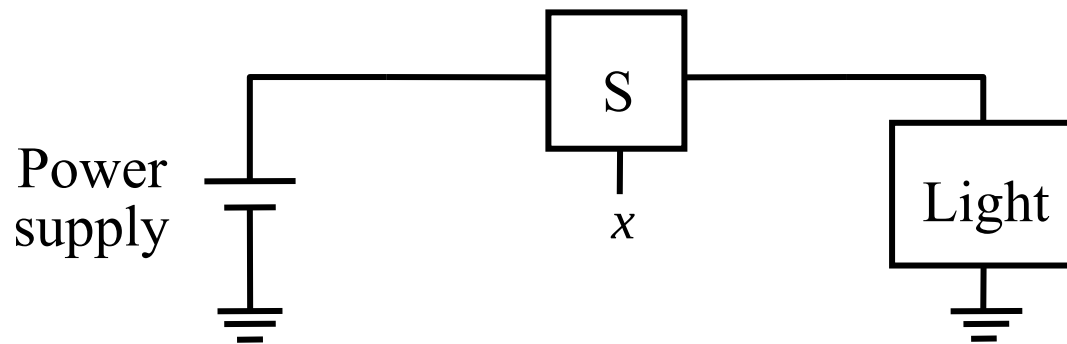
A simple application using a switch : “lightbulb”

State of the switch can be given as a **binary variable** x s.t.:

- $x = 0$ when switch is open, and
- $x = 1$ when switch is closed.



(a) Simple connection to a battery



(b) Using a ground connection as the return path

Figure : A light controlled by a switch.

Logic expression/Logic Function

State of Light is function of input variable x .

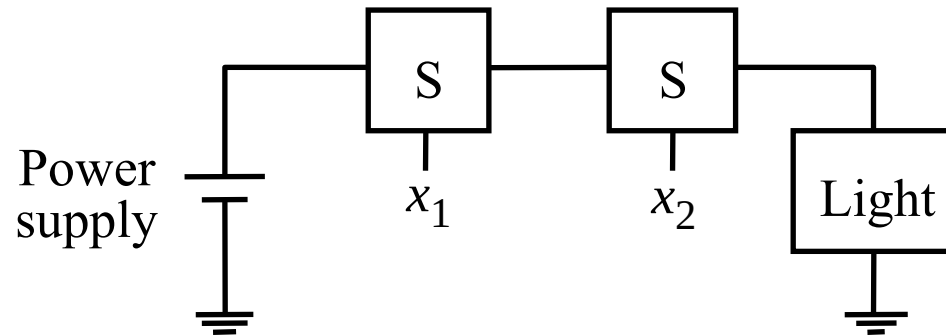
- $L=1$ if $x=1$
- $L=0$ if $x=0$

$$L(x)=x$$

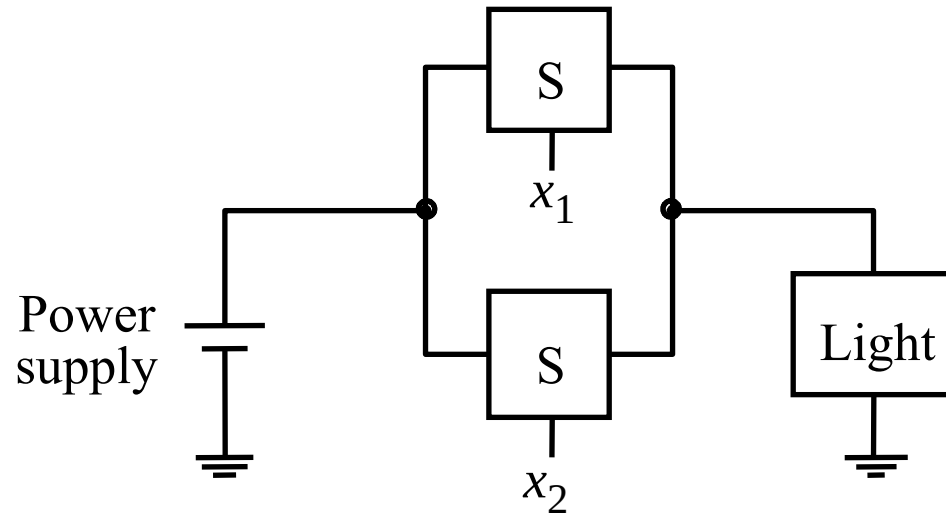
$L(x)=x$ is a logic expression/function

Here only one switch was controlling Light.

If there are two switches then?



(a) The logical AND function (series connection)



(b) The logical OR function (parallel connection)

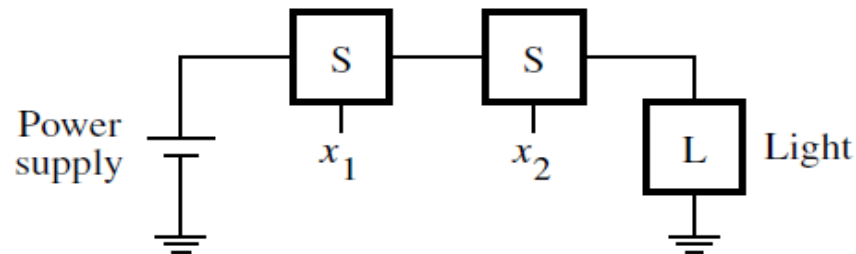
Figure : Two basic functions.

Series Connection => logical AND operation

$$L(x_1, x_2) = x_1 \cdot x_2$$
 where $L = 1$ if $x_1 = 1$ and $x_2 = 1$,
 $L = 0$ otherwise.

The symbol

- Is called **AND** operator



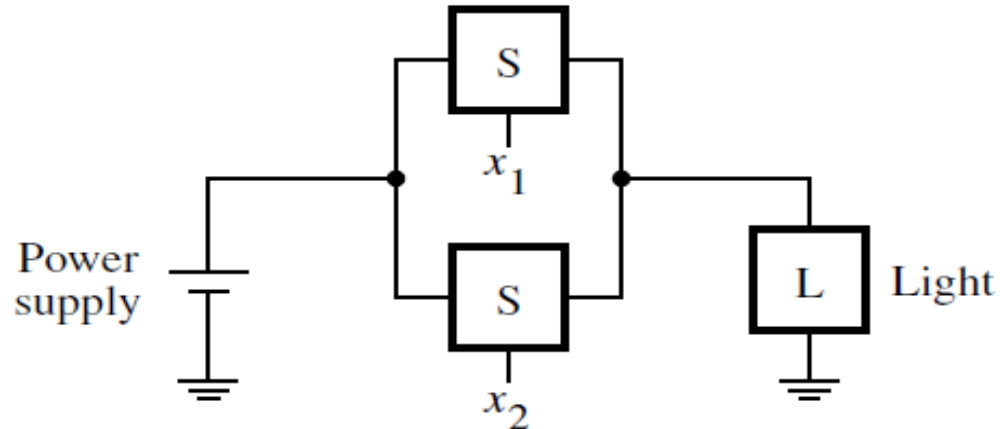
(a) The logical AND function (series connection)

Parallel Connection => logical OR operation

where
$$L(x_1, x_2) = x_1 + x_2$$

 $L = 1$ if $x_1 = 1$ or $x_2 = 1$ or if $x_1 = x_2 = 1$,
 $L = 0$ if $x_1 = x_2 = 0$.

Symbol **+** is
called **OR**
operator



(b) The logical OR function (parallel connection)

Here three switches are controlling the light in more complex way.
 Can you identify the relation?

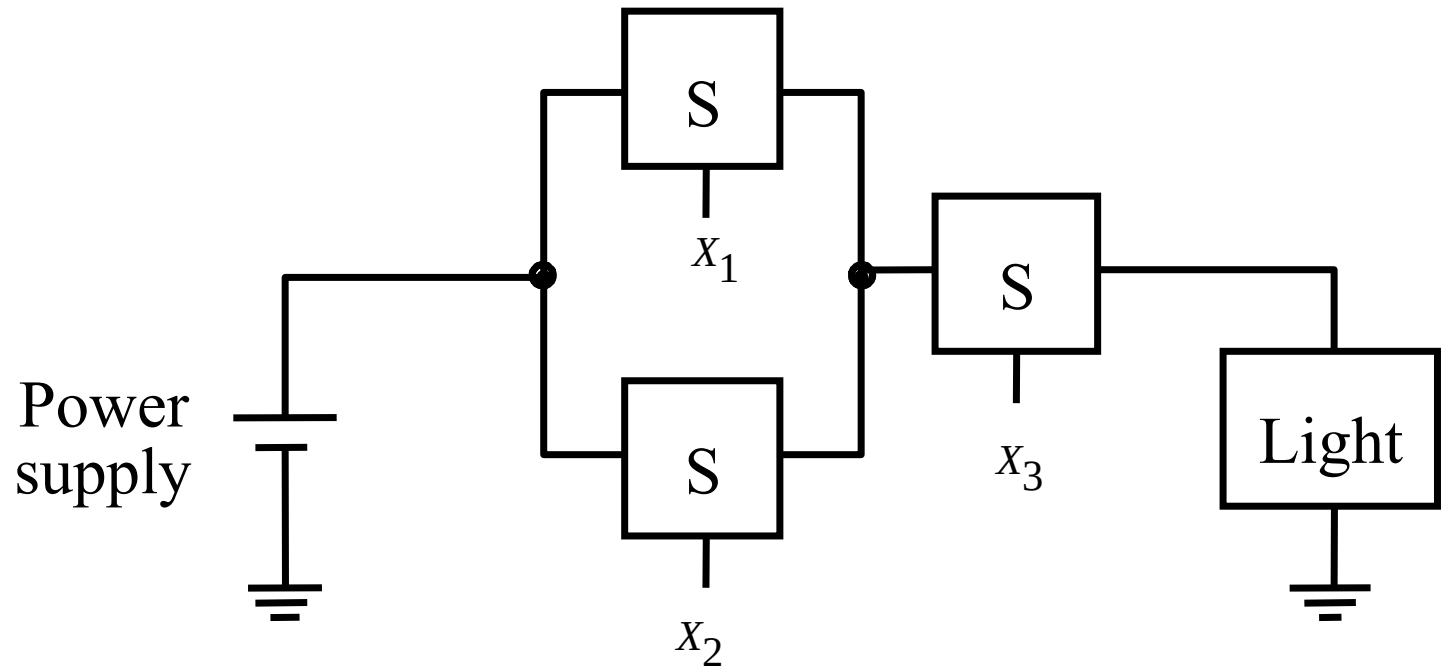
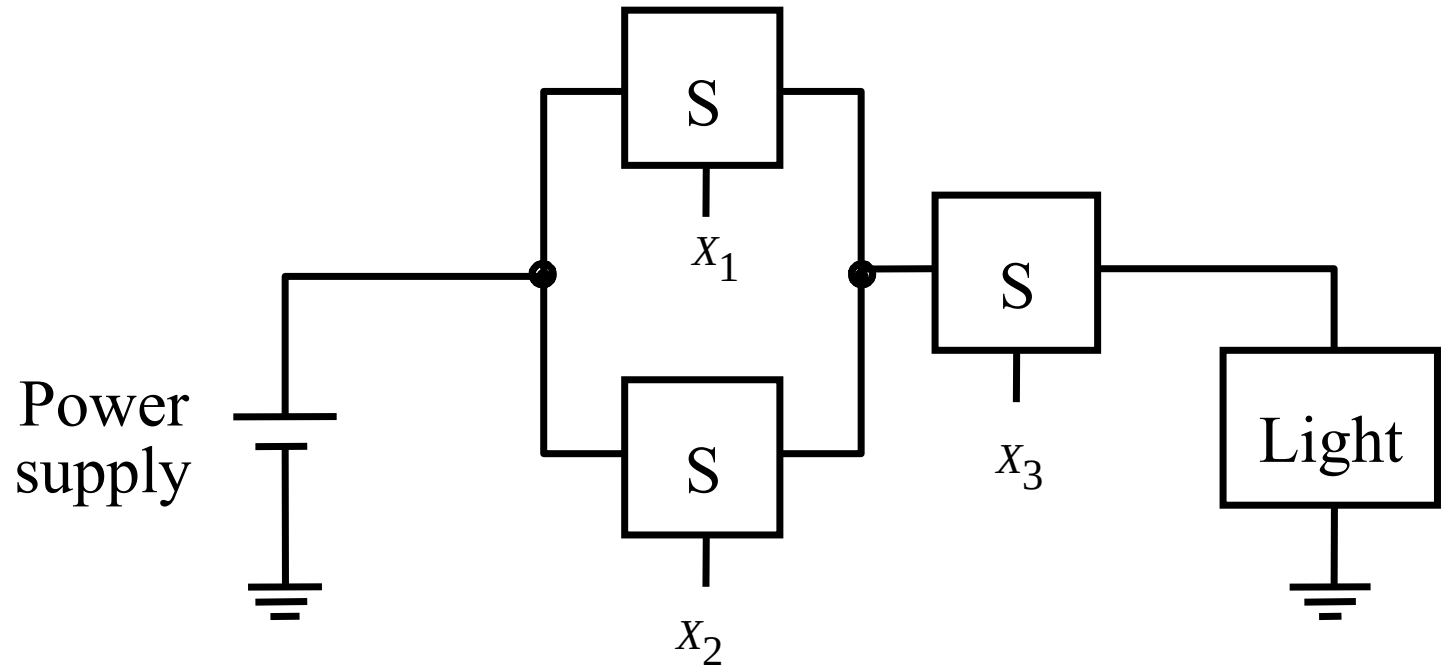


Figure: A series-parallel connection.

Here the series parallel connection of switches realizes the logic function

$$L(x_1, x_2, x_3) = (x_1 + x_2) \cdot x_3$$



The light is on if $x_3 = 1$ and, at the same time, at least one of the x_1 or x_2 inputs is equal to 1.

Figure : A series-parallel connection.

Try implementation using switches

- $(A+B).(C+D)$
- $A.(B+C).D$

- Till now we saw some positive action takes place when a switch is closed, such as turning the light on.
- It is equally interesting and useful to consider the possibility that a positive action takes place when a switch is opened

Complement Operation (Inversion)

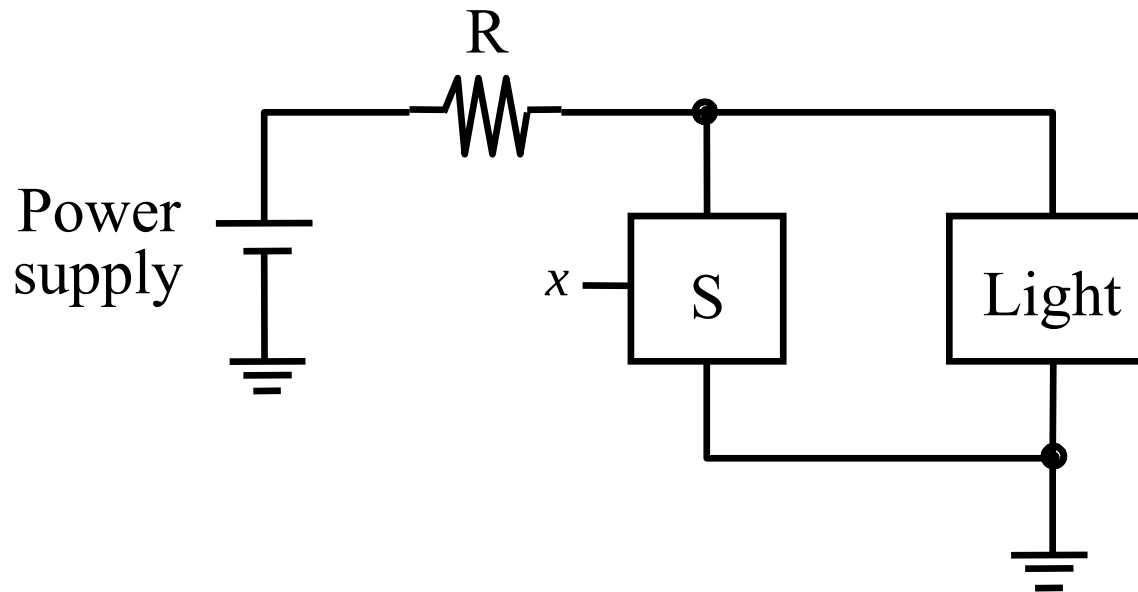


Figure : An inverting circuit.

Complement Operation

$$L(x) = \bar{x}$$

where $L = 1$ if $x = 0$,
 $L = 0$ if $x = 1$

Symbol $\bar{}$ over a variable called **NOT** operator.

$$\bar{x} = x' = !x = \sim x$$

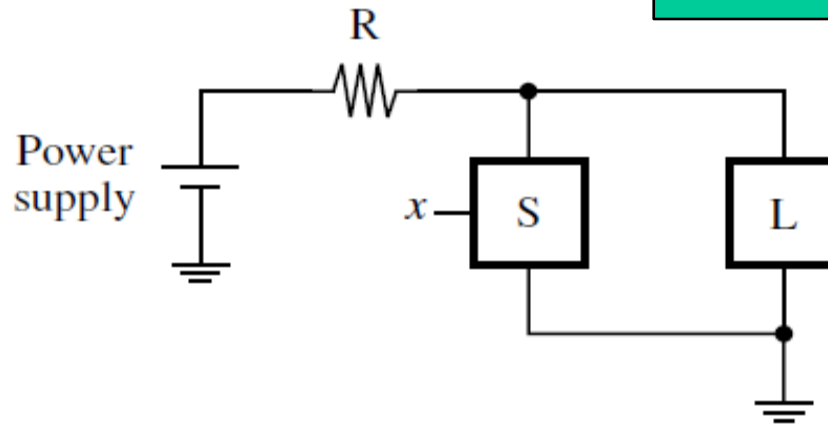


Figure 2.5 An inverting circuit.

Try implementation using switches

- $((A.B)+C).(D)$
- $(A+B)'$

Truth Tables

Useful aid for giving information for a logic function.

x_1	x_2	$x_1 \cdot x_2$	$x_1 + x_2$
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

AND OR

Figure: A truth table for the AND and OR operations.

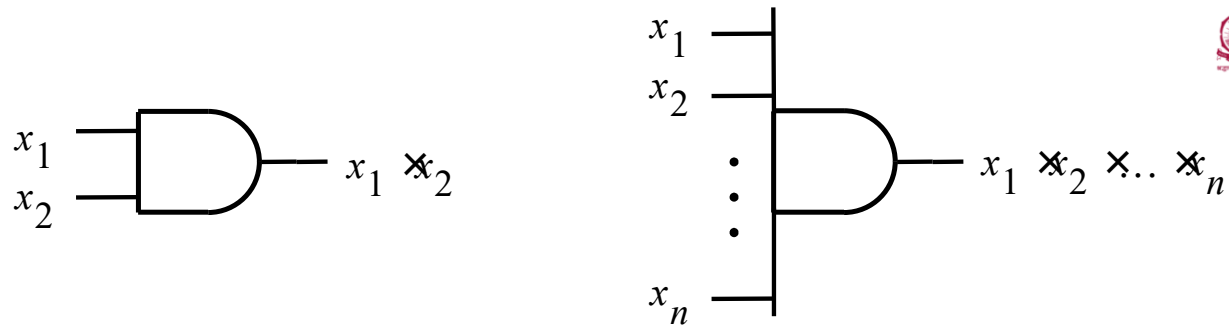
Truth Table for 3 input variables

x_1	x_2	x_3	$x_1 \cdot x_2 \cdot x_3$	$x_1 + x_2 + x_3$
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	0	1
1	0	0	0	1
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

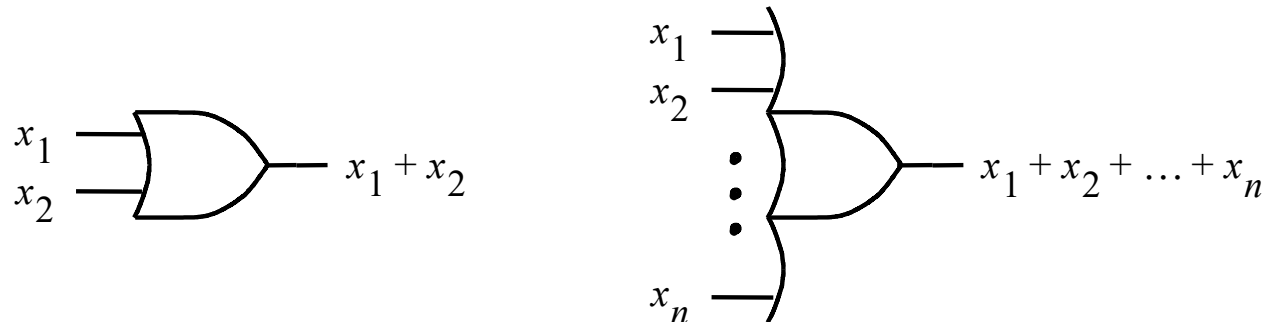
Figure : Three-input AND and OR operations.

Logic Gates

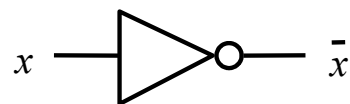
- ❖ Three basic logic operations can be used to implement logic functions of any complexity.
- ❖ Each logic operation can be implemented using Logic Gate.
- ❖ A logic gate has one or more inputs and one output that is a function of its inputs.
- ❖ It is often convenient to describe a logic circuit by drawing a circuit diagram, or *schematic*, consisting of graphical symbols representing the logic gates.



(a) AND gates



(b) OR gates



(c) NOT gate

Figure: The basic gates.

A Larger circuit is implemented by Network of Gates

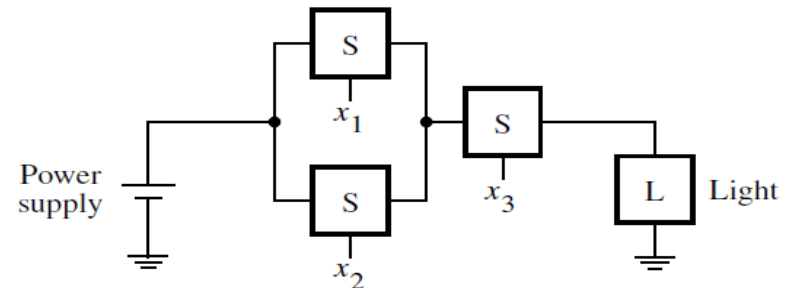
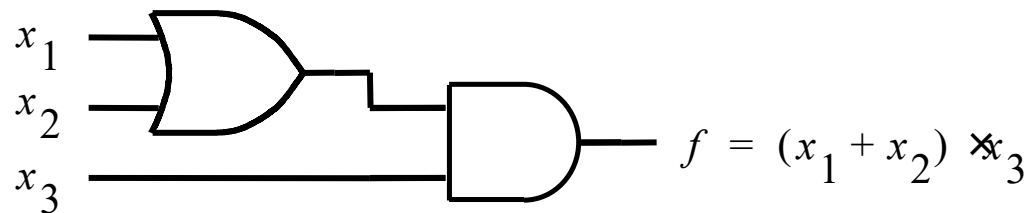
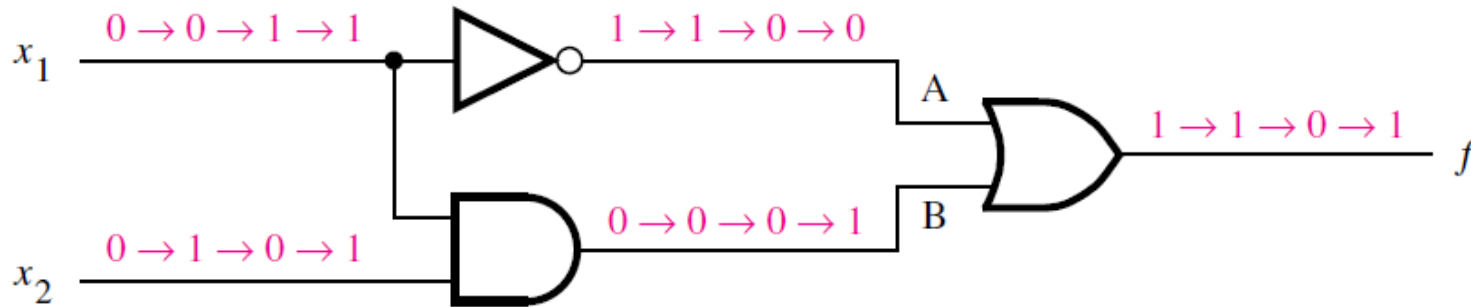


Figure: The function from Figure of switches .

Analysis vs Synthesis of Logic Networks

- **Analysis Process:** For an existing logic network, it must be possible to determine the function performed by the network. This task is referred to as the *analysis* process.
- **Synthesis Process:** The reverse task of designing a new network that implements a desired functional behavior is referred to as the *synthesis* process.
- The analysis process is straightforward and much simpler than the synthesis process

Analysis example



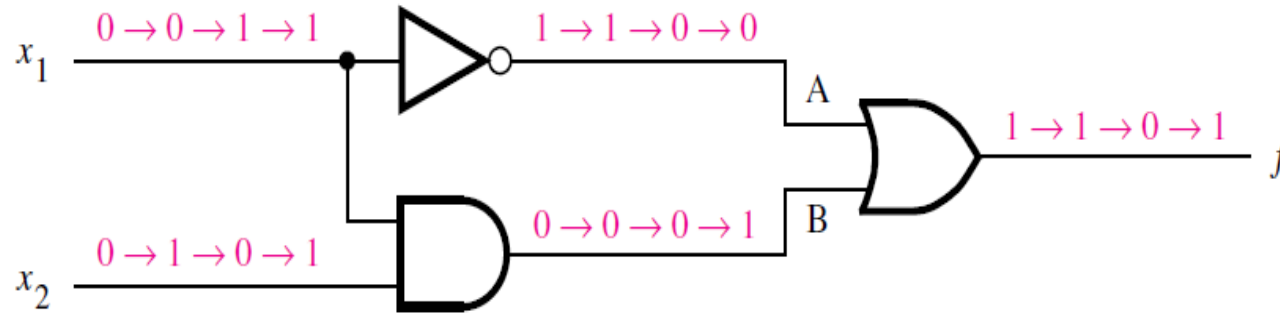
(a) Network that implements $f = \bar{x}_1 + x_1 \cdot x_2$

x_1	x_2	$f(x_1, x_2)$
0	0	1
0	1	1
1	0	0
1	1	1

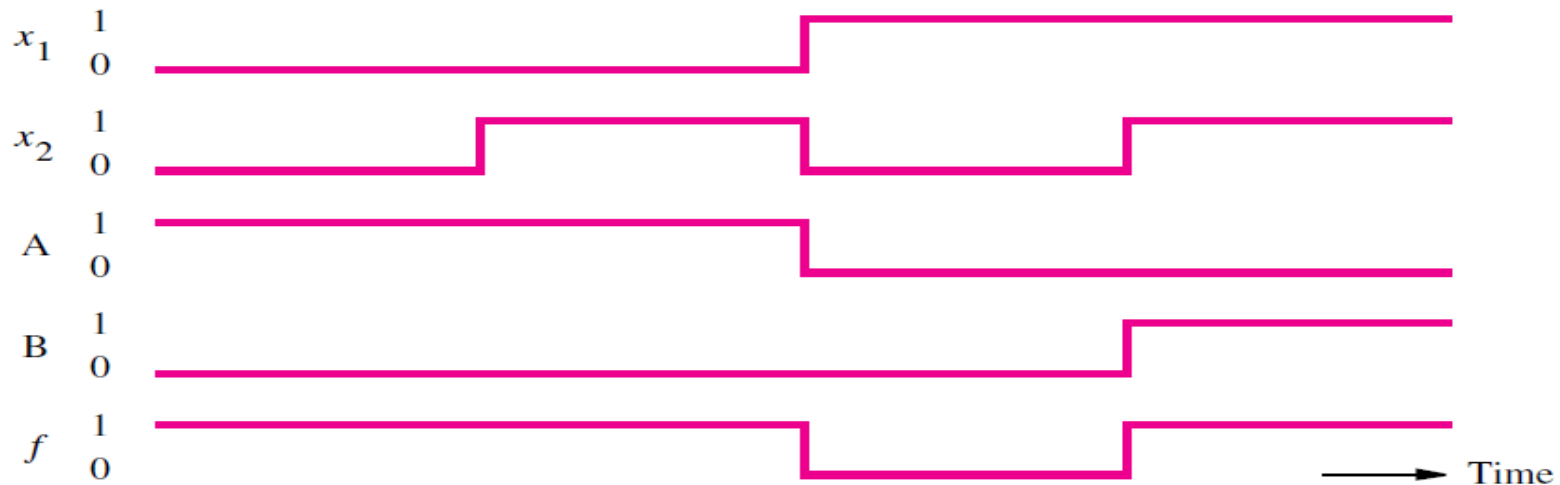
(b) Truth table for f

Check how signal A & B
Are changing with input
 x_1 & x_2

Timing Diagram: Functional Behavior



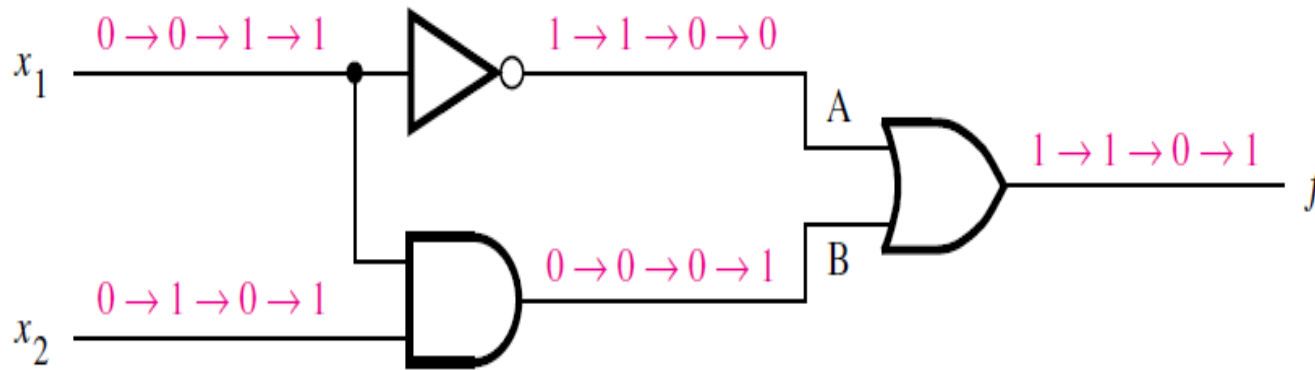
(a) Network that implements $f = \bar{x}_1 + x_1 \cdot x_2$



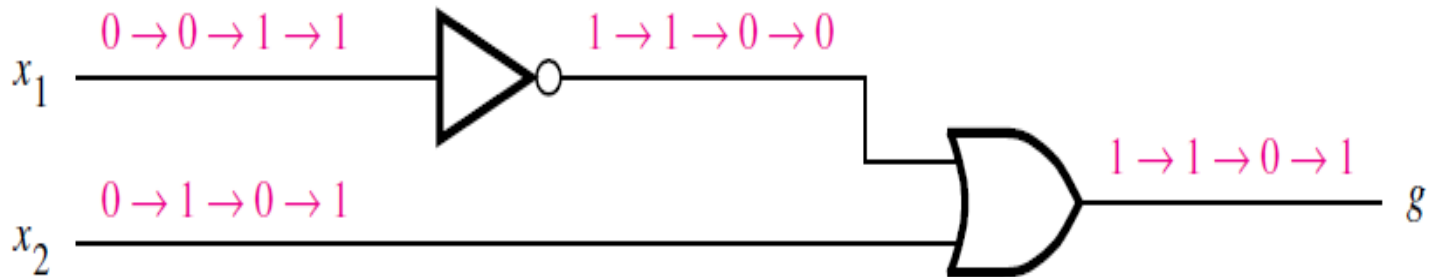
(c) Timing diagram

Check how signal A & B
Are changing with input

Functionally Equivalent Networks



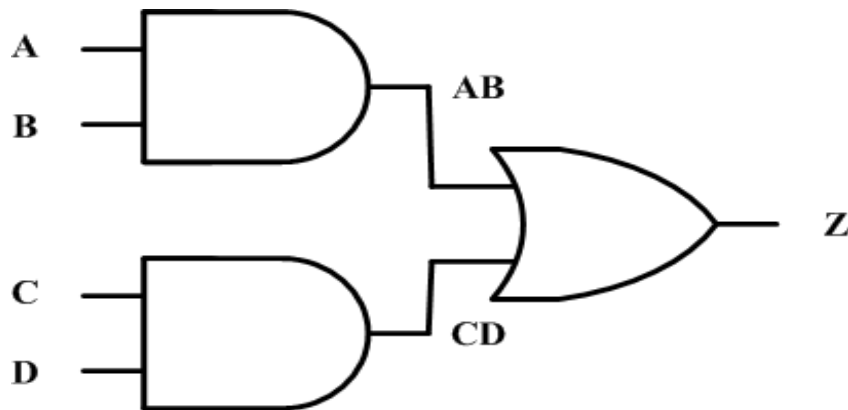
(a) Network that implements $f = \bar{x}_1 + x_1 \cdot x_2$



(d) Network that implements $g = \bar{x}_1 + x_2$

Truth table with 4 input variables

- we can also construct truth tables for any function.



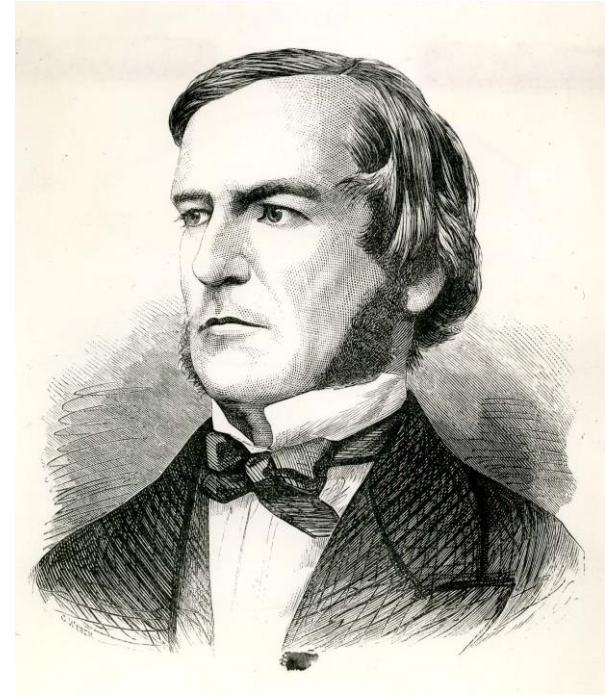
A	B	C	D	Z	AB	CD
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	0	1	0	0	0	0
0	0	1	1	1	0	1
0	1	0	0	0	0	0
0	1	0	1	0	0	0
0	1	1	0	0	0	0
0	1	1	1	1	0	1
1	0	0	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	0	0	0
1	0	1	1	1	0	1
1	1	0	0	1	1	0
1	1	0	1	1	1	0
1	1	1	0	1	1	0
1	1	1	1	1	1	1

Boolean Algebra

- Boolean Algebra
 - Axioms of Boolean Algebra
 - Single Variable Theorem
 - Two and Three variable Properties
 - Complements and Duals

Boolean Algebra

- In 1849 George Boole published a scheme for the algebraic description of processes involved in logical thought and reasoning . Subsequently, this scheme and its further refinements became known as *Boolean algebra*.
- It was almost 100 years later that this algebra found application in the engineering sense. In the late 1930s Claude Shannon showed that Boolean algebra provides an effective means of describing circuits built with switches .The algebra can be used to describe logic circuits.



Boolean Algebra:Axioms

$$1a \quad 0 \cdot 0 = 0 \quad (1)$$

$$1b \quad 1 + 1 = 1 \quad (2)$$

$$2a \quad 1 \cdot 1 = 1 \quad (3)$$

$$2b \quad 0 + 0 = 0 \quad (4)$$

$$3a \quad 0 \cdot 1 = 1 \cdot 0 = 0 \quad (5)$$

$$3b \quad 0 + 1 = 1 + 0 = 1 \quad (6)$$

$$4a \quad \text{If } x = 0 \quad \text{then} \quad \bar{x} = 1 \quad (7)$$

$$4b \quad \text{If } x = 1 \quad \text{then} \quad \bar{x} = 0 \quad (8)$$

Boolean Algebra: Single Variable Theorems

$$\begin{array}{lcl} 5a & x \cdot 0 & = 0 \end{array} \quad (9)$$

$$\begin{array}{lcl} 5b & x + 1 & = 1 \end{array} \quad (10)$$

$$\begin{array}{lcl} 6a & x \cdot 1 & = x \end{array} \quad (11)$$

$$\begin{array}{lcl} 6b & x + 0 & = x \end{array} \quad (12)$$

$$\begin{array}{lcl} 7a & x \cdot x & = x \end{array} \quad (13)$$

$$\begin{array}{lcl} 7b & x + x & = x \end{array} \quad (14)$$

$$\begin{array}{lcl} 8a & x \cdot \bar{x} & = 0 \end{array} \quad (15)$$

$$\begin{array}{lcl} 8b & x + \bar{x} & = 1 \end{array} \quad (16)$$

$$\begin{array}{lcl} 9 & \bar{\bar{x}} & = x \end{array} \quad (17)$$

Principle of Duality

- Given a logic expression, its **dual** is obtained by replacing all + operators with $\cdot$ operators, and vice versa, and by replacing all 0s with 1s, and vice versa.
- The dual of any true statement (axiom or theorem) in Boolean algebra is also a true statement.

$$A + 0 = A$$

OR +

A	0	RESULT
0	0	0
1	0	1

$$(a + b \cdot c) + e$$

$$A \cdot 1 = A$$

AND $\cdot$

A	1	RESULT
0	1	0
1	1	1

$$(a \cdot (b + c)) \cdot e$$

Boolean Algebra: Two & Three variable Properties

Commutativity:

$$10a \quad x \cdot y = y \cdot x \quad (18)$$

$$10b \quad x + y = y + x \quad (19)$$

Associativity:

$$11a \quad x \cdot (y \cdot z) = (x \cdot y) \cdot z \quad (20)$$

$$11b \quad x + (y + z) = (x + y) + z \quad (21)$$

Distributivity:

$$12a \quad x \cdot (y + z) = x \cdot y + x \cdot z \quad (22)$$

$$12b \quad x + y \cdot z = (x + y) \cdot (x + z) \quad (23)$$

Absorption:

$$13a \quad x + x \cdot y = x \quad (24)$$

$$13b \quad x \cdot (x + y) = x \quad (25)$$

Boolean Algebra: Two & Three variable Properties

Proof for 13a

$$x + x.y = x.1 + x.y$$

$$x(1 + y) = x \quad \{1 + y = 1\}$$

Proof for 13b

$$x.(x + y)$$

$$x.x + x.y = x + x.y = x$$

Combining:

$$14a \quad x.y + x.\bar{y} = x \quad (26)$$

$$14b \quad (x + y).(x + \bar{y}) = x \quad (27)$$

Proof for 14a

$$x.y + x.\bar{y} = x.(y + \bar{y}) \quad \{y + \bar{y} = 1\}$$

$$= x$$

Proof for 14b

$$(x + y).(x + \bar{y}) = x + y.\bar{y} = x \quad \{\text{Distributive Law, } y.\bar{y} = 0\}$$

Boolean Algebra: DeMorgans Theorem



Augustus De Morgan (27 June 1806 — 18 March 1871)

15a $\overline{x \cdot y} = \bar{x} + \bar{y}$ (28)

15b $\overline{x + y} = \bar{x} \cdot \bar{y}$ (29)

Proof of DeMorgan's Theorem using TruthTable

$$\overline{x \cdot y} = \bar{x} + \bar{y}$$

$$\overline{x + y} = \bar{x} \cdot \bar{y}$$

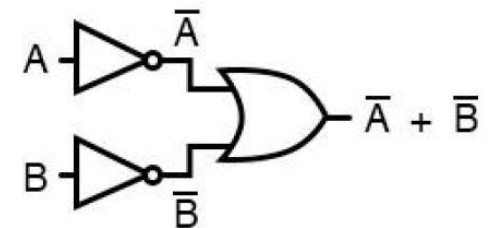
x	y	$x \cdot y$	$\overline{x \cdot y}$	$\bar{x}$	$\bar{y}$	$\bar{x} + \bar{y}$
0	0	0	1	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	1	1
1	1	1	0	0	0	0

$\underbrace{\hspace{10em}}$
LHS

$\underbrace{\hspace{10em}}$
RHS



... is equivalent to ...



$$\overline{AB} = \bar{A} + \bar{B}$$

Proof of DeMorgan's Theorem using TruthTable

$$\overline{x \cdot y} = \bar{x} + \bar{y}$$

$$\overline{x + y} = \bar{x} \cdot \bar{y}$$

X	Y	X+Y	$\overline{X+Y}$
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

X	Y	$\bar{X}$	$\bar{Y}$	$\bar{X} \cdot \bar{Y}$
0	0	1	1	1
0	1	1	0	0
1	0	0	1	0
1	1	0	0	0

Boolean Algebra: Two & Three variable Properties

$$16a \quad x + \bar{x} \cdot y = x + y \quad (30)$$

$$16b \quad x \cdot (\bar{x} + y) = x \cdot y \quad (31)$$

Proof of 16a

$$x + x' y = (x + x')(x + y) = 1 \cdot (x + y) = x + y$$

• Use 12b(distributive) 8b 6a

Proof of 16b

$$x \cdot (x' + y) = x \cdot x' + x \cdot y = 0 + x \cdot y = x \cdot y$$

Use 8a 6b

Boolean Algebra: Two & Three variable Properties

Consensus:

$$17a \quad x \cdot y + y \cdot z + \bar{x} \cdot z = x \cdot y + \bar{x} \cdot z \quad (32)$$

$$17b \quad (x + y) \cdot (y + z) \cdot (\bar{x} + z) = (x + y) \cdot (\bar{x} + z) \quad (33)$$

Proof for 17a

Consensus Theorem gives us the relationship

$$XY + X'Z + YZ = XY + X'Z$$

Note that in doing the reduction the first step is to add 1 to the YZ term. That 1 is in the form, $(X+X')$.

$$\begin{aligned} XY + X'Z + YZ &\rightarrow XY + X'Z + (X + X')YZ \\ &\rightarrow XY + X'Z + XYZ + X'YZ \\ &\rightarrow (XY + XYZ) + (X'Z + X'YZ) \\ &\rightarrow XY(1 + Z) + X'Z(1 + Y) \\ &\rightarrow XY + X'Z \end{aligned}$$

Boolean Algebra: Two & Three variable Properties

Proof for 17b

$$(A + B)(B' + C)(A + C) = (A + B)(B' + C)$$

$$\begin{aligned} \text{LHS} &= (A + B)(B' + C)(A + C) \\ &= (A + B)(B' + C)(A + C + 0) \\ &= (A + B)(B' + C)(A + B.B' + C) \\ &= (A + B)(B' + C)(A + B + C)(A + B' + C) \\ &= [(A + B)(A + B + C)] [(B' + C)(A + B' + C)] \\ &= (A + B)(B' + C) = \text{RHS} \end{aligned}$$

Other examples

Simplify the following-

$$1. XY + XY' = X(Y + Y') = X \cdot 1 = X$$

- Use 12a(distributive) 8b 6a

$$2. X \cdot (X + Y) = X \cdot X + X \cdot Y = X + XY = X(1 + Y) = X \cdot 1 = X$$

- Use 12b(distributive) 7a 12a 5b

$$3. (X + Y) \cdot (X + Y') = XX + XY' + XY + YY' = \\ X + XY' + XY + 0 = X(1 + Y' + Y) = X \cdot 1 = X$$

$$4. X(X' + Y) = XX' + XY = 0 + XY = XY$$

Simplify the logic function-

$$\overline{A}BC + A\overline{B}C + AB\overline{C} + ABC$$



Factoring **BC** out of 1st and 4th terms

$$BC(\overline{A} + A) + A\overline{B}C + AB\overline{C}$$



Applying identity **$A + \overline{A} = 1$**

$$BC(1) + A\overline{B}C + AB\overline{C}$$



Applying identity **$1A = A$**

$$BC + A\overline{B}C + AB\overline{C}$$



Factoring **B** out of 1st and 3rd terms

$$B(C + A\overline{C}) + A\overline{B}C$$



Applying rule **$A + \overline{A}B = A + B$** to the **$C + A\overline{C}$** term

$$B(C + A) + A\overline{B}C$$



Distributing terms

$$BC + AB + A\overline{B}C$$



Factoring **A** out of 2nd and 3rd terms

$$BC + A(B + \overline{B}C)$$



Applying rule **$A + \overline{A}B = A + B$** to the **$B + \overline{B}C$** term

$$BC + A(B + C)$$



Distributing terms

$$BC + AB + AC$$

or

Simplified result

$$AB + BC + AC$$

Examples

Simplify the following logic functions.

$$\begin{aligned} VX'Y + WXZ + VWYZ &= \\ \textcolor{blue}{V}X'Y + \textcolor{red}{W}XZ + \textcolor{blue}{V}\textcolor{red}{W}YZ &= \\ = \textcolor{blue}{V}X'Y + \textcolor{red}{W}XZ \end{aligned}$$

$$\begin{aligned} (A + B + E') (E + F + G') (A + B + F + G') &= \\ (\textcolor{blue}{A} + \textcolor{blue}{B} + E') (E + \textcolor{red}{F} + \textcolor{red}{G}') (\textcolor{blue}{A} + \textcolor{blue}{B} + \textcolor{red}{F} + \textcolor{red}{G}') &= \\ = (A + B + E') (E + F + G) \end{aligned}$$

Examples

- Simplify $F = X'YZ + X'Y\bar{Z} + XZ$

$$F = \bar{X}YZ + \bar{X}Y\bar{Z} + XZ$$

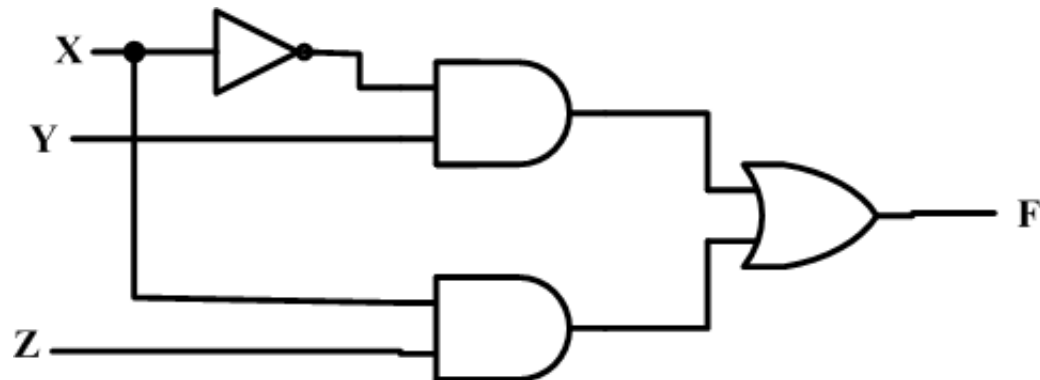
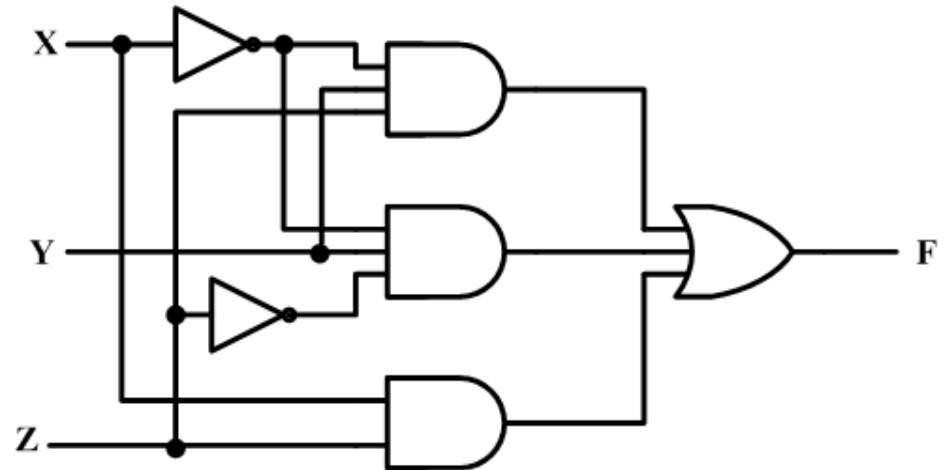
$$= \bar{X}Y(Z + \bar{Z}) + XZ \quad \text{by identity 14}$$

$$= \bar{X}Y \cdot 1 + XZ \quad \text{by identity 7}$$

$$= \bar{X}Y + XZ \quad \text{by identity 2}$$

Affect on implementation

- $F = X'YZ + X'YZ' + XZ$
- Reduces to $F = X'Y + XZ$



Let us prove the validity of the logic equation

$$(x_1 + x_3) \cdot (\bar{x}_1 + \bar{x}_3) = x_1 \cdot \bar{x}_3 + \bar{x}_1 \cdot x_3$$

The left-hand side can be manipulated as follows. Using the distributive property, 12a, gives

$$\text{LHS} = (x_1 + x_3) \cdot \bar{x}_1 + (x_1 + x_3) \cdot \bar{x}_3$$

Applying the distributive property again yields

$$\text{LHS} = x_1 \cdot \bar{x}_1 + x_3 \cdot \bar{x}_1 + x_1 \cdot \bar{x}_3 + x_3 \cdot \bar{x}_3$$

Note that the distributive property allows ANDing the terms in parenthesis in a way analogous to multiplication in ordinary algebra. Next, according to theorem 8a, the terms $x_1 \cdot \bar{x}_1$ and $x_3 \cdot \bar{x}_3$ are both equal to 0. Therefore,

$$\text{LHS} = 0 + x_3 \cdot \bar{x}_1 + x_1 \cdot \bar{x}_3 + 0$$

From 6b it follows that

$$\text{LHS} = x_3 \cdot \bar{x}_1 + x_1 \cdot \bar{x}_3$$

Finally, using the commutative property, 10a and 10b, this becomes

$$\text{LHS} = x_1 \cdot \bar{x}_3 + \bar{x}_1 \cdot x_3$$

which is the same as the right-hand side of the initial equation.

- Try

$$x_1 \cdot \bar{x}_3 + \bar{x}_2 \cdot \bar{x}_3 + x_1 \cdot x_3 + \bar{x}_2 \cdot x_3 = \bar{x}_1 \cdot \bar{x}_2 + x_1 \cdot x_2 + x_1 \cdot \bar{x}_2$$

- Solution:

$$x_1 \cdot \bar{x}_3 + \bar{x}_2 \cdot \bar{x}_3 + x_1 \cdot x_3 + \bar{x}_2 \cdot x_3 = \bar{x}_1 \cdot \bar{x}_2 + x_1 \cdot x_2 + x_1 \cdot \bar{x}_2$$

The left-hand side can be manipulated as follows

$$\begin{aligned} \text{LHS} &= x_1 \cdot \bar{x}_3 + x_1 \cdot x_3 + \bar{x}_2 \cdot \bar{x}_3 + \bar{x}_2 \cdot x_3 && \text{using } 10b \\ &= x_1 \cdot (\bar{x}_3 + x_3) + \bar{x}_2 \cdot (\bar{x}_3 + x_3) && \text{using } 12a \\ &= x_1 \cdot 1 + \bar{x}_2 \cdot 1 && \text{using } 8b \\ &= x_1 + \bar{x}_2 && \text{using } 6a \end{aligned}$$

The right-hand side can be manipulated as

$$\begin{aligned} \text{RHS} &= \bar{x}_1 \cdot \bar{x}_2 + x_1 \cdot (x_2 + \bar{x}_2) && \text{using } 12a \\ &= \bar{x}_1 \cdot \bar{x}_2 + x_1 \cdot 1 && \text{using } 8b \\ &= \bar{x}_1 \cdot \bar{x}_2 + x_1 && \text{using } 6a \\ &= x_1 + \bar{x}_1 \cdot \bar{x}_2 && \text{using } 10b \\ &= x_1 + \bar{x}_2 && \text{using } 16a \end{aligned}$$

Complement of a function

- In real implementation sometimes the complement of a function is needed. Have $F = X'YZ' + X'Y'Z$

$$F = \overline{X}Y\overline{Z} + \overline{X}\overline{Y}Z$$

$$\overline{F} = \overline{\overline{X}Y\overline{Z} + \overline{X}\overline{Y}Z}$$

$$= (\overline{\overline{X}Y\overline{Z}}) \cdot (\overline{\overline{X}\overline{Y}Z})$$

$$= (X + \overline{Y} + Z) \cdot (X + Y + \overline{Z})$$

Demorgan's Theorem

$$\overline{a + b} = \overline{a} \cdot \overline{b}$$

$$\overline{a \cdot b} = \overline{a} + \overline{b}$$

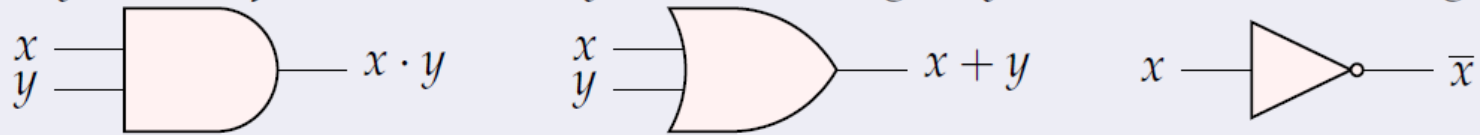
How Duals and Complement are related

- What is meant by the dual of a function?
 - The *dual* of a function is obtained by interchanging OR and AND operations and replacing 1s and 0s with 0s and 1s.
- Shortcut to getting function complement
 - Starting with the equation on the previous slide
 - Generate the dual $F=(X'+Y+Z')(X'+Y'+Z)$
 - Complement each literal to get:
 - $F'=(X+Y'+Z)(X+Y+Z')$

Synthesis of simple circuits

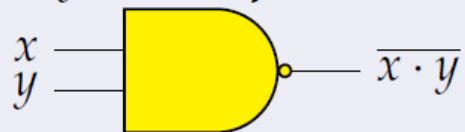
Theorem

Any Boolean function can be synthesized using only AND, OR, and NOT gates.



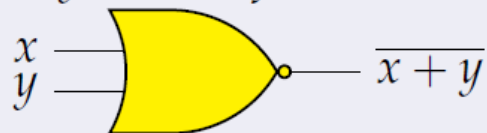
Theorem

Any Boolean function can be synthesized using only NAND gates.



Theorem

Any Boolean function can be synthesized using only NOR gates.



Synthesis using AND, OR, NOT Gates

Given a Boolean function f given in the form of a truth table, the expression that realizes f can be obtained:

- (SUM-OF-PRODUCTS) by considering rows for which $f = 1$, or
- (PRODUCTS-OF-SUMS) by considering rows for which $f = 0$.

Sum of Products Canonical form

- For a function of n variables a **minterm** is a product term in which each of the variable occur exactly once. For example: $x_0 \cdot x_1 \cdot \overline{x_2}$ and $\overline{x_0} \cdot \overline{x_1} \cdot x_2$.
- A function f can be represented by an expression that is sum of **minterms**.
- A logical expression that consists of product terms that are summed (ORed) together is called a **sum-of-product** form.
- If each of the product term is a minterm, then the expression is called **canonical sum-of-product** expression.
- Notice that two logically equivalent functions will have the same canonical representation.

Two Variable Minterms

Minterms			
x	y	term	notation
0	0	$x'.y'$	m0
0	1	$x'.y$	m1
1	0	$x.y'$	m2
1	1	$x.y$	m3

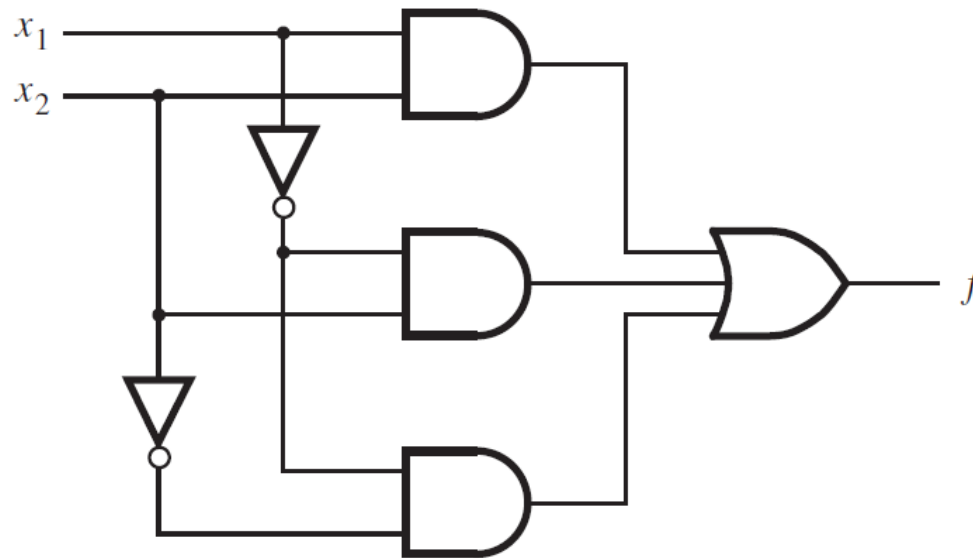
Simple problem of synthesis

- Assume that x_1 and x_2 represent the states of two switches, either of which may produce a 0 or 1.
- The function of the circuit is to continuously monitor the state of the switches and to produce an output logic value 1 whenever the switches (x_1, x_2) are in states (0, 0), (0, 1), or (1, 1). If the state of the switches is (1, 0), the output should be 0.
- We can express the required behavior using a truth table,

x_1	x_2	$f(x_1, x_2)$
0	0	1 ← $\bar{x}_1\bar{x}_2$
0	1	1 ← $\bar{x}_1x_2$
1	0	0
1	1	1 ← x_1x_2

$$f(x_1, x_2) = x_1x_2 + \bar{x}_1\bar{x}_2 + \bar{x}_1x_2$$

Canonical SOP –implementation using Logic Gates



(a) Canonical sum-of-products

$$f(x_1, x_2) = x_1x_2 + \bar{x}_1\bar{x}_2 + \bar{x}_1x_2$$

The process whereby we begin with a description of the desired functional behavior and then generate a circuit that realizes this behavior is called *synthesis*.

Canonical SOP expression can be reduced using Boolean Algebra

$$f(x_1, x_2) = x_1x_2 + \bar{x}_1\bar{x}_2 + \bar{x}_1x_2$$

1. Using 7b, third Product term is replicated

$$f(x_1, x_2) = x_1x_2 + \bar{x}_1\bar{x}_2 + \bar{x}_1x_2 + \bar{x}_1x_2$$

2. Using the commutative property 10b to interchange the second and third product terms gives

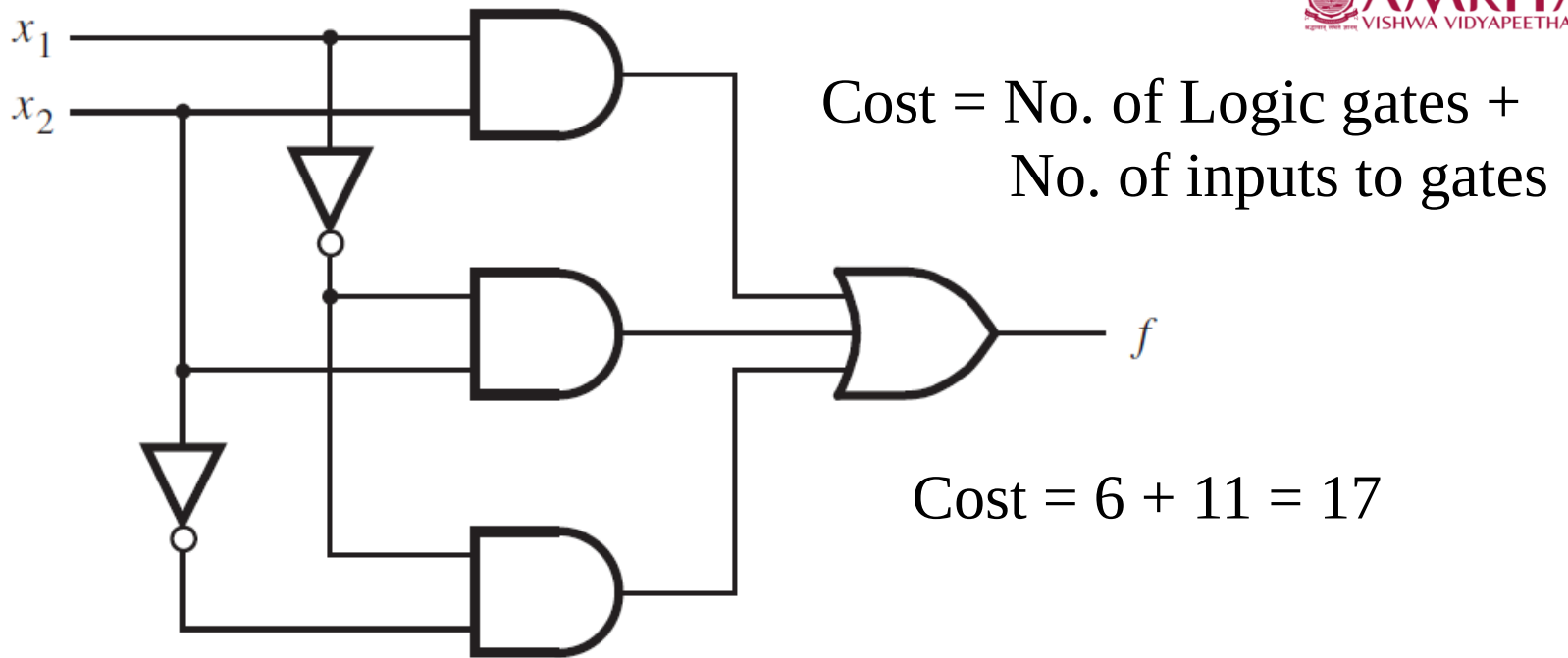
$$f(x_1, x_2) = x_1x_2 + \bar{x}_1x_2 + \bar{x}_1\bar{x}_2 + \bar{x}_1x_2$$

3. The distributive property 12a

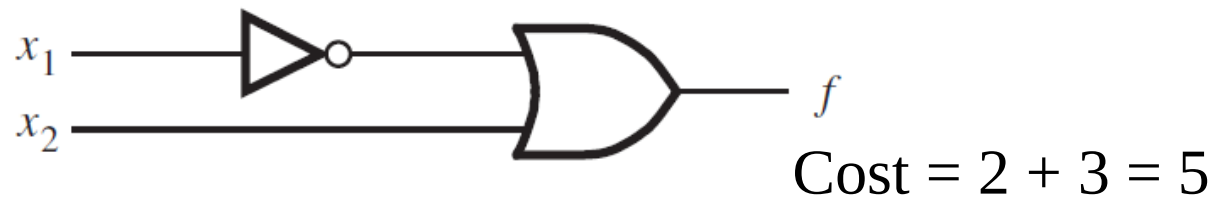
$$f(x_1, x_2) = (x_1 + \bar{x}_1)x_2 + \bar{x}_1(\bar{x}_2 + x_2)$$

4. Using 8b and 6a

$$f(x_1, x_2) = 1 \cdot x_2 + \bar{x}_1 \cdot 1 \quad \Rightarrow \quad f(x_1, x_2) = x_2 + \bar{x}_1$$



(a) Canonical sum-of-products



(b) Minimal-cost realization

Three Variable Minterms

Row number	x_1	x_2	x_3	Minterm
0	0	0	0	$m_0 = \bar{x}_1\bar{x}_2\bar{x}_3$
1	0	0	1	$m_1 = \bar{x}_1\bar{x}_2x_3$
2	0	1	0	$m_2 = \bar{x}_1x_2\bar{x}_3$
3	0	1	1	$m_3 = \bar{x}_1x_2x_3$
4	1	0	0	$m_4 = x_1\bar{x}_2\bar{x}_3$
5	1	0	1	$m_5 = x_1\bar{x}_2x_3$
6	1	1	0	$m_6 = x_1x_2\bar{x}_3$
7	1	1	1	$m_7 = x_1x_2x_3$

- Consider three variable function and synthesize the circuit for it.

Row number	x_1	x_2	x_3	$f(x_1, x_2, x_3)$
0	0	0	0	0
1	0	0	1	1
2	0	1	0	0
3	0	1	1	0
4	1	0	0	1
5	1	0	1	1
6	1	1	0	1
7	1	1	1	0

$$f(x_1, x_2, x_3) = \sum (m_1, m_4, m_5, m_6)$$

$$f(x_1, x_2, x_3) = \sum m(1, 4, 5, 6)$$

- Consider three variable function and synthesize the circuit for it.

Row number	x_1	x_2	x_3	$f(x_1, x_2, x_3)$
0	0	0	0	0
1	0	0	1	1 ← $\bar{x}_1\bar{x}_2x_3$
2	0	1	0	0
3	0	1	1	0
4	1	0	0	1 ← $x_1\bar{x}_2\bar{x}_3$
5	1	0	1	1 ← $x_1\bar{x}_2x_3$
6	1	1	0	1 ← $x_1x_2\bar{x}_3$
7	1	1	1	0

Canonical SOP expression

$$f(x_1, x_2, x_3) = \bar{x}_1\bar{x}_2x_3 + x_1\bar{x}_2\bar{x}_3 + x_1\bar{x}_2x_3 + x_1x_2\bar{x}_3$$

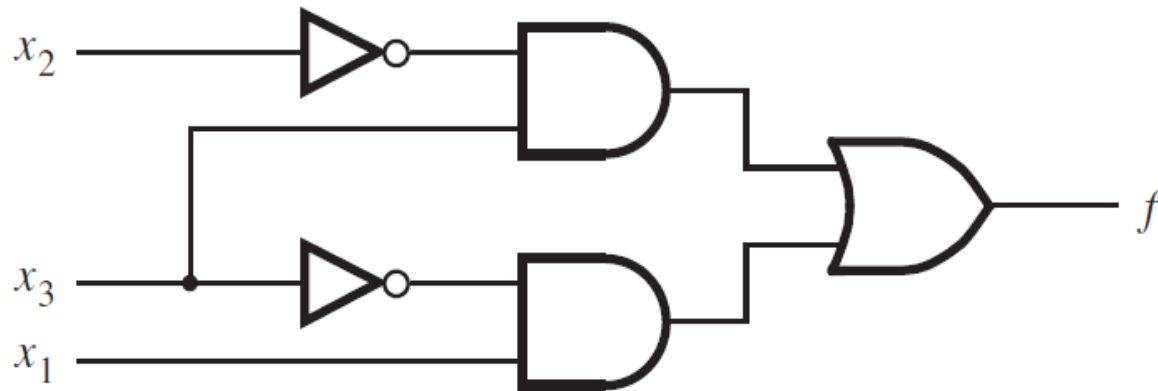
- Draw circuit diagram
- Reduce it and again draw the reduced circuit diagram.
- Compare the cost-A good indication of the *cost* of a logic circuit is the total number of gates plus the total number of inputs to all gates in the circuit.

$$f(x_1, x_2, x_3) = \bar{x}_1\bar{x}_2x_3 + x_1\bar{x}_2\bar{x}_3 + x_1\bar{x}_2x_3 + x_1x_2\bar{x}_3$$



$$\begin{aligned} f &= (\bar{x}_1 + x_1)\bar{x}_2x_3 + x_1(\bar{x}_2 + x_2)\bar{x}_3 \\ &= 1 \cdot \bar{x}_2x_3 + x_1 \cdot 1 \cdot \bar{x}_3 \\ &= \bar{x}_2x_3 + x_1\bar{x}_3 \end{aligned}$$

$$= \bar{x}_2x_3 + x_1\bar{x}_3$$



(a) A minimal sum-of-products realization

Example

Another example:

A	B	Cin	Sum	Cout	Sum = $\overline{A} \overline{B} \text{Cin} + \overline{A} B \overline{\text{Cin}} + A \overline{B} \overline{\text{Cin}} + A B \text{Cin}$				
0	0	0	0	0					
0	0	1	1	0					
0	1	0	1	0					
0	1	1	0	1					
1	0	0	1	0					
1	0	1	0	1					
1	1	0	0	1					
1	1	1	1	1					

$$\text{Cout} = \overline{A} B \text{Cin} + A \overline{B} \text{Cin} + A B \overline{\text{Cin}} + A B \text{Cin}$$

Product of Sum Canonical Form

- For a function of n variables a **maxterm** is a sum term in which each of the variable occur exactly once. For example

$$(x_0 + x_1 + \overline{x_2}) \text{ and } (\overline{x_0} + \overline{x_1} + x_2).$$

- A function f can be represented by an expression that is product of **maxterms**.
- A logical expression that consists of sum terms that are factors of logical product (AND) is called a **product-of-sum** form.
- If each of the sum term is a maxterm, then the expression is called **canonical product-of-sum** expression.
- Notice that two logically equivalent functions will have the same canonical product-of-sum representation.

Maxterms

Row number	x_1	x_2	x_3	Minterm	Maxterm
0	0	0	0	$m_0 = \bar{x}_1 \bar{x}_2 \bar{x}_3$	$M_0 = x_1 + x_2 + x_3$
1	0	0	1	$m_1 = \bar{x}_1 \bar{x}_2 x_3$	$M_1 = x_1 + x_2 + \bar{x}_3$
2	0	1	0	$m_2 = \bar{x}_1 x_2 \bar{x}_3$	$M_2 = x_1 + \bar{x}_2 + x_3$
3	0	1	1	$m_3 = \bar{x}_1 x_2 x_3$	$M_3 = x_1 + \bar{x}_2 + \bar{x}_3$
4	1	0	0	$m_4 = x_1 \bar{x}_2 \bar{x}_3$	$M_4 = \bar{x}_1 + x_2 + x_3$
5	1	0	1	$m_5 = x_1 \bar{x}_2 x_3$	$M_5 = \bar{x}_1 + x_2 + \bar{x}_3$
6	1	1	0	$m_6 = x_1 x_2 \bar{x}_3$	$M_6 = \bar{x}_1 + \bar{x}_2 + x_3$
7	1	1	1	$m_7 = x_1 x_2 x_3$	$M_7 = \bar{x}_1 + \bar{x}_2 + \bar{x}_3$

Maxterms

- The principle of duality suggests that if it is possible to synthesize a function f by considering the rows in the truth table for which $f = 1$, then it should also be possible to synthesize f by considering the rows for which $f = 0$.
- This alternative approach uses the complements of minterms, which are called *maxterms*.

- Consider three variable function and synthesize the circuit for it using Maxterms.

Row number	x_1	x_2	x_3	$f(x_1, x_2, x_3)$
0	0	0	0	0 ← $(x_1 + x_2 + x_3)$
1	0	0	1	1
2	0	1	0	0 ← $(x_1 + \bar{x}_2 + x_3)$
3	0	1	1	0 ← $(x_1 + \bar{x}_2 + \bar{x}_3)$
4	1	0	0	1
5	1	0	1	1
6	1	1	0	1
7	1	1	1	0 ← $(\bar{x}_1 + \bar{x}_2 + \bar{x}_3)$

$$f(x_1, x_2, x_3) = \Pi(M_0, M_2, M_3, M_7)$$

$$f(x_1, x_2, x_3) = \Pi M(0, 2, 3, 7)$$

- Consider three variable function and synthesize the circuit for it using Maxterms

Row number	x_1	x_2	x_3	$f(x_1, x_2, x_3)$
0	0	0	0	0 ← $(x_1 + x_2 + x_3)$
1	0	0	1	1
2	0	1	0	0 ← $(x_1 + \bar{x}_2 + x_3)$
3	0	1	1	0 ← $(x_1 + \bar{x}_2 + \bar{x}_3)$
4	1	0	0	1
5	1	0	1	1
6	1	1	0	1
7	1	1	1	0 ← $(\bar{x}_1 + \bar{x}_2 + \bar{x}_3)$

Canonical POS expression

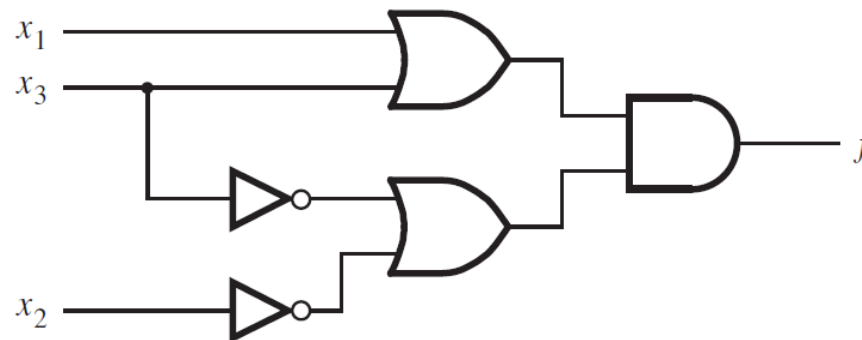
$$\begin{aligned}
 f(x_1, x_2, x_3) &= M_0 \cdot M_2 \cdot M_3 \cdot M_7 \\
 &= (x_1 + x_2 + x_3)(x_1 + \bar{x}_2 + x_3)(x_1 + \bar{x}_2 + \bar{x}_3)(\bar{x}_1 + \bar{x}_2 + \bar{x}_3)
 \end{aligned}$$

Reduce it and draw circuit diagram

$$f(x_1, x_2, x_3) = (x_1 + x_2 + x_3)(x_1 + \bar{x}_2 + x_3)(x_1 + \bar{x}_2 + \bar{x}_3)(\bar{x}_1 + \bar{x}_2 + \bar{x}_3)$$

$$f = ((x_1 + x_3) + x_2)((x_1 + x_3) + \bar{x}_2)(x_1 + (\bar{x}_2 + \bar{x}_3))(\bar{x}_1 + (\bar{x}_2 + \bar{x}_3))$$

$$f = (x_1 + x_3)(\bar{x}_2 + \bar{x}_3) \quad \text{Using 14b-combining property}$$



(b) A minimal product-of-sums realization

Consider the function

$$f(x_1, x_2, x_3) = \sum m(2, 3, 4, 6, 7)$$

Q1> Write the function in canonical SOP form.

Q2> Simplify the function in SOP form.

Q3> Write corresponding Maxterm list-canonical POS form.

Q4> Reduce canonical POS expression.

$$f = m_2 + m_3 + m_4 + m_6 + m_7$$

$$= \bar{x}_1 x_2 \bar{x}_3 + \bar{x}_1 x_2 x_3 + x_1 \bar{x}_2 \bar{x}_3 + x_1 x_2 \bar{x}_3 + x_1 x_2 x_3$$

$$f = \bar{x}_1 x_2 (\bar{x}_3 + x_3) + x_1 (\bar{x}_2 + x_2) \bar{x}_3 + x_1 x_2 (\bar{x}_3 + x_3)$$

$$= \bar{x}_1 x_2 + x_1 \bar{x}_3 + x_1 x_2$$

$$= (\bar{x}_1 + x_1) x_2 + x_1 \bar{x}_3$$

$$= x_2 + x_1 \bar{x}_3$$

$$f(x_1, x_2, x_3) = \Pi M(0, 1, 5)$$

Then, the canonical POS expression is derived as

$$\begin{aligned} f &= M_0 \cdot M_1 \cdot M_5 \\ &= (x_1 + x_2 + x_3)(x_1 + x_2 + \bar{x}_3)(\bar{x}_1 + x_2 + \bar{x}_3) \end{aligned}$$

A simplified POS expression can be derived as

$$\begin{aligned} f &= (x_1 + x_2 + x_3)(x_1 + x_2 + \bar{x}_3)(x_1 + x_2 + \bar{x}_3)(\bar{x}_1 + x_2 + \bar{x}_3) \\ &= ((x_1 + x_2) + x_3)((x_1 + x_2) + \bar{x}_3)(x_1 + (x_2 + \bar{x}_3))(\bar{x}_1 + (x_2 + \bar{x}_3)) \\ &= ((x_1 + x_2) + x_3\bar{x}_3)(x_1\bar{x}_1 + (x_2 + \bar{x}_3)) \\ &= (x_1 + x_2)(x_2 + \bar{x}_3) \end{aligned}$$

Find minimum cost SOP expression for this function

Suppose that a four-variable function is defined by

$$f(x_1, x_2, x_3, x_4) = \sum m(3, 7, 9, 12, 13, 14, 15)$$

The canonical SOP expression for this function is

$$f = \bar{x}_1\bar{x}_2x_3x_4 + \bar{x}_1x_2x_3x_4 + x_1\bar{x}_2\bar{x}_3x_4 + x_1x_2\bar{x}_3\bar{x}_4 + x_1x_2\bar{x}_3x_4 + x_1x_2x_3\bar{x}_4 + x_1x_2x_3x_4$$

A simpler SOP expression can be obtained as follows

$$\begin{aligned} f &= \bar{x}_1(\bar{x}_2 + x_2)x_3x_4 + x_1(\bar{x}_2 + x_2)\bar{x}_3x_4 + x_1x_2\bar{x}_3(\bar{x}_4 + x_4) + x_1x_2x_3(\bar{x}_4 + x_4) \\ &= \bar{x}_1x_3x_4 + x_1\bar{x}_3x_4 + x_1x_2\bar{x}_3 + x_1x_2x_3 \\ &= \bar{x}_1x_3x_4 + x_1\bar{x}_3x_4 + x_1x_2(\bar{x}_3 + x_3) \\ &= \bar{x}_1x_3x_4 + x_1\bar{x}_3x_4 + x_1x_2 \end{aligned}$$

Problem: Design the minimum-cost product-of-sums expression for the function $f(x_1, x_2, x_3, x_4) = \sum m(0, 2, 4, 5, 6, 7, 8, 10, 12, 14, 15)$.

Solution: The function is defined in terms of its minterms. To find a POS expression we should start with the definition in terms of maxterms, which is $f = \prod M(1, 3, 9, 11, 13)$. Thus,

$$\begin{aligned} f &= M_1 \cdot M_3 \cdot M_9 \cdot M_{11} \cdot M_{13} \\ &= (x_1 + x_2 + x_3 + \bar{x}_4)(x_1 + x_2 + \bar{x}_3 + \bar{x}_4)(\bar{x}_1 + x_2 + x_3 + \bar{x}_4)(\bar{x}_1 + x_2 + \bar{x}_3 + \bar{x}_4)(\bar{x}_1 + \bar{x}_2 + x_3 + \bar{x}_4) \end{aligned}$$

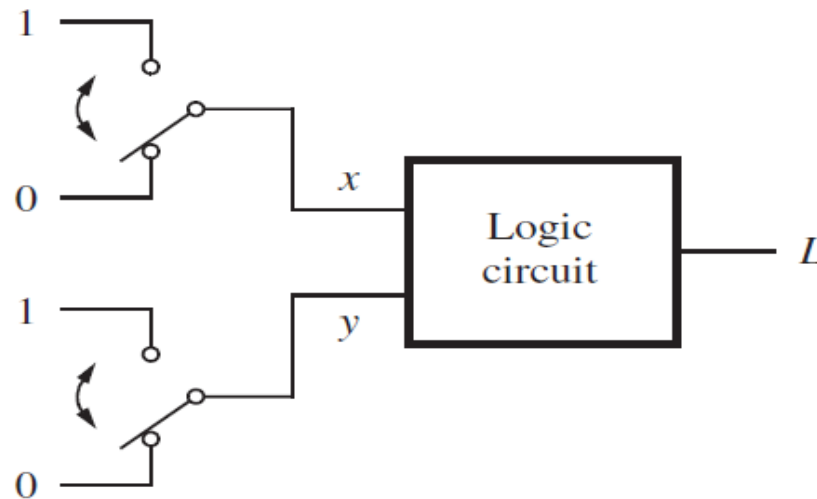
We can rewrite the product of the first two maxterms as

$$\begin{aligned} M_1 \cdot M_3 &= (x_1 + x_2 + \bar{x}_4 + x_3)(x_1 + x_2 + \bar{x}_4 + \bar{x}_3) && \text{using commutative property } 10b \\ &= x_1 + x_2 + \bar{x}_4 + x_3\bar{x}_3 && \text{using distributive property } 12b \\ &= x_1 + x_2 + \bar{x}_4 + 0 && \text{using theorem } 8a \\ &= x_1 + x_2 + \bar{x}_4 && \text{using theorem } 6b \end{aligned}$$

Similarly, $M_9 \cdot M_{11} = \bar{x}_1 + x_2 + \bar{x}_4$. Now, we can use M_9 again, according to property $7a$, to derive $M_9 \cdot M_{13} = \bar{x}_1 + x_3 + \bar{x}_4$. Hence

$$f = (x_1 + x_2 + \bar{x}_4)(\bar{x}_1 + x_2 + \bar{x}_4)(\bar{x}_1 + x_3 + \bar{x}_4)$$

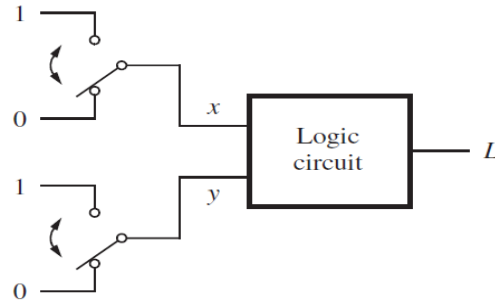
Try this problem: Write Truth Table and draw circuit diagram



(a) Two switches that control a light

The required behavior is that the light should be on only if one, but not both, of the toggle switches is in the top position.

Truth Table and Circuit diagram

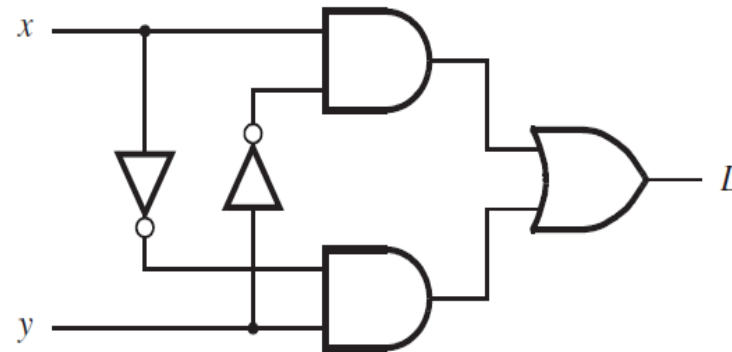


$$L = \bar{x} \cdot y + x \cdot \bar{y}, \text{ we can write } L = x \oplus y.$$

(a) Two switches that control a light

x	y	L
0	0	0
0	1	1
1	0	1
1	1	0

(b) Truth table



(c) Logic network



(d) XOR gate symbol

Synthesis Example

A circuit that controls a given digital system has three inputs: x_1 , x_2 , and x_3 . It has to recognize three different conditions:

- Condition A is true if x_3 is true and either x_1 is true or x_2 is false
- Condition B is true if x_1 is true and either x_2 or x_3 is false
- Condition C is true if x_2 is true and either x_1 is true or x_3 is false

The control circuit must produce an output of 1 if at least two of the conditions A , B , and C are true. Design the simplest circuit that can be used for this purpose.

Solution: Using 1 for true and 0 for false, we can express the three conditions as follows:

$$A = x_3(x_1 + \bar{x}_2) = x_3x_1 + x_3\bar{x}_2$$

$$B = x_1(\bar{x}_2 + \bar{x}_3) = x_1\bar{x}_2 + x_1\bar{x}_3$$

$$C = x_2(x_1 + \bar{x}_3) = x_2x_1 + x_2\bar{x}_3$$

Then, the desired output of the circuit can be expressed as $f = AB + AC + BC$. These product terms can be determined as:

$$\begin{aligned} AB &= (x_3x_1 + x_3\bar{x}_2)(x_1\bar{x}_2 + x_1\bar{x}_3) \\ &= x_3x_1x_1\bar{x}_2 + x_3x_1x_1\bar{x}_3 + x_3\bar{x}_2x_1\bar{x}_2 + x_3\bar{x}_2x_1\bar{x}_3 \\ &= x_3x_1\bar{x}_2 + 0 + x_3\bar{x}_2x_1 + 0 \\ &= x_1\bar{x}_2x_3 \end{aligned}$$

$$\begin{aligned} AC &= (x_3x_1 + x_3\bar{x}_2)(x_2x_1 + x_2\bar{x}_3) \\ &= x_3x_1x_2x_1 + x_3x_1x_2\bar{x}_3 + x_3\bar{x}_2x_2x_1 + x_3\bar{x}_2x_2\bar{x}_3 \\ &= x_3x_1x_2 + 0 + 0 + 0 \\ &= x_1x_2x_3 \end{aligned}$$

$$\begin{aligned} BC &= (x_1\bar{x}_2 + x_1\bar{x}_3)(x_2x_1 + x_2\bar{x}_3) \\ &= x_1\bar{x}_2x_2x_1 + x_1\bar{x}_2x_2\bar{x}_3 + x_1\bar{x}_3x_2x_1 + x_1\bar{x}_3x_2\bar{x}_3 \\ &= 0 + 0 + x_1\bar{x}_3x_2 + x_1\bar{x}_3x_2 \\ &= x_1x_2\bar{x}_3 \end{aligned}$$

Therefore, f can be written as

$$\begin{aligned} f &= x_1\bar{x}_2x_3 + x_1x_2x_3 + x_1x_2\bar{x}_3 \\ &= x_1(\bar{x}_2 + x_2)x_3 + x_1x_2(x_3 + \bar{x}_3) \\ &= x_1x_3 + x_1x_2 \\ &= x_1(x_3 + x_2) \end{aligned}$$

Digital Electronics & Systems

19ECE204

Lecture 10-12

Ms. Bhavana V
Assistant Professor
Dept. of ECE
ASE, Bengaluru

XOR and XNOR Logic Gates

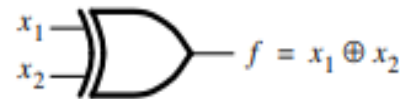
Exclusive-OR (XOR) Gate

- So far we have encountered AND, OR, and NOT gates as the basic elements from which logic circuits can be constructed.
- There is another basic element that is very useful in practice, particularly for building circuits that perform arithmetic operations.
- This element realizes the Exclusive-OR function.

For n-input XOR gate
output is high, when odd
number of inputs are high

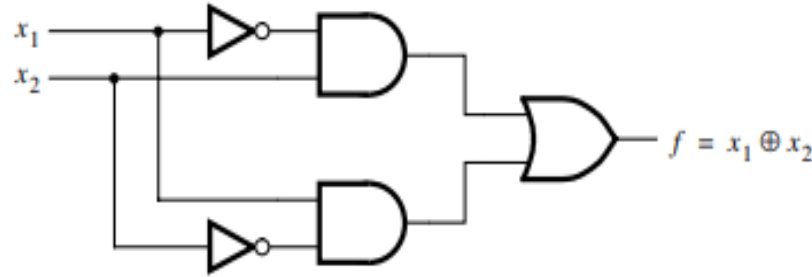
x_1	x_2	$f = x_1 \oplus x_2$
0	0	0
0	1	1
1	0	1
1	1	0

(a) Truth table



(b) Graphical symbol

XOR and XNOR Logic Gates



(c) Sum-of-products implementation

- The truth table for this function is similar to the OR function except that $f = 0$ when both inputs are 1.
- Because of this similarity, the function is called Exclusive-OR, which is commonly abbreviated as XOR.
- The XOR operation is usually denoted with the $\oplus$ symbol. It can be realized in the sum-of-products form as

$$x_1 \oplus x_2 = \overline{x_1}x_2 + x_1\overline{x_2}$$

XOR and XNOR Logic Gates

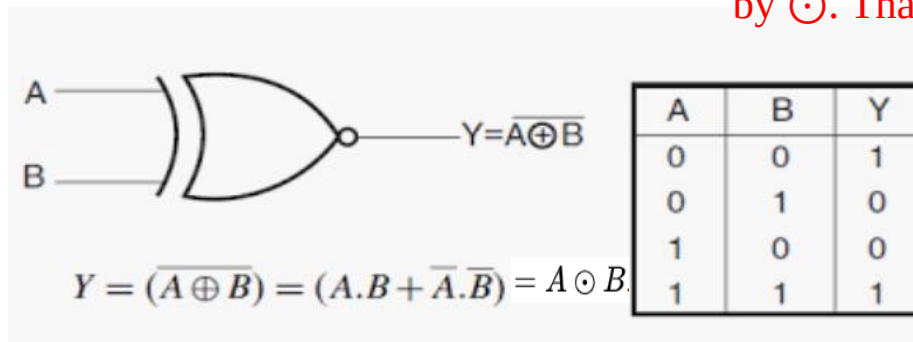
Exclusive-NOR (XNOR) Gate

- The Exclusive-NOR Gate function is a digital logic gate that is the reverse or complementary form of the Exclusive-OR function.
- Basically the “Exclusive-NOR” gate is a combination of the Exclusive-OR gate and the NOT gate.



2-input "Ex-OR" gate plus a "NOT" gate

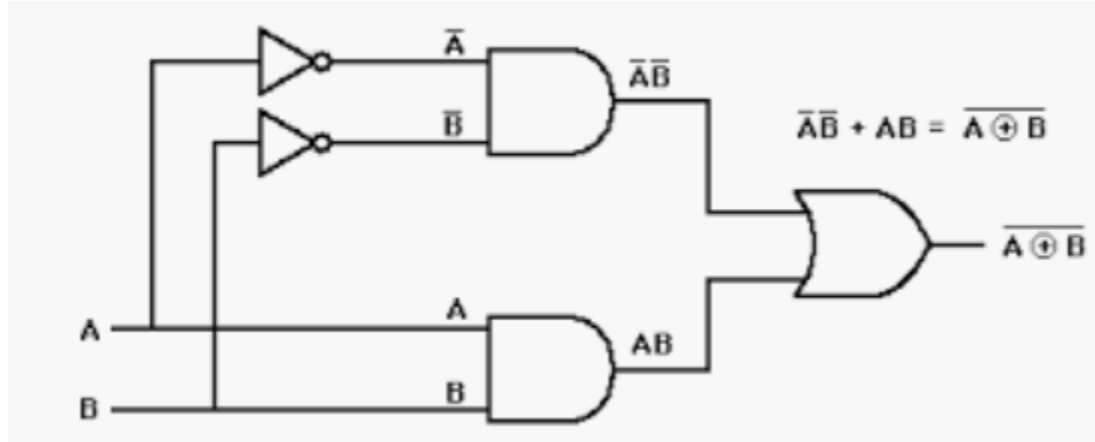
The logical XNOR operation is represented by $\odot$. That is a dot surrounded by circle.



XOR and XNOR Logic Gates

Exclusive-NOR (XNOR) Gate

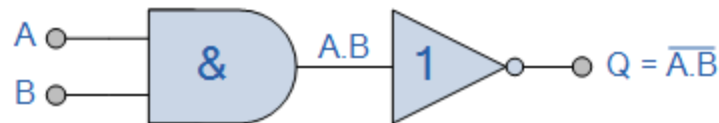
- The output of a 2-input XNOR gate is high when all of its inputs are same, either high or low. If its inputs are different then the output of the XNOR gate is low.
- For n-input XNOR gate, output is high when even number of inputs are high.



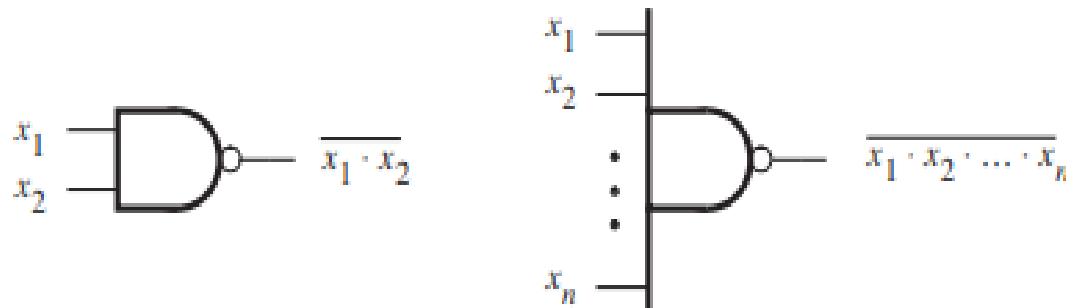
SOP Implementation

NAND and NOR Logic Networks

- The NAND and NOR gates are called Universal gates since with either all other logic gates can be implemented.
- NAND Gate-**
 - NAND gate is equal to AND gate followed by NOT gate.



Logic NAND Gate Equivalence



Symbol for NAND Gate

NAND and NOR Logic Networks

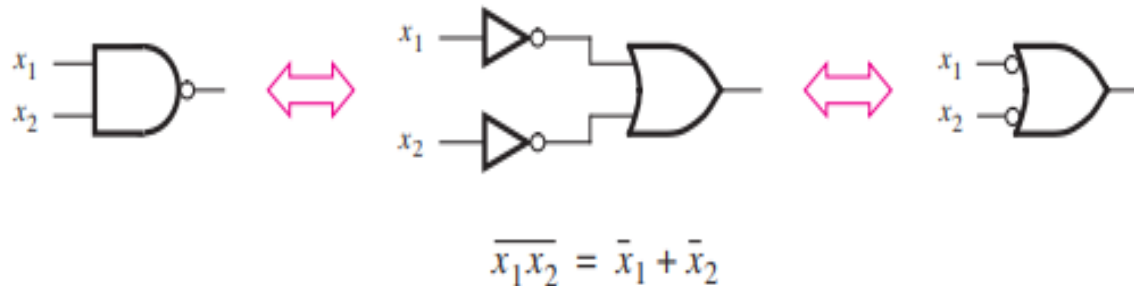
NAND Gate-

Truth Table

2 Input NAND gate		
A	B	$\overline{A \cdot B}$
0	0	1
0	1	1
1	0	1
1	1	0

In NAND gate, if any input is LOW (0), a HIGH (1) output results.

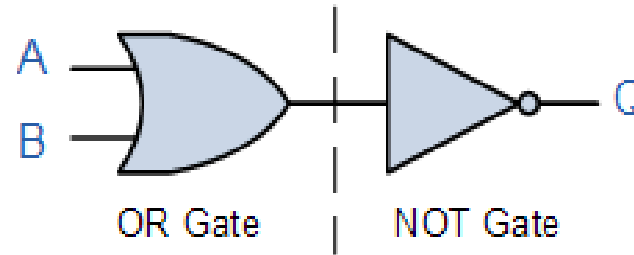
- A 2-input NAND gate is equivalent to 2-input OR gate with complemented inputs.



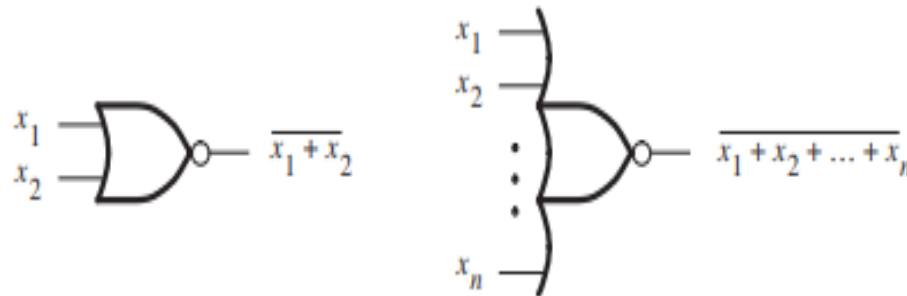
NAND and NOR Logic Network

NOR Gate-

- NOR gate is equal to OR gate followed by NOT gate.



Logic NOR Gate Equivalence



Symbol for NOR Gate

NAND and NOR Logic Networks

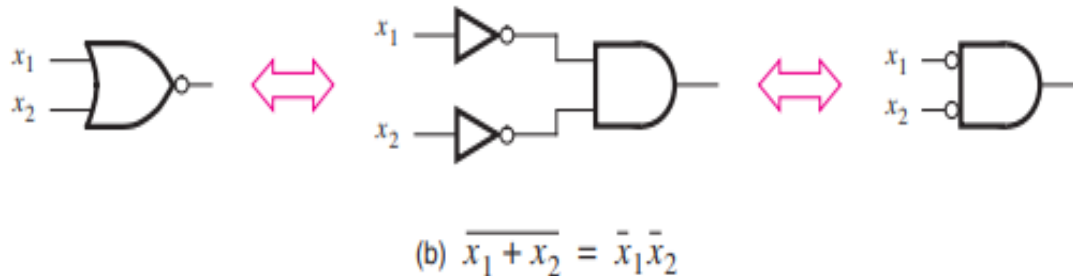
NOR Gate-

Truth Table

2 Input NOR gate		
A	B	$\overline{A+B}$
0	0	1
0	1	0
1	0	0
1	1	0

The output of **NOR gate** is high only if all the inputs are low.

- A 2-input NOR gate is equivalent to 2-input AND gate with complemented inputs.



NAND and NOR Logic Networks

Function Implementation Using NAND Gates Only

- Sum-of-Product form (SOP Form)-

➤ First Method:

- 1) Draw AND-OR Network for the given function.
- 2) Each connection between the AND gate and an OR gate is replaced by a connection that includes two inversions of the signal: one inversion at the output of the AND gate and the other at the input of the OR gate.
- 3) Such double inversion has no effect on the behavior of the network.

Consider a function

$$f = x_1 \cdot x_2 + x_3 \cdot x_4 \cdot x_5$$

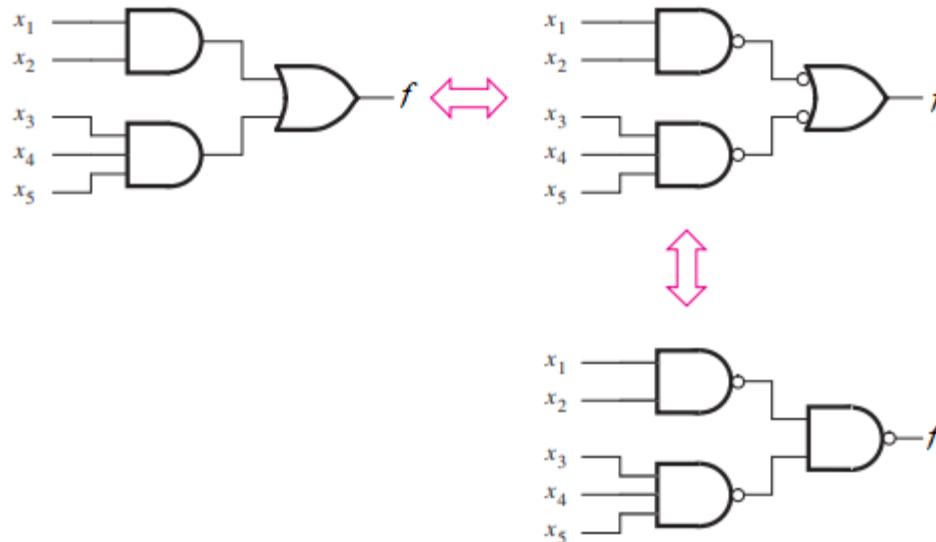
NAND and NOR Logic Networks

Function Implementation Using NAND Gates Only

- Sum-of-Product form (SOP Form)-

➤ First Method:

$$f = x_1 \cdot x_2 + x_3 \cdot x_4 \cdot x_5$$



any AND-OR network
can be implemented as
a NAND-NAND network
having the same topology

Using NAND gates to implement a sum-of-products

NAND and NOR Logic Networks

Function Implementation Using NAND Gates Only

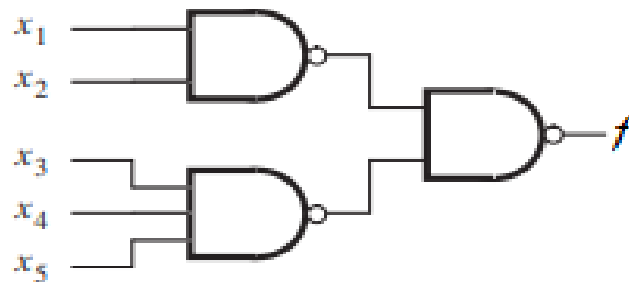
- Sum-of-Product form (SOP Form)-

➤ Second Method:

$$f = x_1 \cdot x_2 + x_3 \cdot x_4 \cdot x_5$$

$$f = \overline{\overline{x_1 \cdot x_2 + x_3 \cdot x_4 \cdot x_5}}$$

$$f = (\overline{x_1 \cdot x_2}) \cdot (\overline{x_3 \cdot x_4 \cdot x_5}) \quad \text{(Using Demorgan's Theorem)}$$



Using NAND gates to implement a sum-of-products

NAND and NOR Logic Networks

Function Implementation Using NOR Gates Only

- Product-of-Sum form (POS Form)-

➤ First Method:

- 1) Draw OR-AND Network for the given function.
- 2) Each connection between the OR gate and AND gate is replaced by a connection that includes two inversions of the signal: one inversion at the output of the OR gate and the other at the input of the AND gate.
- 3) Such double inversion has no effect on the behavior of the network.

Consider a function

$$f = (x_1 + x_2) \cdot (x_3 + x_4 + x_5)$$

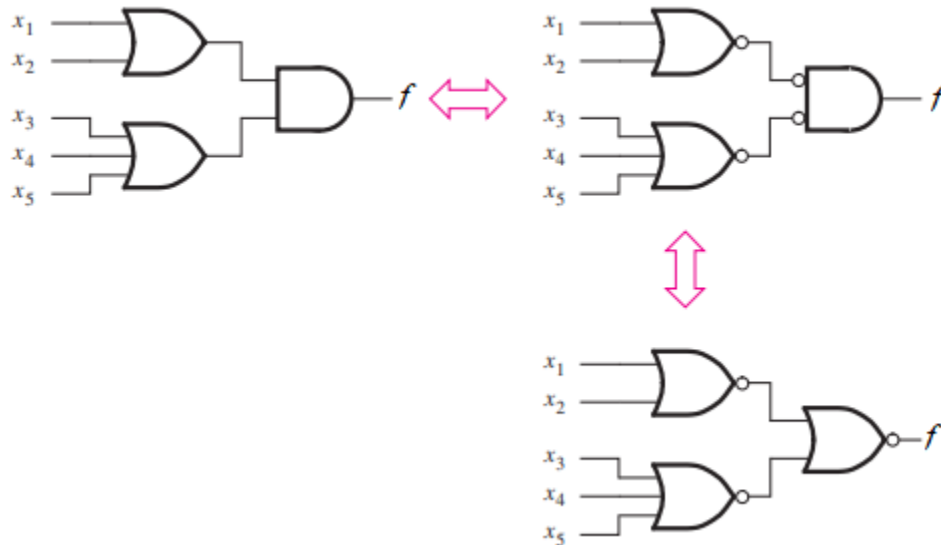
NAND and NOR Logic Networks

Function Implementation Using NOR Gates Only

- Product-of-Sum form (POS Form)-

➤ First Method:

$$f = (x_1 + x_2) \cdot (x_3 + x_4 + x_5)$$



any OR-AND network
can be implemented as
a NOR-NOR network
having the same topology

Using NOR gates to implement a product-of-sums

NAND and NOR Logic Networks

Function Implementation Using NOR Gates Only

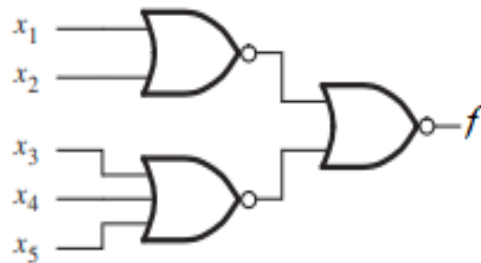
- Product-of-Sum form (POS Form)-

➤ Second Method:

$$f = (x_1 + x_2) \cdot (x_3 + x_4 + x_5)$$

$$f = \overline{\overline{(x_1 + x_2)} \cdot \overline{(x_3 + x_4 + x_5)}}$$

$$f = \overline{\overline{(x_1 + x_2)} + \overline{(x_3 + x_4 + x_5)}} \quad \text{(Using Demorgan's Theorem)}$$

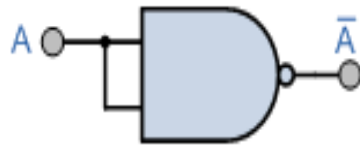


Using NOR gates to implement a product-of-sums

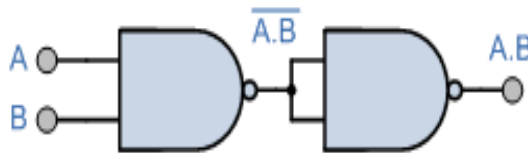
NAND and NOR Logic Networks

Various Logic Gates Using Only NAND Gates-

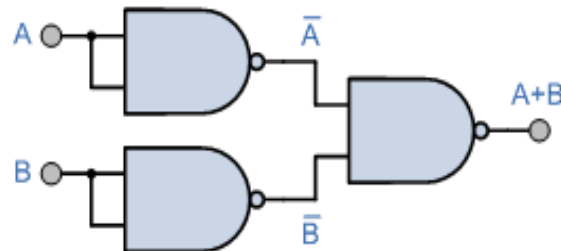
1) NOT Gate



2) AND Gate



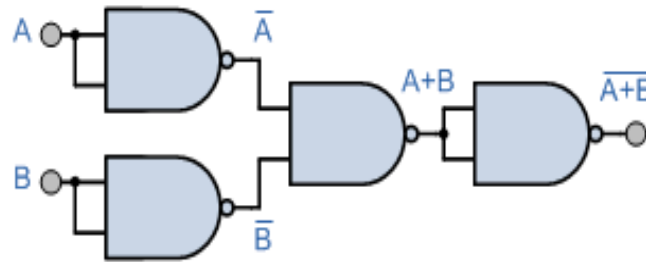
3) OR Gate



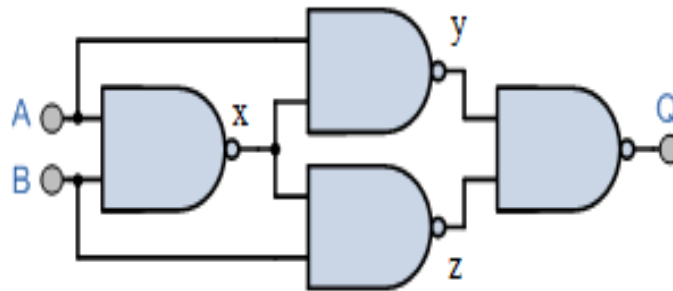
NAND and NOR Logic Networks

Various Logic Gates Using Only NAND Gates-

4) NOR Gate



5) XOR Gate



$$x = \overline{A \cdot B} = \overline{A + B}$$

$$y = \overline{A \cdot x} = A \cdot (\overline{A + B})$$

$$y = \overline{A \cdot B}$$

$$z = \overline{B \cdot x} = B \cdot (\overline{A + B})$$

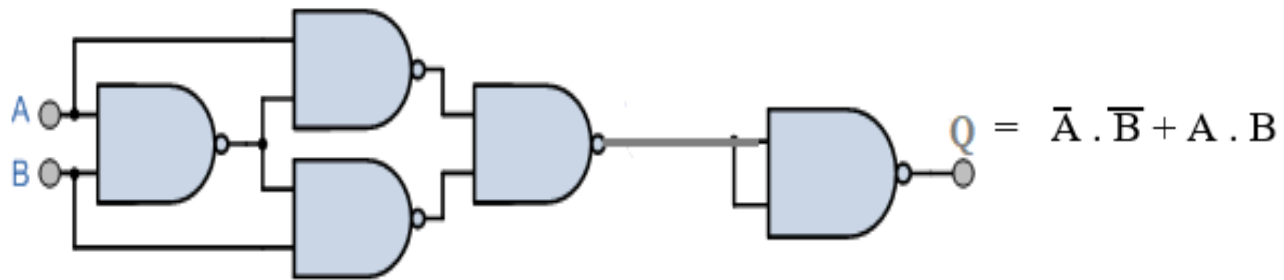
$$z = \overline{A \cdot B}$$

$$Q = \overline{y \cdot z} = A \cdot \overline{B} + \overline{A} \cdot B$$

NAND and NOR Logic Networks

Various Logic Gates Using Only NAND Gates-

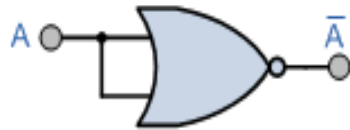
6) XNOR Gate



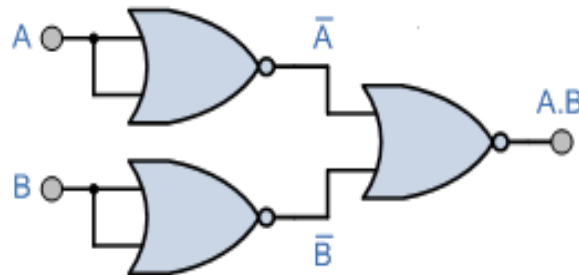
NAND and NOR Logic Networks

Various Logic Gates Using Only NOR Gates-

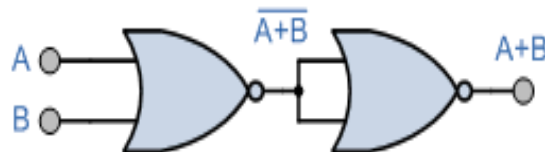
1) NOT Gate



2) AND Gate



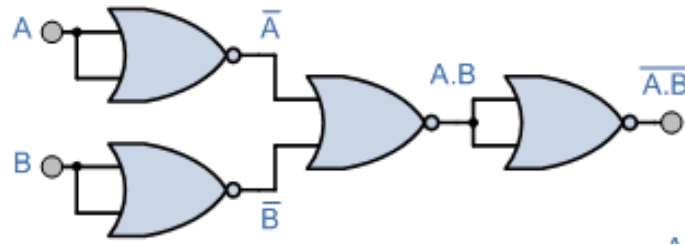
3) OR Gate



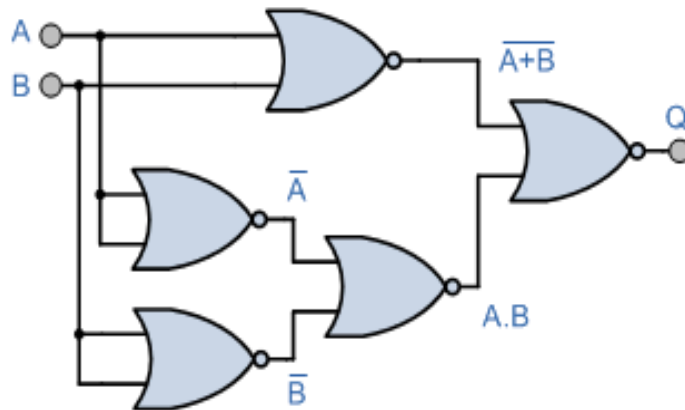
NAND and NOR Logic Networks

Various Logic Gates Using Only NOR Gates-

4) NAND Gate



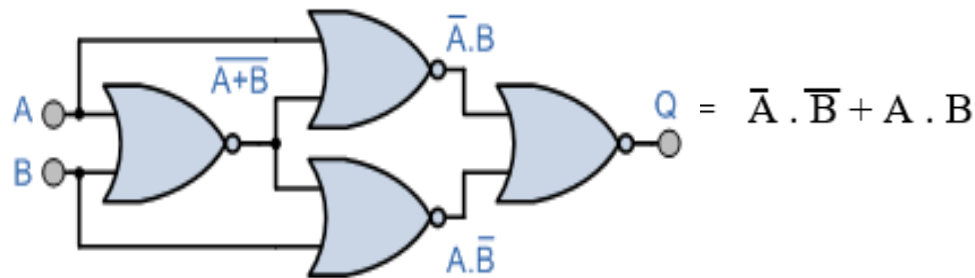
5) XOR Gate



NAND and NOR Logic Networks

Various Logic Gates Using Only NOR Gates-

6) XNOR Gate



NAND and NOR Logic Networks

Function Implementation Using NAND Gates Only

Example 1-

Implement the function $f(a, b, c) = \sum m(2, 3, 4, 6, 7)$ using NAND gates only.

(a) In SOP form

(b) In POS form

Solution- The canonical SOP expression for the function is derived using minterms-

$$f = m_2 + m_3 + m_4 + m_6 + m_7$$

$$f = a'bc' + a'bc + ab'c' + abc' + abc$$

$$f = a'b(c' + c) + ac'(b' + b) + ab(c' + c) \quad \{x + x' = 1\}$$

$$f = a'b + ac' + ab = (a' + a)b + ac'$$

$$f = b + ac'$$

NAND and NOR Logic Networks

Function Implementation Using NAND Gates Only

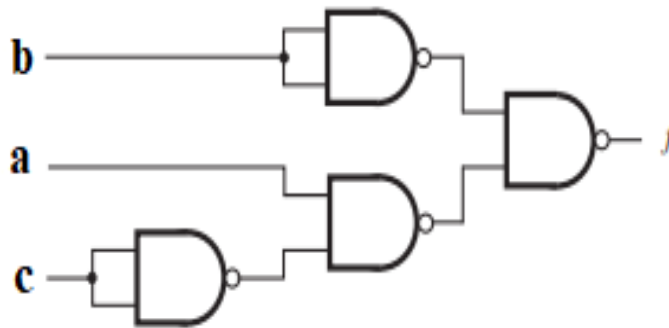
Example 1 Solution-

(a) SOP form

$$f = b + ac' = \overline{\overline{b + ac'}}$$

$x' = \overline{x}$ = complement of x

$$f = \overline{b' \cdot (ac')'}$$



Function implementation using NAND Gates only

NAND and NOR Logic Networks

Function Implementation Using NAND Gates Only

Example 1 Solution-

(b) POS form

$$f = b + \underline{ac'} = \underline{(a + b)}. (b + c')$$

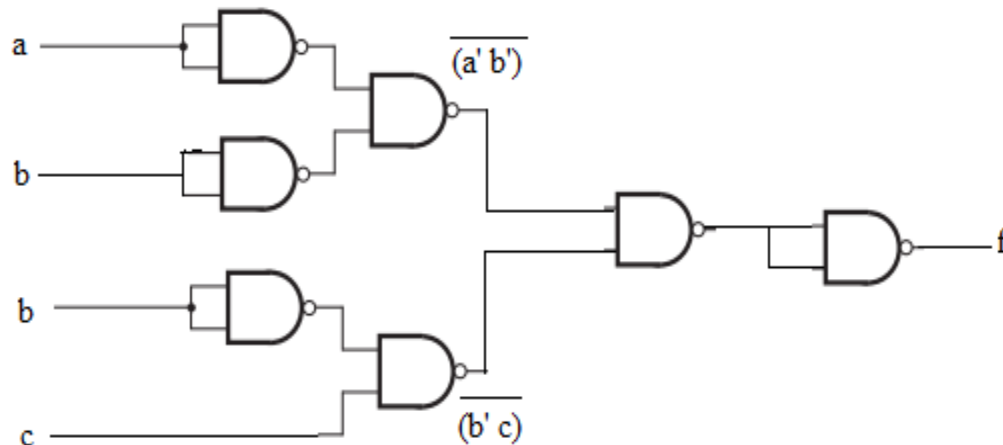
{Distributive Law}

$$f = \underline{(a + b) \cdot (b + c')}$$

{ $x' = \bar{x}$ = complement of x }

$$f = \underline{(a' \cdot b')} + \underline{(b' \cdot c)} = \underline{(a' \cdot b')} \cdot \underline{(b' \cdot c)}$$

{Demorgan's Theorem}



Function implementation using NAND Gates only

NAND and NOR Logic Networks

Function Implementation Using NAND Gates Only

Example 1 Solution-

$$f(a, b, c) = \sum m(2, 3, 4, 6, 7) = b + ac' = (a + b) \cdot (b + c')$$

- Given function implementation in SOP form- No. of NAND gates required – 4
- Given function implementation in POS form- No. of NAND gates required - 7

Conclusion- Any logic function can be implemented using NAND gates. To achieve this, first the logic function has to be written in Sum of Product (SOP) form. Once logic function is converted to SOP, then is very easy to implement using NAND gate and it requires less number of gates.

NAND and NOR Logic Networks

Function Implementation Using NOR Gates Only

Example 2-

Implement the function $f(a, b, c) = \Sigma m(2, 3, 4, 6, 7)$ using NOR gates only.

(a) In SOP form

(b) In POS form

Solution- From Example 1 we got-

$$f = b + a c'$$

(a) In SOP form

$$f = \overline{\overline{b + a c'}} = \overline{b' \cdot a c'}$$

{Demorgan's Theorem}

$$f = b' \cdot \overline{(a' + c)}$$

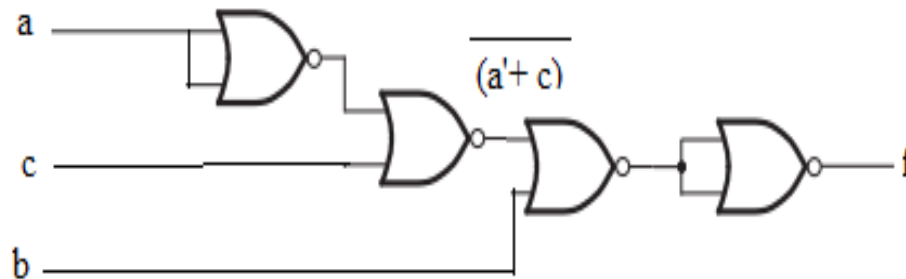
$$f = b + (a' + c)$$

NAND and NOR Logic Networks

Function Implementation Using NOR Gates Only

Example 2 Solution-

(a) In SOP form



Function implementation using NOR Gates only

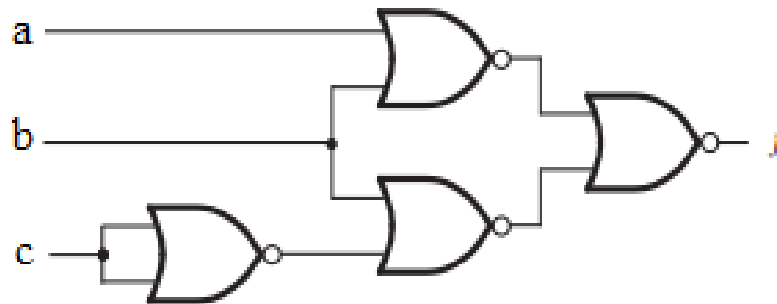
NAND and NOR Logic Networks

Function Implementation Using NOR Gates Only

Example 2 Solution-

(a) In POS form

$$f = b + a c' = \overline{\overline{(a + b)} \cdot \overline{(b + c')}} \\ f = \overline{\overline{(a + b)} + \overline{(b + c')}}$$



Function implementation using NOR Gates only

NAND and NOR Logic Networks

Function Implementation Using NOR Gates Only

Example 2 Solution-

Conclusion- Any logic function can be implemented using NOR gates. To achieve this, first the logic function has to be written in Product of Sum (POS) form. Once logic function is converted to POS, then is very easy to implement using NOR gate and it requires less number of gates.

(in example 2 number of NOR gates, in both SOP and POS form implementations, are same but it is not applicable on all the functions)

NAND and NOR Logic Networks

Example 3-

Design the simplest circuit that implements the given function using NAND gates only.

$$F(x, y, z) = \sum m(1, 2, 3, 4, 5, 7)$$

Solution- The canonical SOP expression for the function is derived using minterms-

$$F = m_1 + m_2 + m_3 + m_4 + m_5 + m_7$$

$$F = x'y'z + x'yz' + x'yz + xy'z' + xy'z + xyz$$

$$F = x'y'z + x'yz + x'yz' + x'yz + xy'z' + xy'z + xy'z + xyz$$

$$F = x'z(y' + y) + x'y(z' + z) + xy'(z' + z) + xz(y' + y) \quad \{x + x' = 1\}$$

$$F = x'z + x'y + xy' + xz = (x' + x)z + x'y + xy'$$

$$F = xy' + x'y + z$$

NAND and NOR Logic Networks

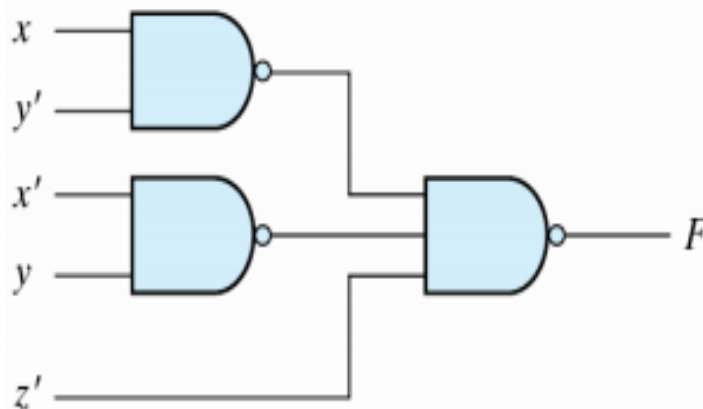
Example 3 Solution-

$$F = xy' + x'y + z$$

$$F(x, y, z) = xy' + x'y + z$$

$$= \overline{\overline{xy' + x'y + z}}$$

$$= \overline{(\overline{xy'}) (\overline{x'y}) z'}$$



(Assuming both normal and complemented inputs are available)

NAND and NOR Logic Networks

Example 4-

Design the simplest circuit that implements the given function using NOR gates only.

$$F(x, y, z) = \sum m(1, 2, 3, 4, 5, 7)$$

Solution-

$$F(x, y, z) = \sum m(1, 2, 3, 4, 5, 7) = \prod M(0, 6)$$

The canonical POS expression for the function is derived using maxterms-

$$F = M_0 \cdot M_6$$

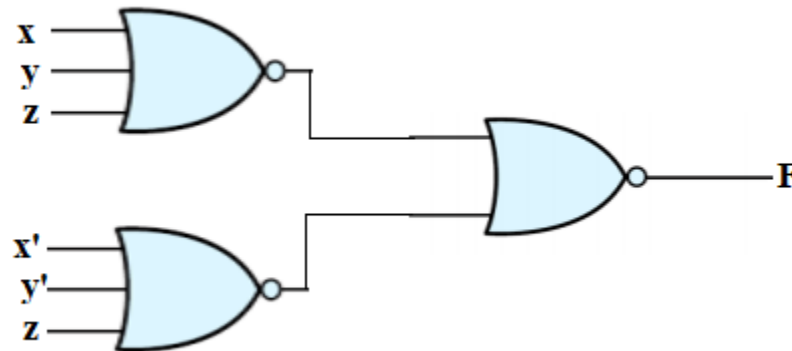
$$F = (x + y + z) \cdot (x' + y' + z) = \overline{\overline{(x + y + z)} \cdot \overline{(x' + y' + z)}}$$

$$F = \overline{\overline{(x + y + z)} + \overline{(x' + y' + z)}}$$

NAND and NOR Logic Networks

Example 4 Solution-

$$F = \overline{(x + y + z)} + \overline{(x' + y' + z)}$$



Digital Electronics & Systems

19ECE204

Lecture 13-17

Ms. Bhavana V

Assistant Professor

Dept. of ECE

ASE, Bengaluru

Optimized Implementation of Logic Functions

- The algebraic manipulation can be used to find the lowest-cost implementations of logic functions.
- It is easy to derive a straightforward realization of a logic function in a canonical form, but it is not at all obvious how to choose and apply the theorems and properties of Boolean algebra to find a minimum-cost circuit.
- Indeed, the algebraic manipulation is rather tedious and quite impractical for functions of many variables.
- Another minimization approach, which provides a neat way to manually derive minimum-cost implementations of logic functions, is known as Karnaugh Map.

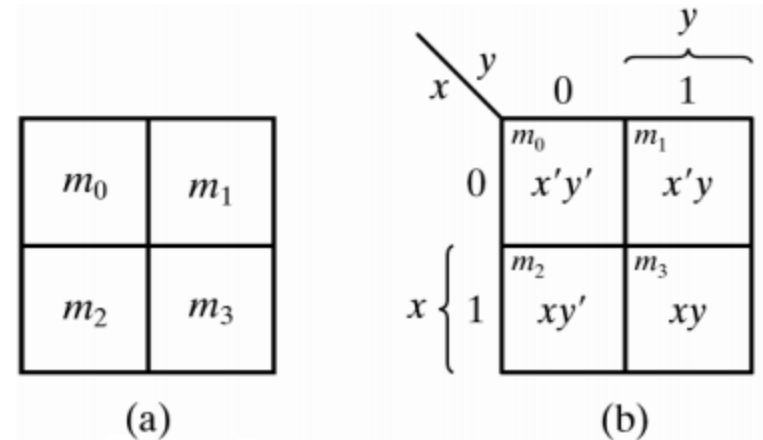
Optimized Implementation of Logic Functions – Karnaugh Map

- The Karnaugh map approach provides a systematic way of performing this optimization.
- The Karnaugh map (K-Map) is an alternative to the truth-table form for representing a function. It is a pictorial form of a truth table.
- The K-Map is a diagram made up of squares. Each square is a cell of K-map. The map consists of cells that correspond to the rows of the truth table.

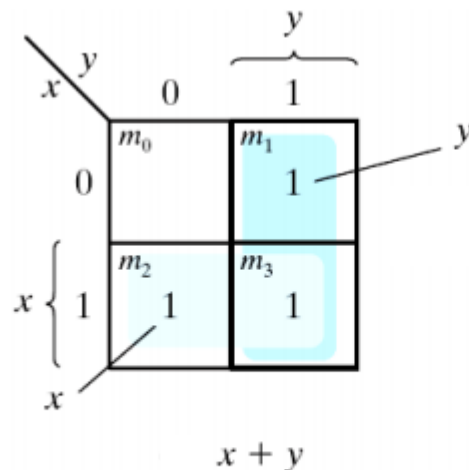
Optimized Implementation of Logic Functions – Karnaugh Map

Two-Variable K-Map

- Four minterms
- $x' = \text{row } 0$; $x = \text{row } 1$
- $y' = \text{column } 0$; $y = \text{column } 1$
- A truth table in square diagram



Two-variable Map



Representation of functions in the map

$$F(x, y) = \sum m(1, 2, 3)$$

$$= x'y + xy' + xy$$

After minimization

$$F(x, y) = x + y$$

Optimized Implementation of Logic Functions – Karnaugh Map

Three-Variable K-Map

- A three-variable Karnaugh map is constructed by placing 2 two-variable maps side by side.
- Eight minterms
- The Gray code sequence
- Any two adjacent squares in the map differ by only one variable.
- In a three-variable map it is possible to combine cells to produce product terms that correspond to
 - a single cell – gives 3 variable term
 - two adjacent cells (pair) – gives 2 variable term
 - a group of four adjacent cells (quad)- gives single variable term

Optimized Implementation of Logic Functions – Karnaugh Map

Three-Variable K-Map

x_1	x_2	x_3	
0	0	0	m_0
0	0	1	m_1
0	1	0	m_2
0	1	1	m_3
1	0	0	m_4
1	0	1	m_5
1	1	0	m_6
1	1	1	m_7

(a) Truth table

		$x_1 x_2$			
		00	01	11	10
x_3	0	m_0	m_2	m_6	m_4
	1	m_1	m_3	m_7	m_5

(b) Karnaugh map

Location of three-variable minterms.

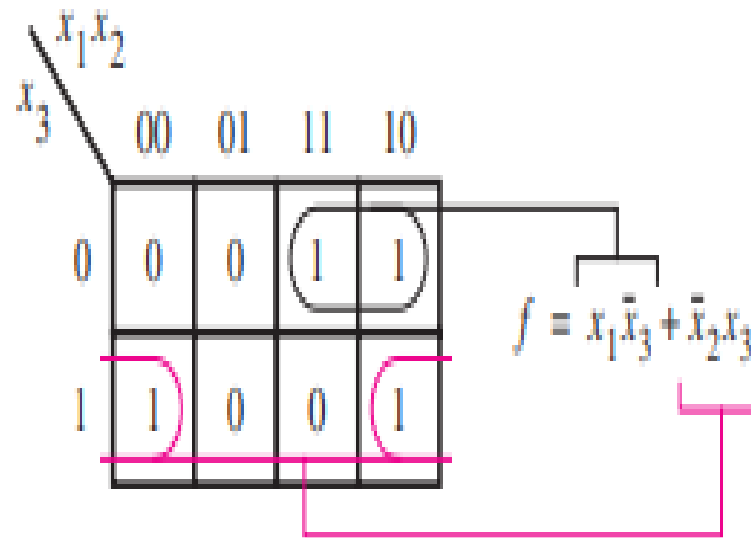
		y			
		x	yz	00	01
0		m_0	m_1	m_3	m_2
		$x'y'z'$	$x'y'z$	$x'yz$	$x'yz'$
1		m_4	m_5	m_7	m_6
		$xy'z'$	$xy'z$	xyz	xyz'
		z			

Optimized Implementation of Logic Functions – Karnaugh Map

Three-Variable K-Map

Example 1. Simplify the Boolean function

$$f(x_1, x_2, x_3) = \sum m(1, 4, 5, 6)$$



Optimized Implementation of Logic Functions – Karnaugh Map

Three-Variable K-Map

Example2 : simplify the Boolean functions:

$$F(x, y, z) = \sum(0, 2, 4, 5, 6)$$

$\begin{array}{c} xy \\ z \end{array}$	xy			
	00	01	11	10
0	1	1	1	1
1	0	0	0	1

Optimized Implementation of Logic Functions – Karnaugh Map

Three-Variable K-Map

Example2 : simplify the Boolean functions:

$$F(x, y, z) = \sum(0, 2, 4, 5, 6)$$

$$F(x, y, z) = \sum(0, 2, 4, 5, 6) = z' + xy'$$

		z'			
		xy 00	01	11	10
z	0	1	1	1	1
	1	0	0	0	1

The Karnaugh map shows two groupings: a horizontal group of four 1s in the top row (z') and a vertical group of two 1s in the rightmost column (xy').

Optimized Implementation of Logic Functions – Karnaugh Map

Three-Variable K-Map

Example 3 : Let the Boolean function

$$F = A'C + A'B + AB'C + BC$$

- a) Express it in sum of minterms.
- b) Find the minimal sum of products expression.

Solution-

$$\begin{aligned} \text{a) } F &= A'C + A'B + AB'C + BC \\ &= A' (B + B') C + A'B (C + C') + AB'C + (A + A')BC \\ &= A'BC + A'B'C + A'BC + A'BC' + AB'C + ABC + A'BC \\ &= m_3 + m_1 + m_3 + m_2 + m_5 + m_7 + m_3 \\ F &= \Sigma m(1, 2, 3, 5, 7) \end{aligned}$$

Optimized Implementation of Logic Functions – Karnaugh Map

Three-Variable K-Map

Solution of Example 3-

b)

C \ AB	00	01	11	10
	0	1	0	0
0	0	1	0	0
1	1	1	1	1

Optimized Implementation of Logic Functions – Karnaugh Map

Three-Variable K-Map

Solution of Example 3-

b)

	AB		A'B		
	00	01	11	10	
C					
0	0	1	0	0	
1	1	1	1	1	

C

$$F(A, B, C) = \sum(1, 2, 3, 5, 7) = C + A'B$$

Optimized Implementation of Logic Functions – Karnaugh Map

Three-Variable K-Map

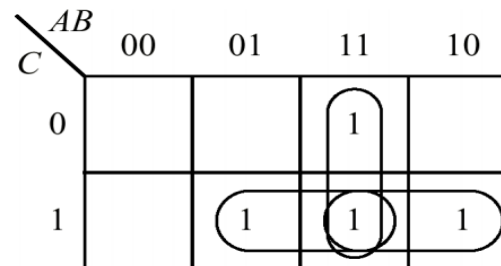
Example 4. Design a 3-bit Majority function.

Solution- **Majority function** is a threshold **function** that produces a 1 if and only if the **majority** of the inputs are 1.

Truth Table

Minterm Index	A	B	C	F
0	0	0	0	0
1	0	0	1	0
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	1

$$F = \sum m(3, 5, 6, 7)$$



$$F = BC + AB + AC$$

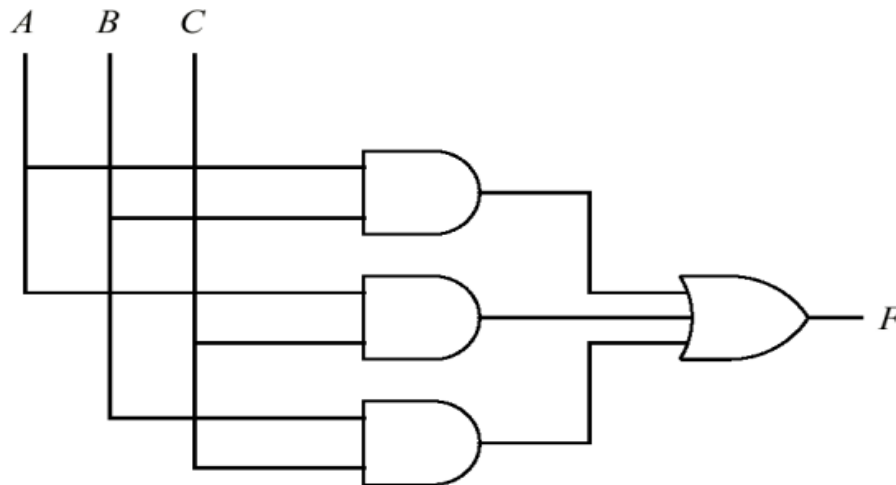
Optimized Implementation of Logic Functions – Karnaugh Map

Three-Variable K-Map

Solution of Example 4-

Logic Circuit for 3-bit Majority function-

$$F = AB + AC + BC$$



Optimized Implementation of Logic Functions – Karnaugh Map

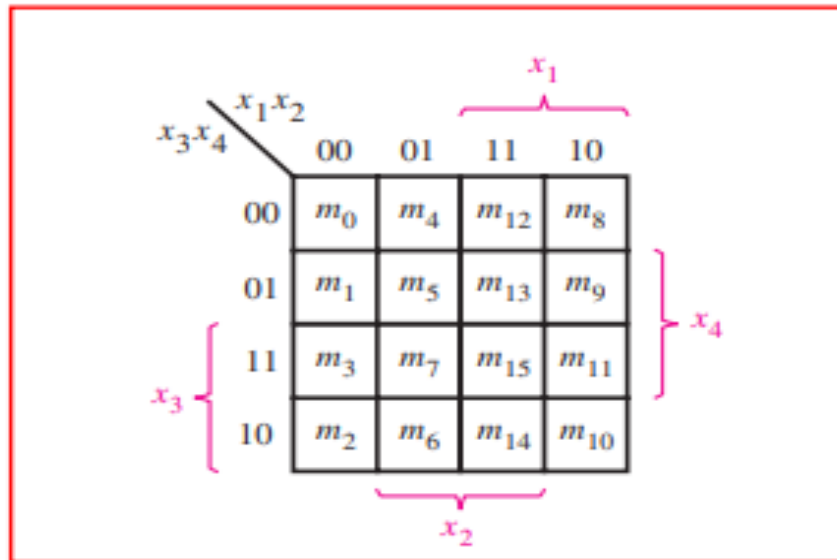
Four-Variable K-Map

- A four-variable map is constructed by placing 2 three-variable maps together to create four rows in the same fashion as we used 2 two-variable maps to form the four columns in a three-variable map.
- In a four-variable map it is possible to combine cells to produce product terms that correspond to a single cell, two adjacent cells (pair), a group of four adjacent cells (quad) and a group of eight adjacent cells(octa).
- Corner cells also can be combined to produce product term that correspond to a pair, quad or octa.

Optimized Implementation of Logic Functions – Karnaugh Map

Four-Variable K-Map

- 16 minterms
- Combinations of 16 adjacent squares



A four-variable Karnaugh map.

Optimized Implementation of Logic Functions – Karnaugh Map

Four-Variable K-Map

Example 1: Simplify the Boolean function-

$$f1(x_1, x_2, x_3, x_4) = \Sigma m(2, 3, 9, 10, 11, 13)$$

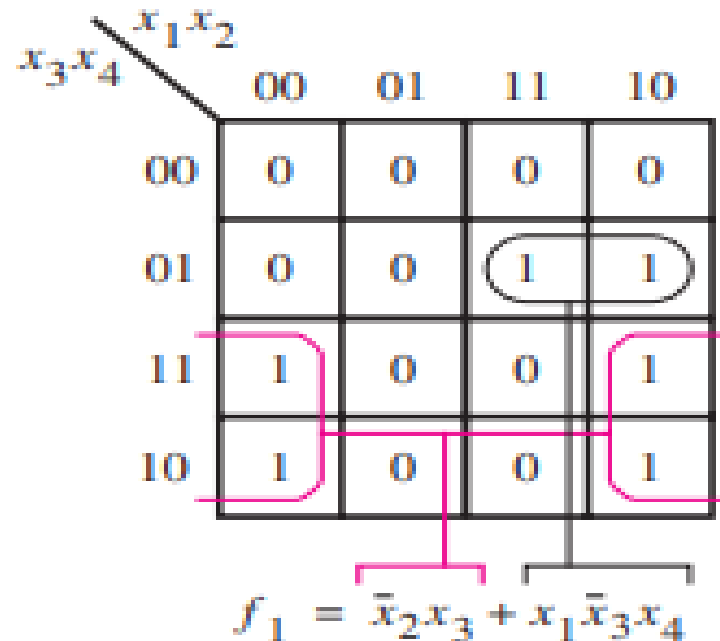
$x_1 x_2$	00	01	11	10
$x_3 x_4$				
00	0	0	0	0
01	0	0	1	1
11	1	0	0	1
10	1	0	0	1

Optimized Implementation of Logic Functions – Karnaugh Map

Four-Variable K-Map

Example 1: Simplify the Boolean function-

$$f_1(x_1, x_2, x_3, x_4) = \sum m(2, 3, 9, 10, 11, 13)$$



Optimized Implementation of Logic Functions – Karnaugh Map

Four-Variable K-Map

Example 2: Simplify the Boolean function-

$$f1(x_1, x_2, x_3, x_4) = \Sigma m(0, 2, 3, 6, 7, 8, 10, 15)$$

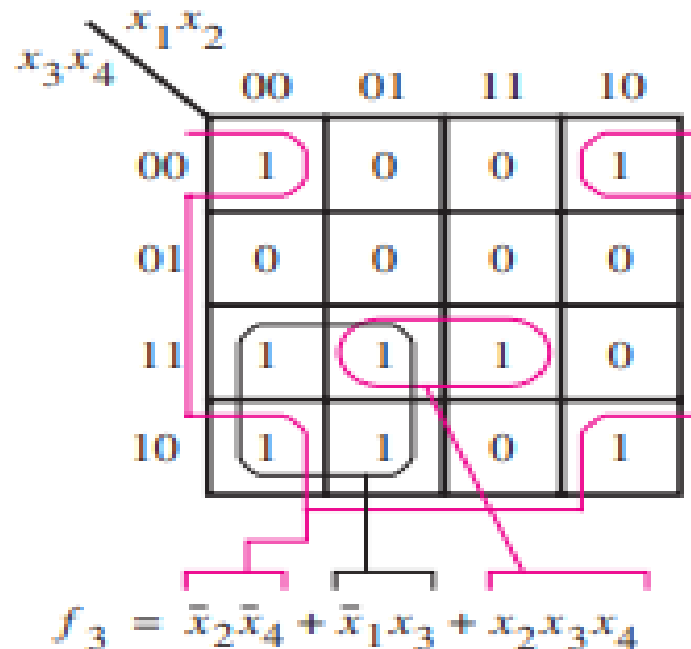
$x_1 x_2$		$x_3 x_4$			
		00	01	11	10
$x_3 x_4$	00	1	0	0	1
	01	0	0	0	0
	11	1	1	1	0
	10	1	1	0	1

Optimized Implementation of Logic Functions – Karnaugh Map

Four-Variable K-Map

Example 2: Simplify the Boolean function-

$$f_1(x_1, x_2, x_3, x_4) = \sum m(0, 2, 3, 6, 7, 8, 10, 15)$$



Optimized Implementation of Logic Functions – Karnaugh Map

Four-Variable K-Map

Example 3: Simplify the Boolean function-

$$f1(x_1, x_2, x_3, x_4) = \Pi M(2, 3, 6, 7, 8, 9)$$

Solution- Write the function in other canonical form

$$f1(x_1, x_2, x_3, x_4) = \Sigma m(0, 1, 4, 5, 10, 11, 12, 13, 14, 15)$$

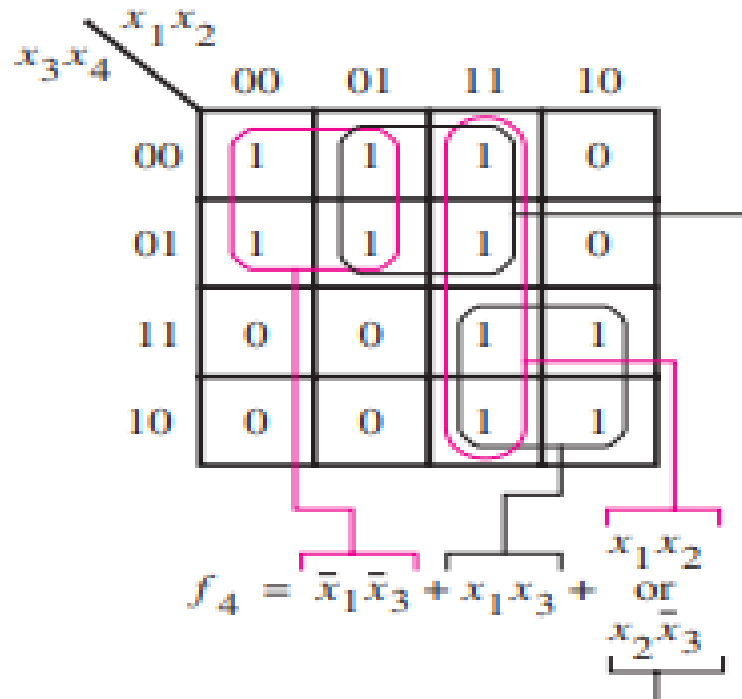
$x_1 x_2$		$x_3 x_4$			
		00	01	11	10
$x_3 x_4$	00	1	1	1	0
	01	1	1	1	0
	11	0	0	1	1
	10	0	0	1	1

Optimized Implementation of Logic Functions – Karnaugh Map

Four-Variable K-Map

Example 3: Simplify the Boolean function-

$$f1(x_1, x_2, x_3, x_4) = \Sigma m(0, 1, 4, 5, 10, 11, 12, 13, 14, 15)$$



Optimized Implementation of Logic Functions – Karnaugh Map

Four-Variable K-Map

Example 4: Simplify the complement of Boolean function-

$$f(x_1, x_2, x_3, x_4) = \Pi M(2, 3, 5, 6, 7, 8, 9, 12, 13)$$

Solution- Take the complement of function both the sides-

$$\overline{f(x_1, x_2, x_3, x_4)} = \overline{\Pi M(2, 3, 5, 6, 7, 8, 9, 12, 13)}$$

$$\begin{aligned} f'(x_1, x_2, x_3, x_4) &= \overline{M_2 \cdot M_3 \cdot M_5 \cdot M_6 \cdot M_7 \cdot M_8 \cdot M_9 \cdot M_{12} \cdot M_{13}} \\ &= \overline{M_2} + \overline{M_3} + \overline{M_5} + \overline{M_6} + \overline{M_7} + \overline{M_8} + \overline{M_9} + \overline{M_{12}} + \overline{M_{13}} \end{aligned}$$

Complement of a maxterm is a minterm and vice versa

$$m_0 = x'y'z' ; M_0 = (x + y + z)$$

$$\overline{m_0} = \overline{x'y'z'} = (x')' + (y')' + (z')' = x + y + z = M_0$$

$$\text{So } f'(x_1, x_2, x_3, x_4) = \Sigma m(2, 3, 5, 6, 7, 8, 9, 12, 13)$$

Optimized Implementation of Logic Functions – Karnaugh Map

Four-Variable K-Map

Solution for example 4-

$$f'(x_1, x_2, x_3, x_4) = \sum m(2, 3, 5, 6, 7, 8, 9, 12, 13)$$

$x_1x_2 \backslash x_3x_4$		x_1x_2			
		00	01	11	10
00				1	1
01			1	1	1
11	1	1			
10	1	1			

$$f'(x_1, x_2, x_3, x_4) = x_1\bar{x}_3 + \bar{x}_1x_3 + x_2\bar{x}_3x_4$$

Optimized Implementation of Logic Functions – Karnaugh Map

Four-Variable K-Map

Example 5. Simplify the given Boolean function using Karnaugh Map.

$$F(w, x, y, z) = w'x'y'z + x'yz + wy'z' + wxy'z + wx'y'z$$

Solution – First we need to express the function in sum of minterms form:

$$F = w'x'y'z + (w + w')x'yz + w(x + x')y'z' + wxy'z + wx'y'z$$

$$= w'x'y'z + wx'yz + w'x'yz + wxy'z' + wx'y'z' + wxy'z + wx'y'z$$

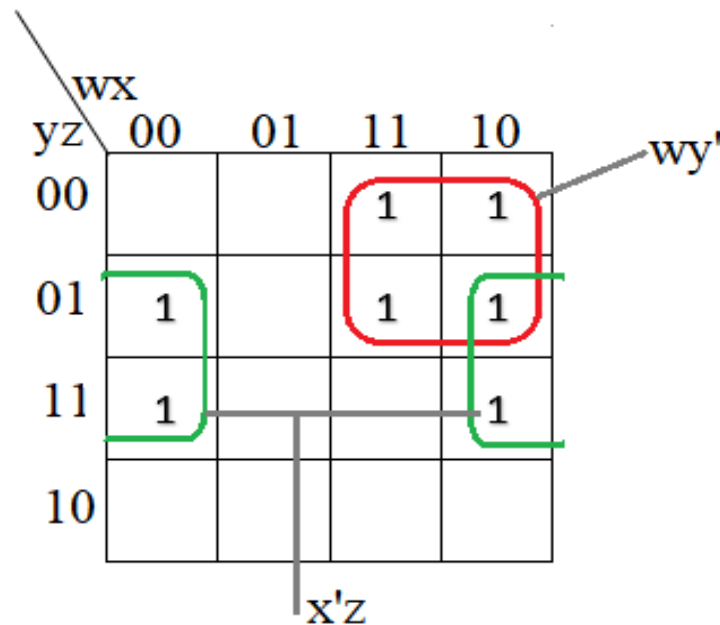
$$= m_1 + m_{11} + m_3 + m_{12} + m_8 + m_{13} + m_9$$

$$F = \Sigma m(1, 3, 8, 9, 11, 12, 13)$$

Optimized Implementation of Logic Functions – Karnaugh Map

Four-Variable K-Map

Solution for Example 5-



$$F = wy' + x'z$$

Optimized Implementation of Logic Functions – Karnaugh Map

Five-Variable K-Map

- We can use 2 four-variable maps to construct a five-variable map.
- It is easy to imagine a structure where one map is directly behind the other, and they are distinguished by $A = 0$ for one map and $A = 1$ for the other map.

		BC			
		00	01	11	10
DE	00	m0	m4	m12	m8
	01	m1	m5	m13	m9
	11	m3	m7	m15	m11
	10	m2	m6	m14	m10

A = 0

		BC			
		00	01	11	10
DE	00	m16	m20	m28	m24
	01	m17	m21	m29	m25
	11	m19	m23	m31	m27
	10	m18	m22	m30	m26

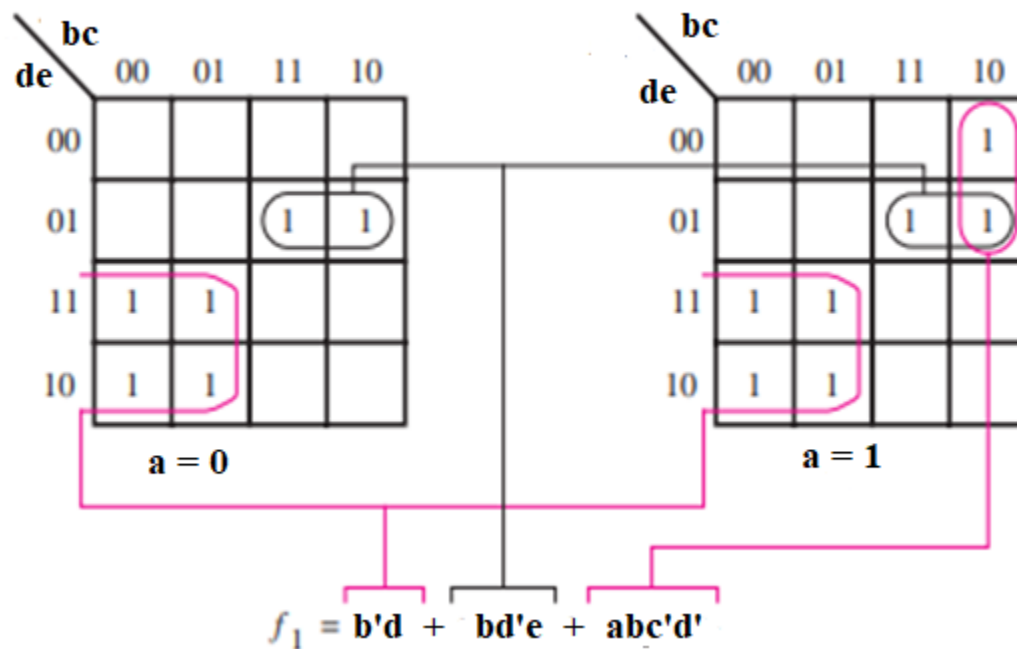
A = 1

Optimized Implementation of Logic Functions – Karnaugh Map

Five-Variable K-Map

Example 1. Simplify the Boolean function

$$f_1(a, b, c, d, e) = \Sigma m(2,3,6,7,9,13,18,19,22,23,24,25,29)$$

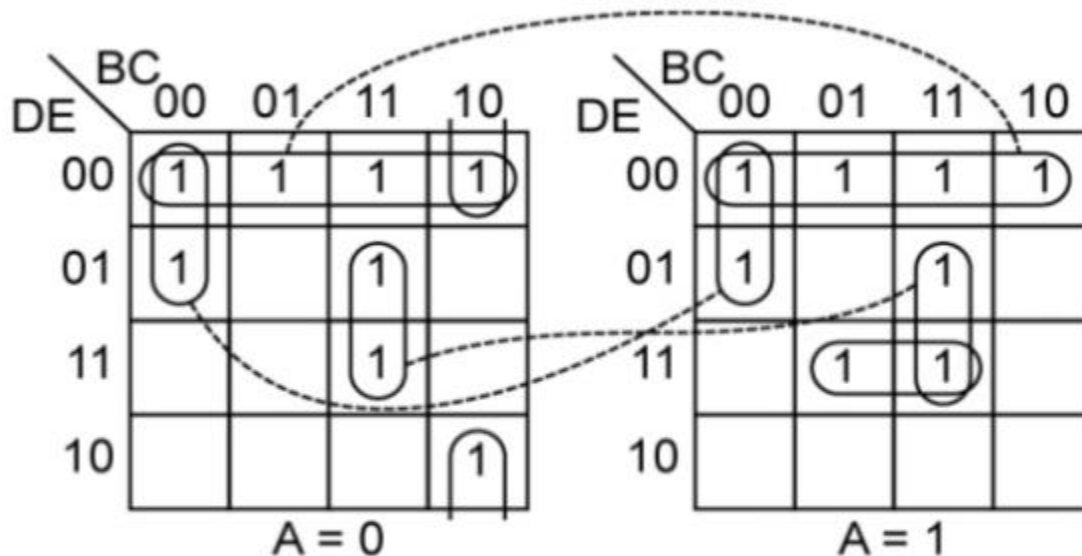


Optimized Implementation of Logic Functions – Karnaugh Map

Five-Variable K-Map

Example 2. Simplify the Boolean function

$$F(A,B,C,D,E) = \sum m(0, 1, 4, 8, 10, 12, 13, 15, 16, 17, 20, 23, 24, 28, 29, 31)$$



$$F = D'E' + B'C'D' + BCE + A'BC'E' + ACDE$$

Optimized Implementation of Logic Functions – Karnaugh Map

of Adjacent Squares and # of Literals

- The relationship between the number of adjacent squares and the number of literals in term-

K	Number of Adjacent Squares 2^k	Number of Literals in a Term in an n-variable Map			
		$n = 2$	$n = 3$	$n = 4$	$n = 5$
0	1	2	3	4	5
1	2	1	2	3	4
2	4	0	1	2	3
3	8		0	1	2
4	16			0	1
5	32				0

Digital Electronics & Systems

19ECE204

Lecture 18-21

Ms. Bhavana V
Assistant Professor
Dept. of ECE
ASE, Bengaluru

Strategy of Minimization

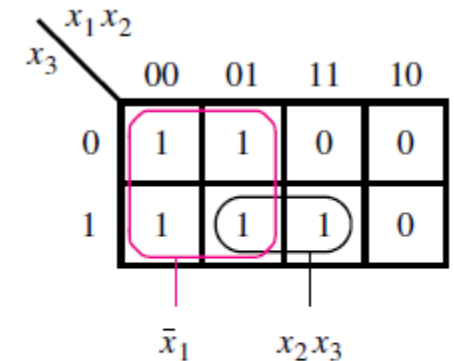
- Requirement:
 - Find few number of groups as possible
 - Find groups as large as possible. Groups of 1s (or 0s) that cover all cases where function has a value of 1 (or 0) -> results in fewer number of input variables in the corresponding product term (or sum term)
- Why Strategy of Minimization?
 - When many input variables are there in a function -> minimum cost implementation need to be derived

Terminology

- **Literal:** Each appearance of a variable, either uncomplemented or complemented form is called a Literal
 - Example: $x_1'x_2'x_3x_4$ has 4 literals
- **Implicant:** A product term that indicates the input valuation for which a given function is equal to 1 is called an implicant of the function.

The most basic implicants are either minterms for Product terms (and maxterms in case of sum terms). For an n-variable function, a minterm is an implicant that consist of n literals.

- Example: five minterms: $x_1'x_2'x_3'$, $x_1'x_2'x_3$, $x_1'x_2x_3'$, $x_1'x_2x_3$, and $x_1x_2x_3$. Finally, there are two implicants: First implicant that covers a group of four minterms results into x_1' and second implicant that covers group of two minterms results into x_2x_3 .



	x_1x_2	00	01	11	10
x_3	0	1	1	0	0
	1	1	1	0	0

$\bar{x}_1$ x_2x_3

Three-variable function $f(x_1, x_2, x_3) = \sum m(0, 1, 2, 3, 7)$.

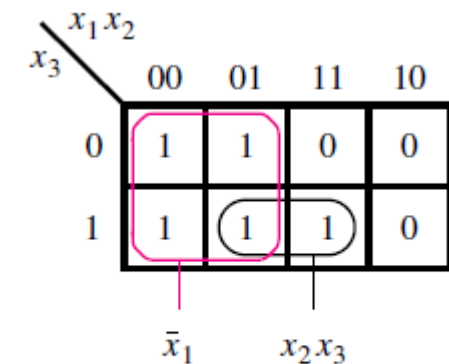
Terminology

- **Prime Implicant:** An implicant is called a *prime implicant* if it cannot be combined into another implicant that has fewer literals. Another way of stating this definition is to say that it is impossible to delete any literal in a prime implicant and still have a valid implicant.

Example: there are two prime implicants: x_1' and x_2x_3 . It is not possible to delete a literal in either of them.

- **Cover:** A collection of implicants that account for all valuations for which a given function is equal to 1 (or 0 in case of POS) is called a *cover* of that function. A number of different covers exist for most functions. A set of all prime implicants is a cover. A cover defines a particular implementation of the function.

$f = x_1' + x_2x_3$ is a cover here in this example.



Three-variable function $f(x_1, x_2, x_3) = \sum m(0, 1, 2, 3, 7)$.

Terminology

- **Cost:** Cost of a logic circuit is the number of gates plus the total number of inputs to all gates in the circuit. we will assume that primary inputs, namely, the input variables, are available in both true and complemented forms at zero cost.
- Example: The expression $f = x_1x_2' + x_3x_4'$ has a cost of 9 because it can be implemented using two AND gates and one OR gate.
- $g = (x_1x_2' + x_3)'(x_4' + x_5)$ *Cost = 9 wires + 5 gates = 14*
- In general, the larger the circuit, the more important the cost issue becomes.
- The main objective is to obtain a minimum-cost circuit.

Strategy for Minimization

- If a prime implicant includes a minterm for which $f = 1$ that is not included in any other prime implicant, then it must be included in the cover and is called an *essential prime implicant*.
- Example: The prime implicants are essential.
- The term x_2x_3 is the only prime implicant that covers the minterm m_7 , and x_1' is the only one that covers the minterms m_0 , m_1 , and m_2 . Notice that the minterm m_3 is covered by both of these prime implicants. The minimum-cost realization of the function is
- $f = x_1' + x_2x_3$

		$x_1 x_2$			
x_3	0	00	01	11	10
	1	00	01	11	10
		1	1	0	0
		1	1	1	0

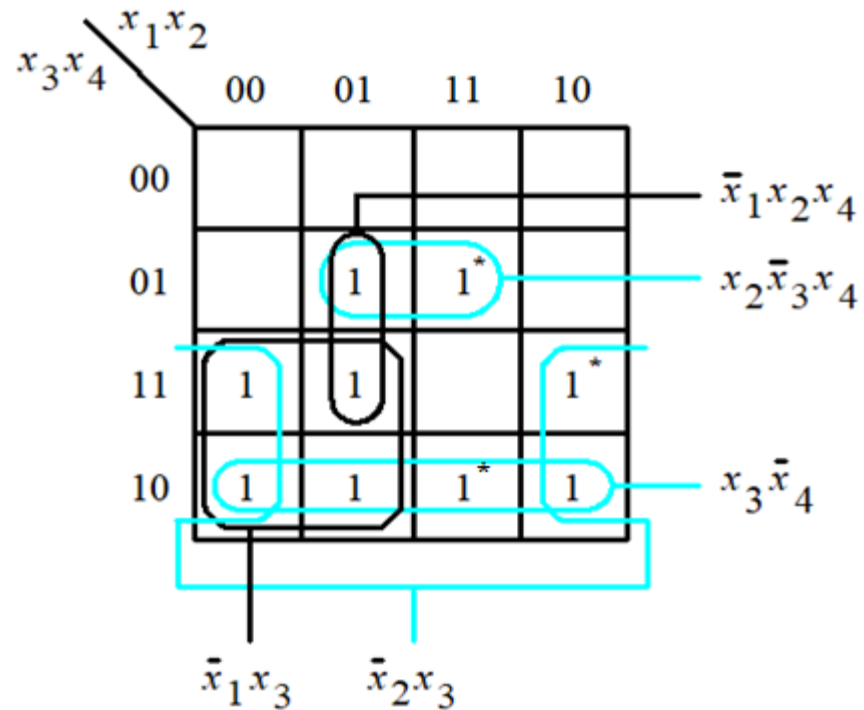
$\bar{x}_1$ x_2x_3

Three-variable function $f(x_1, x_2, x_3) = \sum m(0, 1, 2, 3, 7)$.

Steps for Minimization

- The process of finding a minimum-cost circuit involves the following steps:
 1. Generate all prime implicants for the given function f .
 2. Find the set of essential prime implicants.
 3. If the set of essential prime implicants covers all valuations for which $f = 1$, then this set is the desired cover of f . Otherwise, determine the nonessential prime implicants that should be added to form a complete minimum-cost cover. The choice of nonessential prime implicants to be included in the cover is governed by the cost considerations.

Example 1 for Minimization



$$PI = \{x_1'x_3, x_2'x_3, x_3x_4', x_2x_3'x_4, x_1'x_2x_4\}$$

$$EPI = \{x_3x_4', x_2'x_3, x_2x_3'x_4\}$$

$$Cover = \{x_3x_4', x_2'x_3, x_1'x_3, x_2x_3'x_4\}$$

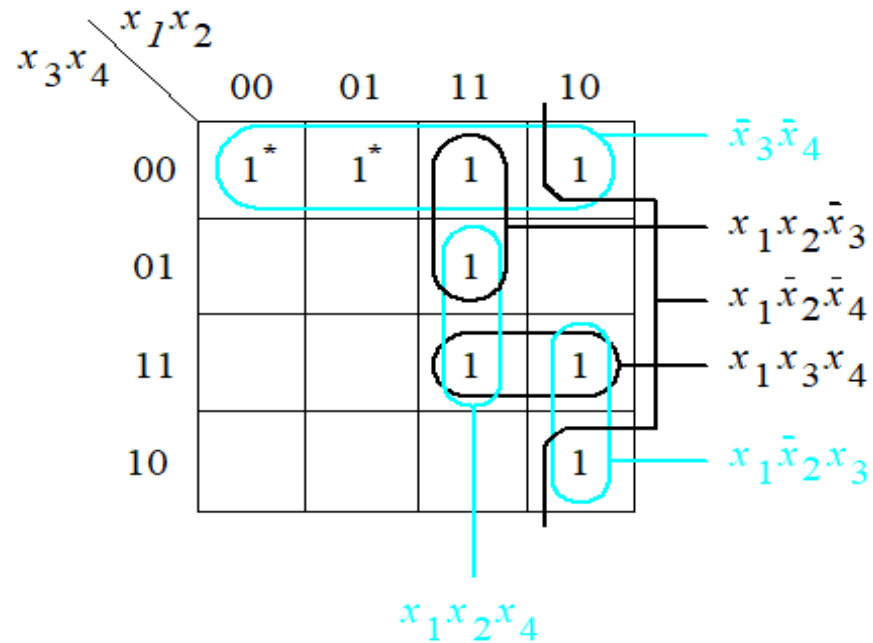
$$f = \bar{x}_2x_3 + x_3\bar{x}_4 + x_2\bar{x}_3x_4 + \bar{x}_1x_3$$

$$Cost = 13 + 5 = 18$$

Four-variable function $f(x_1, \dots, x_4) = \Sigma m(2, 3, 5, 6, 7, 10, 11, 13, 14)$.

Example 2

The function $f(x_1, \dots, x_4) = \sum m(0, 4, 8, 10, 11, 12, 13, 15)$.



$$PI = \{x_3'x_4', x_1x_2x_3', x_1x_2x_4, x_1x_3x_4, x_1x_2'x_3, x_1x_2'x_4'\}$$

$$EPI = \{x_3'x_4'\}$$

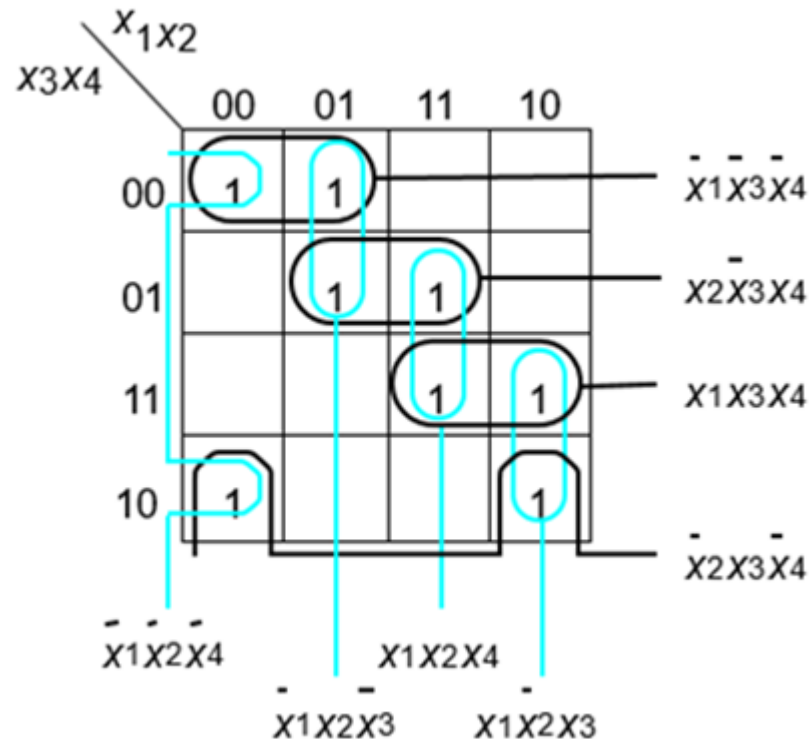
$$Cover = \{x_3'x_4', x_1x_2x_4, x_1x_2'x_3\}$$

$$f = x_3'x_4' + x_1x_2x_4 + x_1x_2'x_3$$

$$Cost = 11 + 4 = 15$$

Example 3

The function $f(x_1, \dots, x_4) = \sum m(0, 2, 4, 5, 10, 11, 13, 15)$



PI = $\{x_1'x_3'x_4', x_2x_3'x_4, x_1x_3x_4, x_2'x_3x_4', x_1'x_2'x_4', x_1'x_2x_3', x_1x_2x_4, x_1x_2'x_3\}$

EPI = {No EPI}

Cover = Either this $\{x_1'x_3'x_4', x_2x_3'x_4, x_1x_3x_4, x_2'x_3x_4'\}$ or this $\{x_1'x_2'x_4', x_1'x_2x_3', x_1x_2x_4, x_1x_2'x_3\}$

Sometimes there may not be any essential prime implicants at all. An example is given in Figure 4.12. Choosing any of the prime implicants and first including it, then excluding it from the cover leads to two alternatives of equal cost. One includes the prime implicants indicated in black, which yields

$$f = \bar{x}_1\bar{x}_3\bar{x}_4 + x_2\bar{x}_3x_4 + x_1x_3x_4 + \bar{x}_2x_3\bar{x}_4$$

The other includes the prime implicants indicated in blue, which yields

$$f = \bar{x}_1\bar{x}_2\bar{x}_4 + \bar{x}_1x_2\bar{x}_3 + x_1x_2x_4 + x_1\bar{x}_2x_3$$

Minimization of Product-of-Sums Forms

- The technique used to find the minimum-cost sum-of-products (SOP) implementations of functions can be used with the principle of duality to obtain minimum cost product-of-sums (POS) implementations. In this case it is the maxterms for which $f = 0$ that have to be combined into sum terms that are as large as possible. Again, a sum term is considered larger if it covers more maxterms, and the larger the term, the less costly it is to implement.

Example 4

$$f = \Pi M(4, 5, 6)$$

$x_1 x_2$					
		00	01	11	10
x_3	0	1	1	0	0
	1	1	1	1	0

$(\bar{x}_1 + x_3)$ (points to the 0s in row $x_3=0$)
 $(\bar{x}_1 + x_2)$ (points to the 0s in column $x_2=10$)

There are three maxterms that must be covered: M_4 , M_5 , and M_6 . They can be covered by two sum terms shown in the figure, leading to the following implementation:

$$f = (\bar{x}_1 + x_2)(\bar{x}_1 + x_3)$$

Example 5

Simplify the function in POS form-

$$F(A, B, C, D) = \Pi M(1, 4, 9, 12, 13)$$

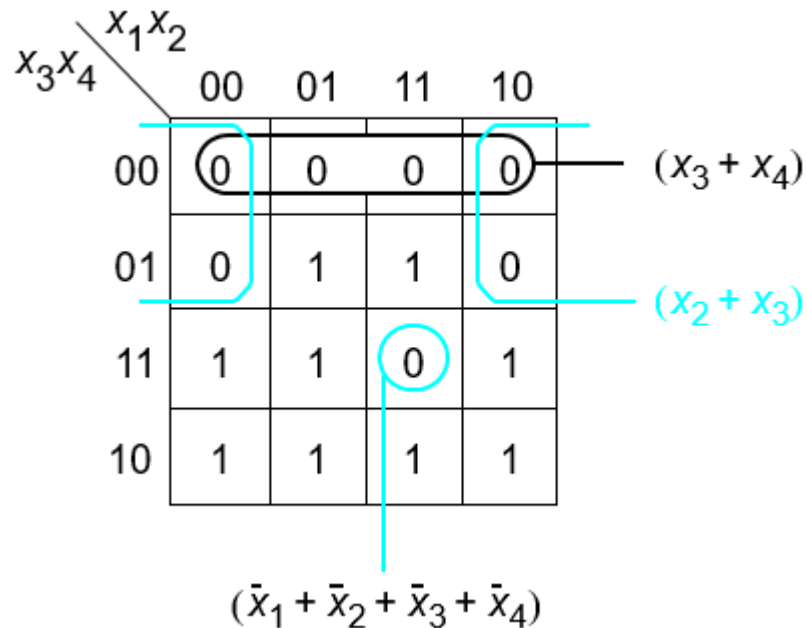
		A			
		00	01	11	10
CD	00	1	0	0	1
	01	0	1	0	0
C	11	1	1	1	1
	10	1	1	1	1
		B			

$$F = (B' + C + D)(B + C + D')(A' + B' + C)$$

Example 6

Simplify the function

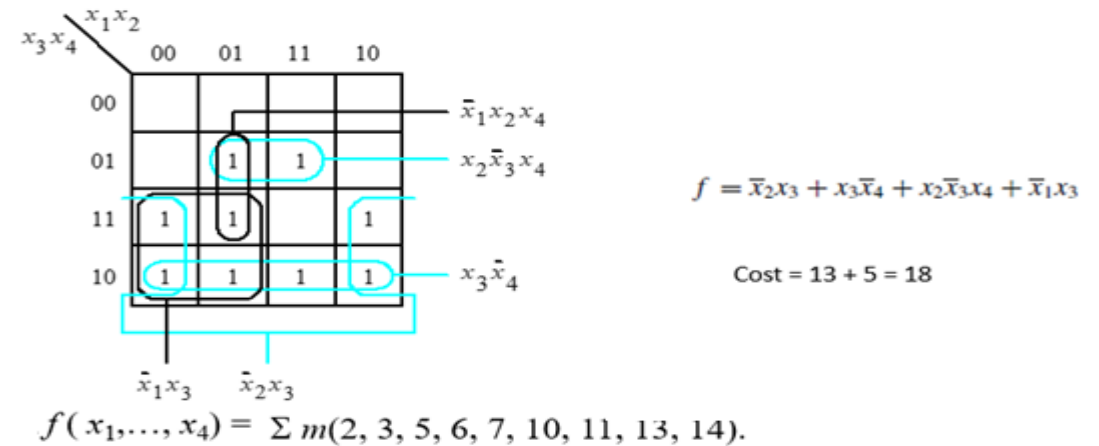
$$f(x_1, \dots, x_4) = \prod M(0, 1, 4, 8, 9, 12, 15)$$



$$f = (x_2 + x_3)(x_3 + x_4)(\bar{x}_1 + \bar{x}_2 + \bar{x}_3 + \bar{x}_4)$$

POS minimization of $f(x_1, \dots, x_4) = \prod M(0, 1, 4, 8, 9, 12, 15)$.

$$f(x_1, \dots, x_4) = \sum m(2, 3, 5, 6, 7, 10, 11, 13, 14)$$



Assuming that both the complemented and uncomplemented versions of the input variables x_1 to x_4 are available at no extra cost, the cost of this circuit in POS is 11 + 4 = 15.

But for the same function in the SOP implementation requires cost = 13 + 5 = 18.

Hence, for the given function, the minimum cost implementation solution is POS

Incompletely Specified Functions

- In digital systems it often happens that certain input conditions can never occur and is a *don't-care condition*. A function that has don't-care condition(s) is said to be *incompletely specified*. Don't-care conditions, or *don't cares* for short, can be used to advantage in the design of logic circuits. Since these input valuations will never occur, the designer may assume that the function value for these valuations is either 1 or 0, whichever is more useful in trying to find a minimum-cost implementation.
- If there are k don't cares, then there are 2^k (2 to the power k) possible ways of assigning 0 or 1 values to them.

Incompletely Specified Functions

Example 7- Simplify the given function using K-map.

$$f(A,B,C,D) = \sum m (1,3,5,7,8,10,12) + d(0,13,14,15)$$

AB \ CD		AB			
		00	01	11	10
CD	00	0 X	4	12 1	8 1
	01	1 1	5 1	13 X	9
	11	3 1	7 1	15 X	11
	10	2	6	14 X	10 1

$$f = A'D + AD'$$

$x_3x_4 \backslash x_1x_2$	00	01	11	10
00	0	1	d	0
01	0	1	d	0
11	0	0	d	0
10	1	1	d	1

(a) SOP implementation

$x_3x_4 \backslash x_1x_2$	00	01	11	10
00	0	1	d	0
01	0	1	d	0
11	0	0	d	0
10	1	1	d	1

(b) POS implementation

Two implementations of the function $f(x_1, \dots, x_4) = \sum m(2, 4, 5, 6, 10) + D(12, 13, 14, 15)$.

Example 8- $f(x_1, \dots, x_4) = \sum m(2, 4, 5, 6, 10) + D(12, 13, 14, 15)$

where D is the set of don't cares.

Part (a) of the figure indicates the best sum-of-products implementation. To form the largest possible groups of 1s, thus generating the lowest-cost prime implicants, it is necessary to assume that the don't cares D_{12} , D_{13} , and D_{14} (corresponding to minterms m_{12} , m_{13} , and m_{14}) have the value of 1 while D_{15} has the value of 0. Then there are only two prime implicants, which provide a complete cover of f . The resulting implementation is

$$f = x_2\bar{x}_3 + x_3\bar{x}_4$$

Part (b) shows how the best product-of-sums implementation can be obtained. The same values are assumed for the don't cares. The result is

$$f = (x_2 + x_3)(\bar{x}_3 + \bar{x}_4)$$

Multiple-Output Circuits

- In practical digital systems it is necessary to implement a number of functions as part of some large logic circuit. Circuits that implement these functions can often be combined into a less-expensive single circuit with multiple outputs by sharing some of the gates needed in the implementation of individual functions.

Example 9

Find minimum cost implementation for the functions-

$$f_1 = \Sigma m(2,3,5,6,7,8,9,12,13)$$

$$f_2 = \Sigma m(2,3,6,7,8,9,12,13,15)$$

$x_3x_4 \backslash x_1x_2$	00	01	11	10
00			1	1
01		1	1	1
11	1	1		
10	1	1		

(a) Function f_1

$x_3x_4 \backslash x_1x_2$	00	01	11	10
00			1	1
01			1	1
11	1	1	1	
10	1	1		

(b) Function f_2

$$f_1 = x_1\bar{x}_3 + \bar{x}_1x_3 + x_2\bar{x}_3x_4$$

$$f_2 = x_1\bar{x}_3 + \bar{x}_1x_3 + x_2x_3x_4$$

- The cost of $f_1 = 4$ gates + 10 inputs wires = 14.
- The cost of f_2 is the same.
- Thus, the total cost is 28 if both functions are implemented by separate circuits.
- Low cost realization is possible if the two circuits are combined into a single circuit with two outputs.
- Because the first two product terms are identical in both expressions, the AND gates that implement them need not be duplicated. The combined circuit is shown in (c). Its cost is six gates and 16 inputs, for a total of 22.

Example 9 continued....

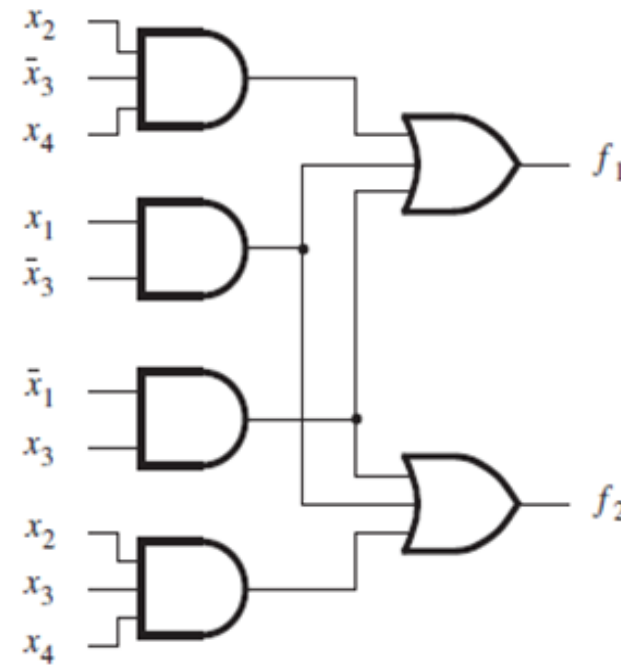
Find minimum cost implementation for the functions-

$$f_1 = \Sigma m (2,3,5,6,7,8,9,12,13)$$

$$f_2 = \Sigma m (2,3,6,7,8,9,12,13,15)$$

$$f_1 = x_1\bar{x}_3 + \bar{x}_1x_3 + x_2\bar{x}_3x_4$$

$$f_2 = x_1\bar{x}_3 + \bar{x}_1x_3 + x_2x_3x_4$$



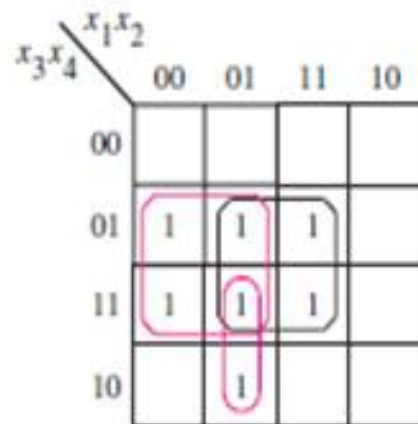
(c)

Example 10

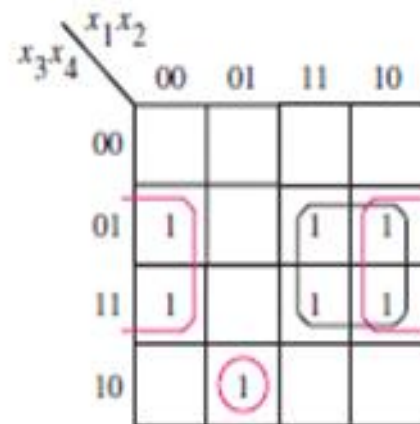
Find minimum cost implementation for the given functions-

$$f_3 = \sum m (1,3,5,6,7,13,15)$$

$$f_4 = \sum m (1,3,6,9,11,13,15)$$



(a) Optimal realization of f_3



(b) Optimal realization of f_4

$$f_3 = \bar{x}_1x_4 + x_2x_4 + \bar{x}_1x_2x_3$$

$$f_4 = x_1x_4 + \bar{x}_2x_4 + \bar{x}_1x_2x_3\bar{x}_4$$

Realizations of the individual functions f_3 and f_4 are obtained from parts (a) and (b). None of the AND gates can be shared, which means that the cost of the combined circuit = 6 AND gates + 2 OR gates + 21 inputs = 29.

Example 10 continued...

$$f_3 = \sum m(1,3,5,6,7,13,15)$$

$$f_4 = \sum m(1,3,6,9,11,13,15)$$

The combined realization of the functions in (c), shows that the first two implicants are identical in both expressions. The resulting circuit is given in (d).

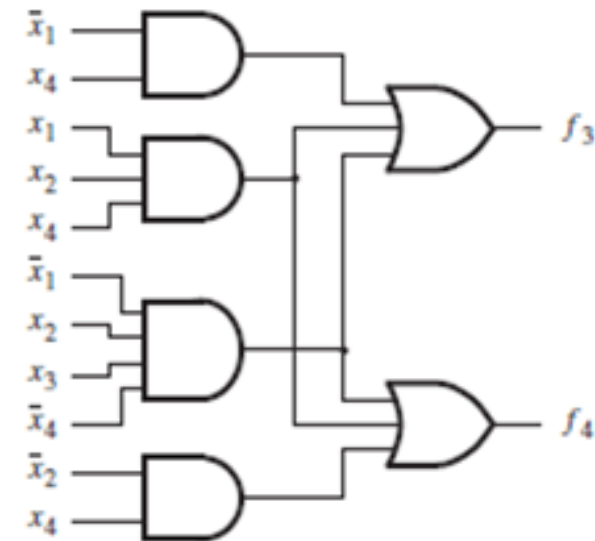
$x_3x_4 \backslash x_1x_2$	00	01	11	10
00				
01	1	1	1	
11	1	1	1	
10		1		

$x_3x_4 \backslash x_1x_2$	00	01	11	10
00				
01	1		1	1
11	1		1	1
10		1		

(c) Optimal realization of f_3 and f_4 together

$$f_3 = x_1x_2x_4 + \bar{x}_1x_2x_3\bar{x}_4 + \bar{x}_1x_4$$

$$f_4 = x_1x_2x_4 + \bar{x}_1x_2x_3\bar{x}_4 + \bar{x}_2x_4$$



(d)

It has the cost = 6 gates + 17 inputs = 23.

Problems for Practice - Tutorial

- Repeat the above two examples for POS implementation
 1. f1 and f2 in POS
 2. f3 and f4 in POS

Digital Electronics & Systems

19ECE204

Lecture 22-25

Ms. Bhavana V
Assistant Professor
Dept. of ECE
ASE, Bengaluru

TABULAR METHOD FOR MINIMIZATION

- K-map method of simplification is convenient as long as the number of variables does not exceed five or six.
- As the number of variables increases it is difficult to make judgement about which combinations form the minimum expression. (Complexity of **K-map** simplification process increases with the increase in the number of variables.(The minimum expression obtained might not be unique.))
- K-map simplification is manual technique and simplification process heavily depends on the human abilities.
- To meet this needs, In 1950s W.V Quine and E.J McCluskey developed an exact tabular method to simplify the Boolean expression. This method is called the Quine-McCluskey, or tabular method.

Algorithm for generating minimum prime implicants using Quine-McCluskey/ Tabular method:

1. List all minterms in the binary form.
2. Arrange the minterms according to number of 1s.
3. Compare each binary number with every term in the adjacent next higher category and if they differ only by one position, put a tick mark and copy the term in the next column with 'X' in the position that they differed.
4. Apply the same process described in step 3 for the resultant column and continue these cycles until a single pass through cycle yields no further elimination of literals.
5. List all prime implicants.
6. Select the minimum number of prime implicants which must cover all the minterms.

Example 1- $F(W,X,Y,Z) = \sum(5,7,9,11,13,15)$

Solution- Generation of Prime implicants Table-

Step 1			Step 2			Step 3		
⊗	5	0101	⊗	5,7	01-1		5,7,13,15	-1-1
⊗	9	1001	⊗	5,13	-101		5,13,7,15	-1-1
			⊗	9,11	10-1		9,11,13,15	1- -1
⊗	7	0111	⊗	9,13	1-01		9,13,11,15	1- -1
⊗	11	1011						
⊗	13	1101	⊗	7,15	-111			
			⊗	11,15	1-11			
⊗	15	1111	⊗	13,15	11-1			

Example 1- $F(W,X,Y,Z) = \sum(5,7,9,11,13,15)$

Finding Essential Prime implicants (EPIs) (Initial Prime implicant cover table)

	Prime Implicants	Covered Minterms	<u>Minterms</u>					
			5	7	9	11	13	15
⊗	- 1 - 1	5,7,13,15	⊗	⊗			X	X
⊗	1 - - 1	9,13,11,15			⊗	⊗	X	X
			⊗	⊗	⊗	⊗	⊗	⊗

Minimum cost cover-

$$F = (X . Z) +(W.Z) = (X +W).Z$$

Example 2- Generate the prime implicants for the function

$$f(x_1, x_2, x_3, x_4) = \sum m(0, 4, 8, 10, 11, 12, 13, 15)$$

List 1

0	0 0 0 0	✓
4	0 1 0 0	✓
8	1 0 0 0	✓
10	1 0 1 0	✓
12	1 1 0 0	✓
11	1 0 1 1	✓
13	1 1 0 1	✓
15	1 1 1 1	✓

List 2

0,4	0 x 0 0	✓
0,8	x 0 0 0	✓
8,10	1 0 x 0	
4,12	x 1 0 0	✓
8,12	1 x 0 0	✓
10,11	1 0 1 x	
12,13	1 1 0 x	
11,15	1 x 1 1	
13,15	1 1 x 1	

List 3

0,4,8,12	x x 0 0
----------	---------

Prime implicants are-

$$P = \{10x0, 101x, 110x, 1x11, 11x1, xx00\}$$

$$= \{p_1, p_2, p_3, p_4, p_5, p_6\}$$

DETERMINATION OF A MINIMUM COVER

Essential prime
implicant is-
= {P6}
= {x3'x4'}

	Prime implicant	Minterm							
		0	4	8	10	11	12	13	15
8,10	$p_1 = 1\ 0\ x\ 0$			✓	✓				
10,11	$p_2 = 1\ 0\ 1\ x$				✓	✓			
12,13	$p_3 = 1\ 1\ 0\ x$						✓	✓	
11,15	$p_4 = 1\ x\ 1\ 1$					✓			✓
13,15	$p_5 = 1\ 1\ x\ 1$							✓	✓
0,4,8,12	$p_6 = x\ x\ 0\ 0$	✓	✓	✓			✓		

(a) Initial prime implicant cover table

single check mark in some column of the cover table, then the prime implicant that covers the minterm of this column is *essential* and it must be included in the final cover. Such is the case with p_6 , which is the only prime implicant that covers minterms 0 and 4. The next step is to remove the row(s) corresponding to the essential prime implicants and the column(s) covered by them. Hence we remove p_6 and columns 0, 4, 8, and 12, which leads to the table in figure b.

	Prime implicant	Minterm			
		10	11	13	15
8,10	P_1	✓			
10,11	P_2	✓	✓		
12,13	P_3			✓	
11,15	P_4		✓		✓
13,15	P_5			✓	✓

Row Dominance
 P_2 dominates P_1
 P_5 dominates P_3
Delete dominated rows i.e P_1 and P_3

(b) After the removal of essential prime implicants

Now, we can use the concept of *row dominance* to reduce the cover table. Observe that p_1 covers only minterm 10 while p_2 covers both 10 and 11. We say that p_2 *dominates* p_1 . Since the cost of p_2 is the same as the cost of p_1 , it is prudent to choose p_2 rather than p_1 , so we will remove p_1 from the table. Similarly, p_5 dominates p_3 , hence we will remove p_3 from the table. Thus, we obtain the table in Figure 4.37c. This table indicates that we must choose p_2 to cover minterm 10 and p_5 to cover minterm 13, which also takes care of covering minterms 11 and 15. Therefore, the final cover is

$$\begin{aligned} C &= \{p_2, p_5, p_6\} \\ &= \{101x, 11x1, xx'00\}. \end{aligned}$$

Prime implicant	Minterm			
	10	11	13	15
10,11 p_2	✓	✓		
11,15 p_4		✓		✓
13,15 p_5			✓	✓

which means that the minimum-cost implementation of the function is

$$f = x_1\bar{x}_2x_3 + x_1x_2x_4 + \bar{x}_3\bar{x}_4$$

Example 3- Use the tabular method to determine the minimum-cost SOP expression for the function $f(x_1, x_2, x_3, x_4) = \sum m(4, 6, 8, 10, 11, 12, 15) + D(3, 5, 7, 9)$

List 1

4	0	1	0	0	✓
8	1	0	0	0	✓
3	0	0	1	1	✓
5	0	1	0	1	✓
6	0	1	1	0	✓
9	1	0	0	1	✓
10	1	0	1	0	✓
12	1	1	0	0	✓
7	0	1	1	1	✓
11	1	0	1	1	✓
15	1	1	1	1	✓

List 2

4,5	0	1	0	x	✓
4,6	0	1	x	0	✓
4,12	x	1	0	0	
8,9	1	0	0	x	✓
8,10	1	0	x	0	✓
8,12	1	x	0	0	
3,7	0	x	1	1	✓
3,11	x	0	1	1	✓
5,7	0	1	x	1	✓
6,7	0	1	1	x	✓
9,11	1	0	x	1	✓
10,11	1	0	1	x	✓
7,15	x	1	1	1	✓
11,15	1	x	1	1	✓

List 3

4,5,6,7	0	1	x	x	
4,6,5,7	0	1	x	x	
8,9,10,11	1	0	x	x	
8,10,9,11	1	0	x	x	
3,7,11,15	x	x	1	1	
3,11,7,15	x	x	1	1	

The prime implicants for this function are:

$$P = \{x100, 1x00, 01xx, 10xx, xx11\}$$

$$= \{p_1, p_2, p_3, p_4, p_5\}$$

Initial prime implicant cover table

Prime Implicant		Minterm						
		4	6	8	10	11	12	15
P1(4,12)	x 1 0 0	✓					✓	
P2(8,12)	1 x 0 0			✓			✓	
P3(4,5,6,7)	0 1 x x	✓	✓					
P4(8,9,10,11)	1 0 x x			✓	✓	✓		
P5(3,7,11,15)	x x 1 1					✓		✓

Essential prime implicants (EPIs)

are-

$$= \{P3, P4, P5\}$$

$$= \{x_1'x_2, x_1x_2', x_3x_4\}$$

Minimum Cost Cover is-

$$\begin{aligned}
 f(x_1, x_2, x_3, x_4) &= \{P2, P3, P4, P5\} \text{ or } \{P1, P3, P4, P5\} \\
 &= x_1x_3'x_4' + x_1'x_2 + x_1x_2' + x_3x_4 \quad \text{or} \\
 &= x_2x_3'x_4' + x_1'x_2 + x_1x_2' + x_3x_4
 \end{aligned}$$

Example 4- Use the tabular method to determine the minimum-cost POS expression for the function $f(x_1, x_2, x_3, x_4) = \Pi M(0, 1, 2, 13, 14) + D(3, 5, 7, 9)$

List 1

0	0 0 0 0	✓
1	0 0 0 1	✓
2	0 0 1 0	✓
3	0 0 1 1	✓
5	0 1 0 1	✓
9	1 0 0 1	✓
7	0 1 1 1	✓
13	1 1 0 1	✓
14	1 1 1 0	

List 2

0,1	0 0 0 x	✓
0,2	0 0 x 0	✓
1,3	0 0 x 1	✓
1,5	0 x 0 1	✓
1,9	x 0 0 1	✓
2,3	0 0 1 x	✓
3,7	0 x 1 1	✓
5,7	0 1 x 1	✓
5,13	x 1 0 1	✓
9,13	1 x 0 1	✓

List 3

0,1,2,3	0 0 x x	
0,2,1,3	0 0 x x	
1,3,5,7	0 x x 1	
1,5,3,7	0 x x 1	
1,5,9,13	x x 0 1	
1,9,5,13	x x 0 1	

The prime implicants for this function are:

$$P = \{1100, 00xx, 0xx1, xx01\}$$

$$= \{P1, P2, P3, P4\}$$

Initial prime implicant cover table

Prime Implicant		Maxterm				
		0	1	2	13	14
P1 (14)	1 1 1 0					✓
P2 (0,1,2,3)	0 0 x x	✓	✓	✓		
P3 (1,3,5,7)	0 x x 1		✓			
P4 (1,5,9,13)	x x 0 1		✓		✓	

Essential prime implicants (EPIs) are-
 $= \{P1, P2, P4\}$
 $= \{(x1' + x2' + x3' + x4), (x1 + x2) (x3 + x4')\}$

Minimum cost cover is-

$$f(x1, x2, x3, x4) = (x1 + x2) (x3 + x4') (x1' + x2' + x3' + x4)$$

Example 5 - $f(x_1, \dots, x_4) = \sum m(0, 2, 5, 6, 7, 8, 9, 13) + D(1, 12, 15)$

List 1

0	0 0 0 0	✓
1	0 0 0 1	✓
2	0 0 1 0	✓
8	1 0 0 0	✓
5	0 1 0 1	✓
6	0 1 1 0	✓
9	1 0 0 1	✓
12	1 1 0 0	✓
7	0 1 1 1	✓
13	1 1 0 1	✓
15	1 1 1 1	✓

List 2

0,1	0 0 0 x	✓
0,2	0 0 x 0	✓
0,8	x 0 0 0	✓
1,5	0 x 0 1	✓
2,6	0 x 1 0	✓
1,9	x 0 0 1	✓
8,9	1 0 0 x	✓
8,12	1 x 0 0	✓
5,7	0 1 x 1	✓
6,7	0 1 1 x	✓
5,13	x 1 0 1	✓
9,13	1 x 0 1	✓
12,13	1 1 0 x	✓
7,15	x 1 1 1	✓
13,15	1 1 x 1	✓

List 3

0,1,8,9	x 0 0 x
1,5,9,13	x x 0 1
8,9,12,13	1 x 0 x
5,7,13,15	x 1 x 1

$$P = \{00x0, 0x10, 011x, x00x, xx01, 1x0x, x1x1\}$$

$$= \{p_1, p_2, p_3, p_4, p_5, p_6, p_7\}$$

	Prime implicant	Minterm							
		0	2	5	6	7	8	9	13
0,2	$p_1 = 0 \ 0 \ x \ 0$	✓	✓						
2,6	$p_2 = 0 \ x \ 1 \ 0$		✓		✓				
6,7	$p_3 = 0 \ 1 \ 1 \ x$				✓	✓			
0,1,8,9	$p_4 = x \ 0 \ 0 \ x$	✓					✓	✓	
1,5,9,13	$p_5 = x \ x \ 0 \ 1$			✓				✓	✓
8,9,12,13	$p_6 = 1 \ x \ 0 \ x$						✓	✓	✓
5,7,13,15	$p_7 = x \ 1 \ x \ 1$			✓		✓			✓

Essential prime implicants (EPIs) - NIL

No Row dominance

Column Dominance
 Column 9 dominates 8
 Column 13 dominates 5
 Delete dominating columns i.e 9 & 13.

(a) Initial prime implicant cover table

Prime implicant	Minterm					
	0	2	5	6	7	8
$p_1 = 0 \ 0 \ x \ 0$	✓	✓				
$p_2 = 0 \ x \ 1 \ 0$		✓		✓		
$p_3 = 0 \ 1 \ 1 \ x$				✓	✓	
$p_4 = x \ 0 \ 0 \ x$	✓					✓
$p_5 = x \ x \ 0 \ 1$			✓			
$p_6 = 1 \ x \ 0 \ x$						✓
$p_7 = x \ 1 \ x \ 1$			✓		✓	

Row Dominance
 P4 dominates P6
 P7 dominates P5
 Delete dominated
 rows i.e P5 and P6

(b) After the removal of columns 9 and 13

Prime implicant	Minterm					
	0	2	5	6	7	8
0,2 P_1	✓	✓				
2,6 P_2		✓		✓		
6,7 P_3				✓	✓	
0,1,8,9 P_4	✓					✓
5,7,13,15 P_7			✓		✓	

(c) After the removal of rows P_5 and P_6

Prime implicant	Minterm	
	2	6
P_1	✓	
P_2	✓	✓
P_3		✓

(d) After including P_4 and P_7 in the cover

After removing columns 9 and 13, we obtain the reduced table in Figure 4.39b. In this table row p_4 dominates p_6 and row p_7 dominates p_5 . This means that p_5 and p_6 can be removed, giving the table in Figure 4.39c. Now, p_4 and p_7 are essential to cover minterms 8 and 5, respectively. Thus, the table in Figure 4.39d is obtained, from which it is obvious that p_2 covers the remaining minterms 2 and 6. Note that row p_2 dominates both rows p_1 and p_3 . The final cover is

$$\begin{aligned} C &= \{p_2, p_4, p_7\} \\ &= \{0x10, x00x, x1x1\} \end{aligned}$$

and the function is implemented as

$$f = \bar{x}_1 x_3 \bar{x}_4 + \bar{x}_2 \bar{x}_3 + x_2 x_4$$

Example 6 - $f(a,b,c,d,e) = \sum m(0,1,2,3,6,8,9,10,11,17,20,21,23,25,28,30,31)$

List 1

0	0 0 0 0 0	✓
1	0 0 0 0 1	✓
2	0 0 0 1 0	✓
8	0 1 0 0 0	✓
3	0 0 0 1 1	✓
6	0 0 1 1 0	✓
9	0 1 0 0 1	✓
10	0 1 0 1 0	✓
17	1 0 0 0 1	✓
20	1 0 1 0 0	✓
11	0 1 0 1 1	✓
21	1 0 1 0 1	✓
25	1 1 0 0 1	✓
28	1 1 1 0 0	✓
23	1 0 1 1 1	✓
30	1 1 1 1 0	✓
31	1 1 1 1 1	✓

List 2

0,1	0 0 0 0 _	✓
0,2	0 0 0 _ 0	✓
0,8	0 _ 0 0 0	✓
1,3	0 0 0 _ 1	✓
1,9	0 _ 0 0 1	✓
1,17	_ 0 0 0 1	✓
2,6	0 0 _ 1 0	
2,10	0 _ 0 1 0	✓
8,9	0 1 0 0 _	✓
2,3	0 0 0 1 _	✓
8,10	0 1 0 _ 0	✓
3,11	0 _ 0 1 1	✓
9,11	0 1 0 _ 1	✓
9,25	_ 1 0 0 1	✓
10,11	0 1 0 1 _	✓
17,21	1 0 _ 0 1	
17,25	1 _ 0 0 1	✓
20,21	1 0 1 0 _	
20,28	1 _ 1 0 0	
21,23	1 0 1 _ 1	
28,30	1 1 1 _ 0	
23,31	1 _ 1 1 1	
30,31	1 1 1 1 _	

List 3

0,1,8,9	0 _ 0 0 _	✓
0,1,2,3	0 0 0 _ _	✓
0,2,8,10	0 _ 0 _ 0	✓
1,3,9,11	0 _ 0 _ 1	✓
1,9,17,25	_ _ 0 0 1	
2,10,3,11	0 _ 0 1 _	✓
8,9,10,11	0 1 0 _ _	✓

List 4

0,1,8,9,2,10,3,11 0 _ 0 _ _

The prime implicants for this function are:

$$P = \{00x10, 10x01, 1010x, 1x100, 101x1, 111x0, 1x111, 1111x, \\ xx001, 0x0xx\}$$

$$= \{P1, P2, P3, P4, P5, P6, P7, P8, P9, P10\}$$

PRIME IMPLICANT		MINTERM																
		0	1	2	3	6	8	9	10	11	17	20	21	23	25	28	30	31
2	P1 = 0 0 x 1 0			✓		✓												
	P2 = 1 0 x 0 1										✓		✓					
4	P3 = 1 0 1 0 x											✓	✓					
	P4 = 1 x 1 0 0											✓				✓		
	P5 = 1 0 1 x 1												✓	✓				
6	P6 = 1 1 1 x 0															✓	✓	
5	P7 = 1 x 1 1 1													✓				✓
	P8 = 1 1 1 1 x																✓	✓
3	P9 = x x 0 0 1		✓					✓			✓				✓			
1	P10 = 0 x 0 x x	✓	✓	✓	✓		✓	✓	✓	✓								

Essential Prime implicants are
 $= \{P10, P1, P9\}$
 $= \{a'c', a'b'de', c'd'e\}$

Minimum cost Cover is-
 $F(a,b,c,d,e) = a'c' + a'b'de' + c'd'e + ab'cd' + acde + abce'$

4.9.3 SUMMARY OF THE TABULAR METHOD

The tabular method can be summarized as follows:

1. Starting with a list of cubes that represent the minterms where $f = 1$ or a don't-care condition, generate the prime implicants by successive pairwise comparisons of the cubes.
2. Derive a cover table which indicates the minterms where $f = 1$ that are covered by each prime implicant.
3. Include the essential prime implicants (if any) in the final cover and reduce the table by removing both these prime implicants and the covered minterms.
4. Use the concept of row and column dominance to reduce the cover table further. A dominated row is removed only if the cost of its prime implicant is greater than or equal to the cost of the dominating row's prime implicant.
5. Repeat steps 3 and 4 until the cover table is either empty or no further reduction of the table is possible.
6. If the reduced cover table is not empty, then use the branching approach to determine the remaining prime implicants that should be included in a minimum cost cover.

1. $f(w,x,y,z) = \sum m(0,3,5,6,7,10,12,13) + \sum d(2,9,15)$

Ans: $f(w,x,y,z) = w'x'z' + w'y + x'yz' + xz + wxy'$

2. $f(w,x,y,z) = \sum m(0,1,3,4,7,11,13,15) + \sum d(9,12,14)$

Ans: $f(w,x,y,z) = w'y'z' + x'z + yz + wz$

Digital Electronics & Systems

19ECE204

Lecture 26-27

Ms. Bhavana V
Assistant Professor
Dept. of ECE
ASE, Bengaluru

Number System

(Number Representation and Conversion)

Number System

Introduction

- Many number systems are in use in digital technology. The most common are :
 - Decimal
 - Binary
 - Octal
 - Hexadecimal
- The **decimal system** is the number system that we use everyday.
- Numbers can be represented in two ways-Signed number and unsigned number.
- Signed numbers use sign flag to represent positive and negative numbers.
- Unsigned numbers only stores positive numbers.

Number System

Introduction

- **Decimal system** uses symbols (*digits*) for *the ten values* 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 so Base or Radix is 10.
- **Binary System** uses digits for *the two values* 0, and 1 so Base or Radix is 2.
- **Octal System** uses digits for *the eight values* 0, 1, 2, 3, 4, 5, 6, 7 so Base or Radix is 8.
- **Hexadecimal System** uses digits for *the sixteen values* 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F so Base or Radix is 16.

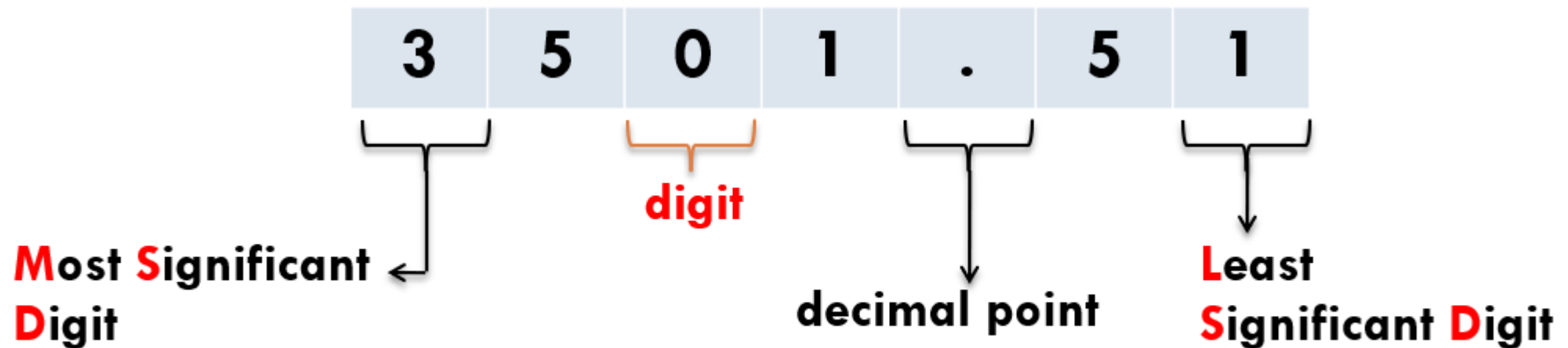
to represent any number, no matter how large or how small.

Number System

Decimal Number System

- **The decimal system** is composed of 10 numerals or symbols. These 10 symbols are 0,1,2,3,4,5,6,7,8,9; using these symbols as digits of a number, we can express any quantity.

Example : 3501.51



Number System

Decimal Number System(Positional weightage)

3 5 0 1 (base-10)

1	$\times 10^0$	=	1
0	$\times 10^1$	=	0
5	$\times 10^2$	=	500
3	$\times 10^3$	=	3000

$$3000 + 500 + 0 + 1 = 3501$$

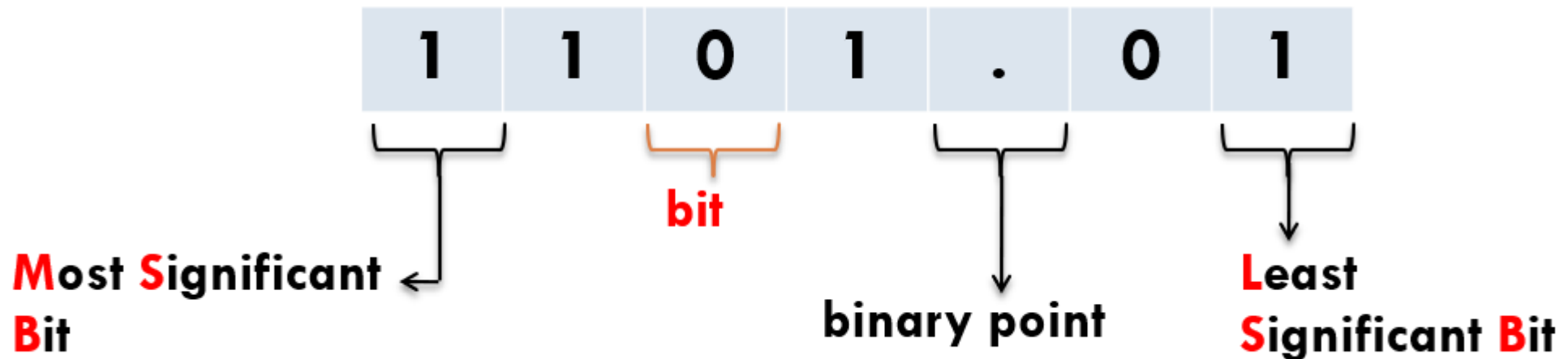
Number System

Binary Number System

- **The binary system** is composed of 2 numerals or symbols 0 and 1; using these symbols as digits of a number, we can express any quantity.

Example : 1101.01

- The binary number system is the most simple number system because it has only two symbols.



Number System

Binary Number System

In digital systems we use the binary, or base-2, system in which digits can be 0 or 1. Each binary digit is called a bit. In the binary number system, the same positional number representation is used so that

$$B = b_{n-1}b_{n-2} \cdots b_1b_0$$

Represents an integer that has the value,

$$\begin{aligned} V(B) &= b_{n-1} \times 2^{n-1} + b_{n-2} \times 2^{n-2} + \cdots + b_1 \times 2^1 + b_0 \times 2^0 \\ &= \sum_{i=0}^{n-1} b_i \times 2^i \end{aligned}$$

For example, the binary number 1101 represents the value, $V = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$

Evaluating the preceding expression for **V** gives, $V = 8 + 4 + 1 = 13$. Hence

$$(1101)_2 = (13)_{10}$$

Number System

Octal Number System

- The octal system provides a convenient way to express binary numbers and codes.
- However, it is used less frequently than hexadecimal in conjunction with computers and microprocessors to express binary quantities for input and output purposes.
- The octal system is composed of eight digits, which are:
0, 1, 2, 3, 4, 5, 6, 7
- To count above 7, begin another column and start over:
10, 11, 12, 13, 14, 15, 16, 17, 20, 21 and so on.
- Counting in octal is similar to counting in decimal, except that the digits 8 and 9 are not used.

Number System

Hexadecimal Number System

- The hexadecimal number system has sixteen digits and is used primarily as a compact way of displaying or writing binary numbers because it is very easy to convert between binary and hexadecimal.
- Hexadecimal is widely used in computer and microprocessor applications. The hexadecimal system has a base of sixteen; it is composed of 16 digits and alphabetic characters.
- The hexadecimal system is composed of nine digits and 6 alphabets; which are:

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F

Where A = 10, B = 11, C = 12, D = 13, E = 14, F = 15

Number System

Decimal-to-Radix-r Conversions (Integer Part)

- Successively divide number by r, taking remainder as result.

Decimal to Binary Conversion (Integer Part)

$(250)_{10}$

$250_{10} = (11111010)_2$

2	250		
2	125	Remainder	0
2	62	Remainder	1
2	31	Remainder	0
2	15	Remainder	1
2	7	Remainder	1
2	3	Remainder	1
	1	Remainder	1



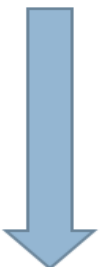
Number System

Decimal-to-Radix-r Conversions (Fractional Part)

- Successively multiply number by r, taking integer part as result and chopping off integer part before next iteration.
- May be unending!

Decimal to Binary Conversion (Fractional Part)

$(0.4375)_{10}$

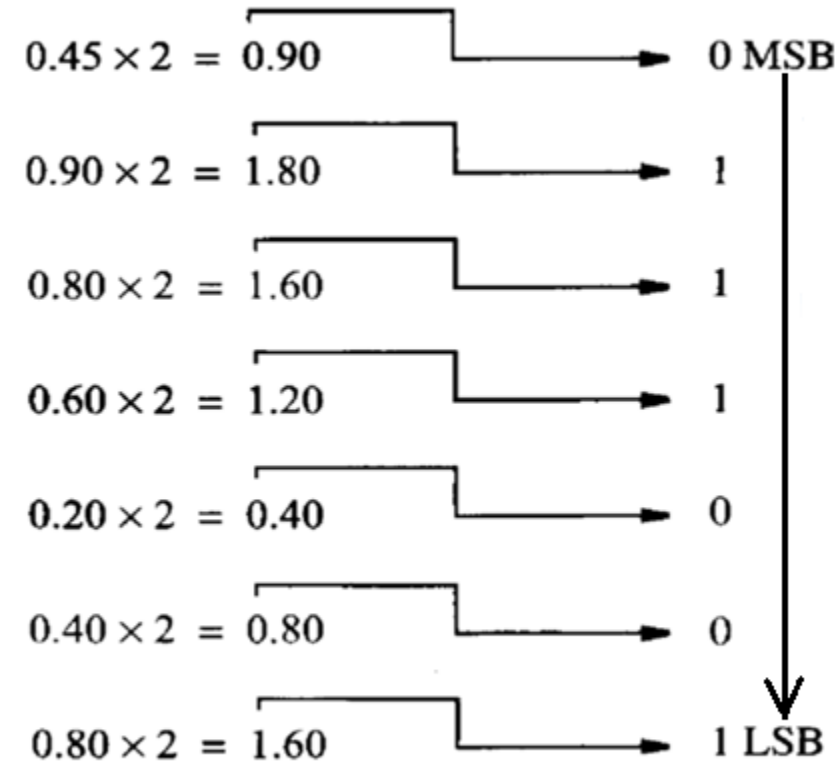
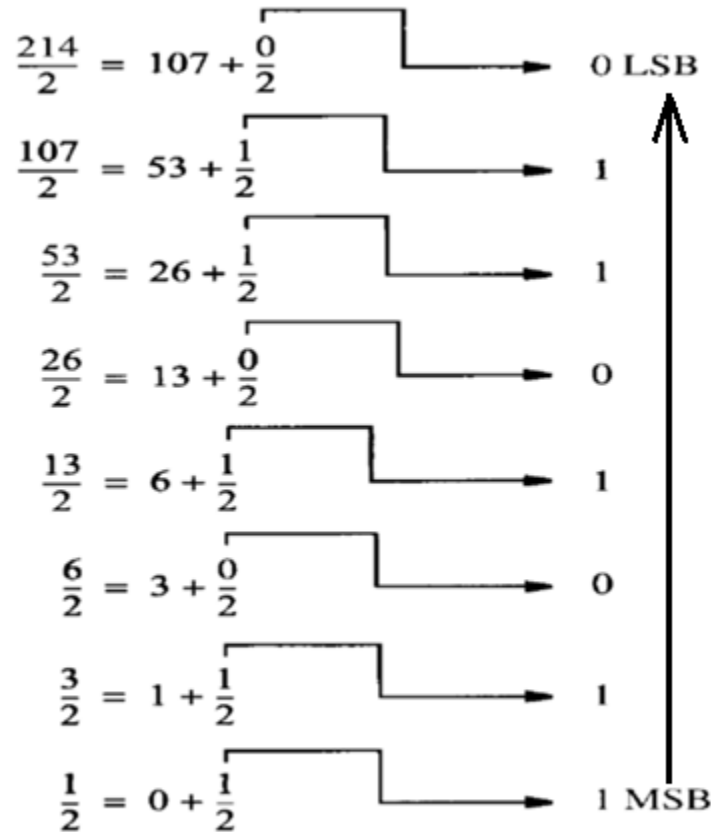
0.4375×2	$=$	0.8750	
0.8750×2	$=$	1.75	
0.75×2	$=$	1.5	
0.5×2	$=$	1.0	

$$(0.4375)_{10} = (0.0111)_2$$

Number System

Decimal to Binary Conversion

Decimal to binary: $(214.45)_{10} = (\dots)_2$



$$(214.45)_{10} = (11010110.0111001 \dots)_2$$

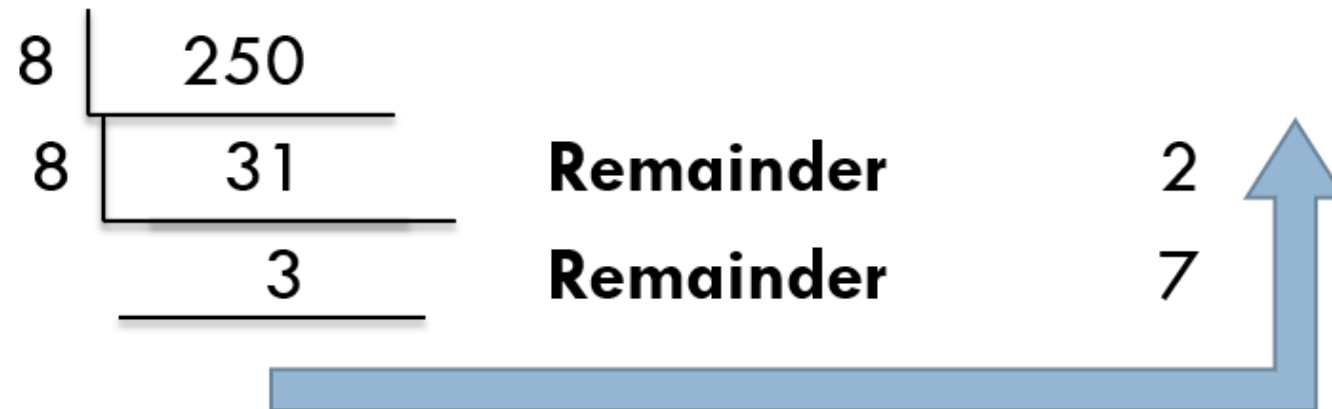
Number System

Decimal-to-Radix-r Conversions (Integer Part)

- Successively divide number by r, taking remainder as result.

Decimal to Octal Conversion (Integer Part)

$(250)_{10}$



$$(250)_{10} = (372)_8$$

Number System

Decimal-to-Radix-r Conversions (Fractional Part)

- Successively multiply number by r, taking integer part as result and chopping off integer part before next iteration.
- May be unending!

Decimal to Octal Conversion (Fractional Part)

$$(0.4375)_{10}$$

$$\begin{array}{rcl} 0.4375 \times 8 & = & 3.5 \\ 0.5 \quad \times 8 & = & 4.0 \end{array} \quad \downarrow$$

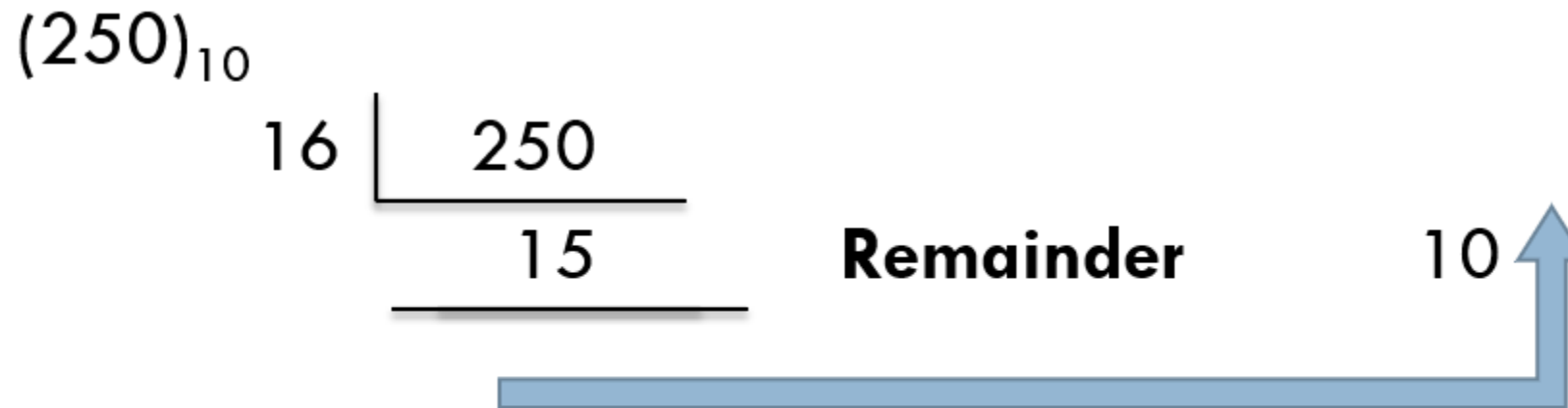
$$(0.4375)_{10} = (0.34)_8$$

Number System

Decimal-to-Radix-r Conversions (Integer Part)

- Successively divide number by r, taking remainder as result.

Decimal to Hexadecimal Conversion (Integer Part)



$$(250)_{10} = (\mathbf{FA})_{16}$$

Number System

Decimal-to-Radix-r Conversions (Fractional Part)

- Successively multiply number by r, taking integer part as result and chopping off integer part before next iteration.
- May be unending!

Decimal to Hexadecimal Conversion (Fractional Part)

$$(0.4375)_{10}$$

$$0.4375 \times 16 = 7.0$$


$$0.4375_{10} = 0.7_{16}$$

Number System

Decimal	Binary	Octal	Hexadecimal
00	00000	00	00
01	00001	01	01
02	00010	02	02
03	00011	03	03
04	00100	04	04
05	00101	05	05
06	00110	06	06
07	00111	07	07
08	01000	10	08
09	01001	11	09
10	01010	12	0A
11	01011	13	0B
12	01100	14	0C
13	01101	15	0D
14	01110	16	0E
15	01111	17	0F
16	10000	20	10
17	10001	21	11
18	10010	22	12

Number System

Radix-r –to-Decimal Conversions

- Multiply each digit by r^n , where r is the radix or base of the number to be converted in decimal and n is the weight of the digit.
- The weight is the position of the digit, starting from the zero on the right.
- Add the results.

Binary to Decimal Conversion (Integer Part)

$(1\ 1\ 0\ 1)_2$ (base-2)

1	1	0	1	
			→	$1 \times 2^0 = 1$
		→	$0 \times 2^1 = 0$	
	→	$1 \times 2^2 = 4$		
→	$1 \times 2^3 = 8$			

$$8 + 4 + 0 + 1 = 13$$

$$(1101)_2 = (13)_{10}$$

Number System

Radix-r –to-Decimal Conversions

Binary to Decimal Conversion (Fractional Part)

$(.101)_2$ (base-2)


$$\begin{aligned} 1 \times 2^{-1} &= 1 \times 0.5 = 0.5 \\ 0 \times 2^{-2} &= 0 \times 0.25 = 0 \\ 1 \times 2^{-3} &= 1 \times 0.125 = 0.125 \end{aligned}$$

$$0.5 + 0 + 0.125 = 0.625$$

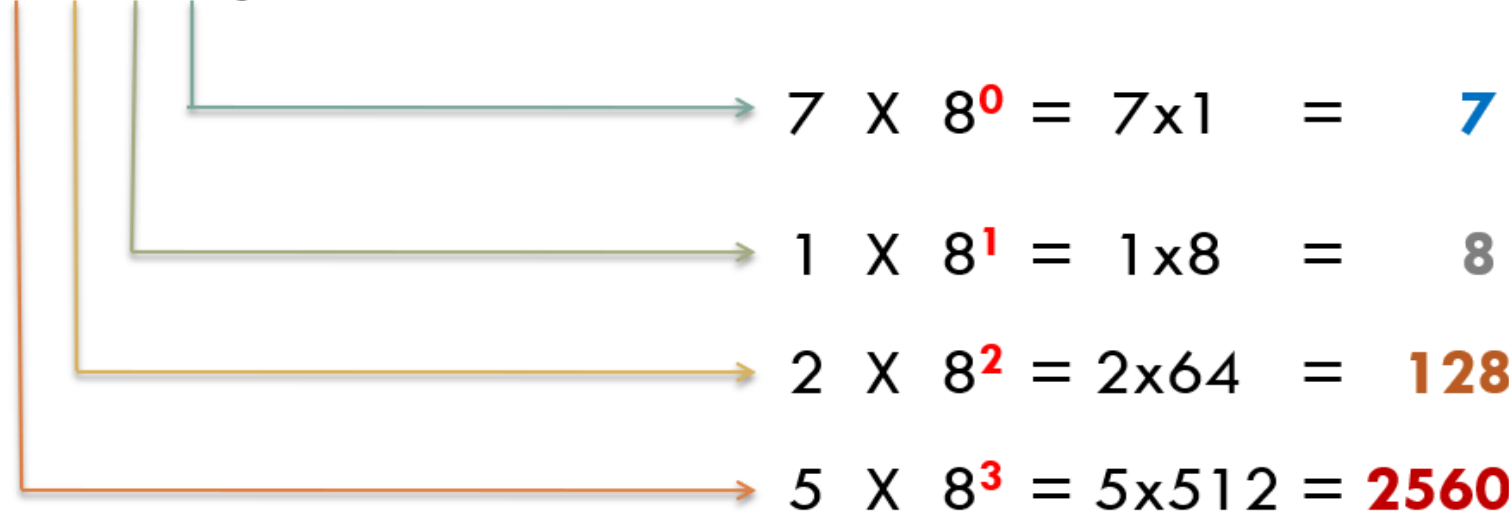
$$(0.101)_2 = (0.625)_{10}$$

Number System

Radix-r –to-Decimal Conversions

Octal to Decimal Conversion (Integer Part)

$(5\ 2\ 1\ 7)_8$ (base-8)



7	$\times 8^0$	$= 7 \times 1$	$= 7$
1	$\times 8^1$	$= 1 \times 8$	$= 8$
2	$\times 8^2$	$= 2 \times 64$	$= 128$
5	$\times 8^3$	$= 5 \times 512$	$= 2560$

$$2560 + 128 + 8 + 7 = 2703$$

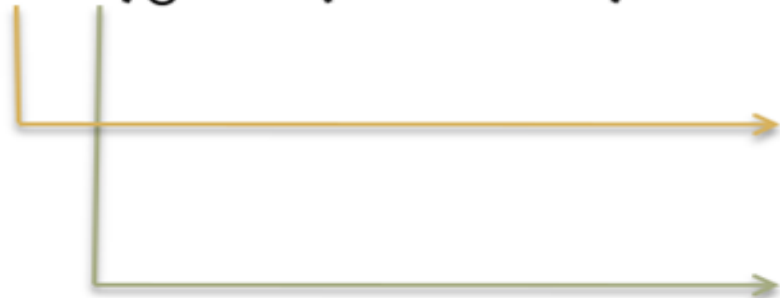
$$(5217)_8 = (2703)_{10}$$

Number System

Radix-r –to-Decimal Conversions

Octal to Decimal Conversion (Fractional Part)

$(.2\ 5)_8$ (base-8)


$$2 \times 8^{-1} = 2 \times 0.125 = \mathbf{0.25}$$
$$5 \times 8^{-2} = 5 \times 0.015625 = 0.078125$$

$$\mathbf{0.25} + 0.078125 = 0.328125$$

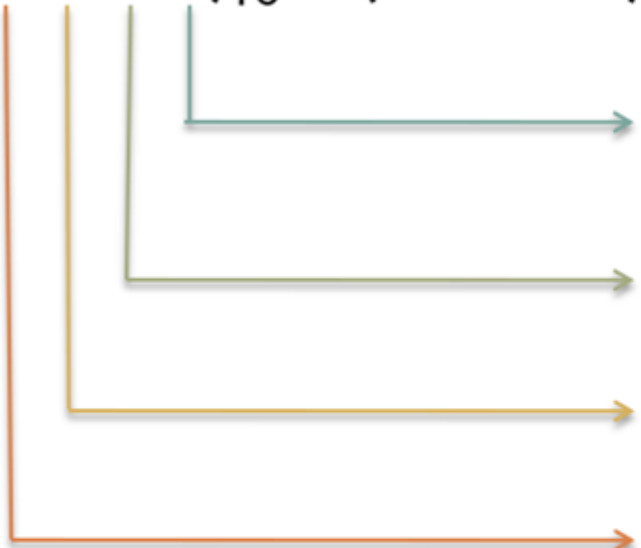
$$(0.25)_8 = (0.328125)_{10}$$

Number System

Radix-r –to-Decimal Conversions

Hexadecimal to Decimal Conversion (Integer Part)

$(5ACF)_{16}$ (base-16) [A = 10, B = 11, C = 12, D = 13, E = 14, F = 15]


$$\begin{aligned} & \rightarrow 15 \times 16^0 = 15 \times 1 = 15 \\ & \rightarrow 12 \times 16^1 = 12 \times 16 = 192 \\ & \rightarrow 10 \times 16^2 = 10 \times 256 = 2560 \\ & \rightarrow 5 \times 16^3 = 5 \times 4096 = 20480 \end{aligned}$$

$$20480 + 2560 + 192 + 15 = 23247$$

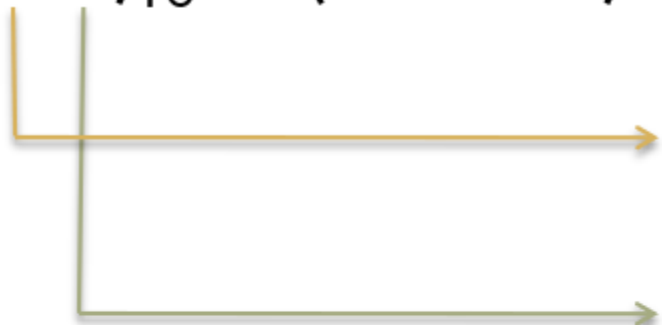
$$(5ACF)_{16} = (23247)_{10}$$

Number System

Radix-r –to-Decimal Conversions

Hexadecimal to Decimal Conversion (Fractional Part)

$(.F\ 5)_{16}$ (base-16)


$$15 \times 16^{-1} = 15 \times 0.0625 =$$
$$\mathbf{0.9375}$$
$$5 \times 16^{-2} = 5 \times 0.00390625$$
$$= \mathbf{0.01953125}$$

$$\mathbf{0.9375} + 0.01953125 = 0.95703125$$

$$(0.F5)_{16} = (0.95703125)_{10}$$

Number System

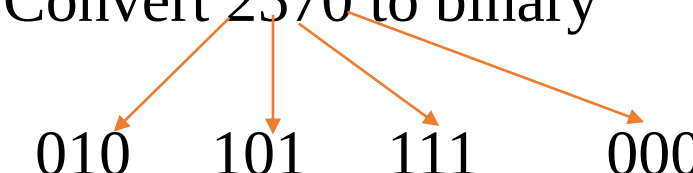
Octal to Binary Conversion

- Octal to Binary is just as straightforward; each octal digit is simply replaced by three bits that denote the same value. (Hint: Use 4-2-1 code)

Example:

Convert 2570 to binary

010 101 111 000



$$(2570)_8 = (010101111000)_2$$

Number System

Octal to Binary Conversion

Example: Convert $(765.31)_8$ to Binary.

$$7 = 111$$

$$6 = 110$$

$$5 = 101$$

$$3 = 011$$

$$1 = 001$$

$$(765.31)_8 = (111110101.011001)_2$$

Number System

Binary to Octal Conversion

- Group into 3 bits starting from least significant bit (if the number of bits is not evenly divisible by 3, then add 0's at the most significant end).
- Write 1 octal digit for each group.

Example: Convert $(1001\ 1110\ 0111\ 0000)_2$ to Octal.

1|001|111|001|110|000

add two zeros here

001|001|111|001|110|000

1 1 7 1 6 0

$$(1001\ 1110\ 0111\ 0000)_2 = (117160)_8$$

Number System

Binary to Octal Conversion

- If the binary number has both integer and fractional part then the group of 3 bits , for both integer and fractional part, will be formed from binary point.

Example: Convert $(11101111.0011101)_2$ to Octal.

11|101|111|.001|110|1

add zeros in the MSB side of integer part and LSB side of fractional part

011|101|111|.001|110|100

3 5 7 . 1 6 4

$$(11101111.0011101)_2 = (357.164)_8$$

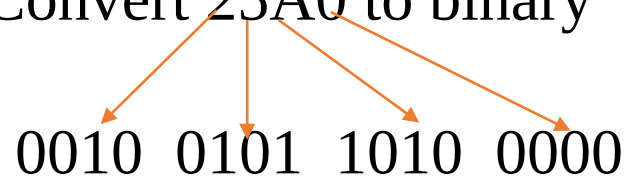
Number System

Hexadecimal to Binary Conversion

- Each hexadecimal digit is simply replaced by four bits that denote the same value. (Hint: Use 8-4-2-1 code)

Example:

Convert 25A0 to binary



0010 0101 1010 0000

$$(2570)_{16} = (0010010110100000)_2$$

Number System

Hexadecimal to Binary Conversion

Example- Convert $(B76A.C1)_{16}$ to Binary.

B = 1011

7 = 0111

6 = 0110

A = 1010

C = 1100

1 = 0001

$$(B76A.C1)_{16} = (1011011101101010.11000001)_2$$

Number System

Binary to Hexadecimal Conversion

- Group into 4 bits starting from least significant bit (if the number of bits is not evenly divisible by 4, then add 0's at the most significant end)
- Write 1 hexadecimal digit for each group.

Example: Convert $(1001\ 1110\ 0111\ 0000)_2$ to Hex.

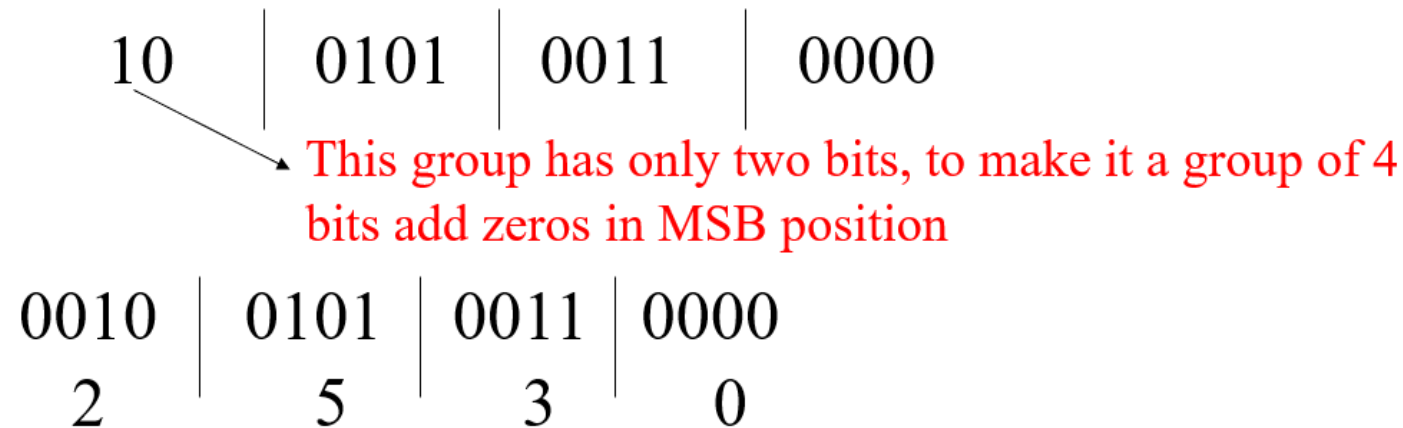
1001		1110		0111		0000
9		E		7		0

$$(1001\ 1110\ 0111\ 0000)_2 = (9E70)_{16}$$

Number System

Binary to Hexadecimal Conversion

Example: Convert 100 1010 011 0000 to Hex



$$(10010100110000)_2 = (2530)_{16}$$

Number System

Binary to Hexadecimal Conversion

Example- Convert $(1111101111.0011101)_2$ to Hexadecimal.

11 | 1110 | 1111 | .0011 | 101

Add zeros to the MSB side of integer part and LSB side of fractional part.

0011 | 1110 | 1111 | .0011 | 1010
 3 E F . 3 A

$$(1111101111.0011101)_2 = (3EF.3A)_{16}$$

Number System

Octal to Hexadecimal Conversion

Example: Convert $(5763.24)_8$ to Hexadecimal.

First convert $(5763.24)_8$ to Binary

5	7	6	3	.	2	4
101	111	110	011	.	010	100

$$(5673.24)_8 = (101111110011.010100)_2$$

Now convert Binary number to Hexadecimal by making group of 4 bits.

1011	1111	0011	0101	00	
1011	1111	0011	0101	0000	
B	F	3	.	5	0

adding two 0's to get group of 4 bits

$$(5673.24)_8 = (\mathbf{BF3.50})_{16}$$

Number System

Hexadecimal to Octal Conversion

Example: Convert $(FA79.C5)_{16}$ to Octal.

First convert $(FA79.C5)_{16}$ to Binary

F	A	7	9	.	C	5
1111	1010	0111	1001	.	1100	0101

$$(FA79.C5)_8 = (1111101001111001.11000101)_2$$

Now convert Binary number to Octal by making group of 3 bits.

1	111	101	001	111	001	.110	001	01	
001	111	101	001	111	001	.110	001	010	adding 0's in MSB and LSB sides to
1	7	5	1	7	1	.	6	1	2
									get group of 3 bits

$$(FA79.C5)_{16} = (175171.612)_{16}$$

Number System

Write next two consecutive numbers after given number.

1. $(98)_{10}$
 $98 \longrightarrow 99 \longrightarrow 100$

2. $(10)_2$
 $10 \longrightarrow 11 \longrightarrow 100$

3. $(76)_8$
 $76 \longrightarrow 77 \longrightarrow 100$

4. $(99)_{16}$
 $99 \longrightarrow 9A \longrightarrow 9B$

5. $(FE)_{16}$
 $FE \longrightarrow FF \longrightarrow 100$

Digital Electronics & Systems

19ECE204

Lecture 28-30

Arithmetic Circuits

Ms. Bhavana V
Assistant Professor
Dept. of ECE
ASE, Bengaluru

Unsigned Arithmetic

- Numbers that are positive only are called unsigned, and numbers that can also be negative are called signed.
- We will deal with binary arithmetic in this course.
- Unsigned Binary Addition and Subtraction
 - Binary addition is performed in the same way as decimal addition except that the values of individual digits can be only 0 or 1.

Unsigned Arithmetic - Addition

- Single Digit Addition, all possible combinations

$$\begin{array}{r} 0 \\ + 0 \\ \hline 00 \end{array}$$

$$\begin{array}{r} 0 \\ + 1 \\ \hline 01 \end{array}$$

$$\begin{array}{r} 1 \\ + 0 \\ \hline 01 \end{array}$$

$$\begin{array}{r} 1 \\ + 1 \\ \hline 10 \end{array}$$

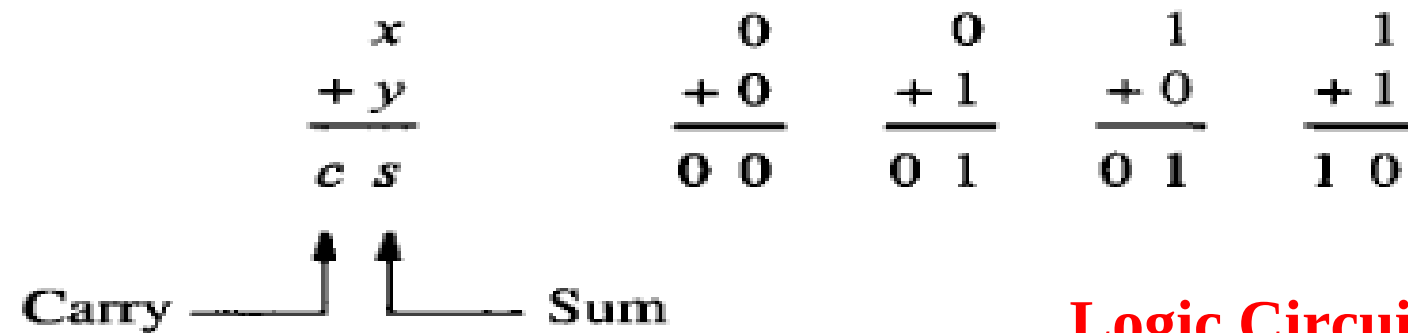
$$\begin{array}{r} x \\ + y \\ \hline c \ s \end{array}$$

Carry $\xrightarrow{\quad}$ $\uparrow$ $\uparrow$ Sum

Can we get a truth table for one bit unsigned addition ??

Unsigned Arithmetic - Half Adder

This circuit, which implements the addition of only two bits, is called a half-adder.



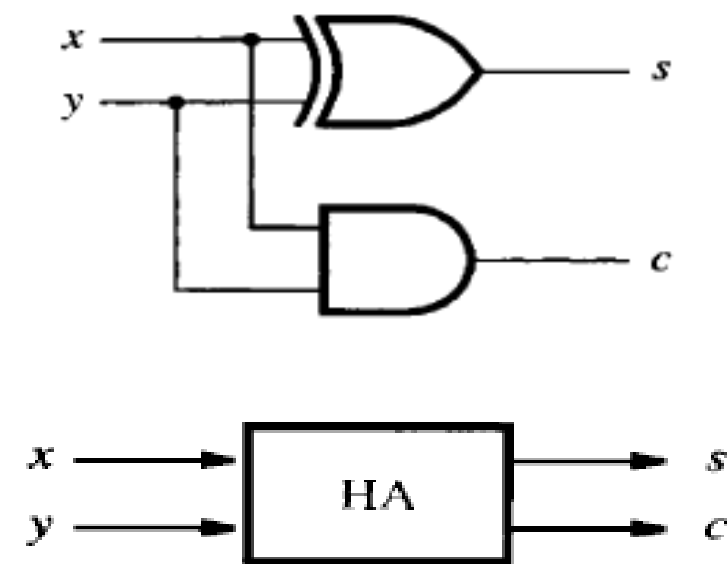
Truth Table

x	y	Carry c	Sum s
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$$s = xy' + x'y$$

$$c = xy$$

Logic Circuit



Unsigned Arithmetic - Multi bit

By Hand Method

Generated carries	1 1 1 0	
$X = x_4x_3x_2x_1x_0$	0 1 1 1 1	$(15)_{10}$
$+ Y = y_4y_3y_2y_1y_0$	0 1 0 1 0	$(10)_{10}$
$S = s_4s_3s_2s_1s_0$	1 1 0 0 1	$(25)_{10}$

How to design a digital circuit for this ???

Unsigned Arithmetic - Multi bit

	Generated carries	1 1 1 0	
$X = x_4x_3x_2x_1x_0$		0 1 1 1 1	$(15)_{10}$
$+ Y = y_4y_3y_2y_1y_0$		0 1 0 1 0	$(10)_{10}$
$S = s_4s_3s_2s_1s_0$		1 1 0 0 1	$(25)_{10}$

- For bit position 0, there is no carry-in, and hence the addition is the same as Half Adder.
- For each other bit position i the addition involves bits X_i and Y_i , and a carry-in C_i .
- This is a three variable function, for which truth table can be derived.

Unsigned Arithmetic - Full Adder

This circuit, which implements the addition of only three bits, is called a Full-adder.

Truth Table

c_i	x_i	y_i	c_{i+1}	s_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$s_i = \sum m(1,2,4,7)$$

$$c_{i+1} = \sum m(3,5,6,7)$$

K-map simplification

$c_i \backslash x_i y_i$	00	01	11	10
0		1		1
1	1		1	

$$s_i = \bar{x}_i \bar{y}_i \bar{c}_i + x_i \bar{y}_i \bar{c}_i + \bar{x}_i \bar{y}_i c_i + x_i y_i c_i$$

$$s_i = (\bar{x}_i y_i + x_i \bar{y}_i) \bar{c}_i + (\bar{x}_i \bar{y}_i + x_i y_i) c_i$$

$$= (x_i \oplus y_i) \bar{c}_i + \overline{(x_i \oplus y_i)} c_i$$

$$= (x_i \oplus y_i) \oplus c_i$$

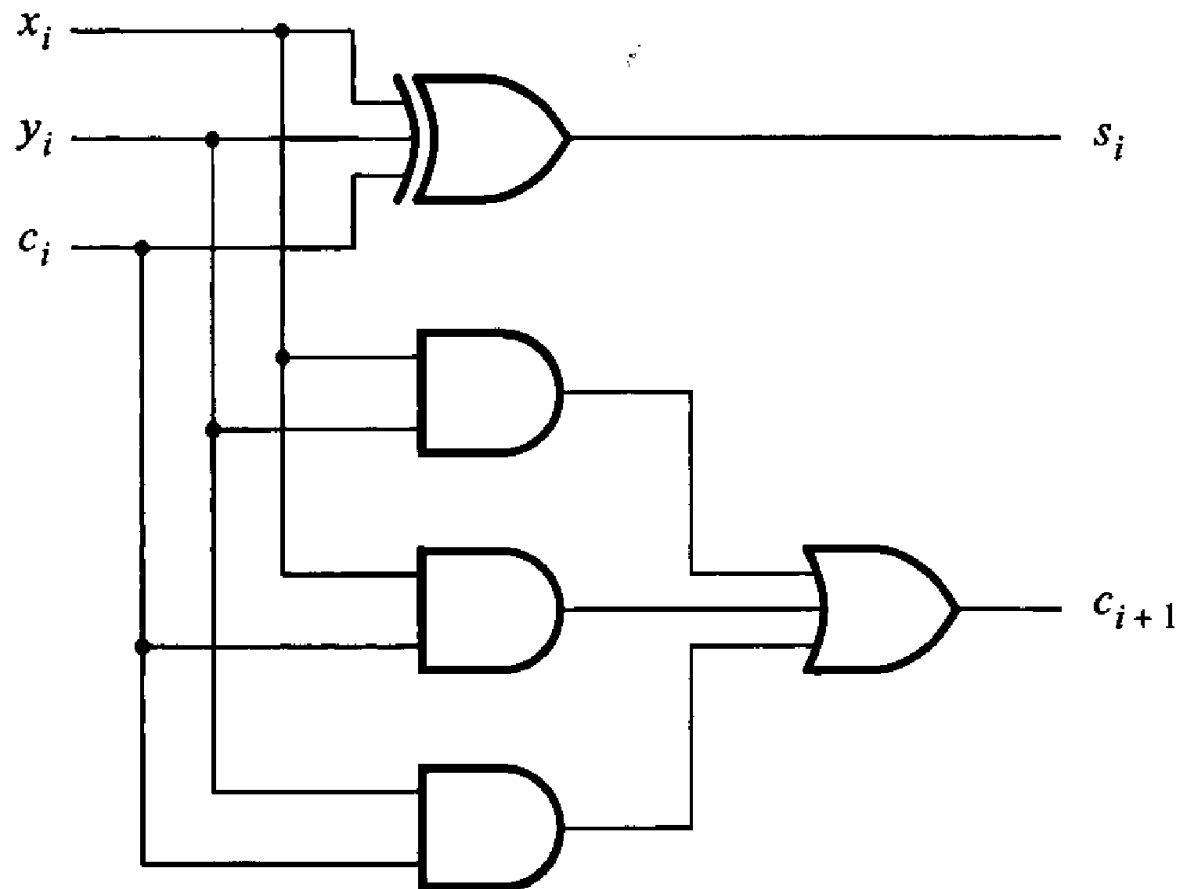
$$s_i = x_i \oplus y_i \oplus c_i$$

$c_i \backslash x_i y_i$	00	01	11	10
0			1	
1		1	1	1

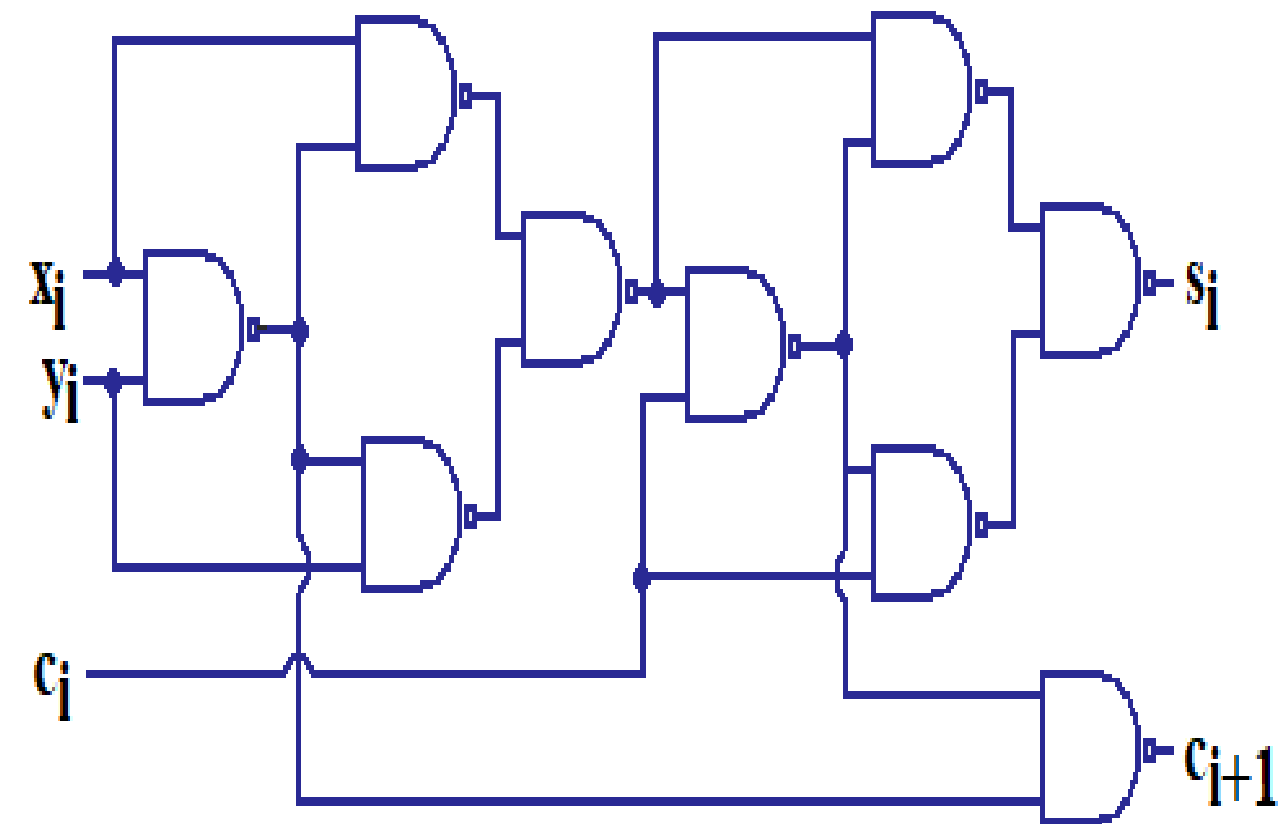
$$c_{i+1} = x_i y_i + x_i c_i + y_i c_i$$

Unsigned Arithmetic - Full Adder

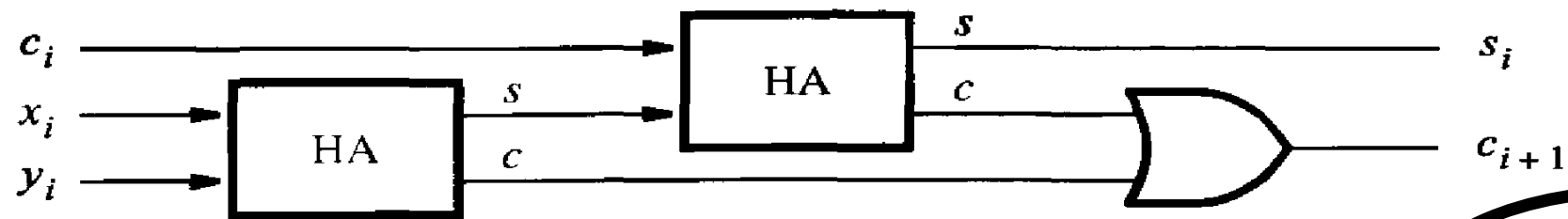
Logic Circuit



Using NAND Gates only

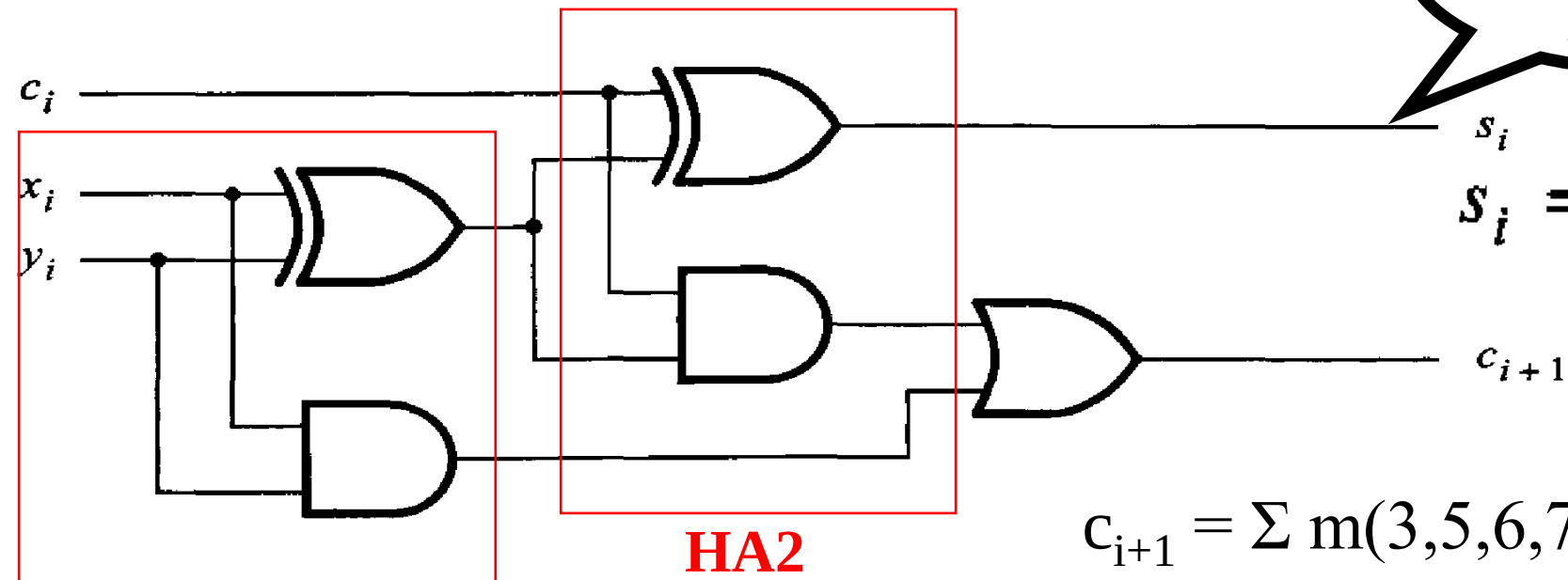


Unsigned Arithmetic - Decomposed Full Adder



(a) Block diagram

2 Half Adder
= 1 Full
Adder



HA1

HA2

(b) Detailed diagram

$$s_i = x_i \oplus y_i \oplus c_i$$

$$c_{i+1} = \Sigma m(3,5,6,7)$$

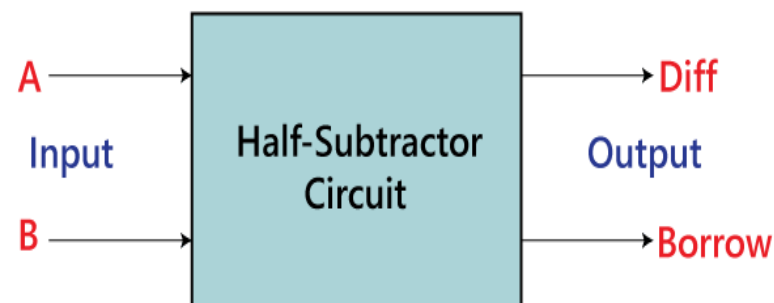
$$= x_i' y_i c_i + x_i y_i' c_i + x_i y_i c_i' + x_i y_i c_i$$

$$= x_i' y_i c_i + x_i y_i' c_i + x_i y_i$$

Unsigned Arithmetic - Half Subtractor

- The half subtractor is a building block for subtracting two binary numbers.
- It has two inputs and two outputs. This circuit is used to subtract two single bit binary numbers A and B. The '**diff**' and '**borrow**' are two output states of the half subtractor.

Block diagram



Truth Table

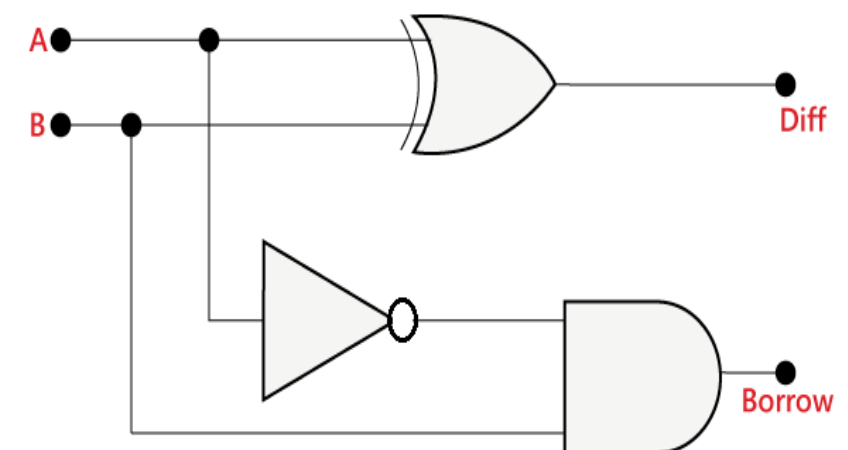
Inputs		Outputs	
A	B	Diff	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

The SOP form of the Diff and Borrow is as follows:

$$\text{Diff} = A'B + AB'$$

$$\text{Borrow} = A'B$$

Half-Subtractor logical circuit



Unsigned Arithmetic - Full Subtractor

- The full subtractor is a combinational circuit which is used to perform subtraction of three input bits: the minuend X , subtrahend Y and borrow in B_{in} .
- The full subtractor generates two output bits: the difference D and borrow out B_{out} .

Truth Table

Inputs			Outputs	
X	Y	B_{in}	D	B_{out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

$$D = X \oplus Y \oplus B_{in}$$

$$B_{out} = \bar{X}B_{in} + \bar{X}Y + YB_{in}$$

Full Subtractor

Full subtractor-

The full subtractor is a combinational circuit which is used to perform subtraction of three input bits.

Truth Table:

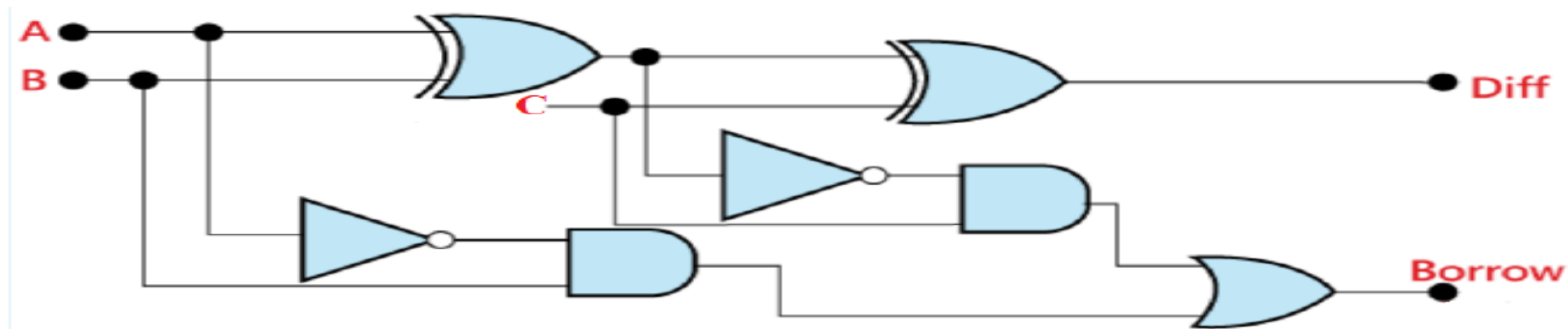
A	B	C	Diff	Borrow
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Logic expression:

$$\text{Diff} = A \oplus B \oplus C$$

$$\text{Borrow} = A'B + A'C + BC$$

Logic Diagram:

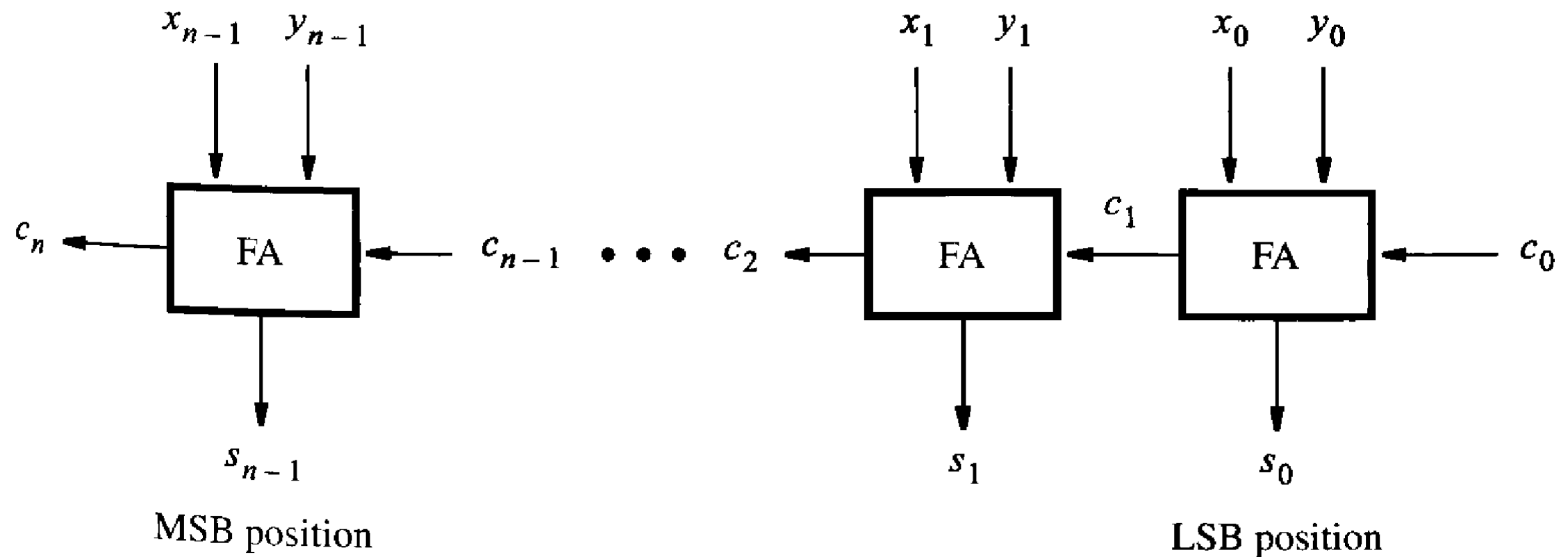


Unsigned Arithmetic - Ripple Carry Adder

- To perform addition by hand, we start from the least-significant digit and add pairs of digits, progressing to the most-significant digit.
- If a carry is produced in position i , then this carry is added to the operands in position $i + 1$.
- The same arrangement can be used in a logic circuit that performs addition.

Unsigned Arithmetic - Ripple Carry Adder

	Generated carries	1 1 1 0	
$X = x_4x_3x_2x_1x_0$		0 1 1 1 1	$(15)_{10}$
$+ Y = y_4y_3y_2y_1y_0$		0 1 0 1 0	$(10)_{10}$
$S = s_4s_3s_2s_1s_0$		1 1 0 0 1	$(25)_{10}$



--

Unsigned Arithmetic - Ripple Carry Adder

- When the operands X and Y are applied as inputs to the adder, it takes some time before the output sum, S , is valid.
- Each full-adder introduces a certain delay before its S_i and C_{i+1} outputs are valid. Let this delay be denoted as Δt .
- Thus the carry-out from the first stage, C_1 , arrives at the second stage Δt after the application of the x_0 and y_0 inputs. The carry-out from the second stage, C_2 , arrives at the third stage with a $2\Delta t$ delay, and so on.
- The signal C_{n-1} is valid after a delay of $(n - 1)\Delta t$, which means that the complete sum is available after a delay of $n\Delta t$.

Unsigned Arithmetic - Design Example

- Design a multiplier that multiplies an eight bit unsigned number by 3.

$$P = 3A \quad \text{where } A \text{ is 8-bit unsigned number.}$$

$$A = a_7a_6\dots a_0$$

$$P = p_7p_6\dots p_0$$

$$3A = A + A + A$$

$$A = a_7a_6\dots a_0 \quad 8\text{-bits}$$

$$+ A = a_7a_6\dots a_0 \quad 8\text{-bits}$$

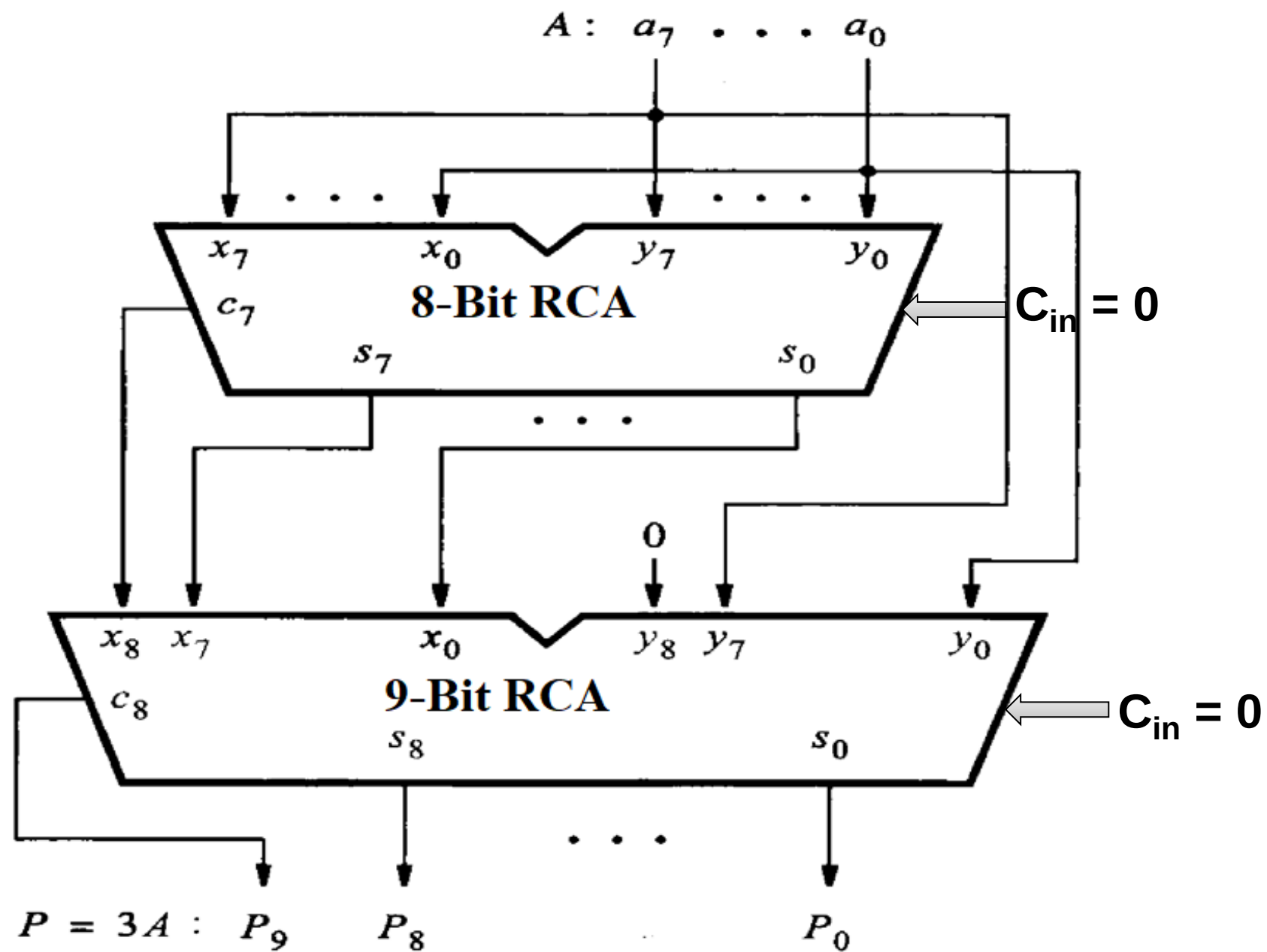
$$2A = c_8 s_7s_6\dots s_0 \quad 9\text{-bits}$$

$$A = 0 a_7a_6\dots a_0 \quad 9\text{-bits}$$

$$P = p_9p_8p_7\dots p_0 \quad 10\text{-bits}$$

Unsigned Arithmetic - Design Example

Normal approach



Unsigned Arithmetic - Design Example

- Design a multiplier that multiplies an eight bit unsigned number by 3.

$P = 3A$ where A is 8-bit unsigned number.

$$A = a_7a_6 \dots a_0$$

$$P = p_7p_6 \dots p_0$$

$$3A = 2A + A$$

$$A = a_7a_6 \dots a_0 \quad 8\text{-bits}$$

$$2A = a_7a_6 \dots a_0 0 \quad 9\text{-bits}$$

$$+ A = 0a_7a_6 \dots a_0 \quad 9\text{-bits}$$

$$P = p_9p_8p_7 \dots p_0 \quad 10\text{-bits}$$

$$2 = 10$$

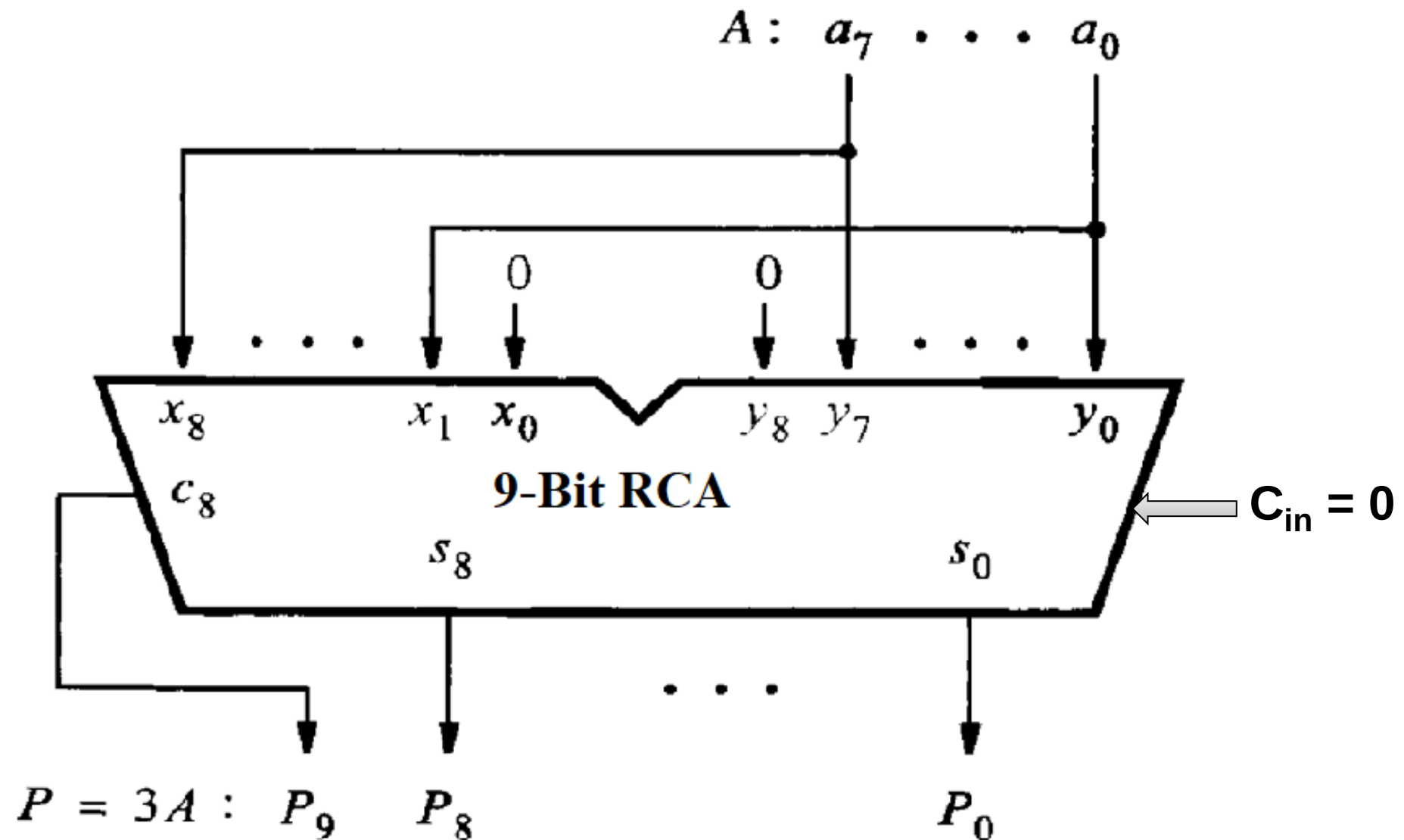
$$4 = 100$$

$$8 = 1000$$

In Binary number system
left shift by 1-bit is equal
to multiplied by 2

Unsigned Arithmetic - Design Example

Efficient Approach



Digital Electronics & Systems

19ECE204

Lecture 31-35

Arithmetic Circuits

Ms. Bhavana V
Assistant Professor(Sr.Gr.)
Dept. of ECE
ASE, Bengaluru

Fast Adders

- The performance of a large digital system is dependent on the speed of circuits that form its various functional units.
- Obviously, better performance can be achieved using faster circuits.
- This can be accomplished by using superior (usually newer) technology in which the delays in basic gates are reduced.
- But it can also be accomplished by changing the overall structure of a functional unit, which may lead to even more impressive improvement.
- To reduce the delay caused by the effect of carry propagation through the ripple-carry adder, we can attempt to evaluate quickly for each stage whether the carry-in from the previous stage will have a value 0 or 1.
- If a correct evaluation can be made in a relatively short time, then the performance of the complete adder will be improved.

Fast Adders - Carry Look Ahead Adder

- Based on Full adder circuit the carry-out function for stage i can be realized as-

$$C_{i+1} = x_i y_i + x_i C_i + y_i C_i$$

If we factor this expression as

$$C_{i+1} = x_i y_i + (x_i + y_i) C_i$$

then it can be written as

$$C_{i+1} = g_i + p_i C_i$$

where

$$g_i = x_i y_i \quad \text{Generate function}$$

$$p_i = x_i + y_i \quad \text{Propagate function}$$

—

Fast Adders - Carry Look Ahead Adder

$$C_{i+1} = g_i + p_i C_i$$

Substituting $i = 0, 1, 2, 3, \dots$

$$C_1 = g_0 + p_0 C_0$$

$$C_2 = g_1 + p_1 C_1 = g_1 + p_1(g_0 + p_0 C_0) = g_1 + p_1 g_0 + p_1 p_0 C_0$$

$$C_3 = g_2 + p_2 C_2 = g_2 + p_2(g_1 + p_1 g_0 + p_1 p_0 C_0)$$

$$= g_2 + p_2 g_1 + p_2 p_1 g_0 + p_2 p_1 p_0 C_0$$

$$C_4 = g_3 + p_3 C_3 = g_3 + p_3(g_2 + p_2 g_1 + p_2 p_1 g_0 + p_2 p_1 p_0 C_0)$$

$$= g_3 + p_3 g_2 + p_3 p_2 g_1 + p_3 p_2 p_1 g_0 + p_3 p_2 p_1 p_0 C_0$$

Where $g_0 = x_0 y_0$ and $p_0 = x_0 + y_0$

$g_1 = x_1 y_1$ and $p_1 = x_1 + y_1$

$g_2 = x_2 y_2$ and $p_2 = x_2 + y_2$

Fast Adders - Carry Look Ahead Adder

- Expanding in term of $i-1$ gives

$$\begin{aligned}c_{i+1} &= g_i + p_i(g_{i-1} + p_{i-1}c_{i-1}) \\ &= g_i + p_i g_{i-1} + p_i p_{i-1} c_{i-1}\end{aligned}$$

The same expansion for other stages, ending with stage 0, gives

$$c_{i+1} = g_i + p_i g_{i-1} + p_i p_{i-1} g_{i-2} + \cdots + p_i p_{i-1} \cdots p_2 p_1 g_0 + p_i p_{i-1} \cdots p_1 p_0 c_0$$

- This expression represents a two-level AND-OR circuit in which C_{i+1} is evaluated very quickly. An adder based on this expression is called a carry-lookahead adder.

Fast Adders - Carry Look Ahead Adder

Critical Path : From x_0 - y_0 to c_2

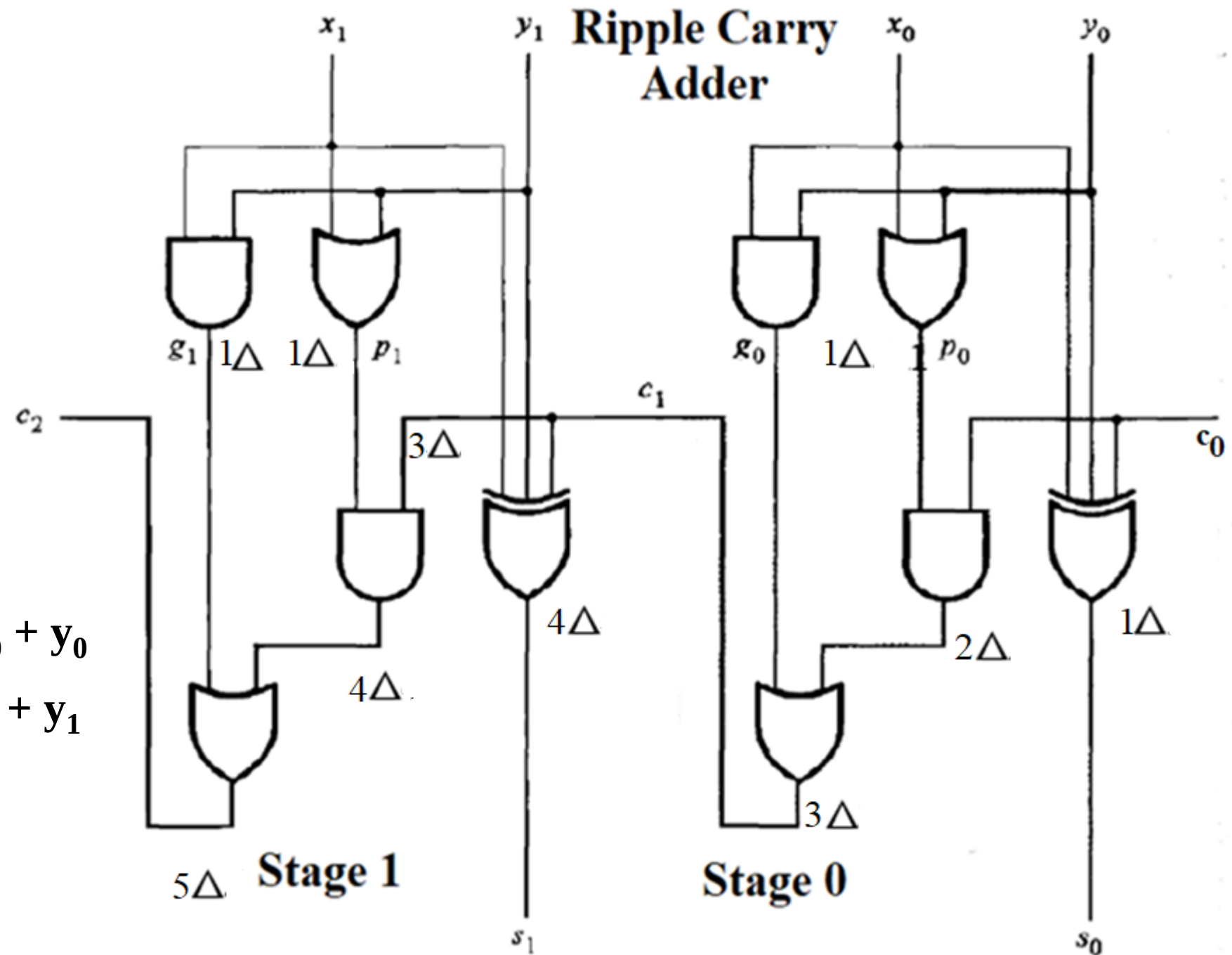
For any n bit adder gate delay for carry = $2n+1$

$$c_1 = g_0 + p_0 c_0$$

$$c_2 = g_1 + p_1 c_1$$

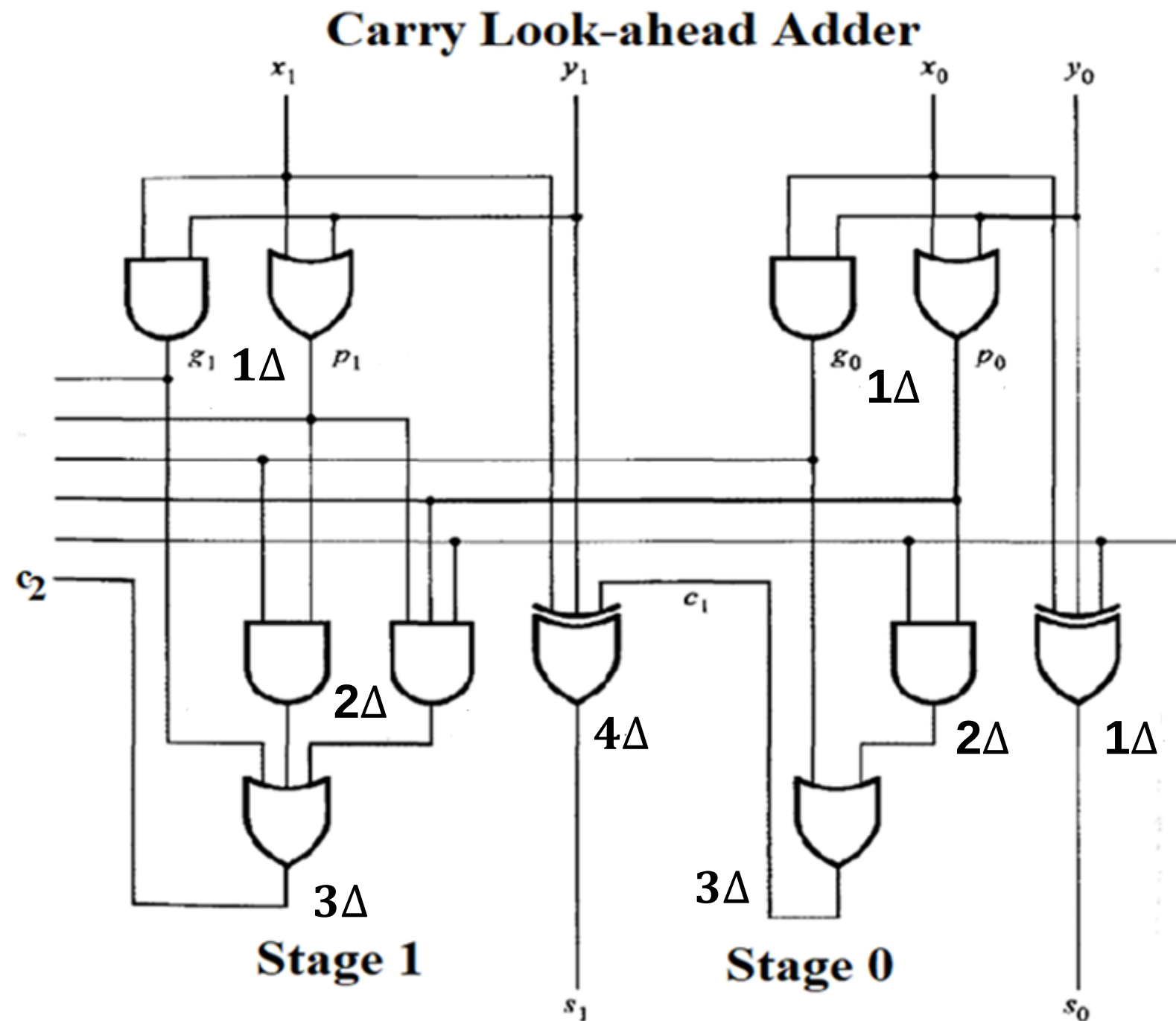
Where $g_0 = x_0 y_0$ and $p_0 = x_0 + y_0$

$g_1 = x_1 y_1$ and $p_1 = x_1 + y_1$



Ripple Carry adder

Fast Adders - Carry Look Ahead Adder



Critical Path: From x_0 - y_0 to c_2
Gate Delay for c_2 - 3

For n-bit adder, gate delay for carry = 3

For n-bit adder, gate delay for sum = 4

$$c_1 = g_0 + p_0 c_0$$

$$c_2 = g_1 + p_1 g_0 + p_1 p_0 c_0$$

Where $g_0 = x_0 y_0$ and $p_0 = x_0 + y_0$

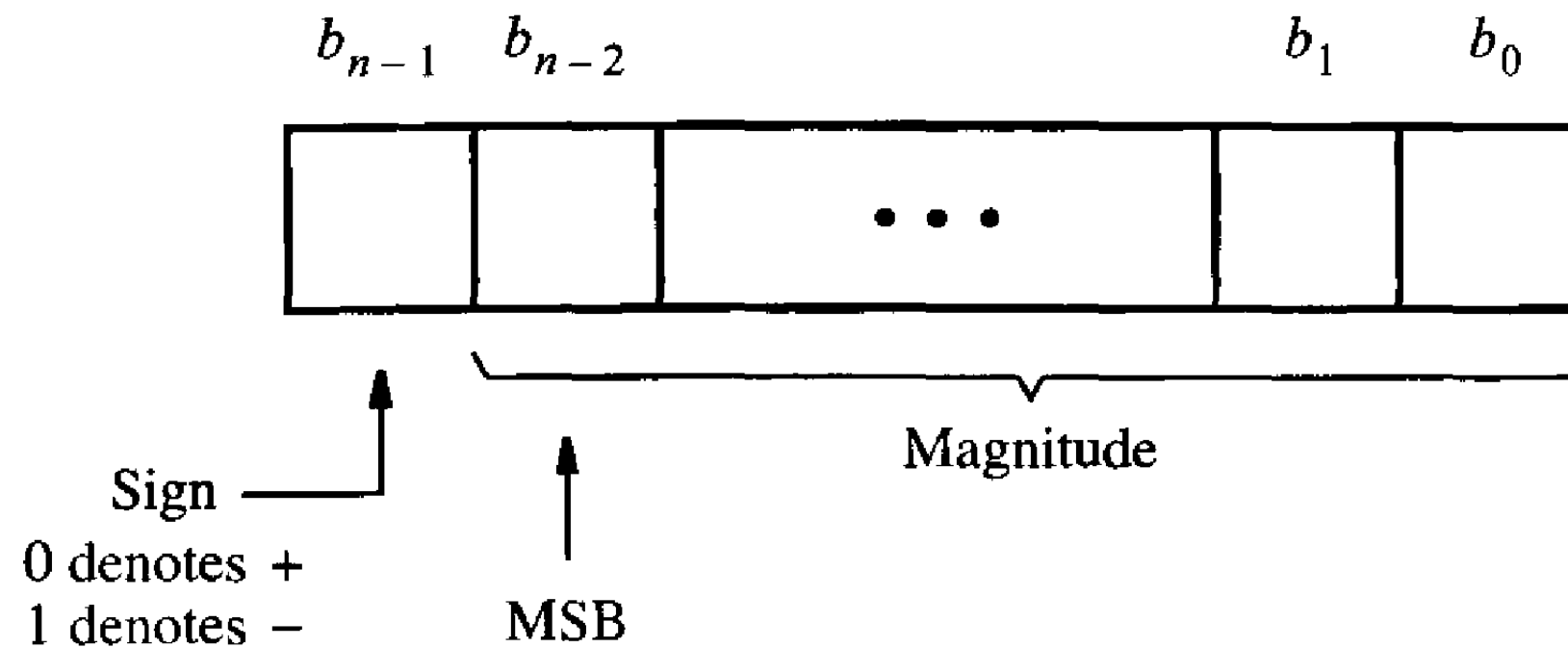
$g_1 = x_1 y_1$ and $p_1 = x_1 + y_1$

Carry look Ahead Adder

Signed Number Representation

- Positive numbers are represented using the positional number representation as explained in the previous section.
- Negative numbers can be represented in three different ways:
 - Sign-and-magnitude
 - 1 's complement
 - 2's complement.

Signed Number Representation- Sign Magnitude Representation



Example:

+5 = 0101

-5 = 1101

Drawbacks

- Range
- Circuit complexity

Signed Number Representation - 1's Complement

- In a complementary number system, the negative numbers are defined according to a subtraction operation involving positive numbers.
- In the 1's complement scheme. an n-bit negative number, K, is obtained by subtracting its equivalent positive number, P, from $2^n - 1$
 - $K = (2^n - 1) - P$

Example:

$$-5 = (2^4 - 1) - 0101 = 1111 - 0101 = 1010$$

$$-3 = (2^4 - 1) - 0011 = 1111 - 0011 = 1100$$

Simply it is complement of its positive number

- To get 1's complement of any negative binary number, simply invert each bit of the corresponding positive number.

$$(+11)_{10} \Rightarrow \text{sign-magnitude} = (0 \ 1011)_2 \quad 1's \text{ complement} = (0 \ 1011)_2$$

$$(-11)_{10} \Rightarrow \text{sign-magnitude} = (1 \ 1011)_2 \quad 1's \text{ complement} = (1 \ 0100)_2$$

Signed Number Representation - 2's Complement

- In the 2's complement scheme, a negative number, K, is obtained by subtracting its equivalent positive number, P, from 2^n
 - $K = 2^n - P$

Example:

$$-5 = 2^4 - 0101 = 10000 - 0101 = 1011$$

$$-3 = 2^4 - 0011 = 10000 - 0011 = 1101$$

Simply it is 1's complement plus one

- To get 2's complement of any negative binary number, add 1 to the 1's complement.

$$(+11)_{10} \Rightarrow \text{sign-magnitude} = (0 \ 1011)_2 \quad 2's \text{ complement} = (0 \ 1011)_2$$

$$(-11)_{10} \Rightarrow \text{sign-magnitude} = (1 \ 1011)_2 \quad 1's \text{ complement} = (1 \ 0100)_2$$

$$2's \text{ complement} = (1 \ 0101)_2$$

Signed Number Representation

$b_3b_2b_1b_0$	Sign and magnitude	1's complement	2's complement
0111	+7	+7	+7
0110	+6	+6	+6
0101	+5	+5	+5
0100	+4	+4	+4
0011	+3	+3	+3
0010	+2	+2	+2
0001	+1	+1	+1
0000	+0	+0	+0
1000	-0	-7	-8
1001	-1	-6	-7
1010	-2	-5	-6
1011	-3	-4	-5
1100	-4	-3	-4
1101	-5	-2	-3
1110	-6	-1	-2
1111	-7	-0	-1

Signed Number Representation

- Represent (+23) in 8-bit 1's complement format.

$$(+23) \rightarrow S-M = 0\ 0010111$$

$$1's\ comp = 0\ 0010111$$

- Represent (-23) in 8-bit 1's complement format.

$$(-23) \rightarrow S-M = 1\ 0010111$$

$$1's\ comp = 1\ 1101000$$

Signed Arithmetic - Sign Magnitude

- If both operands have the same sign, then the addition of sign-and-magnitude numbers is simple. The magnitudes are added, and the resulting sum is given the sign of the operands.
- However, if the operands have opposite signs, the task becomes more complicated. Then it is necessary to subtract the smaller number from the larger one.
- This means that logic circuits that compare and subtract numbers are also needed.
- For this reason, the sign-and-magnitude representation is not used in computers.

Signed Arithmetic - 1's Complement Addition

- An obvious advantage of the 1's complement representation is that a negative number is generated simply by complementing all bits of the corresponding positive number..

$$(+2) \Rightarrow S-M = 1's \text{ comp} = 2's \text{ comp} = 0 \ 010$$

$$(-2) \Rightarrow S-M = 1 \ 010,$$

$$1's \text{ comp} = 1 \ 101,$$

$$2's \text{ comp} = 1 \ 110$$

$$(+5) \Rightarrow S-M = 1's \text{ comp} = 2's \text{ comp} = 0 \ 101$$

$$(-5) \Rightarrow S-M = 1 \ 101,$$

$$1's \text{ comp} = 1 \ 010,$$

$$2's \text{ comp} = 1 \ 011$$

Signed Arithmetic - 1's Complement Addition

$$\begin{array}{r}
 (+5) \quad 0101 \\
 + (+2) \quad 0010 \\
 \hline
 (+7) \quad 0111
 \end{array}$$

$$\begin{array}{r}
 (-5) \quad 1010 \\
 + (+2) \quad 0010 \\
 \hline
 (-3) \quad 1100
 \end{array}$$

$$\begin{array}{r}
 (+5) \quad 0101 \\
 + (-2) \quad 1101 \\
 \hline
 (+3) \quad 10010 \\
 \text{Carry } 1 \rightarrow \\
 \hline
 0011
 \end{array}$$

$$\begin{array}{r}
 (-5) \quad 1010 \\
 + (-2) \quad 1101 \\
 \hline
 (-7) \quad 10111 \\
 \text{Carry } 1 \rightarrow \\
 \hline
 1000
 \end{array}$$

Adding the extra carry is an overhead

Signed Arithmetic - 2's Complement Addition

$$\begin{array}{r}
 (+5) \quad 0101 \\
 + (+2) \quad 0010 \\
 \hline
 (+7) \quad 0111
 \end{array}$$

$$\begin{array}{r}
 (-5) \quad 1011 \\
 + (+2) \quad 0010 \\
 \hline
 (-3) \quad 1101
 \end{array}$$

$$\begin{array}{r}
 (+5) \quad 0101 \\
 + (-2) \quad 1110 \\
 \hline
 (+3) \quad 10011
 \end{array}$$

↑
ignore

$$\begin{array}{r}
 (-5) \quad 1011 \\
 + (-2) \quad 1110 \\
 \hline
 (-7) \quad 11001
 \end{array}$$

↑
ignore

No extra addition of carry is required, hence this is more suitable for addition operation

Signed Arithmetic - 2's Complement Subtraction



$\begin{array}{r} (+5) \\ - (+2) \\ \hline (+3) \end{array}$	$\begin{array}{r} (+5) \\ + (-2) \\ \hline (+3) \end{array}$	$\Rightarrow$	$\begin{array}{r} 0101 \\ + 1110 \\ \hline 10011 \end{array}$
			$\uparrow$ ignore
$\begin{array}{r} (-5) \\ - (+2) \\ \hline (-7) \end{array}$	$\begin{array}{r} (-5) \\ + (-2) \\ \hline (-7) \end{array}$	$\Rightarrow$	$\begin{array}{r} 1011 \\ + 1110 \\ \hline 11001 \end{array}$
			$\uparrow$ ignore

$\begin{array}{r} (+5) \\ - (-2) \\ \hline (+7) \end{array}$	$\begin{array}{r} (+5) \\ + (+2) \\ \hline (+7) \end{array}$	$\Rightarrow$	$\begin{array}{r} 0101 \\ + 0010 \\ \hline 0111 \end{array}$
$\begin{array}{r} (-5) \\ - (-2) \\ \hline (-3) \end{array}$	$\begin{array}{r} (-5) \\ + (+2) \\ \hline (-3) \end{array}$	$\Rightarrow$	$\begin{array}{r} 1011 \\ + 0010 \\ \hline 1101 \end{array}$

Hence, Subtraction can be done by adder itself

Signed Arithmetic - 2's Complement

- Perform the following

1) $(+10)_{10} + (-12)_{10}$

2) $(+10)_{10} - (+15)_{10}$

1) $(+10)_{10} + (-12)_{10}$

+10	2's comp form	0 1010
-12	2's comp form	<u>1 0100</u>
-2	2's comp form	<u>1 1110</u>

2) $(+10) - (+15) = (+10) + (-15)$

+10	2's comp form	0 1010
+(-15)	2's comp form	<u>1 0001</u>
-5	2's comp form	<u>1 1011</u>

Adder and Subtractor Unit for Unsigned Numbers

- Assume two unsigned 4 bit numbers A and B where

$$A = 1011$$

$$B = 0110$$

- To perform $A + B$ we use 4-bit Ripple carry adder where the carry in to the full adder, which will add LSB, is 0.
- For subtraction $A - B$ in form of addition A is added with 2's complement of B i.e. $A + (-B)$

$$\begin{array}{r} A = 1011 \\ B = 0110 \\ + \quad \quad 0 \leftarrow c_0 \\ \hline 10001 \end{array}$$

$$\begin{array}{r} A = 1011 \\ (-B) = 1001 \quad \text{1's comp of} \\ \quad \quad \quad 1 \quad \text{1's comp} + 1 = \text{2's comp} \\ \hline 10101 \\ \text{Discard} \end{array}$$

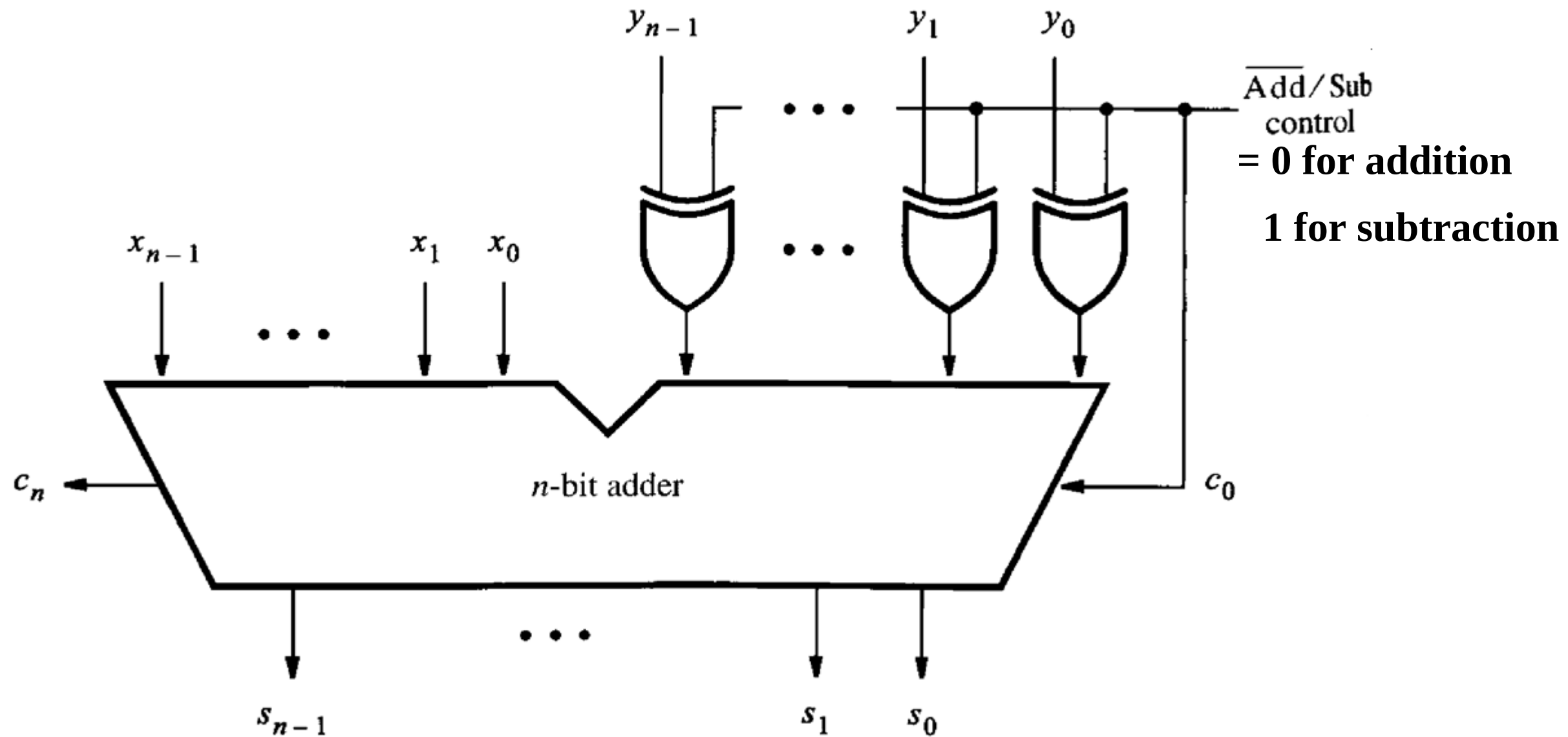
Adder and Subtractor Unit for Unsigned Numbers

- To perform both the operation $A + B$ and $A - B$ with the same circuit, for addition we need B as normal b but for subtraction we need 1's complement of B .
- This can be achieved if we will apply each bit of B to 2-input XOR gate where the second input for XOR gate will be a control signal which will be 0 for addition and 1 for subtraction.

$$\begin{array}{rcccc}
 B = & 0 & 1 & 1 & 0 \\
 & \oplus & \oplus & \oplus & \oplus \\
 & 0 & 0 & 0 & 0 \\
 \hline
 & 0 & 1 & 1 & 0 \leftarrow B
 \end{array}$$

$$\begin{array}{rcccc}
 B = & 0 & 1 & 1 & 0 \\
 & \oplus & \oplus & \oplus & \oplus \\
 & 1 & 1 & 1 & 1 \\
 \hline
 & 1 & 0 & 0 & 1 \leftarrow 1's \text{ comp of } B
 \end{array}$$

Adder and Subtractor Unit for Unsigned Numbers



Adder and Subtractor Unit

Design minimum cost circuits to perform following operation on two 3-bit unsigned numbers A and B-

- a) $A + 2B$
- b) $A - 2B$ using 2's complement
- c) $A \pm 2B$

Solution-

$$A = a_2a_1a_0 \quad \text{-- 3 bits}$$

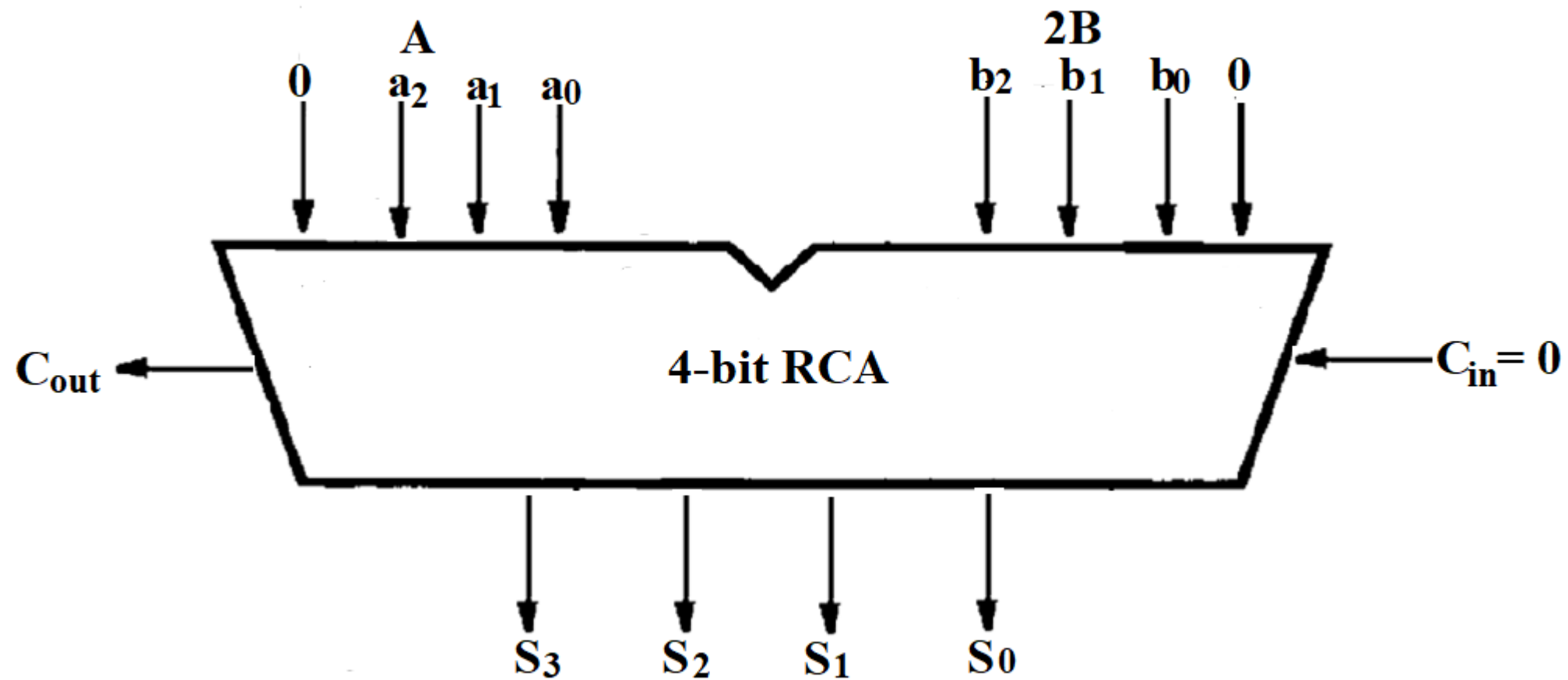
$$B = b_2b_1b_0 \quad \text{-- 3 bits}$$

$$2B = b_2b_1b_00 \quad \text{-- 4 bits}$$

$$A = 0a_2a_1a_0 \quad \text{-- 4 bits}$$

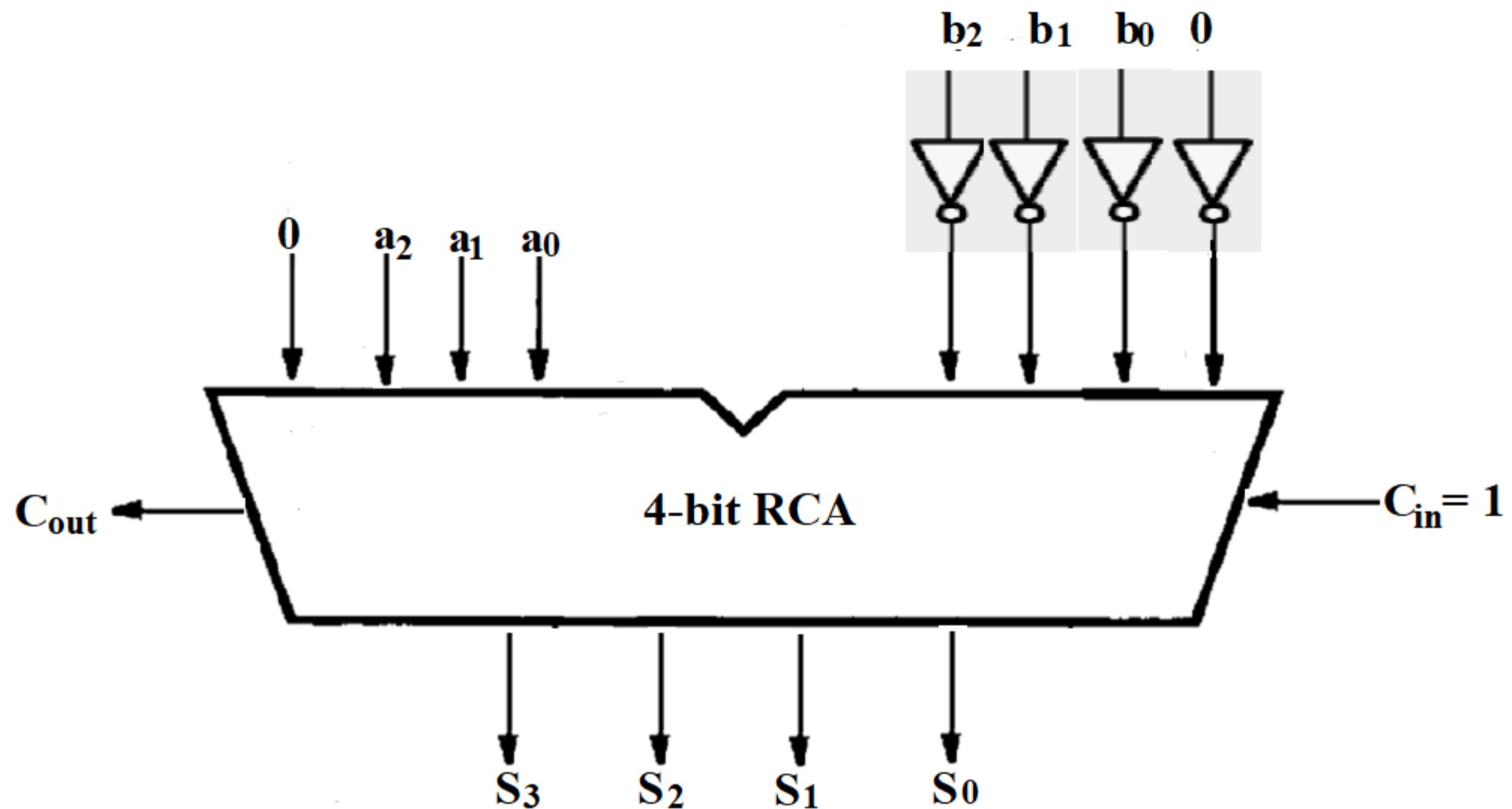
Adder and Subtractor Unit

a) $A + 2B$



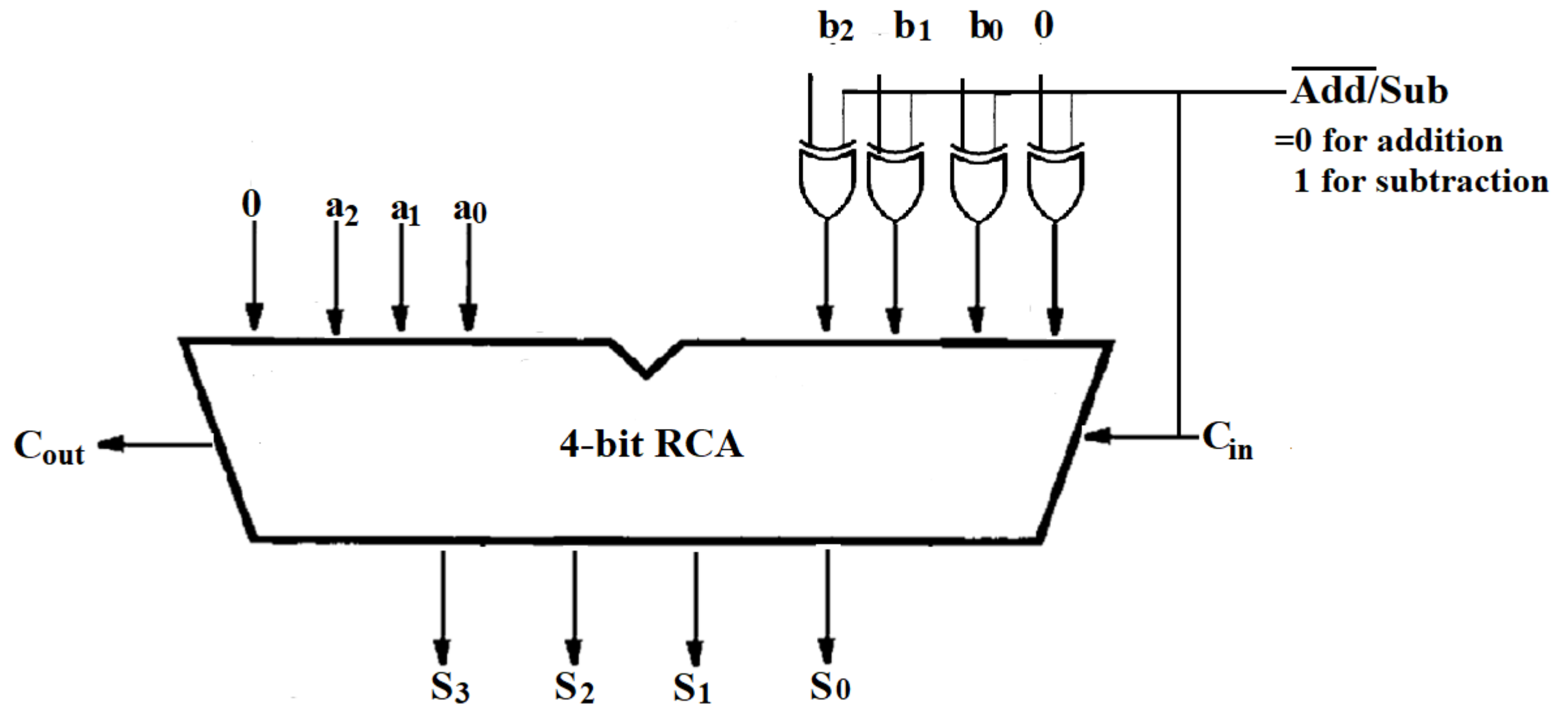
Adder and Subtractor Unit

b) $A - 2B$ using 2's complement



Adder and Subtractor Unit

b) $A \pm 2B$



Arithmetic Overflow

- The result of addition or subtraction is supposed to fit within the significant bits used to represent the numbers.
- If n bits are used to represent signed numbers, then the result must be in the range -2^{n-1} to $2^{n-1} - 1$.
- If the result does not fit in this range, then we say that arithmetic overflow has occurred.
- To ensure the correct operation of an arithmetic circuit, it is important to be able to detect the occurrence of overflow.

Arithmetic Overflow

$$\begin{array}{r}
 (+7) \quad 0111 \\
 + (+2) \quad 0010 \\
 \hline
 (+9) \quad 1001 \\
 c_4 = 0 \\
 c_3 = 1
 \end{array}$$

$$\begin{array}{r}
 (-7) \quad 1001 \\
 + (+2) \quad 0010 \\
 \hline
 (-5) \quad 1011 \\
 c_4 = 0 \\
 c_3 = 0
 \end{array}$$

$$\begin{array}{r}
 (+7) \quad 0111 \\
 + (-2) \quad 1110 \\
 \hline
 (+5) \quad 10101 \\
 c_4 = 1 \\
 c_3 = 1
 \end{array}$$

$$\begin{array}{r}
 (-7) \quad 1001 \\
 + (-2) \quad 1110 \\
 \hline
 (-9) \quad 10111 \\
 c_4 = 1 \\
 c_3 = 0
 \end{array}$$

$$\begin{aligned}
 \text{Overflow} &= c_3 \bar{c}_4 + \bar{c}_3 c_4 \\
 &= c_3 \oplus c_4
 \end{aligned}$$

$$\text{Overflow} = c_{n-1} \oplus c_n$$

c_{n-1} - carry-out from the magnitude MSB position
 c_n - carry-out from the sign-bit position

Arithmetic Overflow

- To solve the problem of arithmetic overflow we need to increase the size of input numbers by 1-bit.
- For example $(+7) + (+2)$

$$\begin{array}{r}
 (+7) \\
 + (+2) \\
 \hline
 (+9)
 \end{array}
 \quad
 \begin{array}{r}
 0111 \\
 + 0010 \\
 \hline
 1001 \\
 c_4 = 0 \\
 c_3 = 1
 \end{array}
 \quad
 \begin{aligned}
 \text{Overflow} &= c_3 \bar{c}_4 + \bar{c}_3 c_4 \\
 &= c_3 \oplus c_4 \\
 &= 0 \oplus 1 = 1
 \end{aligned}$$

So arithmetic overflow exists here. To solve this we will represent $(+7)$ and $(+2)$ with 5 bits.

$$\begin{array}{r}
 (+7) \quad 00111 \\
 + (+2) \quad +0010 \\
 \hline
 (+9) \quad 01001
 \end{array}$$

Now $c_3 = 0$ and $c_4 = 0$

$$\text{Overflow} = 0 \oplus 0 = 0$$

Digital Electronics & Systems

19ECE204

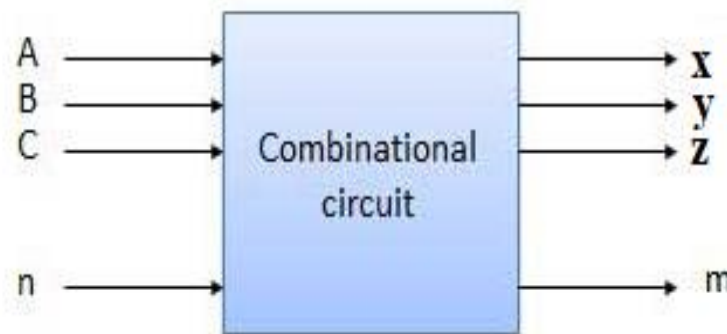
Lecture 36-40

Combinational Circuits

Ms. Bhavana V
Assistant Professor (Sr.Gr.)
Dept. of ECE
ASE, Bengaluru

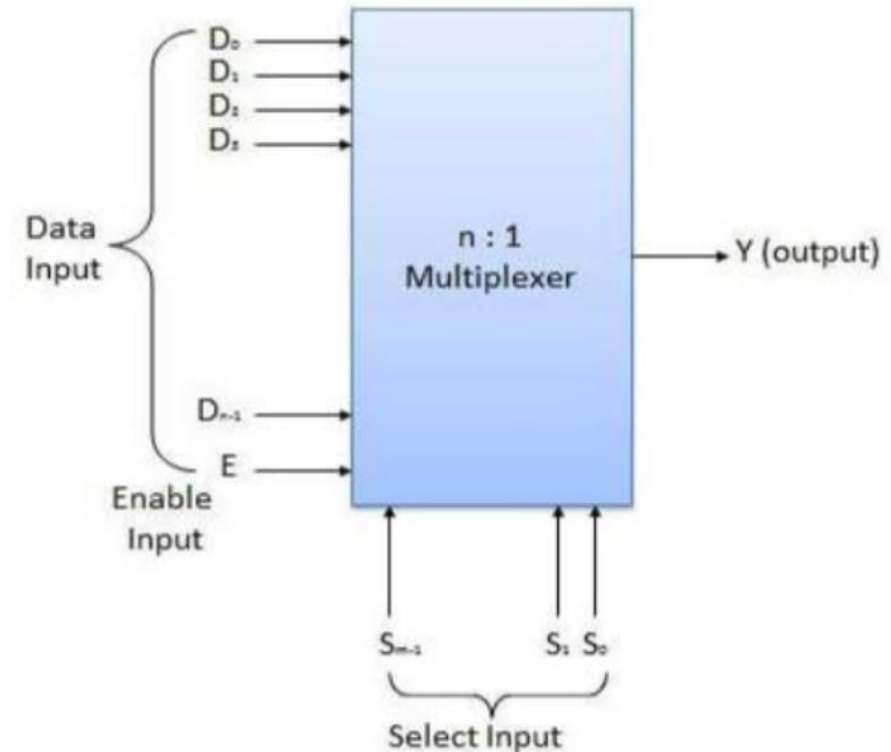
Combinational circuit is a circuit in which we combine the different gates in the circuit, for example encoder, decoder, multiplexer and demultiplexer. Some of the characteristics of combinational circuits are following –

- The output of combinational circuit at any instant of time, depends only on the levels present at input terminals.
- The combinational circuit do not use any memory. The previous state of input does not have any effect on the present state of the circuit.
- A combinational circuit can have an n number of inputs and m number of outputs.



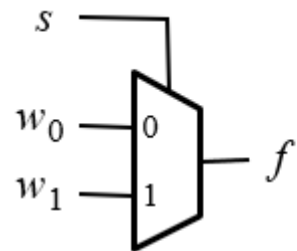
Multiplexers

- A multiplexer has
 - n Selection inputs
 - 2^n data inputs
 - 1 output
- A multiplexer routes (or connects) the selected data input to the output.
- The value of the selection inputs determines the data input that is selected.



Multiplexers

2-to-1 Multiplexer

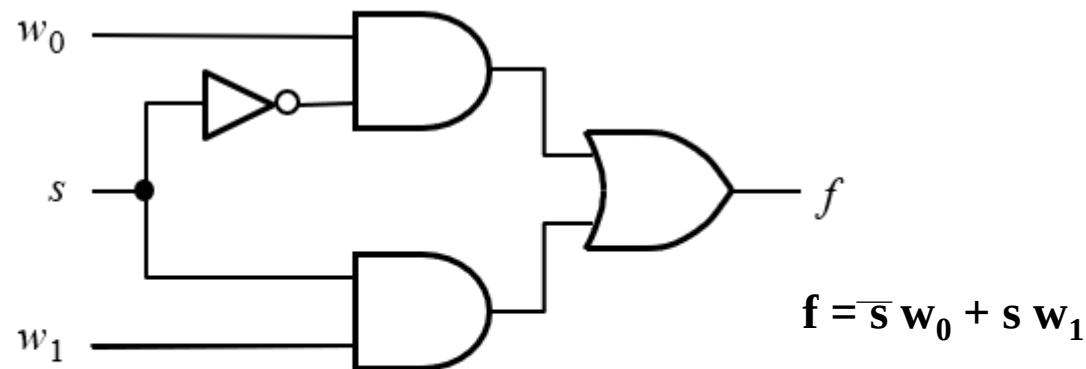


- 2 - Data inputs
- 1- selection input
- 1 output

s	f
0	w_0
1	w_1

(a) Graphical symbol

(b) Truth table-compact



(c) Sum-of-products circuit

Multiplexers

2-to-1 Multiplexer

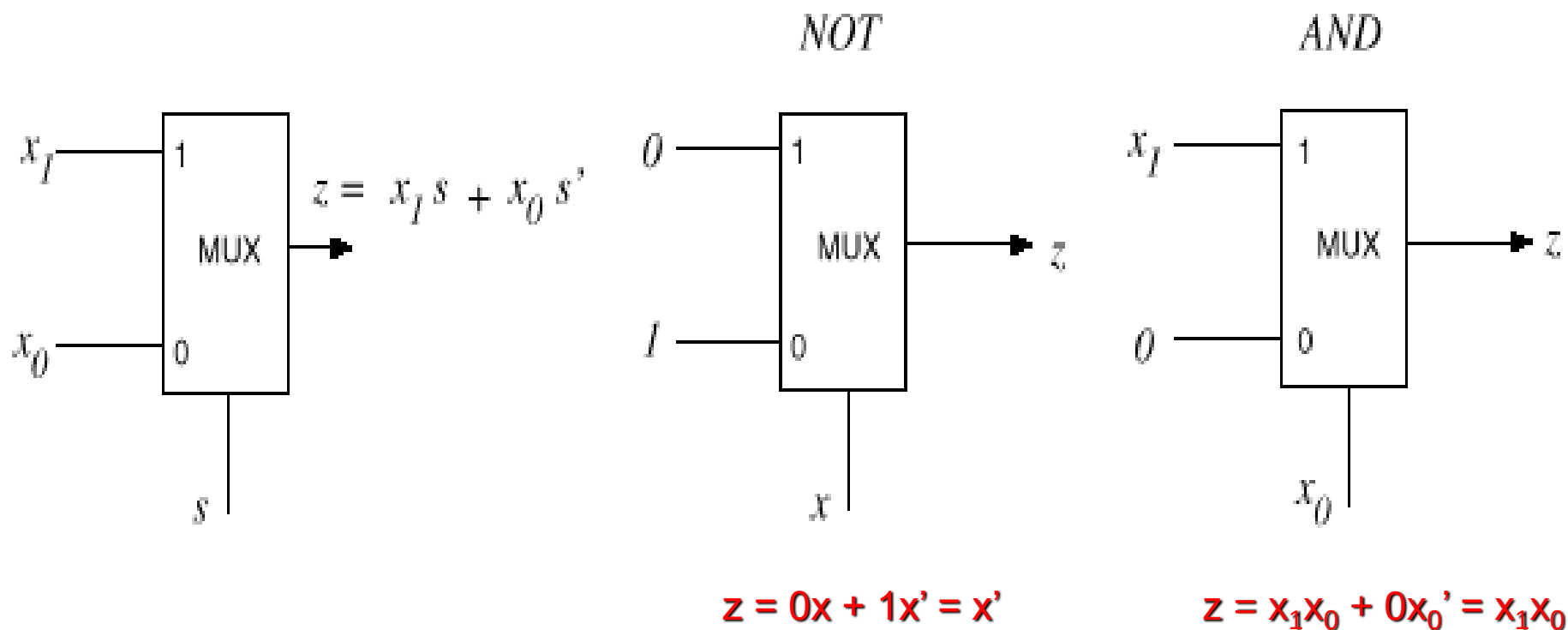
$s \ w_0 \ w_1$	$f(s, w_0, w_1)$
0 0 0	0
0 0 1	0
0 1 0	1
0 1 1	1
1 0 0	0
1 0 1	1
1 1 0	0
1 1 1	1

$$f = \bar{s} w_0 + s w_1$$

Multiplexers

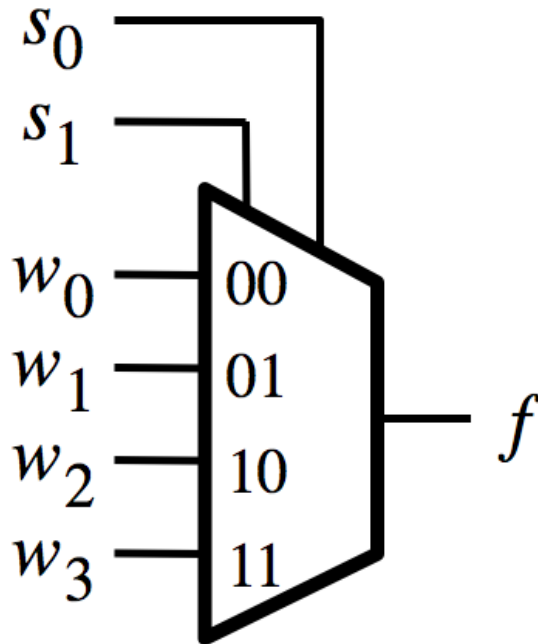
Multiplexer as Universal Gate

- We can construct AND and NOT gates using 2-to-1 MUXs. Thus, 2-to-1 MUX is a universal gate.
- Try OR Gate and NAND Gate



Multiplexers

4-to-1 Multiplexer



s_1	s_0	f
0	0	w_0
0	1	w_1
1	0	w_2
1	1	w_3

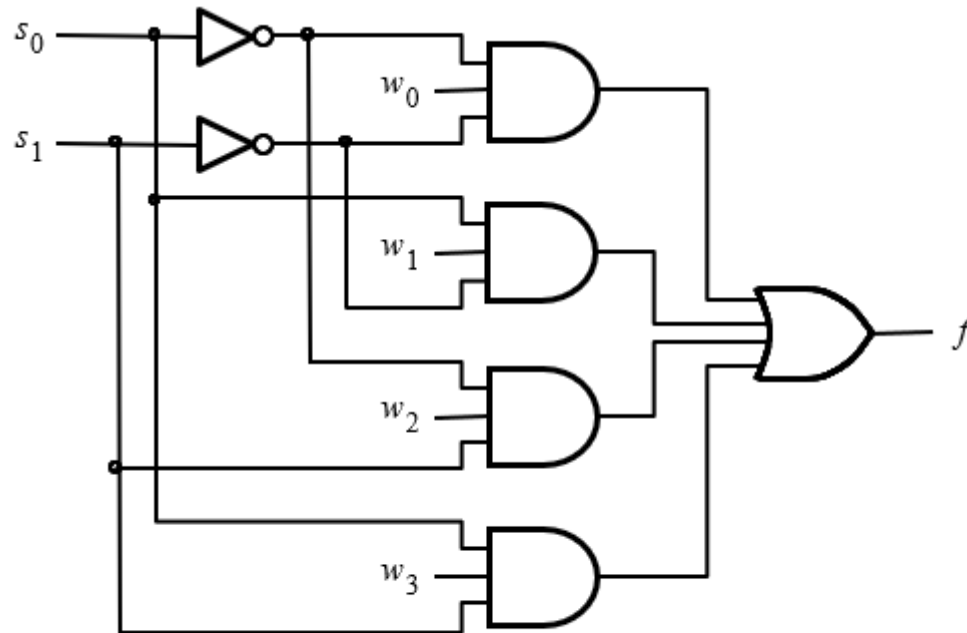
(a) Graphic symbol

(b) Truth table

$$f = \overline{s_1} \overline{s_0} w_0 + \overline{s_1} s_0 w_1 + s_1 \overline{s_0} w_2 + s_1 s_0 w_3$$

Multiplexers

4-to-1 Multiplexer

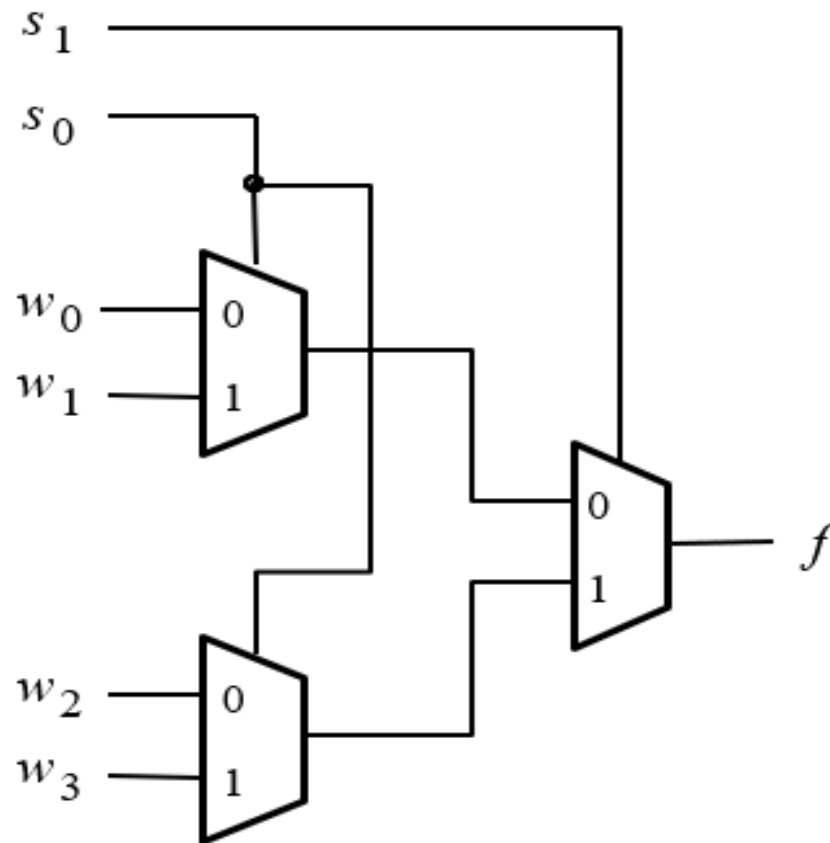


(c) Circuit

$$f = \overline{s_1} \overline{s_0} w_0 + \overline{s_1} s_0 w_1 + s_1 \overline{s_0} w_2 + s_1 s_0 w_3$$

Multiplexers

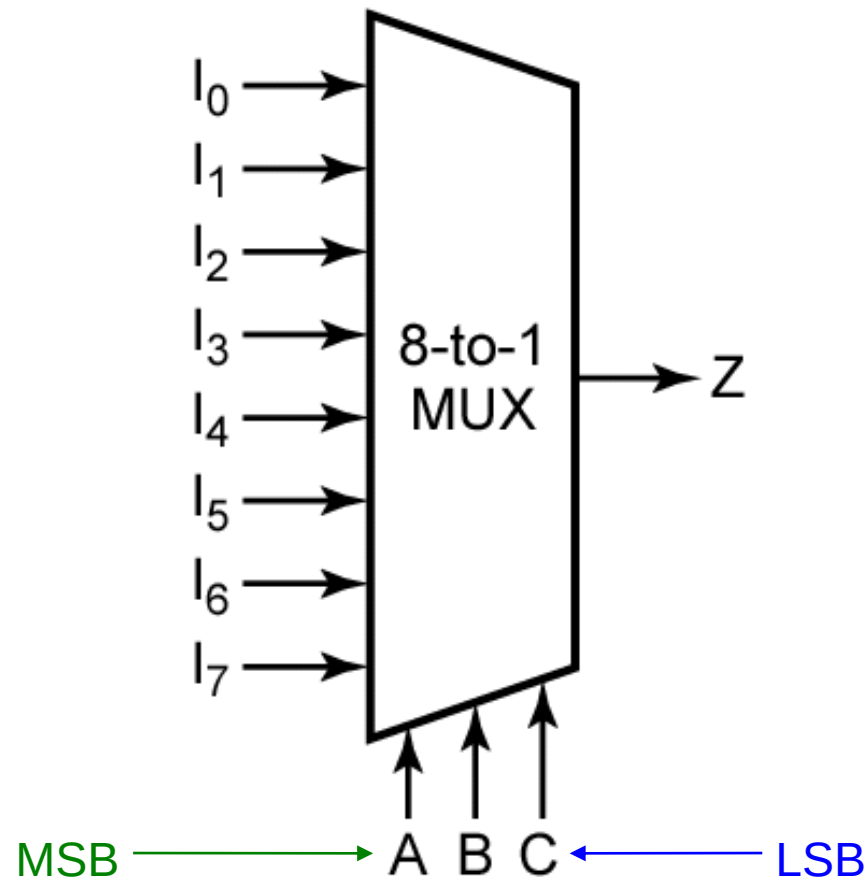
4-to-1 Multiplexer using 2-to-1 Multiplexers



s_1	s_0	f
0	0	w_0
0	1	w_1
1	0	w_2
1	1	w_3

Multiplexers

8 to 1 Multiplexers



A	B	C	F
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	I_4
1	0	1	I_5
1	1	0	I_6
1	1	1	I_7

$$Z = A'.B'.C'.I_0 + A'.B'.C.I_1 + A'.B.C'.I_2 + A'.B.C.I_3 + A.B'.C'.I_4 + A.B'.C.I_5 + A.B.C'.I_6 + A.B.C.I_7$$

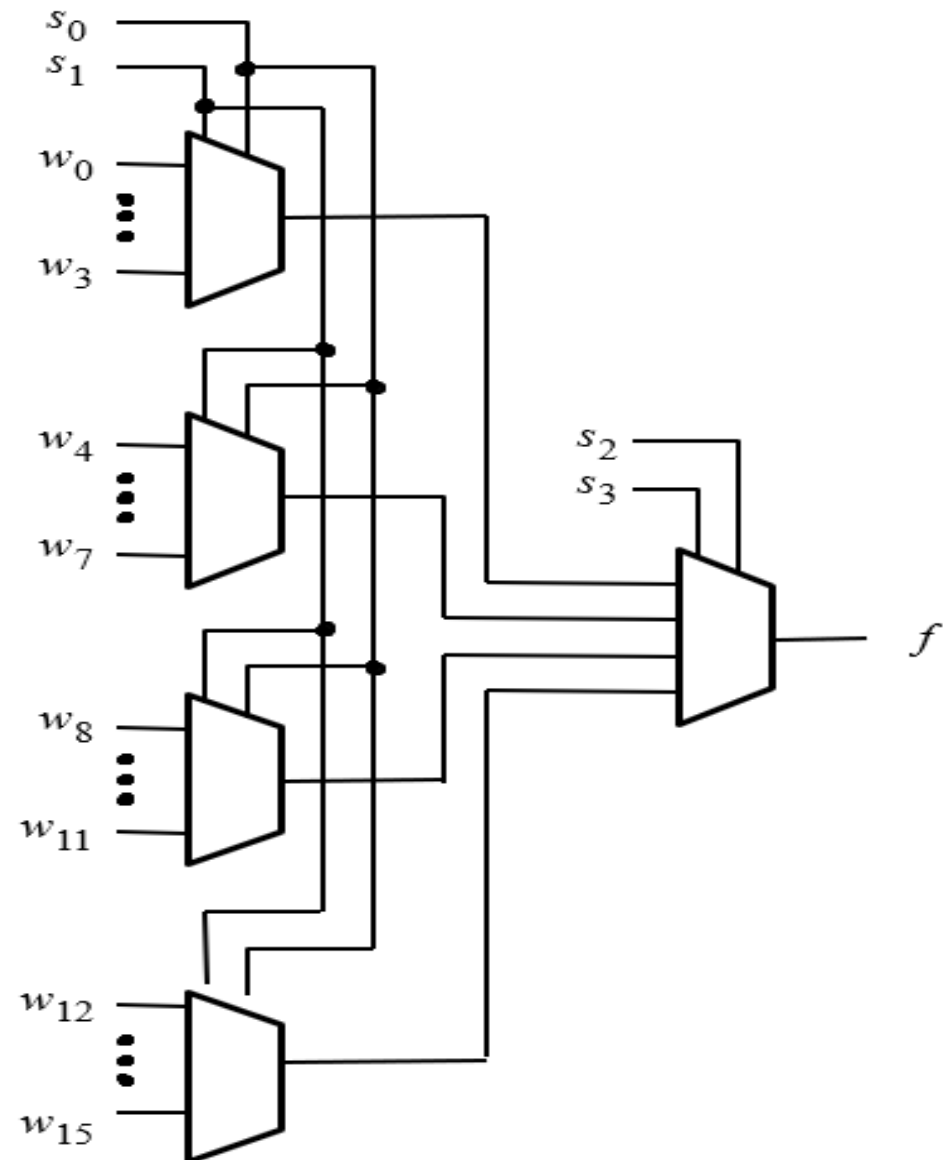
Multiplexers

Exercise: Design an 8-to-1 multiplexer using 4-to-1 and 2-to-1 multiplexers only.

Multiplexers

16-to-1 Multiplexer using 4-to-1 Multiplexers only

S3	S2	S1	S0	Output f
0	0	0	0	w0
0	0	0	1	w1
0	0	1	0	w2
0	0	1	1	w3
0	1	0	0	w4
0	1	0	1	w5
0	1	1	0	w6
0	1	1	1	w7
1	0	0	0	w8
1	0	0	1	w9
1	0	1	0	w10
1	0	1	1	w11
1	1	0	0	w12
1	1	0	1	w13
1	1	1	0	w14
1	1	1	1	w15



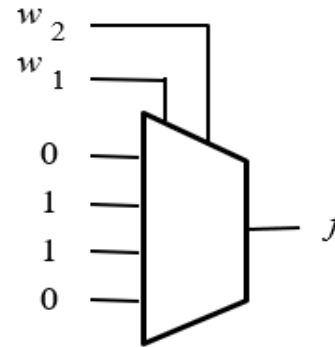
Synthesis of Logic Circuits Using Multiplexers

Function Synthesis using Multiplexer

Truth table Method

2-input XOR Gate

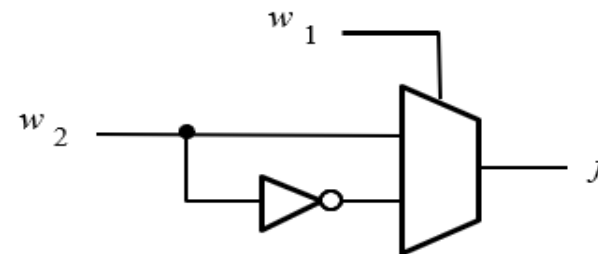
w_1	w_2	f
0	0	0
0	1	1
1	0	1
1	1	0



(a) Implementation using a 4-to-1 multiplexer

w_1	w_2	f		w_1	f
0	0	0	}	0	w_2
0	1	1		1	$\overline{w_2}$
1	0	1	}		
1	1	0			

(b) Modified truth table



(c) Circuit using 2:1 mux

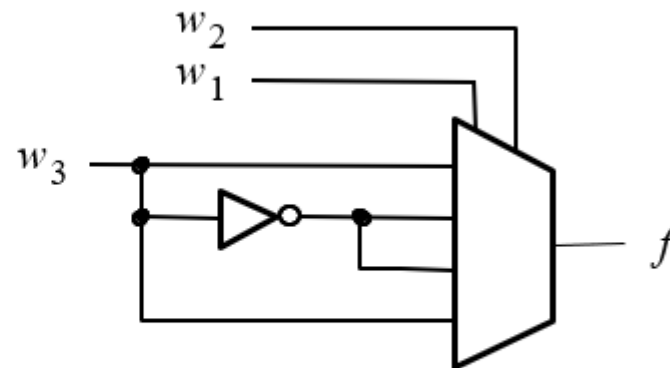
Function Synthesis using Multiplexer

Truth table Method

3-input XOR Gate using 4:1 MUX

w_1	w_2	w_3	f
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

(a) Truth table



(b) Circuit

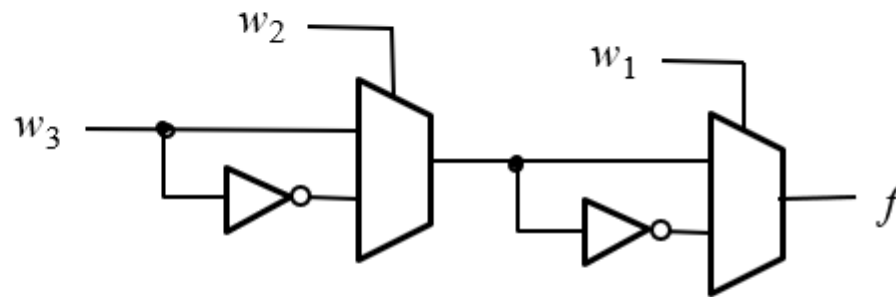
Function Synthesis using Multiplexer

Truth table Method

3-input XOR Gate using 2:1 MUX

w_1	w_2	w_3	f
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

(a) Truth table



(b) Circuit

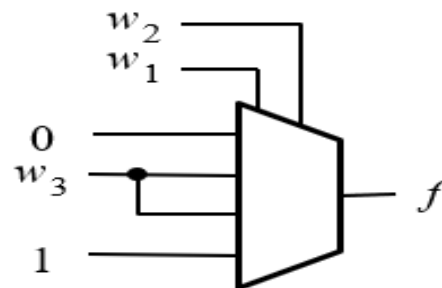
Function Synthesis using Multiplexer

Truth table Method

3-input Majority function using 4:1 MUX

w_1	w_2	w_3	f		w_1	w_2	f
0	0	0	0	}	0	0	0
0	0	1	0		0	1	w_3
0	1	0	0	}	1	0	w_3
0	1	1	1		1	1	1
1	0	0	0	}			
1	0	1	1				
1	1	0	1	}			
1	1	1	1				

(a) Modified truth table



(b) Circuit

Function Synthesis using Multiplexer

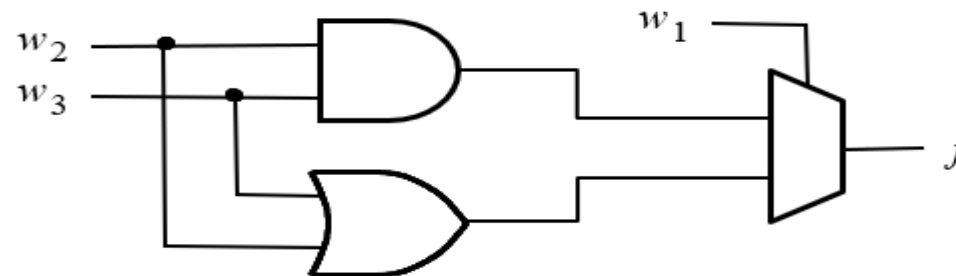
Truth table Method

3-input Majority function using 2:1 MUX

w_1	w_2	w_3	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

w_1	f
0	$w_2 w_3$
1	$w_2 + w_3$

(b) Truth table



(b) Circuit

Function Synthesis using Multiplexer

Shannon's Expansion Theorem

Shannon's Expansion Theorem:

Any Boolean function $f(w_1, w_2, \dots, w_n)$ can be written in the form:

$$f(w_1, w_2, \dots, w_n) = w_1' \cdot f(0, w_2, \dots, w_n) + w_1 \cdot f(1, w_2, \dots, w_n)$$

- The term $f(0, w_2, \dots, w_n)$ is known as the cofactor of f with respect to w_1' (also called **negative cofactor** with respect to w_1)
- It is obtained by replacing all occurrences of w_1 in f with 0
- It is denoted as $f_{w_1'}$

- The term $f(1, w_2, \dots, w_n)$ is known as the cofactor of f with respect to w_1 (also called **positive cofactor** with respect to w_1)
- It is obtained by replacing all occurrences of w_1 in f with 1
- It is denoted as f_{w_1}

$$f(w_1, w_2, \dots, w_n) = w_1' \cdot f_{w_1'} + w_1 \cdot f_{w_1}$$

Function Synthesis using Multiplexer

Shannon's Expansion Theorem

Problem: Example 1

Do Shannon's expansion of: $f = w_1 \oplus w_2$ and implement in terms of 2-to-1 MUX.

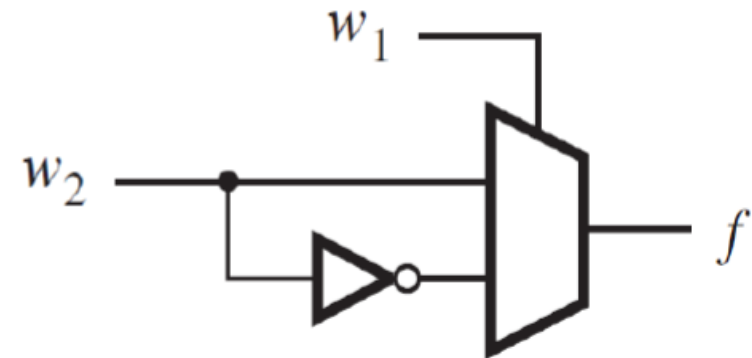
Solution:

$$f = w_1 \oplus w_2 = w_1' \cdot w_2 + w_1 w_2'$$

$$f_{w_1'} = w_2$$

$$f_{w_1} = w_2'$$

$$f = w_1' \cdot f_{w_1'} + w_1 \cdot f_{w_1}$$



Function Synthesis using Multiplexer

Shannon's Expansion method

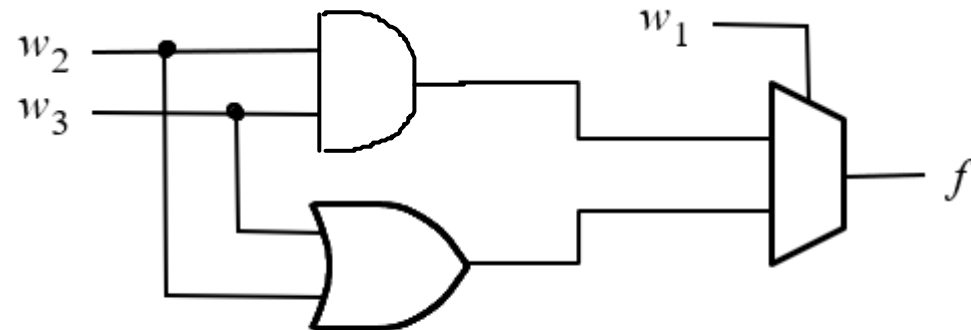
3-input Majority function using 2:1 MUX

w_1	w_2	w_3	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

$$f = \bar{w}_1 w_2 w_3 + w_1 \bar{w}_2 w_3 + w_1 w_2 \bar{w}_3 + w_1 w_2 w_3$$

$$f = \bar{w}_1 (w_2 w_3) + w_1 (\bar{w}_2 w_3 + w_2 \bar{w}_3 + w_2 w_3)$$

$$= \bar{w}_1 (w_2 w_3) + w_1 (w_2 + w_3)$$



Circuit

Function Synthesis using Multiplexer

Example 2(a)- Factor and implement the following function with a 2x1 multiplexer using w_1 as selection input

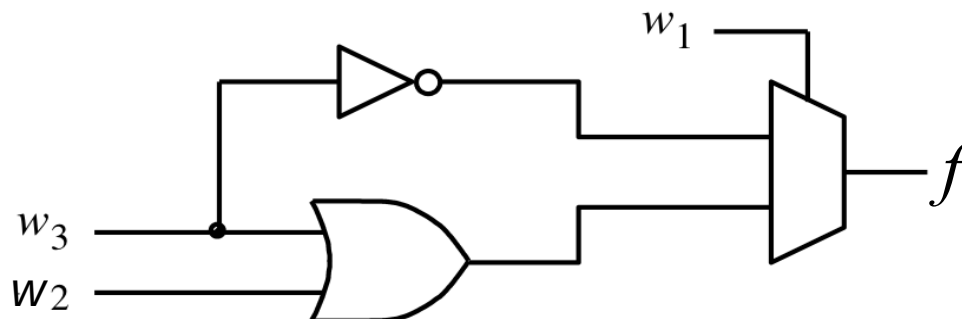
$$f = \overline{w}_1 \overline{w}_3 + w_1 w_2 + w_1 w_3$$

Solution- w_1 is the selection input so decompose the given function w.r.t w_1 -

$$f = \overline{w}_1 \overline{w}_3 + w_1 w_2 + w_1 w_3$$

$$f = \overline{w}_1 f_{\overline{w}_1} + w_1 f_{w_1}$$

$$= \overline{w}_1 (\overline{w}_3) + w_1 (w_2 + w_3)$$



Function Synthesis using Multiplexer

Example 2(b)- Factor and implement the following function with a 2x1 multiplexer using w_2 as selection input

$$f = \overline{w}_1 \overline{w}_3 + w_1 w_2 + w_1 w_3$$

Solution- w_2 is the selection input so each term of f should consist of w_2 either in normal form or complemented form.

$$\begin{aligned} f &= \overline{w}_1 \overline{w}_3 + w_1 w_2 + w_1 w_3 \\ &= \overline{w}_1 \overline{w}_3 (\overline{w}_2 + w_2) + w_1 w_2 + w_1 w_3 (\overline{w}_2 + w_2) \\ &= \overline{w}_1 \overline{w}_2 \overline{w}_3 + \overline{w}_1 w_2 \overline{w}_3 + w_1 w_2 + w_1 \overline{w}_2 w_3 + \\ &\quad w_1 w_2 w_3 \end{aligned}$$

Decompose the function w.r.t w_2

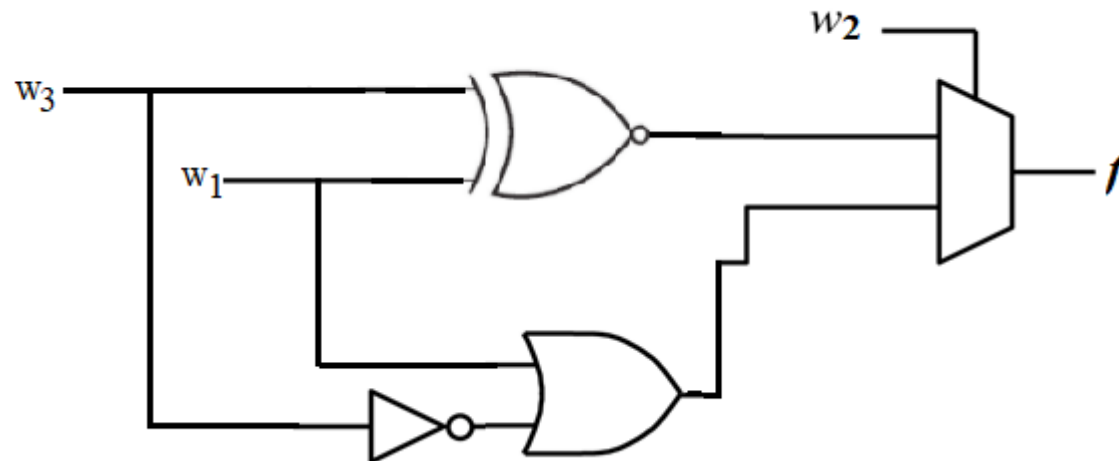
$$\begin{aligned} f &= \overline{w}_2 (\overline{w}_1 \overline{w}_3 + w_1 w_3) + w_2 (\overline{w}_1 \overline{w}_3 + w_1 + w_1 w_3) \\ &= \overline{w}_2 (\overline{w}_1 \overline{w}_3 + w_1 w_3) + w_2 (\overline{w}_1 \overline{w}_3 + w_1 (1 + w_3)) \\ &= \overline{w}_2 (\overline{w}_1 \overline{w}_3 + w_1 w_3) + w_2 (\overline{w}_3 + w_1) \quad \{\text{Distributive law}\} \end{aligned}$$

Function Synthesis using Multiplexer

Example 2(b)- Factor and implement the following function with a 2x1 multiplexer using w_2 as selection input

$$f = \overline{w_1}\overline{w_3} + w_1w_2 + w_1w_3$$

Solution-



Shannon's Expansion Theorem

(In terms of more than one variable)

$$f(w_1, \dots, w_n) = \bar{w}_1 \bar{w}_2 \cdot f(0, 0, w_3, \dots, w_n) + \bar{w}_1 w_2 \cdot f(0, 1, w_3, \dots, w_n) \\ + w_1 \bar{w}_2 \cdot f(1, 0, w_3, \dots, w_n) + w_1 w_2 \cdot f(1, 1, w_3, \dots, w_n)$$

This form is suitable for implementation with a 4x1 multiplexer.

Function Synthesis using Multiplexer

Example 3-Factor and implement the following function with a 4x1 multiplexer

$$f = \overline{w}_1 \overline{w}_3 + w_1 w_2 + w_1 w_3$$

Solution- Assume w_1 and w_2 are selection inputs. So

$$f = \overline{w}_1 \overline{w}_2 f_{\overline{w}_1 \overline{w}_2} + \overline{w}_1 w_2 f_{\overline{w}_1 w_2} + w_1 \overline{w}_2 f_{w_1 \overline{w}_2} + w_1 w_2 f_{w_1 w_2}$$

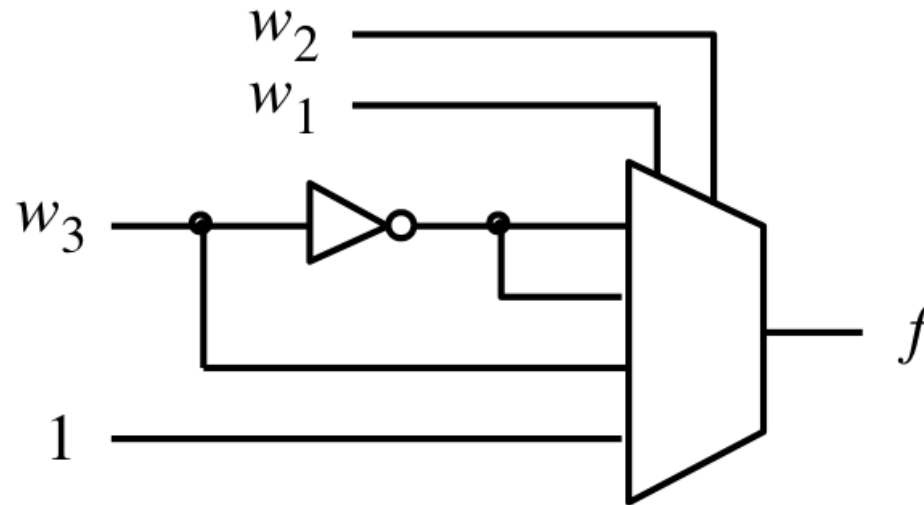
$$f = \overline{w}_1 (\overline{w}_2 + w_2) \overline{w}_3 + w_1 w_2 + w_1 (\overline{w}_2 + w_2) w_3$$

$$f = \overline{w}_1 \overline{w}_2 \overline{w}_3 + \overline{w}_1 w_2 \overline{w}_3 + w_1 w_2 + w_1 \overline{w}_2 w_3 + w_1 w_2 w_3$$

$$f = \overline{w}_1 \overline{w}_2 \overline{w}_3 + \overline{w}_1 w_2 \overline{w}_3 + w_1 \overline{w}_2 w_3 + w_1 w_2 + w_1 w_2 w_3$$

$$= \overline{w}_1 \overline{w}_2 (\overline{w}_3) + \overline{w}_1 w_2 (\overline{w}_3) + w_1 \overline{w}_2 (w_3) + w_1 w_2 (1)$$

Function Synthesis using Multiplexer



$$\begin{aligned}
 f &= \overline{w_1}\overline{w_2}f_{\overline{w_1}\overline{w_2}} + \overline{w_1}w_2f_{\overline{w_1}w_2} + w_1\overline{w_2}f_{w_1\overline{w_2}} + w_1w_2f_{w_1w_2} \\
 &= \overline{w_1}\overline{w_2}(\overline{w_3}) + \overline{w_1}w_2(\overline{w_3}) + w_1\overline{w_2}(w_3) + w_1w_2(1)
 \end{aligned}$$

Function Synthesis using Multiplexer

Example 4- Factor and implement the following function with a 2x1 multiplexer.

$$f = w_1w_2 + w_1w_3 + w_2w_3$$

Solution- Assume w_1 is the selection input so decompose the given function w.r.t w_1 -

$$f = \overline{w}_1f_{\overline{w}_1} + w_1f_{w_1}$$

$$f = w_1w_2 + w_1w_3 + (w_1 + \overline{w}_1)w_2w_3$$

$$\begin{aligned} f &= \overline{w}_1(w_2w_3) + w_1(w_2 + w_3 + w_2w_3) \\ &= \overline{w}_1(w_2w_3) + w_1(w_2 + w_3) \end{aligned}$$

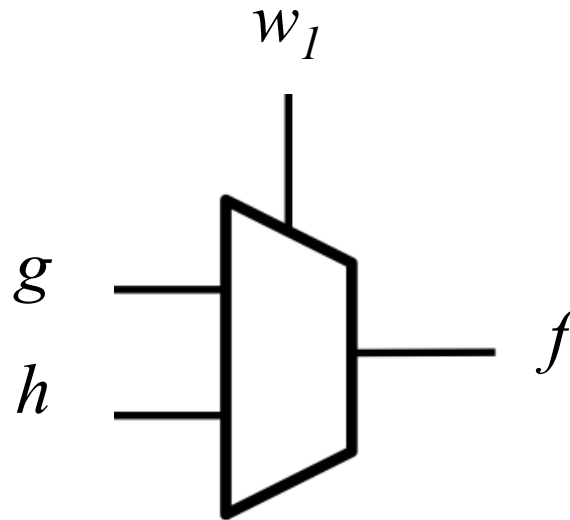
Function Synthesis using Multiplexer

$$f = w_1 w_2 + w_1 w_3 + w_2 w_3$$

$$\begin{aligned} f &= \bar{w}_1 (w_2 w_3) + w_1 (w_2 + w_3 + w_2 w_3) \\ &= \bar{w}_1 (w_2 w_3) + w_1 (w_2 + w_3) \end{aligned}$$

$$g = w_2 w_3 \quad h = w_2 + w_3$$

Function Synthesis using Multiplexer



$$\begin{aligned}
 f &= \bar{w}_1(w_2w_3) + w_1(w_2 + w_3 + w_2w_3) \\
 &= \bar{w}_1(\underbrace{w_2w_3}) + w_1(\underbrace{w_2 + w_3})
 \end{aligned}$$

$$g = w_2w_3 \quad h = w_2 + w_3$$

Function Synthesis using Multiplexer

$$g = w_2 w_3$$

$$h = w_2 + w_3$$

Function Synthesis using Multiplexer

$$g = w_2 w_3$$



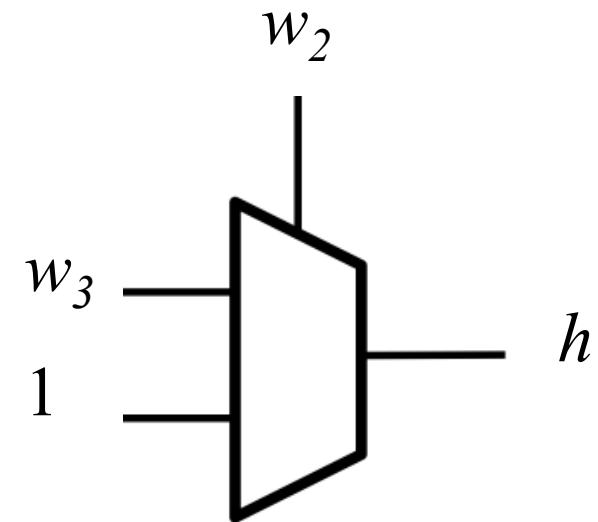
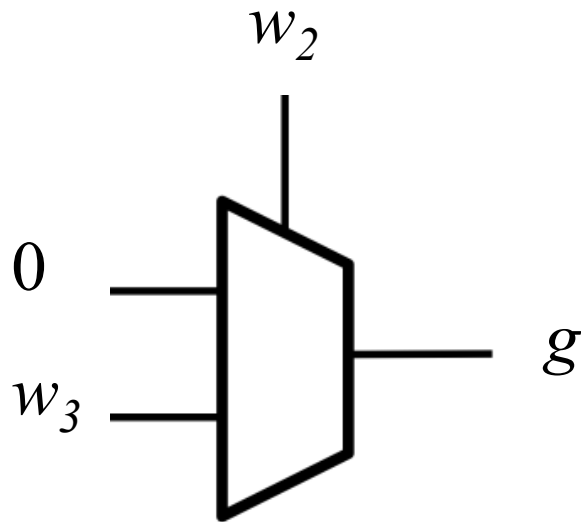
$$g = \overline{w}_2(0) + w_2(w_3)$$

$$h = w_2 + w_3$$



$$h = \overline{w}_2(w_3) + w_2(1)$$

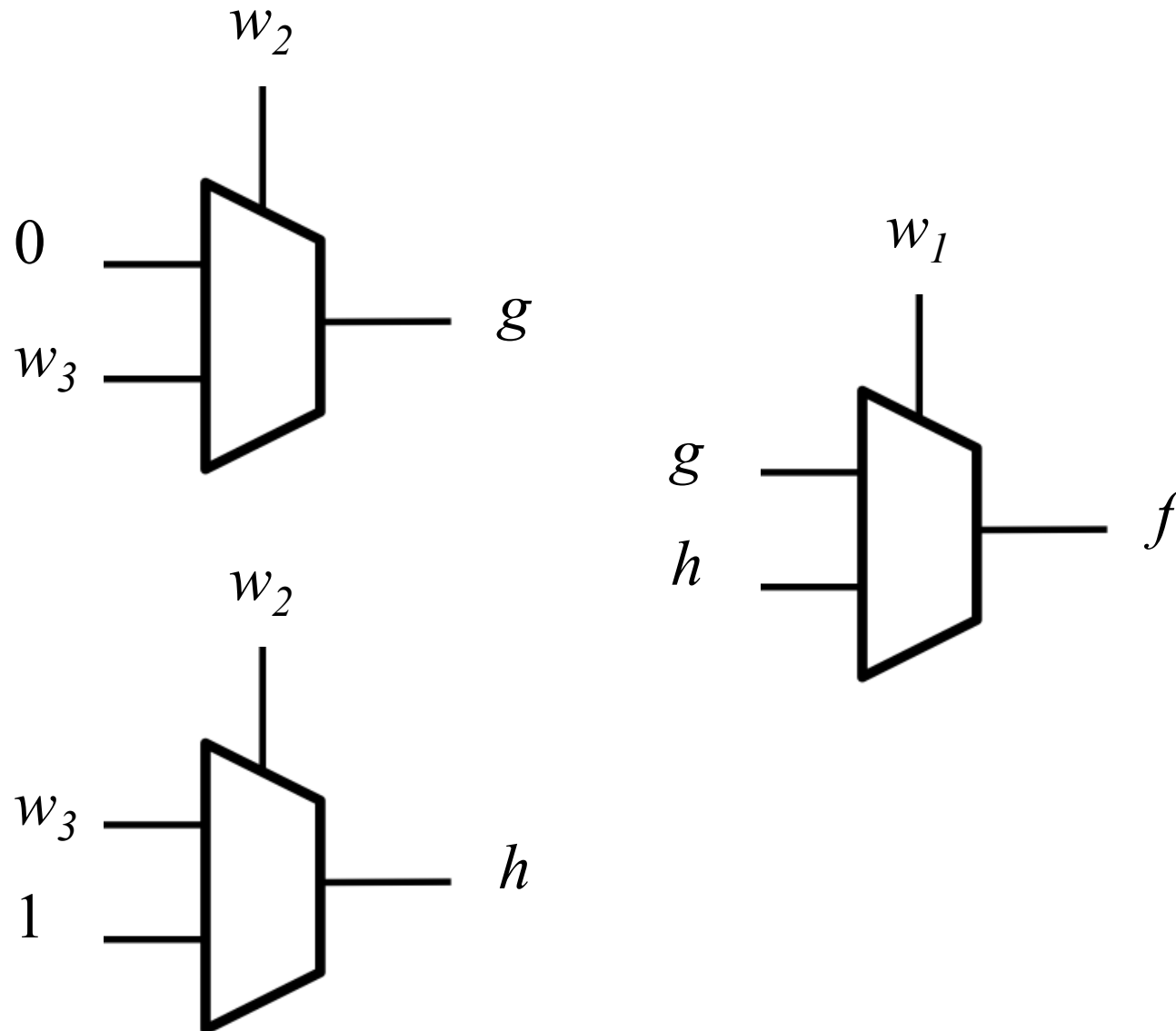
Function Synthesis using Multiplexer



$$g = \overline{w_2}(0) + w_2(w_3)$$

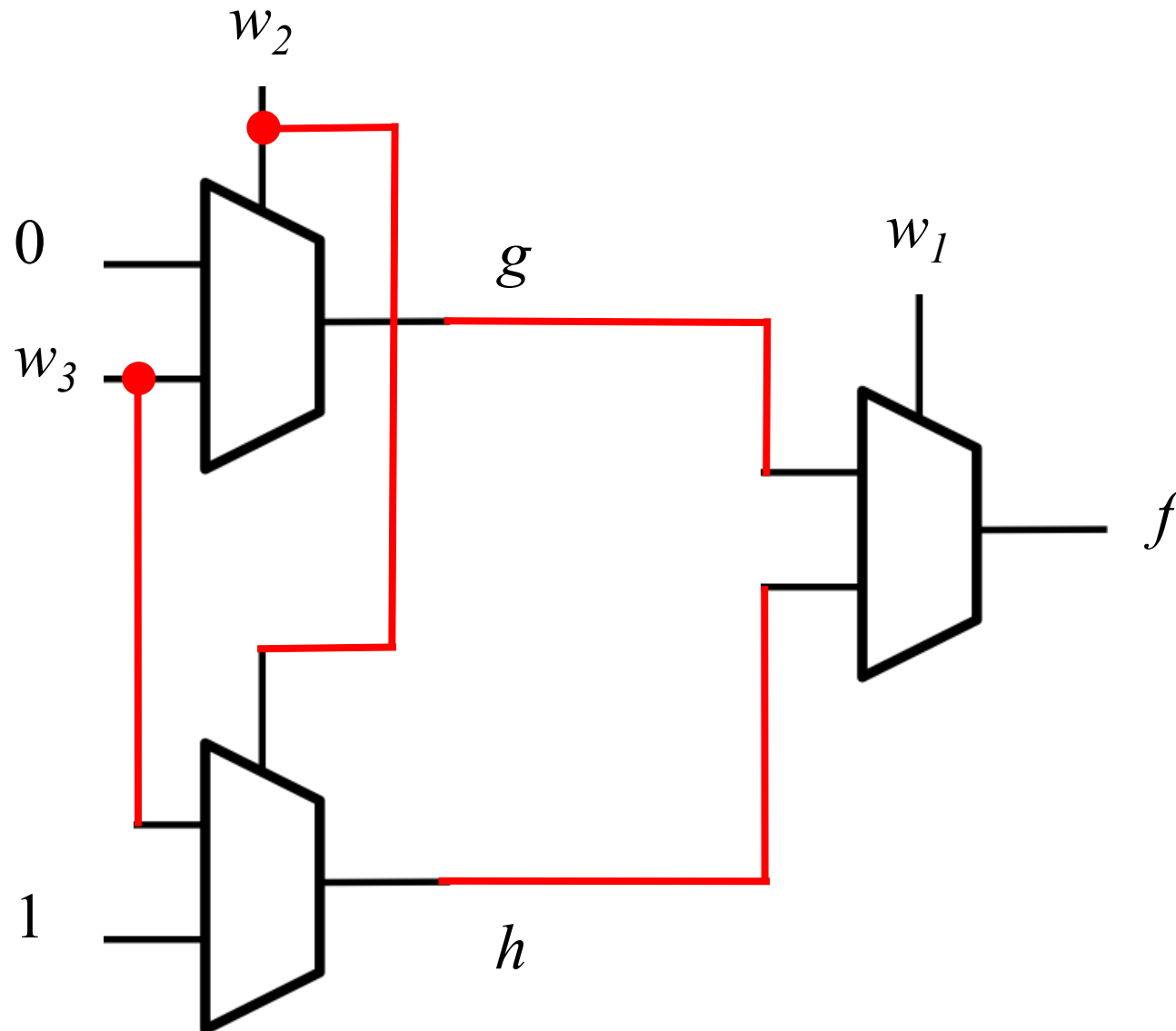
$$h = \overline{w_2}(w_3) + w_2(1)$$

Function Synthesis using Multiplexer

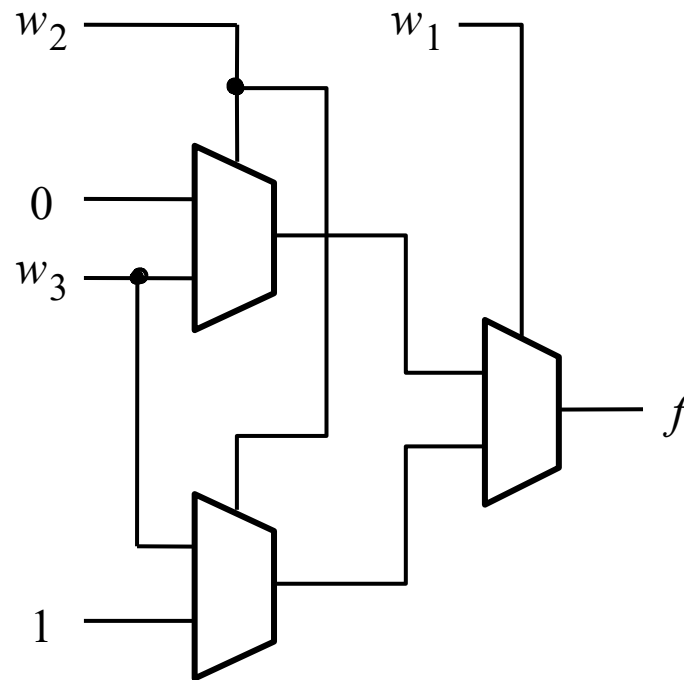


Function Synthesis using Multiplexer

Finally, we are ready to draw the circuit



Function Synthesis using Multiplexer



Function Synthesis using Multiplexer

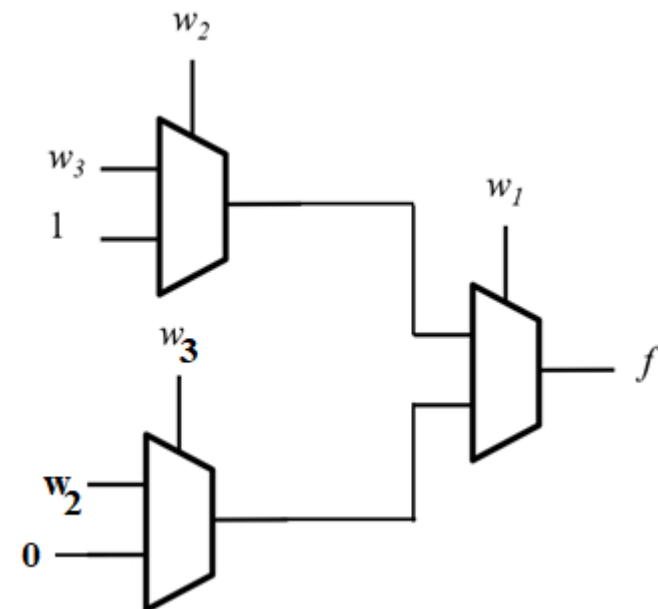
Shannon's Expansion Theorem: Significance of Variables

Problem: Compute the cost of the implementation of the function $f = w_1'w_3 + w_2w_3'$ using only 2-to-1 MUX, when the function is expanded with respect to i) w_1 ii) w_2 iii) w_3

Solution-

i) When the function is expanded with respect to w_1

$$\begin{aligned} f &= w_1'w_3 + w_2w_3' \\ &= w_1'w_3 + (w_1' + w_1) w_2w_3' \\ &= w_1'w_3 + w_1'w_2w_3' + w_1w_2w_3' \\ &= w_1' (w_3 + w_2w_3') + w_1(w_2w_3') \\ &= w_1'(w_2 + w_3) + w_1(w_2w_3') \end{aligned}$$



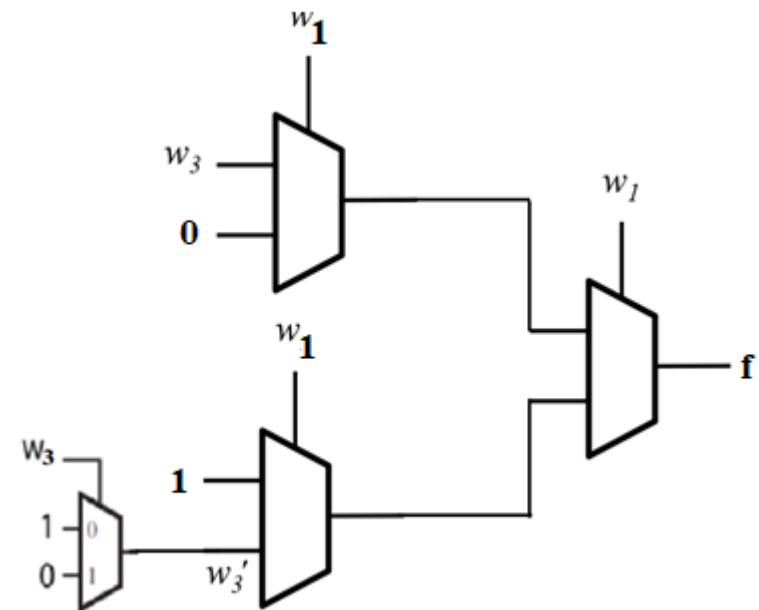
Function Synthesis using Multiplexer

Shannon's Expansion Theorem: Significance of Variables

Problem: Compute the cost of the implementation of the function $f = w_1'w_3 + w_2w_3'$ using only 2-to-1 MUX, when the function is expanded with respect to i) w_1 ii) w_2 iii) w_3

ii) When the function is expanded with respect to w_2

$$\begin{aligned} f &= w_1' (w_2' + w_2) w_3 + w_2 w_3' \\ &= w_1' w_2' w_3 + w_1' w_2 w_3 + w_2 w_3' \\ &= w_2' (w_1' w_3) + w_2 (w_1' w_3 + w_3') \\ &= w_2' (w_1' w_3) + w_2 (w_1' + w_3') \end{aligned}$$



Shannon's Expansion Theorem: Significance of Variables

Problem: Compute the cost of the implementation of the function $f = w_1'w_3 + w_2w_3'$ using only 2-to-1 MUX, when the function is expanded with respect to i) w_1 ii) w_2 iii) w_3

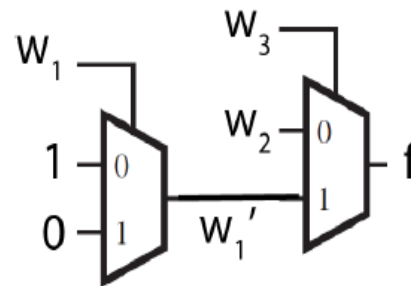
Solution:

iii) Expand with respect to w_3

$$f = w_3' \cdot f_{w_3'} + w_3 \cdot f_{w_3}$$

$$f_{w_3'} = w_2$$

$$f_{w_3} = w_1'$$



Two 2-to-1 MUX are sufficient

Conclusions:

- The order of variables chosen in the recursive Shannon's Expansion has a big impact on the cost of implementation
- In practice, tools try out various orders and come up with a minimized implementation

Implementing Boolean Functions using Multiplexers

Problem 1: Implement the function $f(x, y, z) = \Sigma m(3, 5, 6, 7)$ using minimum 2-to-1 MUX (do not use any other gates)

Solution:

i) Draw the K-map

x \ yz	yz			
	00	01	11	10
0	0	0	1	0
1	0	1	1	1

$$F(x, y, z) = x'yz + xy'z + xyz' + xyz$$

ii) Take x as the select input of the MUX:

$$f(x, y, z) = x'(yz) + x(y + z)$$

Can be implemented using three 2-to-1 MUX

iii) Take y as the select input of the MUX:

$$f(x, y, z) = y'(xz) + y(x + z)$$

Can be implemented using three 2-to-1 MUX

iv) Take z as the select input of the MUX:

$$f(x, y, z) = z'(xy) + z(x + y)$$

Can be implemented using three 2-to-1 MUX

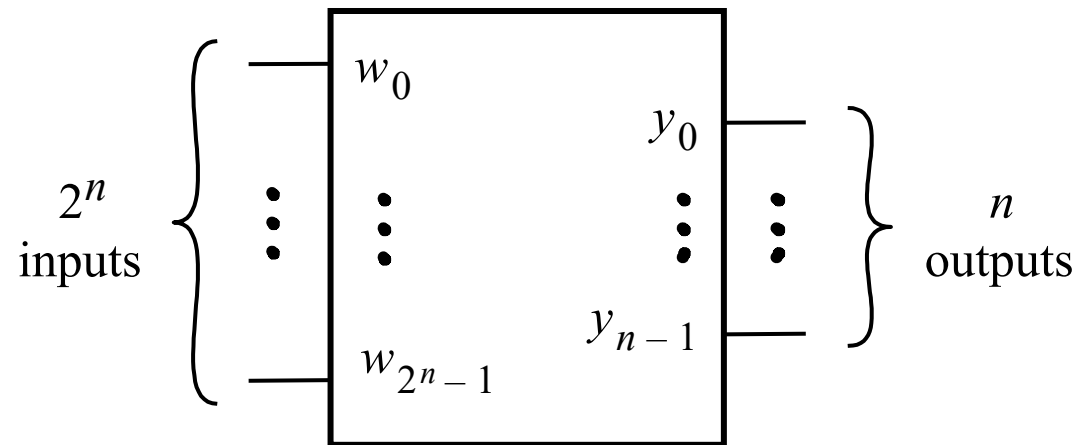
Encoders

Encoders

Encoders

- An **encoder** performs the opposite function of a decoder
- A **binary encoder** encodes information (data) from 2^n inputs into an n -bit code (output)
 - Exactly one of the inputs should have a value of one
 - The outputs represent the binary number that identifies which input is equal to 1
- Encoders reduce the number of bits needed to represent given information
- Practical use: transmitting information in a digital system

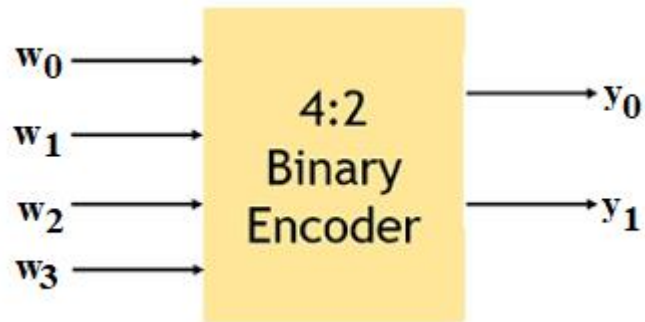
Encoders



A 2^n -to- n binary encoder.

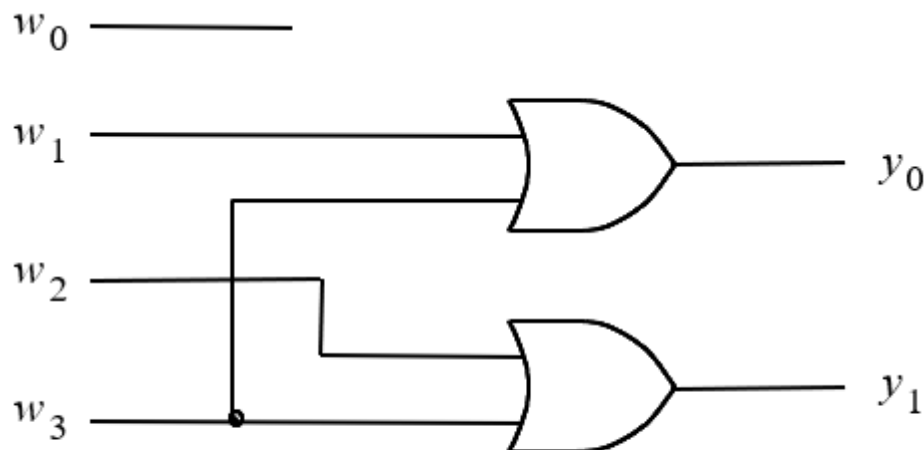
Encoders

4-to-2 binary encoder



w_3	w_2	w_1	w_0	y_1	y_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

(a) Truth table



(b) Circuit

Logical Expression-

$$y_1 = w_2 + w_3$$

$$y_0 = w_1 + w_3$$

Encoders

8-to-3 binary encoder (octal-to-binary)

Inputs								Outputs		
D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0	A_2	A_1	A_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

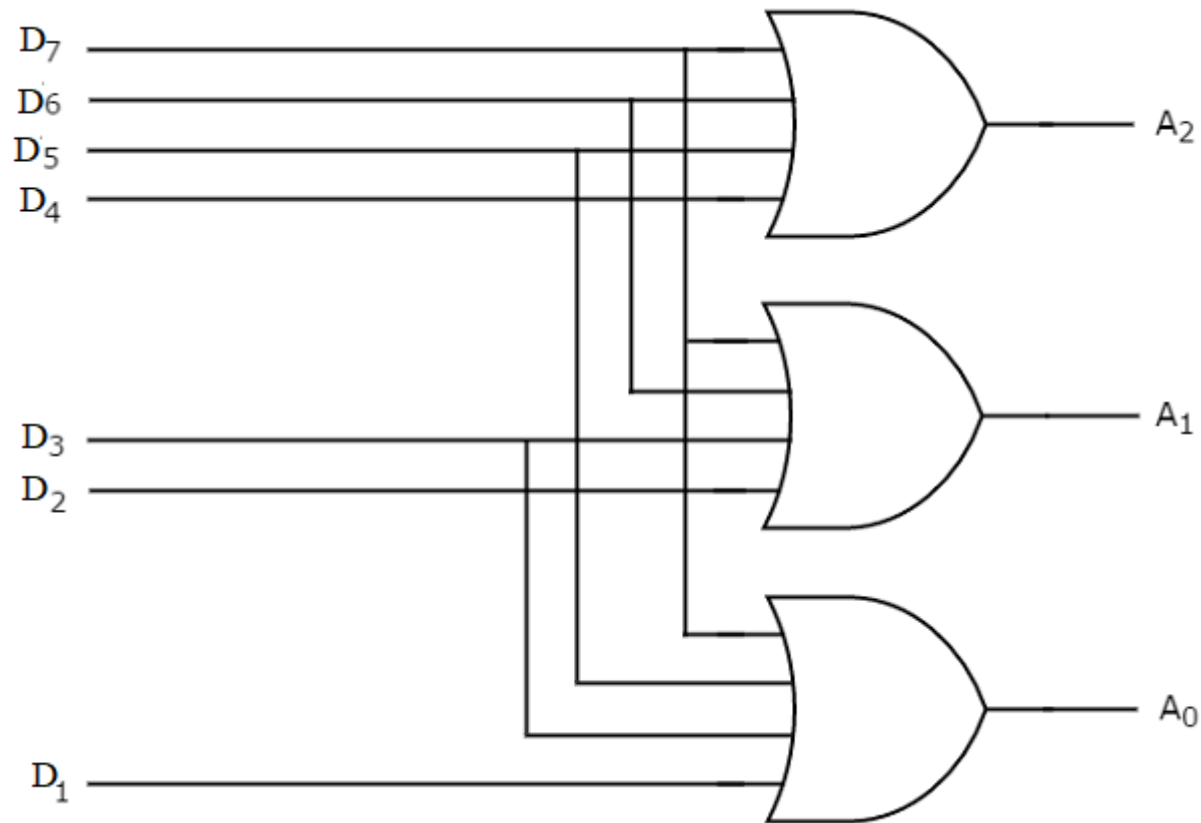
$$A_0 = D_1 + D_3 + D_5 + D_7$$

$$A_1 = D_2 + D_3 + D_6 + D_7$$

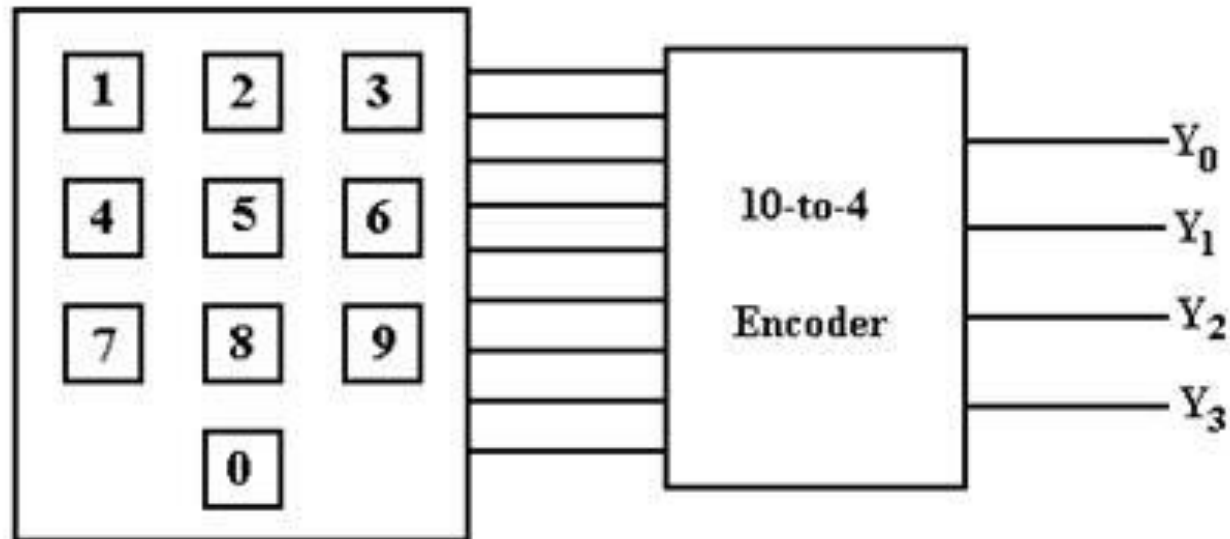
$$A_2 = D_4 + D_5 + D_6 + D_7$$

Encoders

8-to-3 binary encoder (octal-to-binary)



Encoders



Encoders

Priority encoders

- Another useful class of encoders is based on the ***priority*** of the input signals
- In a priority encoder, each input has a priority level associated with it
- The encoder outputs indicate the active input that has the highest priority
 - When an input with a high priority is asserted, the other lower priority inputs are ignored

Priority Encoder

4-to-2 priority encoder

- Assume that w_0 has the lowest priority and w_3 has the highest
- The output z indicates when none of the inputs are 1
- Letting
 - $i_0 = w_3'w_2'w_1'w_0$
 - $i_1 = w_3'w_2'w_1$
 - $i_2 = w_3'w_2$
 - $i_3 = w_3$

$$Y_0 = i_1 + i_3, \quad Y_1 = i_2 + i_3$$

$$Z = i_0 + i_1 + i_2 + i_3$$

w_3	w_2	w_1	w_0	y_1	y_0	z
0	0	0	0	d	d	0
0	0	0	1	0	0	1
0	0	1	x	0	1	1
0	1	x	x	1	0	1
1	x	x	x	1	1	1

$$y_0 = i_1 + i_3 = w_3'w_2'w_1 + w_3$$

$$= (w_3' + w_3)(w_2'w_1 + w_3)$$

$$y_0 = (w_2'w_1 + w_3)$$

$$y_1 = i_2 + i_3$$

$$= w_3'w_2 + w_3 = (w_3' + w_3)(w_2 + w_3)$$

$$y_1 = w_2 + w_3$$

$$Z = w_0 + w_1 + w_2 + w_3$$

Priority Encoder

4-to-2 priority encoder

$$y_0 = i_1 + i_3 = w_3'w_2'w_1 + w_3$$

$$= (w_3' + w_3) (w_2'w_1 + w_3)$$

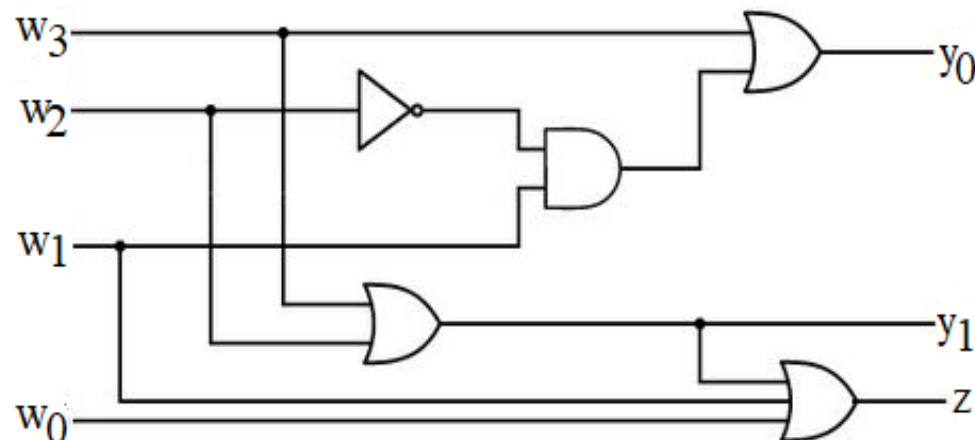
$$y_0 = (w_2'w_1 + w_3)$$

$$y_1 = i_2 + i_3$$

$$= w_3'w_2 + w_3 = (w_3' + w_3) (w_2 + w_3)$$

$$y_1 = w_2 + w_3$$

$$z = w_0 + w_1 + w_2 + w_3$$



Priority Encoder

4-to-2 priority encoder

Design a 4-to-2 priority with w_3, w_2, w_1 and w_0 inputs where w_3 is MSB and w_0 is LSB. The LSB input has highest priority.

Priority Encoder

8-to-3 Priority Encoder

Table 4.6 A priority encoder.

A_0	A_1	A_2	A_3	A_4	A_5	A_6	A_7	Z_0	Z_1	Z_2	NR
0	0	0	0	0	0	0	0	X	X	X	1
X	X	X	X	X	X	X	1	1	1	1	0
X	X	X	X	X	X	1	0	1	1	0	0
X	X	X	X	X	1	0	0	1	0	1	0
X	X	X	X	1	0	0	0	1	0	0	0
X	X	X	1	0	0	0	0	0	1	1	0
X	X	1	0	0	0	0	0	0	1	0	0
X	1	0	0	0	0	0	0	0	0	1	0
1	0	0	0	0	0	0	0	0	0	0	0

Digital Electronics & Systems

19ECE204

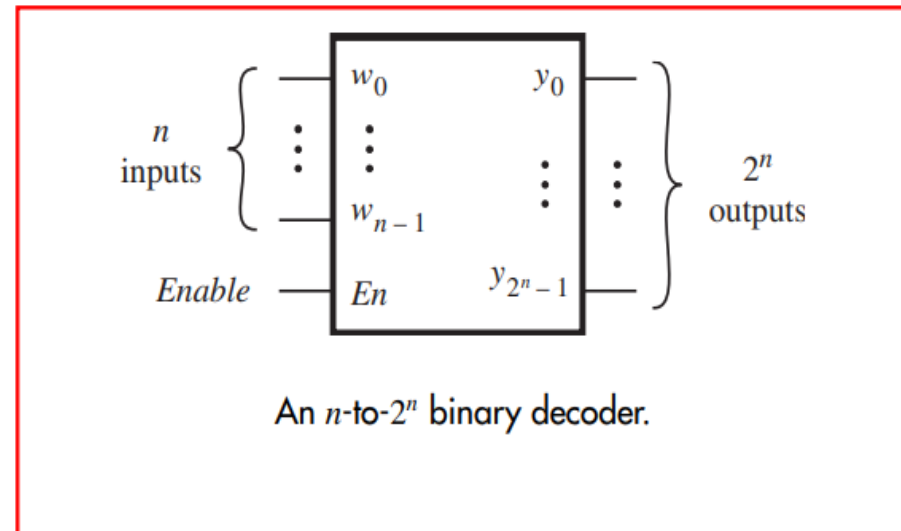
Lecture 41-44

Combinational Circuits

Ms. Bhavana V
Assistant Professor (Sr.Gr.)
Dept. of ECE
ASE, Bengaluru

Decoder

- Decoder circuits are used to decode encoded information.
- A binary decoder is a logic circuit with n inputs and 2^n outputs.
- Only one output is asserted at a time, and each output corresponds to one valuation of the inputs.
- The decoder also has an enable input, En , that is used to disable the outputs;
 - If $En = 0$, then none of the decoder outputs is asserted.
 - If $En = 1$, the valuation of $w_{n-1} \cdots w_1 w_0$ determines which of the outputs is asserted.



Decoder

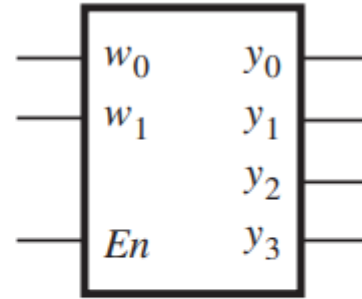
- An n -bit binary code in which exactly one of the bits is set to 1 at a time is referred to as one-hot encoded, meaning that the single bit that is set to 1 is deemed to be “hot.”
- The outputs of a binary decoder are one-hot encoded.
- A decoder can be designed to have either active-high or active-low outputs.

Decoder

2-to-4 Decoder (Active High Output)-

En	w_1	w_0	y_0	y_1	y_2	y_3
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1
0	x	x	0	0	0	0

(a) Truth table



(b) Graphical symbol

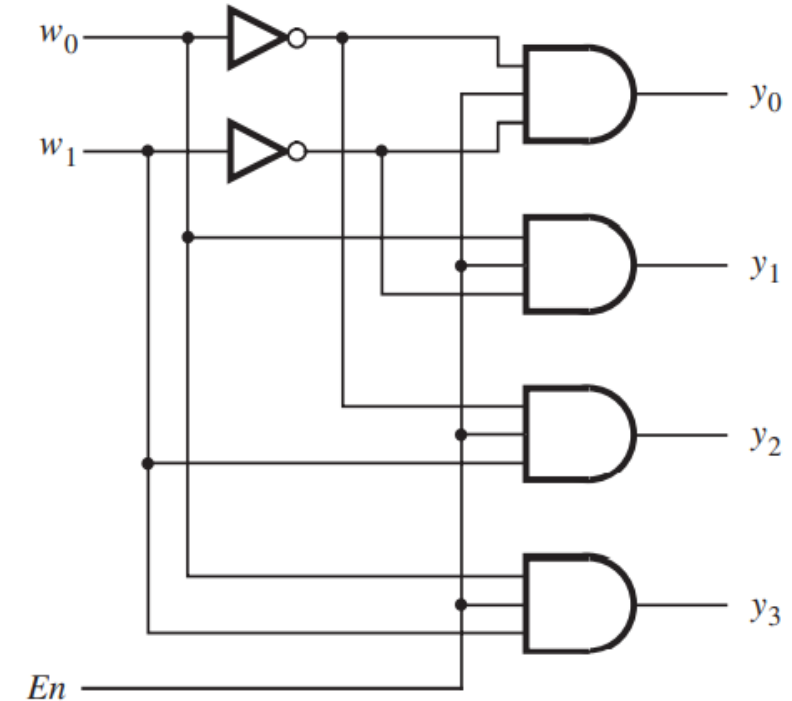
Logical Expressions-

$$y_0 = En \cdot w_1' \cdot w_0'$$

$$y_1 = En \cdot w_1' \cdot w_0$$

$$y_2 = En \cdot w_1 \cdot w_0'$$

$$y_3 = En \cdot w_1 \cdot w_0$$



(c) Logic circuit

Decoder

2-to-4 Decoder (Active Low Output)-

E	A	B	D_0	D_1	D_2	D_3
1	X	X	1	1	1	1
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	0

(a) Truth table

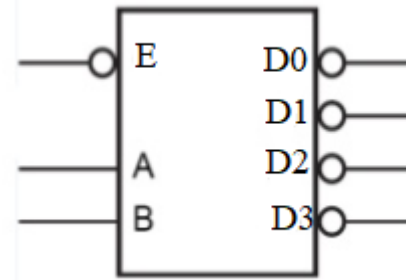
Logical Expressions-

$$D_0 = \overline{E' \cdot A' \cdot B'}$$

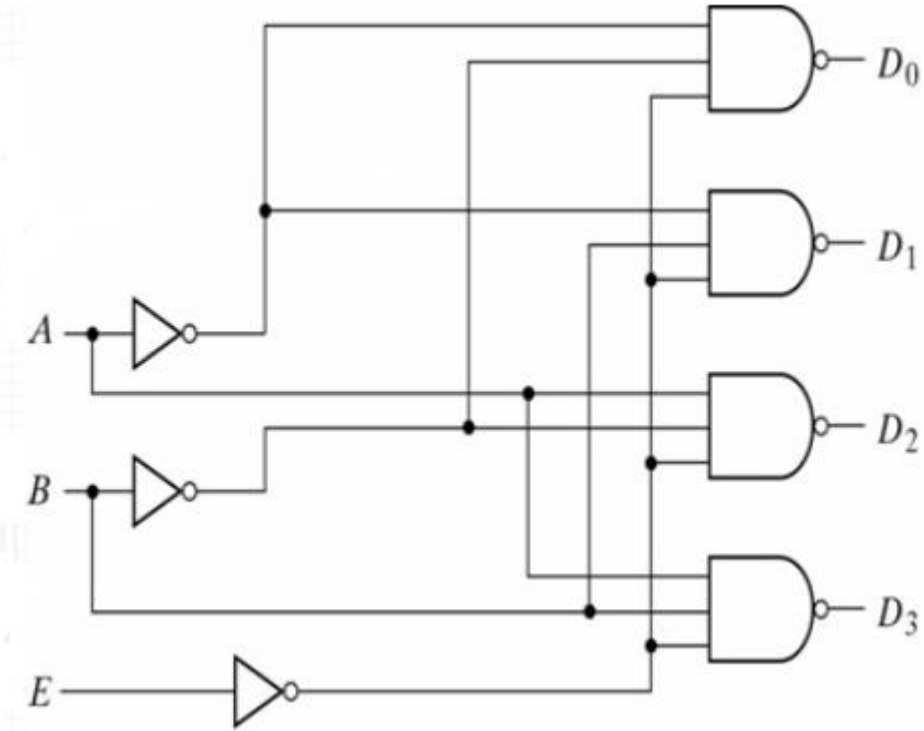
$$D_1 = \overline{E' \cdot A' \cdot B}$$

$$D_2 = \overline{E' \cdot A \cdot B'}$$

$$D_3 = \overline{E' \cdot A \cdot B}$$



(b) Graphical symbol



(c) Logic circuit

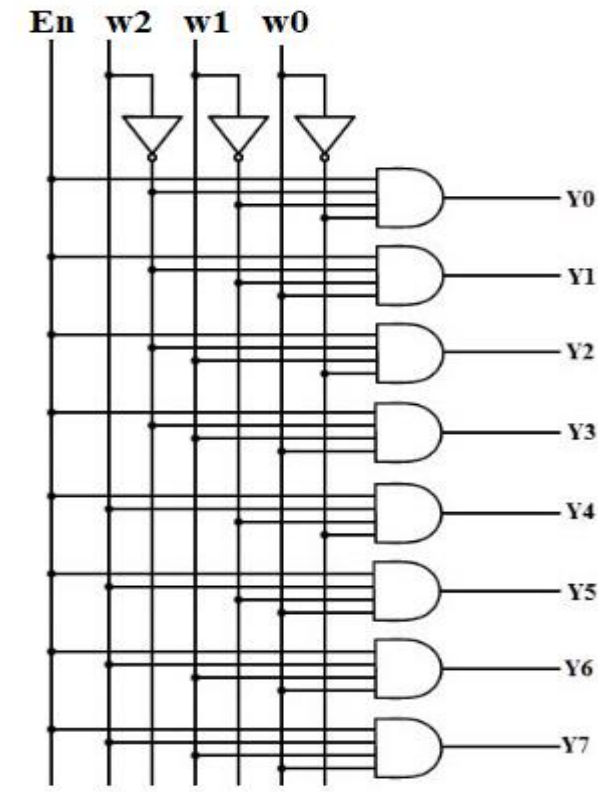
Decoder

3-to-8 Decoder-

- Larger decoders can be built using the sum-of-products structure as shown for 2-to-4 decoder.

Inputs				Outputs							
EN	w ₂	w ₁	w ₀	y ₇	y ₆	y ₅	y ₄	y ₃	y ₂	y ₁	y ₀
0	x	x	x	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1
1	0	0	1	0	0	0	0	0	0	1	0
1	0	1	0	0	0	0	0	0	1	0	0
1	0	1	1	0	0	0	0	1	0	0	0
1	1	0	0	0	0	0	1	0	0	0	0
1	1	0	1	0	0	1	0	0	0	0	0
1	1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	1	0	0	0	0	0	0	0

Truth Table

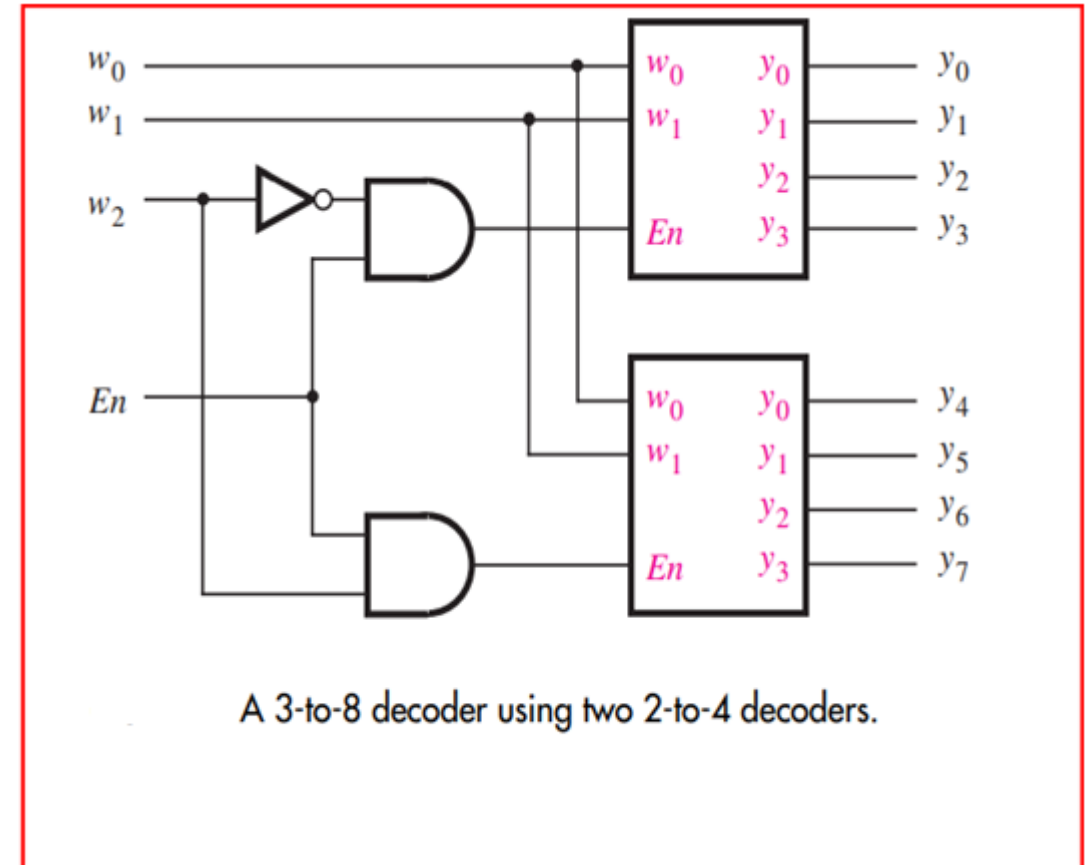


Logic Circuit

Decoder

3-to-8 Decoder-

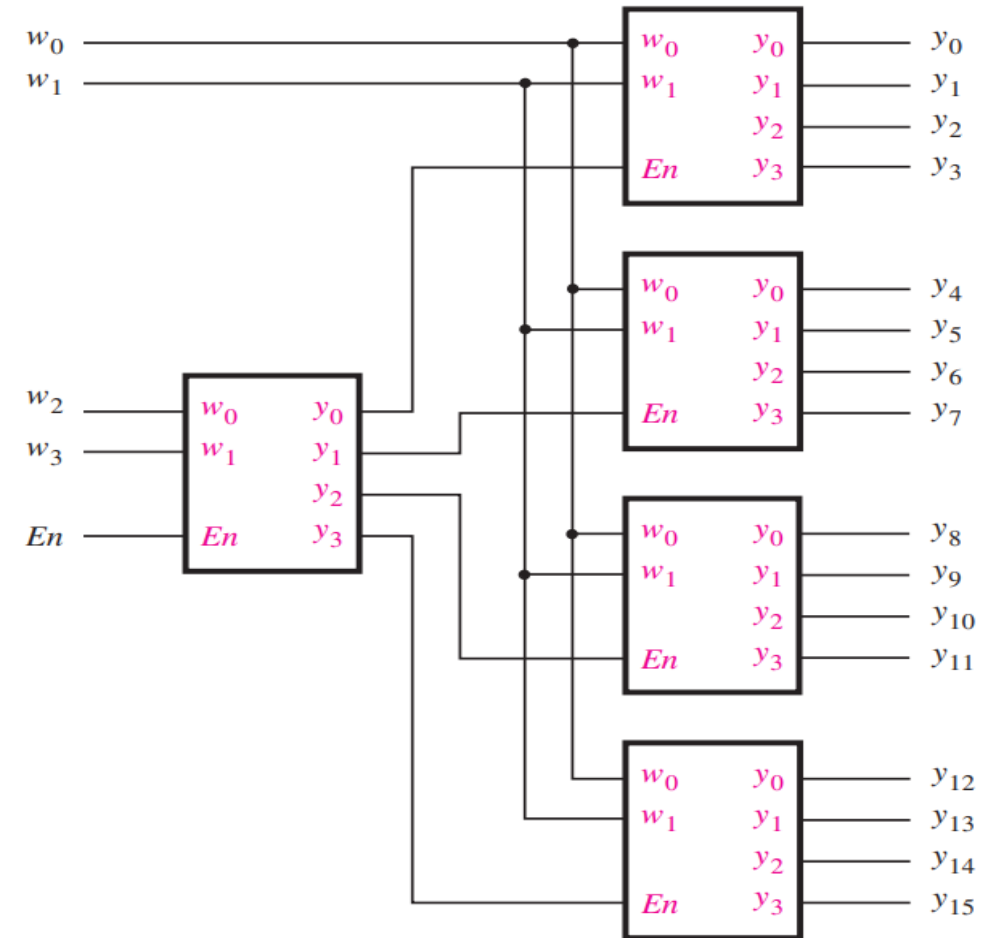
- A larger decoder can be constructed from smaller decoders.
- Figure shows how a 3-to-8 decoder is built with two 2-to-4 decoders.
- The w_2 input drives the enable inputs of the two decoders. The top decoder is enabled if $w_2 = 0$, and the bottom decoder is enabled if $w_2 = 1$.
- This concept can be applied for decoders of any size.



Decoder

4-to-16 Decoder-

- Five 2-to-4 decoders can be used to construct a 4-to-16 decoder.
- Because of its treelike structure, this type of circuit is often referred to as a decoder tree.

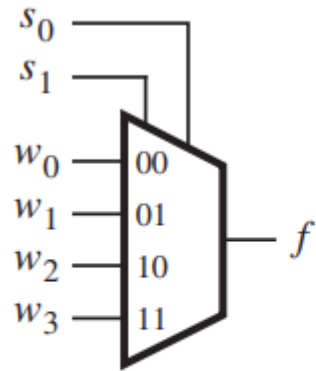


A 4-to-16 decoder built using a decoder tree.

Decoder

4-to-1 Multiplexer using Decoder-

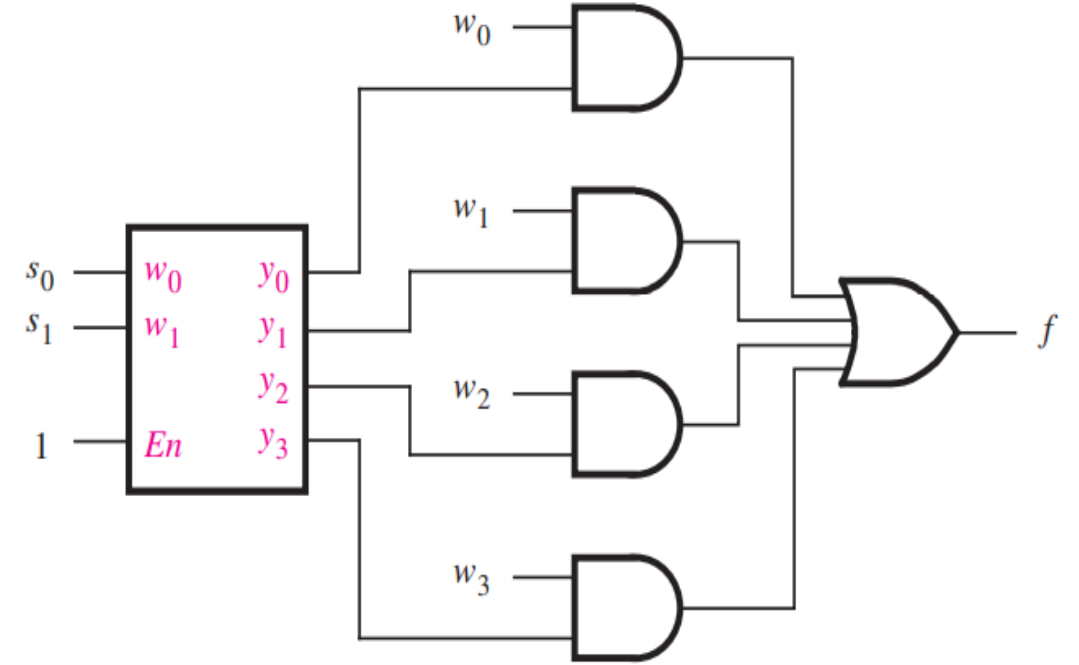
4-to-1 Multiplexer



(a) Graphical symbol

s_1	s_0	f
0	0	w_0
0	1	w_1
1	0	w_2
1	1	w_3

(b) Truth table



A 4-to-1 multiplexer built using a decoder.

Decoder

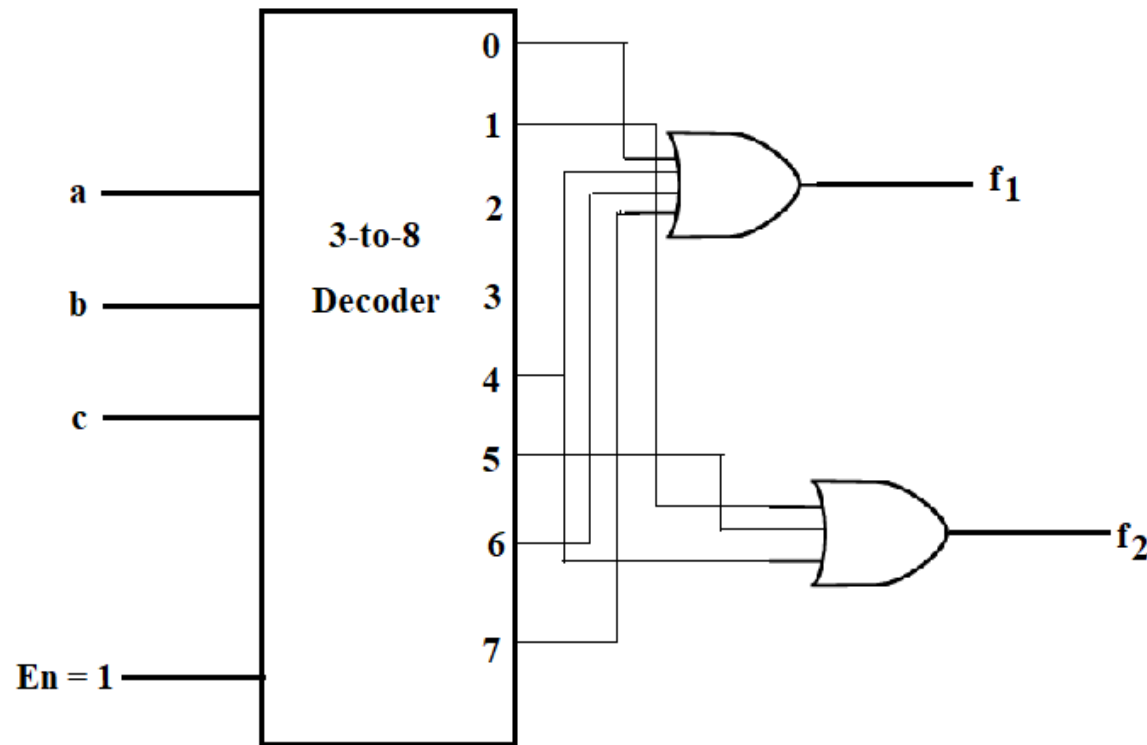
Function Implementation using Decoder (Active High Output)-

SOP Form-

Implement the following functions using a 3 to 8 line decoder and required logic gates.

$$f_1(a, b, c) = \sum m(0, 4, 6, 7)$$

$$f_2(a, b, c) = \sum m(1, 4, 5)$$



Decoder

Function Implementation using Decoder (Active High Output)-

POS Form-

Implement the following functions using a decoder minimizing the number of inputs to be summed.

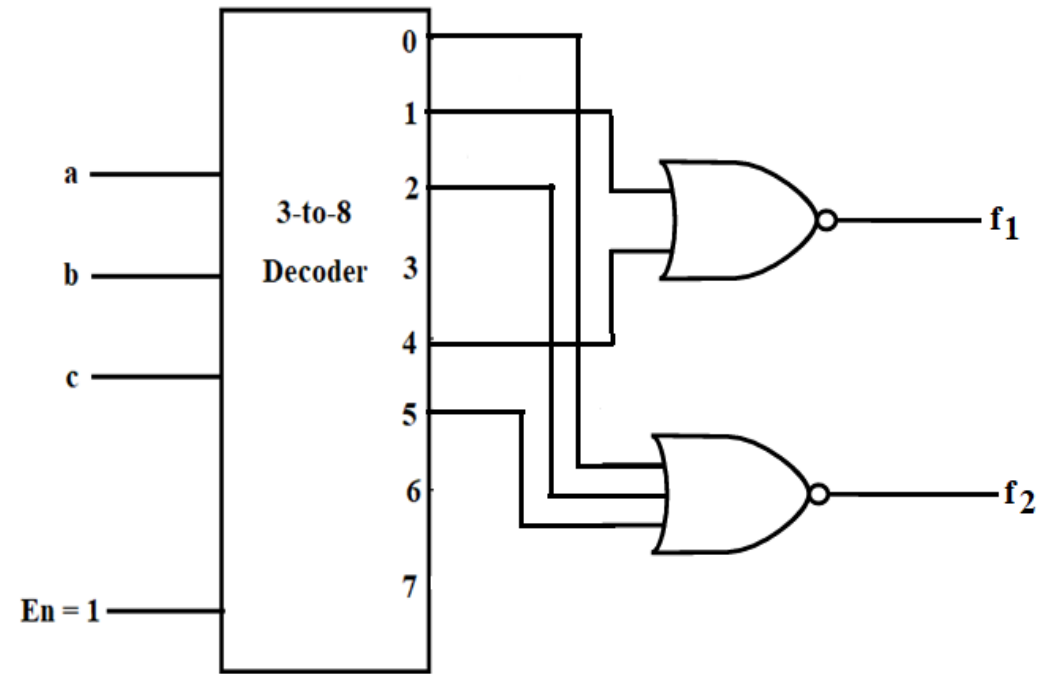
$$f_1(a, b, c) = \sum m(0, 2, 3, 5, 6, 7)$$

$$f_2(a, b, c) = \sum m(1, 3, 4, 6, 7)$$

Solution- In POS form-

$$f_1(a, b, c) = \prod M(1, 4) = \overline{\sum m(1, 4)}$$

$$f_2(a, b, c) = \prod M(0, 2, 5) = \overline{\sum m(0, 2, 5)}$$



Decoder

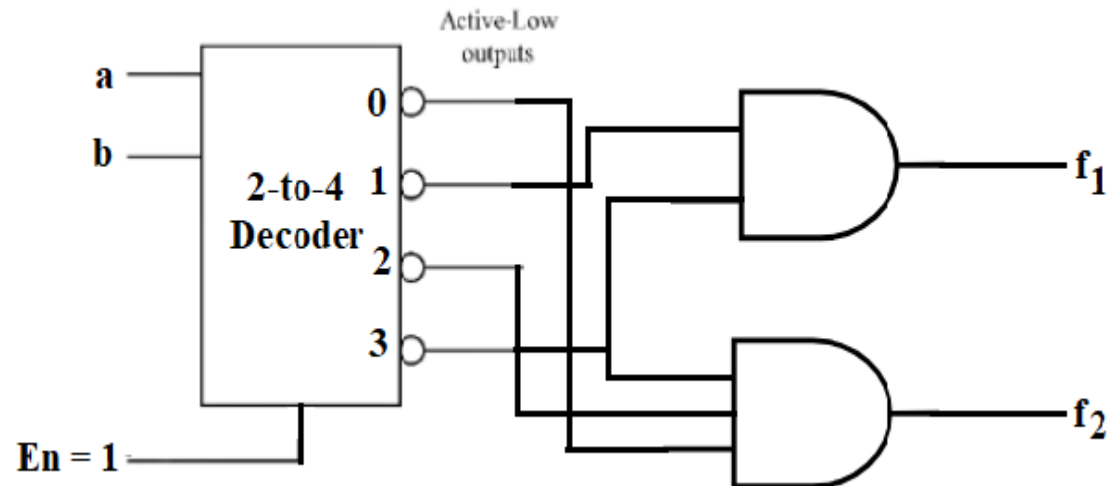
Function Implementation using Decoder (Active Low Output)-

POS Form-

Implement the following functions in maxterm canonical form using a decoder with active low outputs and required logic gates.

$$f_1(a, b) = \Pi M (1,3)$$

$$f_2(a, b) = \Pi M (0,2,3)$$



Decoder

Function Implementation using Decoder (Active Low Output)-

SOP Form-

Implement the following functions using a 3 to 8 decoder minimizing the number of inputs to the logic gates with active low outputs.

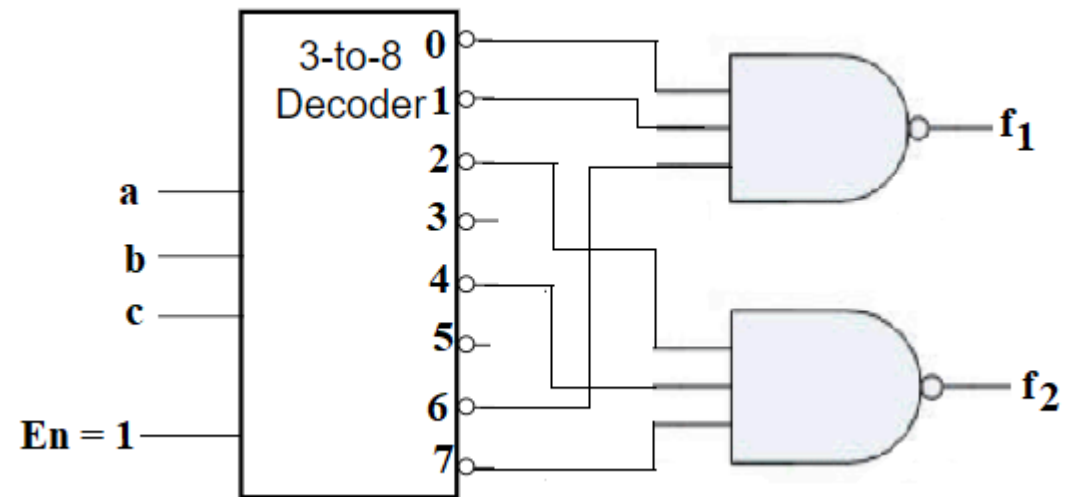
$$f_1(a, b, c) = \Pi M (0,1,3,5,6)$$

$$f_2(a, b, c) = \Pi M (2,3,4,5,7)$$

Solution- In SOP form

$$f_1(a, b, c) = \Sigma m (2,4,7) = \overline{\Pi M (2,4,7)}$$

$$f_2(a, b, c) = \Sigma m (0,1,6) = \overline{\Pi M (0,1,6)}$$



Decoder

Function Implementation using Decoder-

Implement full adder using a decoder and required logic gates.

Solution- To implement a full adder, we need a 3:8 decoder and OR gates. The input to the full adder, first and second bits and carry bit, are used as input to the decoder. Let x, y and z represent these three bits. Sum and Carry outputs of a full adder have the following truth table-

x	y	z	S	C
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$S = \sum(1, 2, 4, 7)$$
$$C = \sum(3, 5, 6, 7)$$

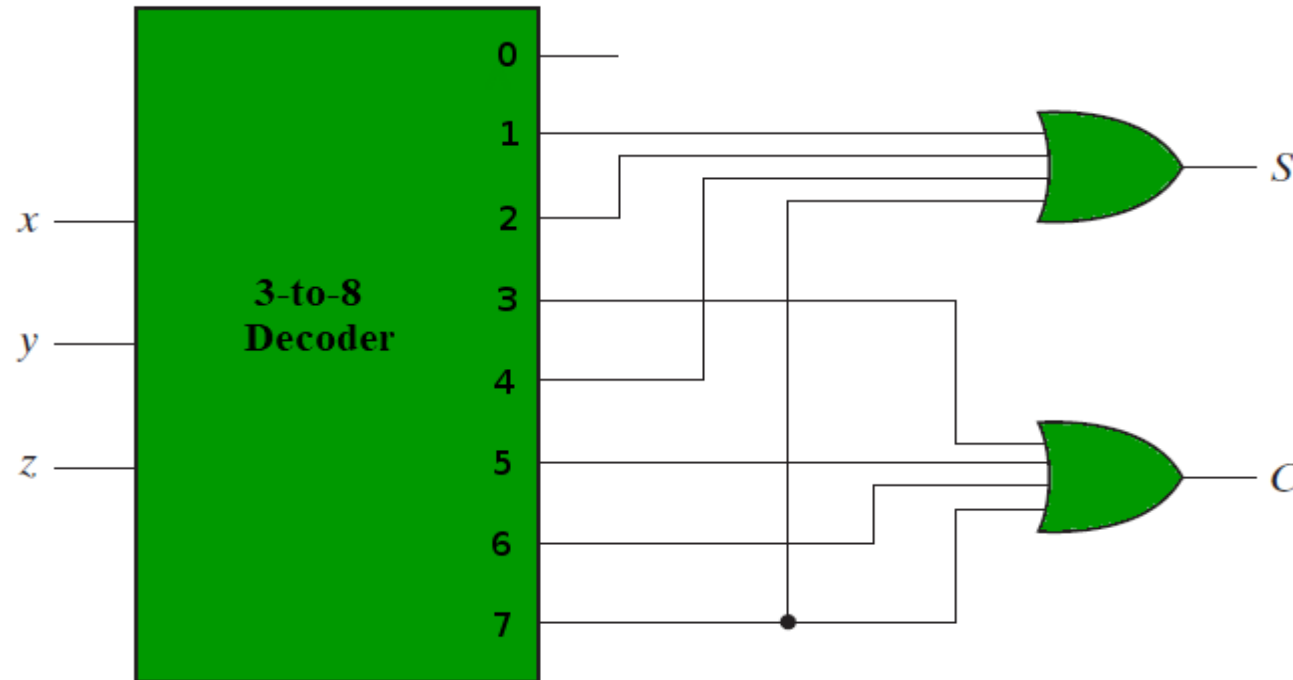
Decoder

Function Implementation using Decoder-

Implement full adder using a decoder and required logic gates.

$$S = \sum(1, 2, 4, 7)$$

$$C = \sum(3, 5, 6, 7)$$



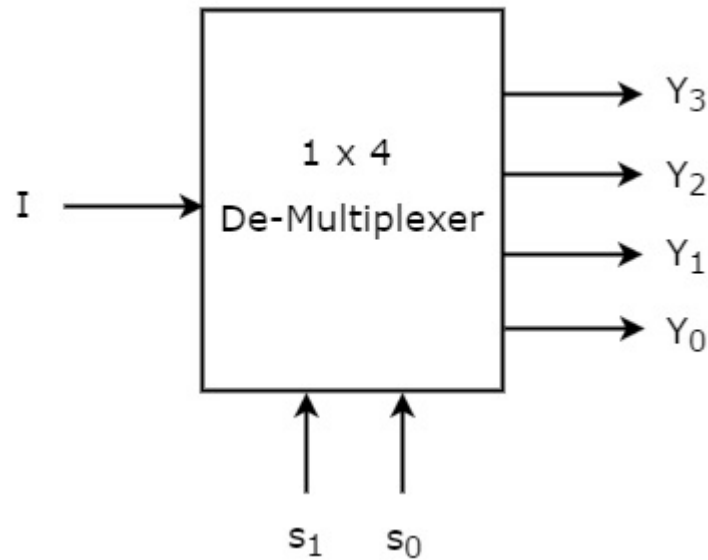
Demultiplexer

- **De-Multiplexer** is a combinational circuit that performs the reverse operation of Multiplexer.
- It has single input, 'n' selection lines and maximum of 2^n outputs. The input will be connected to one of these outputs based on the values of selection lines.
- Since there are 'n' selection lines, there will be 2^n possible combinations of zeros and ones. So, each combination can select only one output.
- The demultiplexer can be implemented using a decoder circuit.
- In general, an n-to- 2^n decoder circuit can be used as a 1-to-n demultiplexer.

Demultiplexer

1-to-4 Demultiplexer

- 1x4 De-Multiplexer has one input I , two selection lines, s_1 & s_0 and four outputs Y_3 , Y_2 , Y_1 & Y_0 .



Block Diagram

Selection Inputs		Outputs			
s_1	s_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	I
0	1	0	0	I	0
1	0	0	I	0	0
1	1	I	0	0	0

Truth Table

$$Y_3 = s_1 s_0 I$$

$$Y_2 = s_1 s_0' I$$

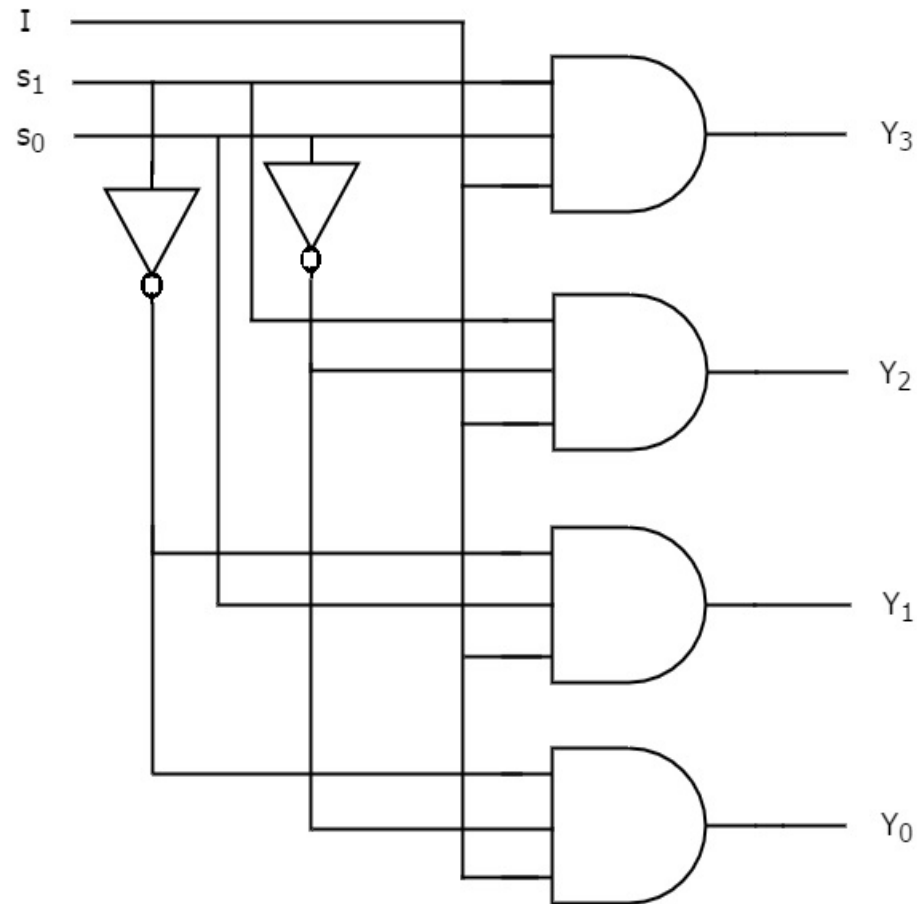
$$Y_1 = s_1' s_0 I$$

$$Y_0 = s_1' s_0' I$$

Logical
Expression

Demultiplexer

1-to-4 Demultiplexer



Logic Circuit

Code Converter

Binary to Gray code converter

Gray code-

- **Gray code-** also known as **Cyclic Code**, **Reflected Binary Code (RBC)**, **Reflected Binary (RB)** – is defined as an ordering of the binary number system such that each incremental value can only differ by one bit.
- In gray code, while traversing from one step to another step only one bit in the code group changes. That is to say that two adjacent code numbers differ from each other by only one bit.

How to Convert Binary to Gray Code

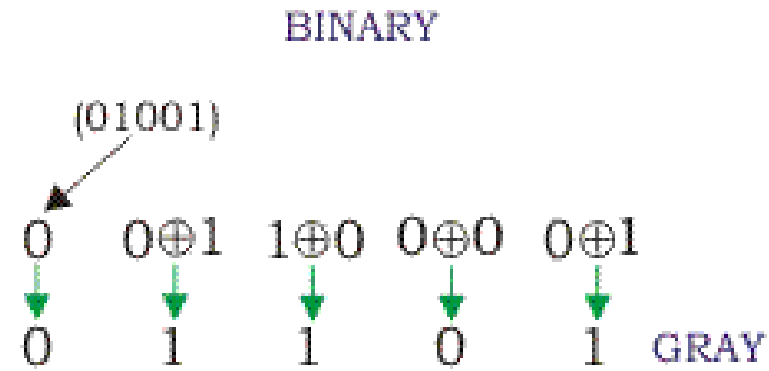
1. The MSB (Most Significant Bit) of the gray code will be exactly equal to the first bit of the given binary number.
2. The second bit of the gray code will be exclusive-or (XOR) of the first and second bit of the given binary number, i.e if both the bits are same the result will be 0 and if they are different the result will be 1.
3. The third bit of gray code will be equal to the exclusive-or (XOR) of the second and third bit of the given binary number. Thus the binary to gray code conversion goes on. An example is given below to illustrate these steps.

Code converter

Binary to Gray code converter

How to Convert Binary to Gray Code

we have a binary number 01001 which we wish to convert to gray code-



Code converter

Binary to Gray code converter

Truth Table

Binary				Gray Code			
b ₃	b ₂	b ₁	b ₀	g ₃	g ₂	g ₁	g ₀
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	0
0	1	0	1	0	1	1	1
0	1	1	0	0	1	0	1
0	1	1	1	0	1	0	0
1	0	0	0	1	1	0	0
1	0	0	1	1	1	0	1
1	0	1	0	1	1	1	1
1	0	1	1	1	1	1	0
1	1	0	0	1	0	1	0
1	1	0	1	1	0	1	1
1	1	1	0	1	0	0	1
1	1	1	1	1	0	0	0

$$g_3 = \sum m (8,9,10,11,12,13,14,15)$$

$$g_2 = \sum m (4,5,6,7,8,9,10,11)$$

$$g_1 = \sum m (2,3,4,5,10,11,12,13)$$

$$g_0 = \sum m (1,2,5,6,9,10,13,14)$$

Code converter

Binary to Gray code converter

K-map Simplification

K-map for g_3 -

		b ₁ ,b ₀			
		00	01	11	10
b ₃ ,b ₂	00	0	0	0	0
	01	0	0	0	0
	11	1	1	1	1
	10	1	1	1	1

$$g_3 = b_3$$

K-map for g_2 -

		b ₁ ,b ₀			
		00	01	11	10
b ₃ ,b ₂	00	0	0	0	0
	01	1	1	1	1
	11	0	0	0	0
	10	1	1	1	1

$$g_2 = b_2b'_3 + b_3b'_2 = b_2 \oplus b_3$$

K-map for g_1 -

		b ₁ ,b ₀			
		00	01	11	10
b ₃ ,b ₂	00	0	0	1	1
	01	1	1	0	0
	11	1	1	0	0
	10	0	0	1	1

$$g_1 = b_2b'_1 + b_1b'_2 = b_1 \oplus b_2$$

K-map for g_0 -

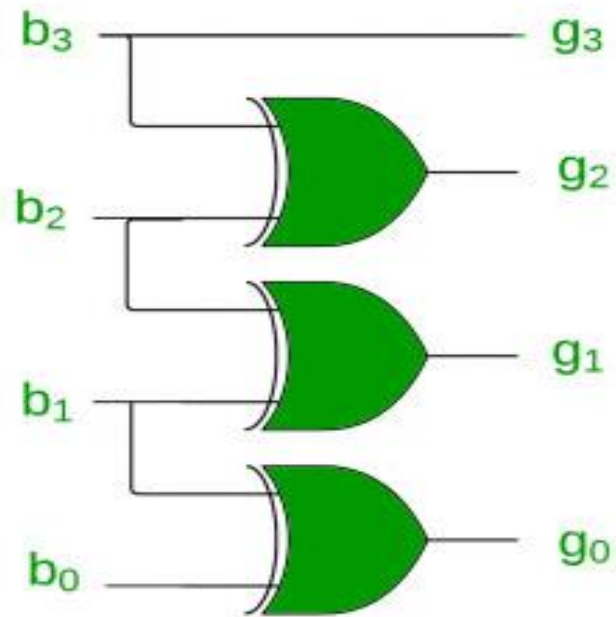
		b ₁ ,b ₀			
		00	01	11	10
b ₃ ,b ₂	00	0	1	0	1
	01	0	1	0	1
	11	0	1	0	1
	10	0	1	0	1

$$g_0 = b_0b'_1 + b_1b'_0 = b_0 \oplus b_1$$

Code converter

Binary to Gray code converter

Logic Circuit-

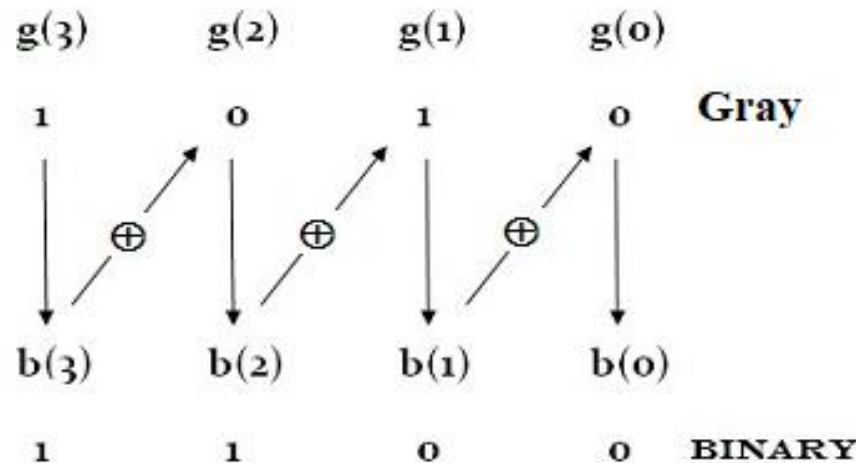


Code Converter

Gray to Binary code converter

How to Convert Gray Code to Binary-

1. The MSB of the binary number will be equal to the MSB of the given gray code.
2. Now if the second gray bit is 0, then the second binary bit will be the same as the previous or the first bit. If the gray bit is 1 the second binary bit will alter. If it was 1 it will be 0 and if it was 0 it will be 1.
3. This step is continued for all the bits to do **Gray code to binary conversion**.



Code Converter

Gray to Binary code converter

Truth Table

Gray Code				Binary			
g ₃	g ₂	g ₁	g ₀	b ₃	b ₂	b ₁	b ₀
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	1
0	1	0	1	0	1	1	0
0	1	1	0	0	1	0	0
0	1	1	1	0	1	0	1
1	0	0	0	1	1	1	1
1	0	0	1	1	1	1	0
1	0	1	0	1	1	0	0
1	0	1	1	1	1	0	1
1	1	0	0	1	0	0	0
1	1	0	1	1	0	0	1
1	1	1	0	1	0	1	1
1	1	1	1	1	0	1	0

$$b_3 = \sum m(8,9,10,11,12,13,14,15)$$

$$b_2 = \sum m(4,5,6,7,8,9,10,11)$$

$$b_1 = \sum m(2,3,4,5,8,9,14,15)$$

$$b_0 = \sum m(1,2,4,7,8,11,13,14)$$

Code Converter

Gray to Binary code converter

K-map Simplification

K-map for b_3 -

		g_1, g_0			
		00	01	11	10
g_3, g_2	00	0	0	0	0
	01	0	0	0	0
	11	1	1	1	1
	10	1	1	1	1

$$b_3 = g_3$$

K-map for b_2 -

		g_1, g_0			
		00	01	11	10
g_3, g_2	00	0	0	0	0
	01	1	1	1	1
	11	0	0	0	0
	10	1	1	1	1

$$b_2 = g'_3 g_2 + g_3 g'_2$$

$$= g_3 \oplus g_2$$

$$b_2 = b_3 \oplus g_2$$

K-map for b_1 -

		g_1, g_0			
		00	01	11	10
g_3, g_2	00	0	0	1	1
	01	1	1	0	0
	11	0	0	1	1
	10	1	1	0	0

$$b_1 = g'_3 g'_2 g_1 + g'_3 g_2 g'_1 + g_3 g_2 g_1 + g_3 g'_2 g'_1$$

$$= g'_3 (g'_2 g_1 + g_2 g'_1) + g_3 (g_2 g_1 + g'_2 g'_1)$$

$$= g'_3 (g_2 \oplus g_1) + g_3 (g_2 \odot g_1)$$

$$= g_3 \oplus g_2 \oplus g_1 = b_2 \oplus g_1$$

Code Converter

Gray to Binary code converter

K-map Simplification

K-map for b_0 –

		g_1, g_0			
		00	01	11	10
g_3, g_2	00	0	1	0	1
	01	1	0	1	0
	11	0	1	0	1
	10	1	0	1	0

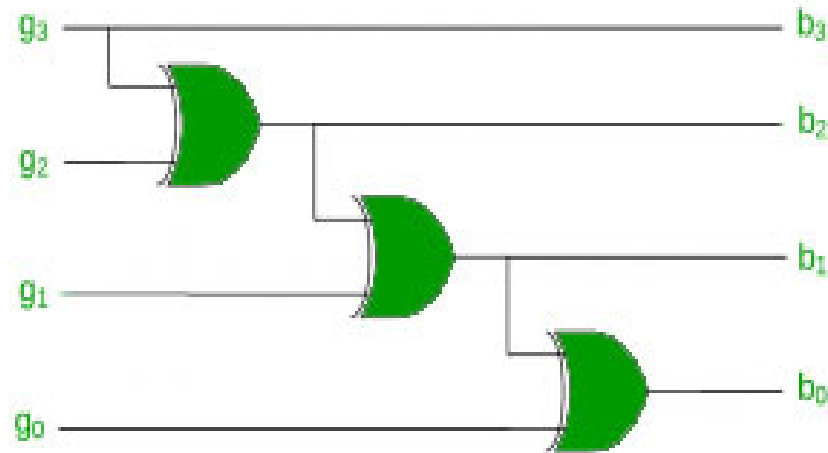
$$\begin{aligned}
 b_0 &= g'_3 g'_2 g'_1 g_0 + g'_3 g'_2 g_1 g'_0 + g'_3 g_2 g'_1 g'_0 + g'_3 g_2 g_1 g_0 + g_3 g'_2 g'_1 g'_0 + g_3 g'_2 g_1 g_0 \\
 &\quad + g_3 g_2 g'_1 g_0 + g_3 g_2 g_1 g'_0 \\
 &= g'_3 g'_2 (g'_1 g_0 + g_1 g'_0) + g'_3 g_2 (g'_1 g'_0 + g_1 g_0) + g_3 g'_2 (g'_1 g'_0 + g_1 g_0) \\
 &\quad + g_3 g_2 (g'_1 g_0 + g_1 g'_0) \\
 &= g'_3 g'_2 (g_0 \oplus g_1) + g'_3 g_2 (g_0 \odot g_1) + g_3 g'_2 (g_0 \odot g_1) + g_3 g_2 (g_0 \oplus g_1) \\
 &= (g_0 \oplus g_1)(g_2 \odot g_3) + (g_0 \odot g_1)(g_2 \oplus g_3) \\
 &= g_3 \oplus g_2 \oplus g_1 \oplus g_0
 \end{aligned}$$

$$b_0 = b_1 \oplus g_0$$

Code converter

Gray to binary code converter

Logic Circuit-



Code converter

BCD to Excess-3 code converter

BCD code

- **Binary Coded Decimal** or **BCD** is simply the 4-bit binary code representation of a decimal digit with each decimal digit replaced in the integer and fractional parts with its binary equivalent. BCD Code uses four bits to represent the 10 decimal digits of 0 to 9.
- To display decimal numbers in the range of 0-to-9, (one digit) we would need 4 data bits (a nibble), decimal numbers in the range of 0-to-99, (two digits) we would need 8 bits (one byte), decimal numbers in the range of 0-to-999, (three digits) we would need 12 bits, and so on

Excess-3 code

- **Excess-3** codes are unweighted and can be obtained by adding 3 to each BCD digit then it can be represented by using 4 bit binary number for each digit.

Code converter

BCD and Binary Representation for Decimal numbers-

1. Find the binary and BCD representation for following numbers-

(a) $(6)_{10}$

$$\text{Binary} = (110)_2$$

$$\text{BCD} = (0110)_{\text{BCD}}$$

(b) $(13)_{10}$

$$\text{Binary} = (1101)_2$$

$$\text{BCD} = (0001\ 0011)_{\text{BCD}}$$

(c) $(28)_{10}$

$$\text{Binary} = (11100)_2$$

$$\text{BCD} = (0010\ 1000)_{\text{BCD}}$$

Code converter

BCD to Excess-3 code converter

Truth Table

BCD(8421)				Excess-3			
A	B	C	D	w	x	y	z
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0	X	X	X	X
1	0	1	1	X	X	X	X
1	1	0	0	X	X	X	X
1	1	0	1	X	X	X	X
1	1	1	0	X	X	X	X
1	1	1	1	X	X	X	X

$$w = \sum m(5,6,7,8,9) + d(10,11,12,13,14,15)$$

$$x = \sum m(1,2,3,4,9) + d(10,11,12,13,14,15)$$

$$y = \sum m(0,3,4,7,8) + d(10,11,12,13,14,15)$$

$$z = \sum m(0,2,4,6,8) + d(10,11,12,13,14,15)$$

Code converter

BCD to Excess-3 code converter

K-map Simplification-

K-map for w

CD \ AB	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

Groupings: BC (blue dashed box), BD (purple dashed box), A (green dashed box)

$$w = A + BC + BD$$

$$w = A + B(C + D)$$

K-map for x

CD \ AB	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

Groupings: $B\bar{C}\bar{D}$ (blue dashed box), BD (purple dashed box), BC (green dashed box)

$$x = B'C + B'D + BC'D'$$

$$x = B'(C + D) + B(C + D)'$$

$$x = B \oplus (C + D)$$

K-map for y

CD \ AB	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

Groupings: $\bar{C}\bar{D}$ (blue dashed box), CD (green dashed box), C (green dashed box)

$$y = CD + C'D'$$

$$y = (C \oplus D)'$$

K-map for z

CD \ AB	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

Groupings: $\bar{D}$ (green dashed box)

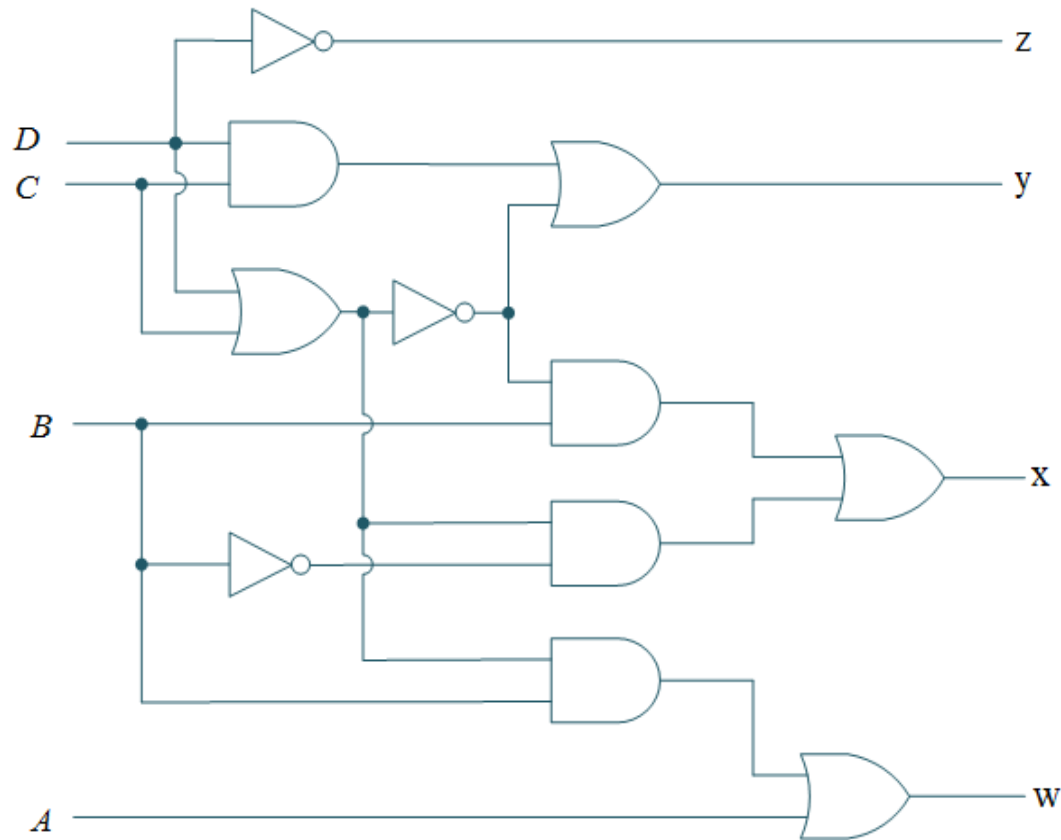
$$z = D'$$

$$z = D'$$

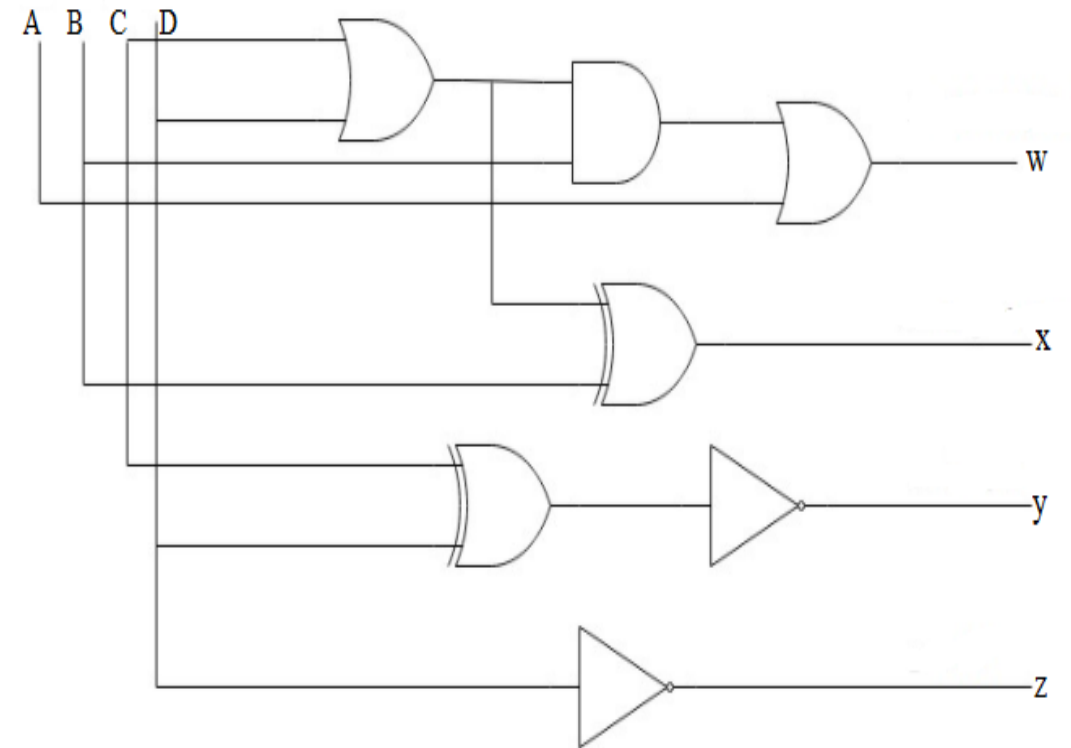
Code converter

BCD to Excess-3 code converter

Logic Circuit-



OR



Code converter

Excess-3 to BCD code converter

Truth Table

Excess-3				BCD			
w	x	y	z	A	B	C	D
0	0	0	0	X	X	X	X
0	0	0	1	X	X	X	X
0	0	1	0	X	X	X	X
0	0	1	1	0	0	0	0
0	1	0	0	0	0	0	1
0	1	0	1	0	0	1	0
0	1	1	0	0	0	1	1
0	1	1	1	0	1	0	0
1	0	0	0	0	1	0	1
1	0	0	1	0	1	1	0
1	0	1	0	0	1	1	1
1	0	1	1	1	0	0	0
1	1	0	0	1	0	0	1
1	1	0	1	X	X	X	X
1	1	1	0	X	X	X	X
1	1	1	1	X	X	X	X

$$A = \sum m(11,12) + d(0,1,2,13,14,15)$$

$$B = \sum m(7,8,9,10) + d(0,1,2,13,14,15)$$

$$C = \sum m(5,6,9,10) + d(0,1,2,13,14,15)$$

$$D = \sum m(4,6,8,10) + d(0,1,2,13,14,15)$$

Code converter

Excess-3 to BCD code converter

K-map Simplification

K-Map for A-

wx \ yz	00	01	11	10
00	X	X	0	X
01	0	0	0	0
11	1	X	X	X
10	0	0	1	0

$$A = wx + wyz$$

$$A = w(x + yz)$$

K-Map for B-

wx \ yz	00	01	11	10
00	X	X	0	X
01	0	0	1	0
11	0	X	X	X
10	1	1	0	1

$$B = x'y' + x'z' + xyz$$

$$B = x'(y' + z') + x(y' + z')$$

$$B = x \oplus (y' + z')$$

K-Map for C-

wx \ yz	00	01	11	10
00	X	X	0	X
01	0	1	0	1
11	0	X	X	X
10	0	1	0	1

$$C = y'z + yz'$$

$$C = y \oplus z$$

K-Map for D-

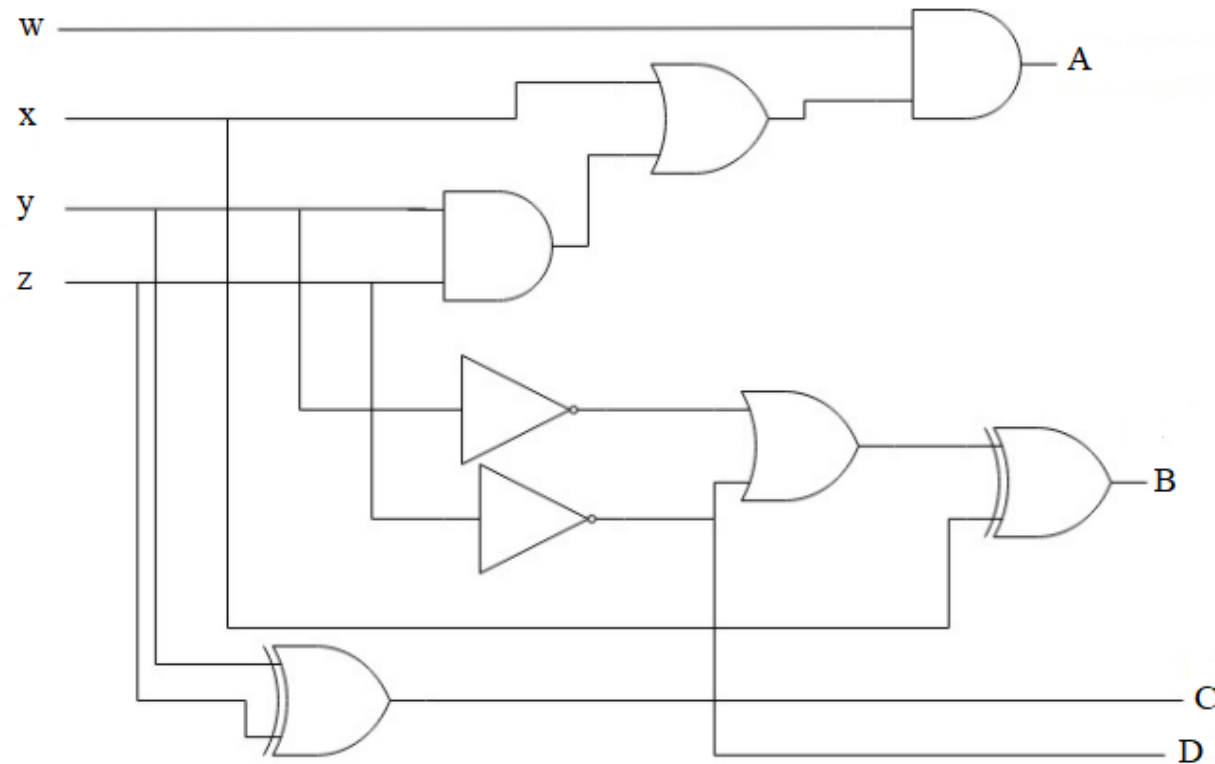
wx \ yz	00	01	11	10
00	X	X	0	X
01	1	0	0	1
11	1	X	X	X
10	1	0	0	1

$$D = z'$$

Code converter

Excess-3 to BCD code converter

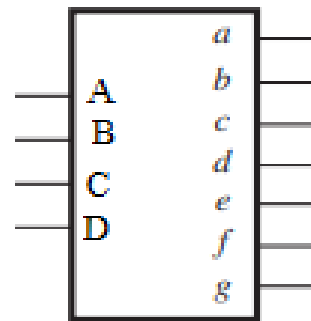
Logic Circuit



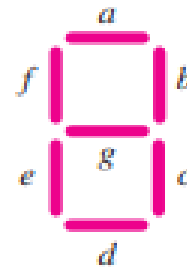
Code converter

BCD to 7-segment Display decoder

- A BCD-to-7-segment decoder converts one binary-coded decimal (BCD) digit into information suitable for driving a digit-oriented display.
- The circuit converts the BCD digit into seven signals that are used to drive the segments in the display.
- Each segment is a small light-emitting diode (LED), which glows when driven by an electrical signal. The segments are labeled from a to g.



(a) Code converter



(b) 7-segment display

Code converter

BCD to 7-segment Display decoder

Truth Table

Decimal Digit	Input lines				Output lines							Display pattern
	A	B	C	D	a	b	c	d	e	f	g	
0	0	0	0	0	1	1	1	1	1	1	0	0
1	0	0	0	1	0	1	1	0	0	0	0	1
2	0	0	1	0	1	1	0	1	1	0	1	2
3	0	0	1	1	1	1	1	1	0	0	1	3
4	0	1	0	0	0	1	1	0	0	1	1	4
5	0	1	0	1	1	0	1	1	0	1	1	5
6	0	1	1	0	1	0	1	1	1	1	1	6
7	0	1	1	1	1	1	1	0	0	0	0	7
8	1	0	0	0	1	1	1	1	1	1	1	8
9	1	0	0	1	1	1	1	1	0	1	1	9

$$a = \sum m(0,2,3,5,6,7,8,9) + d(10,11,12,13,14,15)$$

$$b = \sum m(0,1,2,3,4,7,8,9) + d(10,11,12,13,14,15)$$

$$c = \sum m(0,1,3,4,5,6,7,8,9) + d(10,11,12,13,14,15)$$

$$d = \sum m(0,2,3,5,6,8,9) + d(10,11,12,13,14,15)$$

$$e = \sum m(0,2,6,8) + d(10,11,12,13,14,15)$$

$$f = \sum m(0,4,5,6,8,9) + d(10,11,12,13,14,15)$$

$$g = \sum m(2,3,4,5,6,8,9) + d(10,11,12,13,14,15)$$

Code converter

BCD to 7-segment Display decoder

K-map Simplification-

K-map for a

AB \ CD	00	01	11	10
00	1	0	1	1
01	0	1	1	1
11	x	x	x	x
10	1	1	x	x

$$a = A + C + BD + \bar{B}\bar{D}$$

K-map for b

AB \ CD	00	01	11	10
00	1	1	1	1
01	1	0	1	0
11	x	x	x	x
10	1	1	x	x

$$b = \bar{B} + \bar{C}\bar{D} + CD$$

K-map for c

AB \ CD	00	01	11	10
00	1	1	1	0
01	1	1	1	1
11	x	x	x	x
10	1	1	x	x

$$c = B + \bar{C} + D$$

K-map for d

AB \ CD	00	01	11	10
00	1	0	1	1
01	0	1	0	1
11	x	x	x	x
10	1	1	x	x

$$d = \bar{B}\bar{D} + C\bar{D} + B\bar{C}D + \bar{B}C + A$$

K-map for e

AB \ CD	00	01	11	10
00	1	0	0	1
01	0	0	0	1
11	x	x	x	x
10	1	0	x	x

$$e = \bar{B}\bar{D} + C\bar{D}$$

K-map for f

AB \ CD	00	01	11	10
00	1	0	0	0
01	1	1	0	1
11	x	x	x	x
10	1	1	x	x

$$f = A + \bar{C}\bar{D} + B\bar{C} + B\bar{D}$$

K-map for g

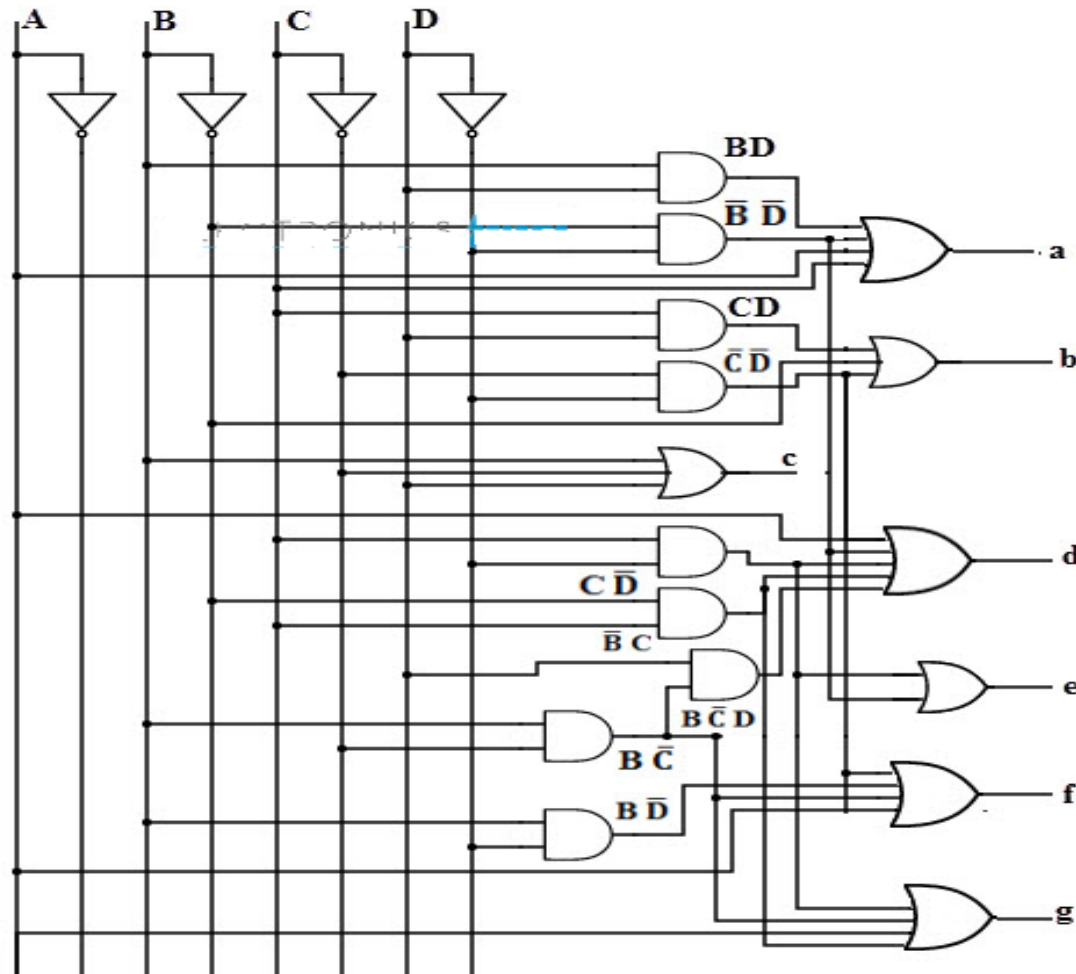
AB \ CD	00	01	11	10
00	0	0	1	1
01	1	1	0	1
11	x	x	x	x
10	1	1	x	x

$$g = A + B\bar{C} + \bar{B}C + C\bar{D}$$

Code converter

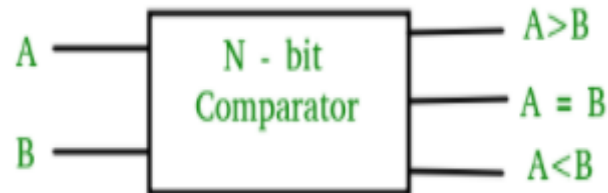
BCD to 7-segment Display decoder

Logic Circuit



Arithmetic Comparison Circuit (Magnitude Comparator)

- An arithmetic comparison circuit or a magnitude Comparator is a combinational circuit that **compares two digital or binary numbers** in order to find out whether one binary number is equal, less than or greater than the other binary number.
- We logically design a circuit for which we will have two inputs one for A and other for B and have three output terminals, one for $A > B$ condition, one for $A = B$ condition and one for $A < B$ condition.



Arithmetic Comparison Circuit (Magnitude Comparator)

1-bit Magnitude Comparator

- A comparator used to compare two bits is called a single bit comparator. It consists of two inputs each for two single bit numbers and three outputs to generate less than, equal to and greater than between two binary numbers.

Truth Table

Inputs		Outputs		
A	B	A<B	A=B	A>B
0	0	0	1	0
0	1	1	0	0
1	0	0	0	1
1	1	0	1	0

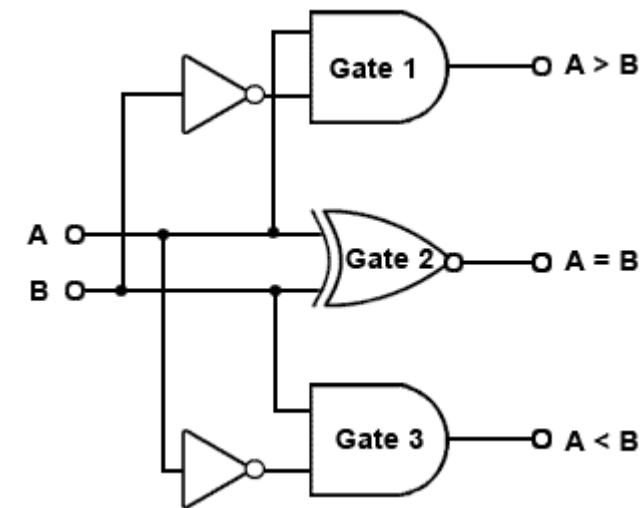
Logical expressions

$$A < B = A'B$$

$$A = B = A'B' + AB = A \odot B$$

$$A > B = AB'$$

Logic circuit

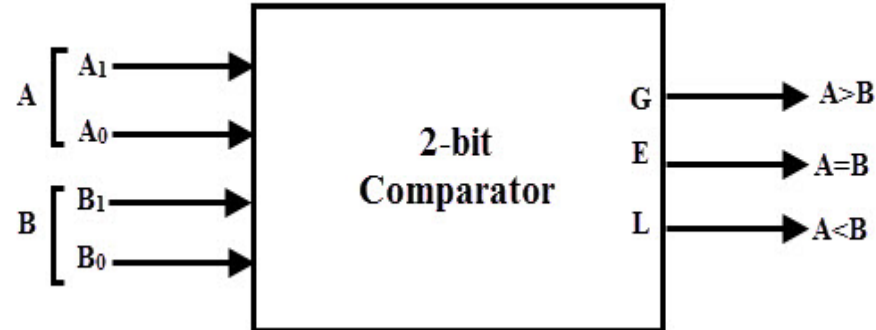


Arithmetic Comparison Circuit (Magnitude Comparator)

2-bit Magnitude Comparator

- A 2-bit comparator compares two binary numbers, each of two bits and produces their relation such as one number is equal or greater than or less than the other.

Block diagram



Arithmetic Comparison Circuit (Magnitude Comparator)

2-bit Magnitude Comparator

Truth table

Inputs				Outputs		
A ₁	A ₀	B ₁	B ₀	A>B	A=B	A<B
0	0	0	0	0	1	0
0	0	0	1	0	0	1
0	0	1	0	0	0	1
0	0	1	1	0	0	1
0	1	0	0	1	0	0
0	1	0	1	0	1	0
0	1	1	0	0	0	1
0	1	1	1	0	0	1
1	0	0	0	1	0	0
1	0	0	1	1	0	0
1	0	1	0	0	1	0
1	0	1	1	0	0	1
1	1	0	0	1	0	0
1	1	0	1	1	0	0
1	1	1	0	1	0	0
1	1	1	1	0	1	0

K-map Simplification

		B ₁ B ₀			
		A>B			
A ₁ A ₀	00	00	01	11	10
	00	0	0	0	0
	01	1	0	0	0
	11	1	1	0	1
	00	1	1	0	0

$$A>B = A_0 \overline{B_1} \overline{B_0} + A_1 \overline{B_1} + A_1 A_0 \overline{B_0}$$

Arithmetic Comparison Circuit (Magnitude Comparator)

2-bit Magnitude Comparator

K-map Simplification

		B1B0			
		00	01	11	10
A1A0	00	1	0	0	0
	01	0	1	0	0
	11	0	0	1	0
	00	0	0	0	1

$$A = B = \overline{A_1} \overline{A_0} \overline{B_1} \overline{B_0} + \overline{A_1} A_0 \overline{B_1} B_0 + A_1 A_0 B_1 B_0 + A_1 \overline{A_0} B_1 \overline{B_0}$$

$$= \overline{A_1} \overline{B_1} (\overline{A_0} \overline{B_0} + A_0 B_0) + A_1 B_1 (A_0 B_0 + \overline{A_0} \overline{B_0})$$

$$= (A_0 B_0 + \overline{A_0} \overline{B_0}) (A_1 B_1 + \overline{A_1} \overline{B_1})$$

$$= (A_0 \text{ Ex-NOR } B_0) (A_1 \text{ Ex-NOR } B_1)$$

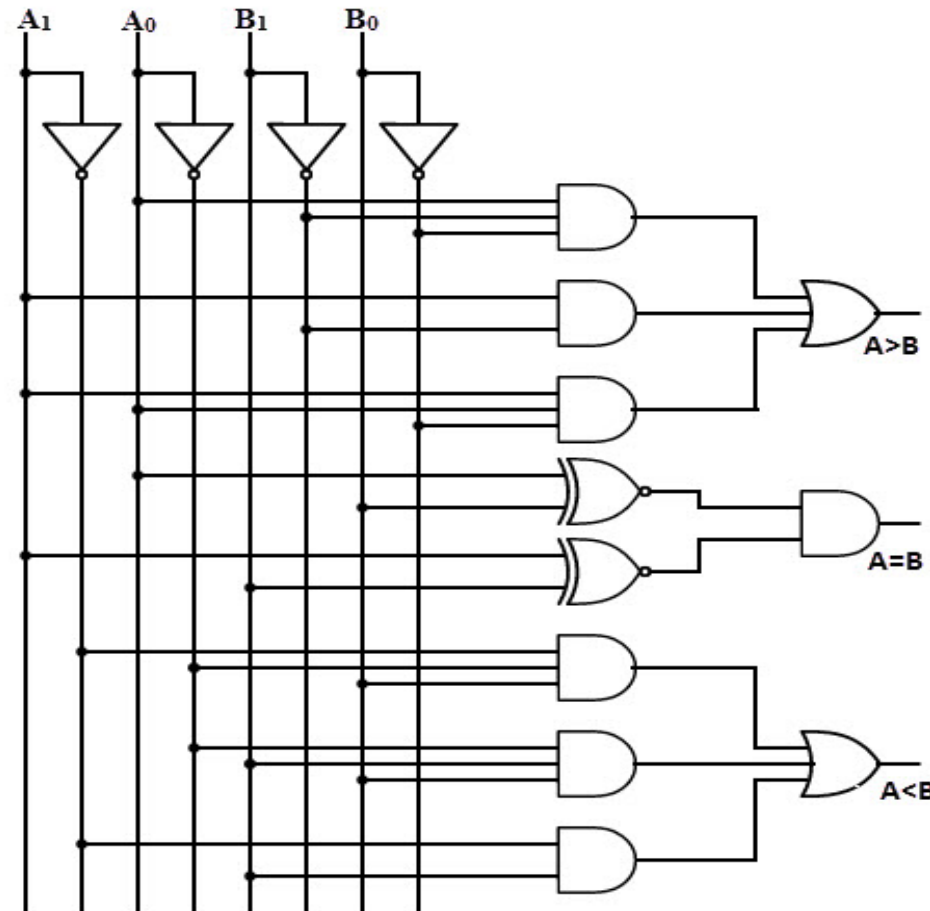
		B1B0			
		00	01	11	10
A1A0	00	0	1	1	1
	01	0	0	1	1
	11	0	0	0	0
	00	0	0	1	0

$$A < B = \overline{A_1} B_1 + \overline{A_0} B_1 B_0 + \overline{A_1} \overline{A_0} B_0$$

Arithmetic Comparison Circuit (Magnitude Comparator)

2-bit Magnitude Comparator

Logic Circuit



Arithmetic Comparison Circuit (Magnitude Comparator)

4-bit Magnitude Comparator

- A comparator used to compare two binary numbers each of four bits is called a 4-bit magnitude comparator. It consists of eight inputs each for two four bit numbers and three outputs to generate less than, equal to and greater than between two binary numbers.

$$\text{Let } A = a_3a_2a_1a_0 \quad \text{and} \quad B = b_3b_2b_1b_0$$

- Define a set of intermediate signals called i_3 , i_2 , i_1 , and i_0 . Each signal, i_k , is 1 if the bits of A and B with the same index are equal. That is,

$$i_k = \overline{a_k \oplus b_k}$$

The comparator's AeqB output is then given by

$$\boxed{AeqB = i_3i_2i_1i_0}$$

Arithmetic Comparison Circuit (Magnitude Comparator)

4-bit Magnitude Comparator

- An expression for the $AgtB$ output can be derived by considering the bits of A and B in the order from the most-significant bit to the least-significant bit.
- In a 4-bit comparator the condition of $A > B$ can be possible in the following four cases:
 1. If $a_3 = 1$ and $b_3 = 0$
 2. If $a_3 = b_3$ and $a_2 = 1$ and $b_2 = 0$
 3. If $a_3 = b_3$, $a_2 = b_2$ and $a_1 = 1$ and $b_1 = 0$
 4. If $a_3 = b_3$, $a_2 = b_2$, $a_1 = b_1$ and $a_0 = 1$ and $b_0 = 0$

So the expression for $A > B$ is-

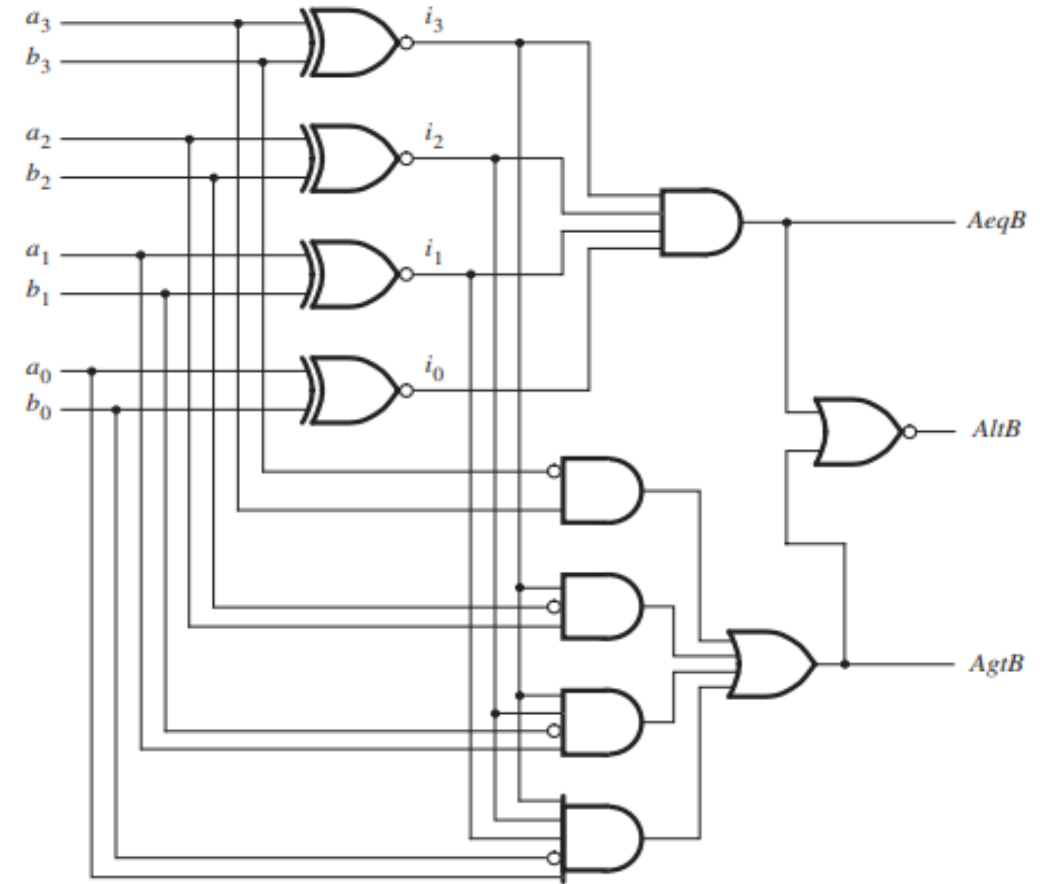
$$AgtB = a_3b_3' + i_3a_2b_2' + i_3i_2a_1b_1' + i_3i_2i_1a_0b_0'$$

Arithmetic Comparison Circuit (Magnitude Comparator)

4-bit Magnitude Comparator

- The $A \neq B$ output can be derived by using the other two outputs as

$$A \neq B = \overline{A = B} + A > B$$
- This approach can be used to design a comparator for any value of n .



A four-bit comparator circuit.

Tutorial

1. Design a 4-to-16 Decoder using 3-to-8 decoders and required logic gates.
2. Implement a minimum cost circuit for the following functions using a decoder and required logic gates.

$$f1(x, y, z) = \Pi M (2,3,4,5,7)$$

$$f2(x, y, z) = \Pi M (0,2,4,6,7)$$

(Hint: Choose decoder with active high outputs or active low outputs according to cost)

3. Design a 3-bit Binary to Gray code converter using a decoder with active high outputs and required logic gates.
4. Design an Excess-3 to BCD code converter using a 4-to-16 decoder with an enable input E' and construct using NAND gates, so as to minimize the gate inputs.
5. Design and implement a full subtractor using a decoder with active high outputs and NOR gates.
6. Design a 2-bit magnitude comparator using 2-bit Adder/Subtractor circuit and obtain the expressions for all possible outputs of comparator. Use 2's complement method.

Digital Electronics & Systems

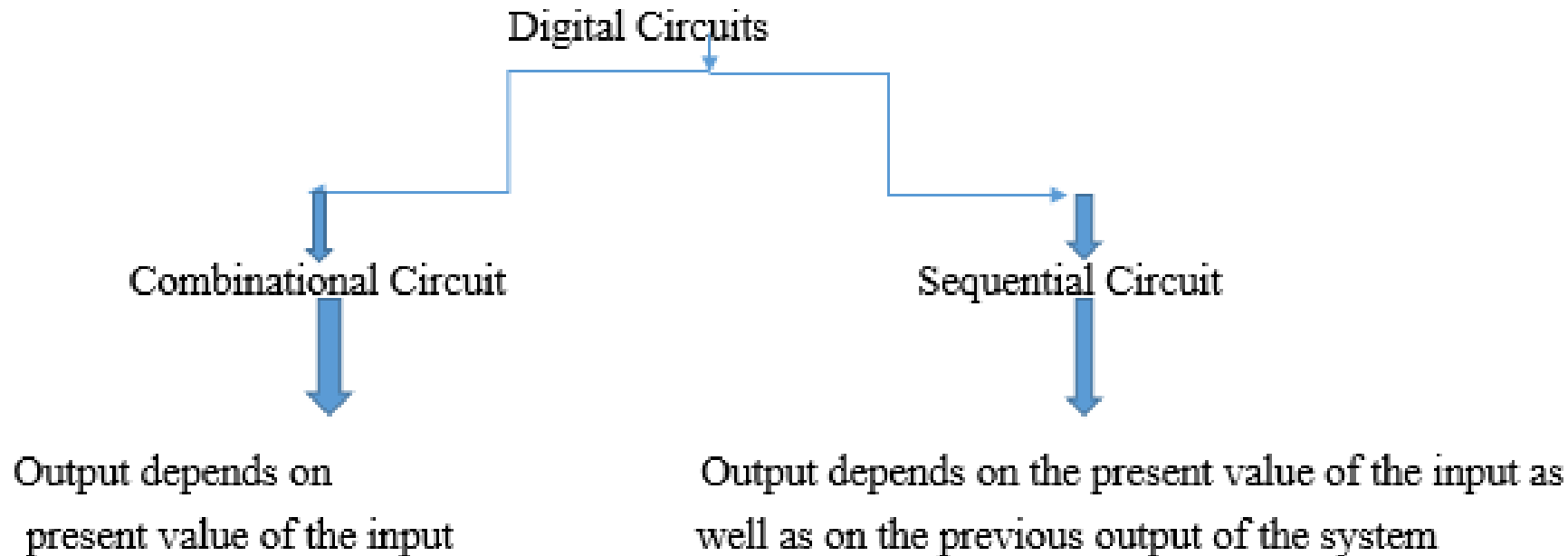
19ECE204

Lecture 45-50

Latches and Flip Flops

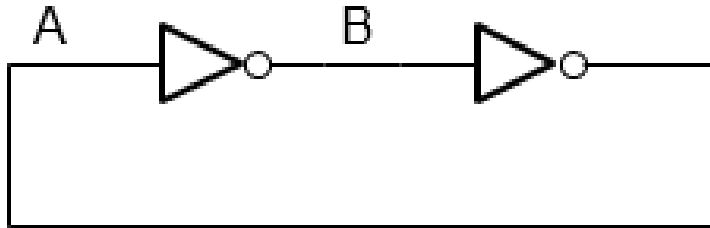
Ms. Bhavana V
Assistant Professor (Sr.Gr.)
Dept. of ECE
ASE, Bengaluru

Combinational Circuit and Sequential Circuit



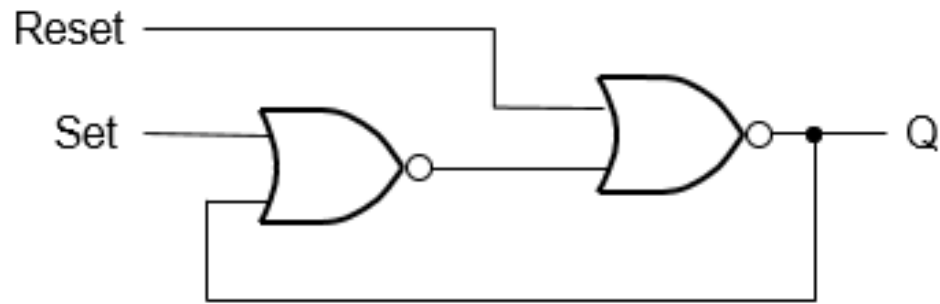
Sequential circuits include storage elements that store the values of the previous state of the system. The contents of the storage elements are said to represent the *state* of the circuit.

A Simple Memory Element



If we assume that $A = 0$, then $B = 1$. The circuit will maintain these values indefinitely. If we assume that $A = 1$, then $B = 0$, and the circuit will remain in this second state indefinitely. Thus, the circuit has two possible states. This circuit is not useful, because it lacks some practical means for changing its state.

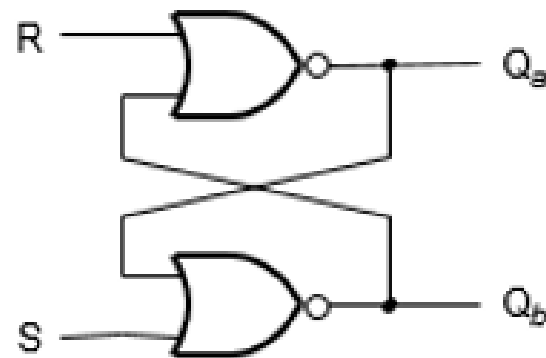
A Memory Element with NOR gates



Set (S)	Reset (R)	Q	Remarks
0	0	Q	Previous State
0	1	0	Reset State
1	0	1	Set State
1	1	0	But this is invalid state which will be discussed soon

A Latch built with NOR gates

Required: Q_a and Q_b are complement of each other



(a) Circuit

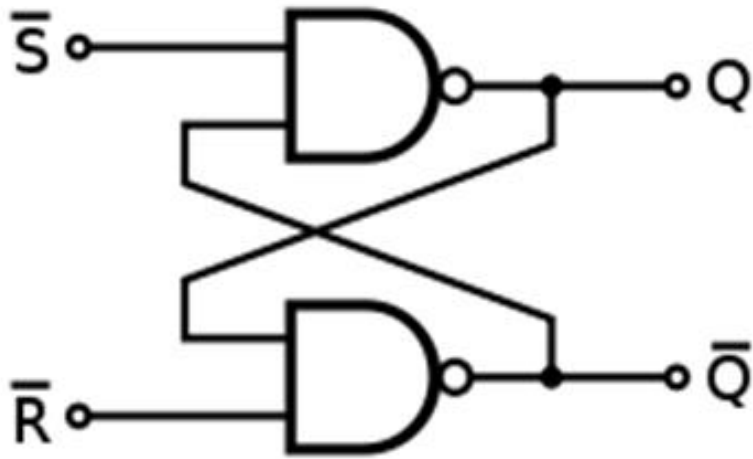
S	R	Q_a	Q_b	
0	0	0/1	1/0	(no change)
0	1	0	1	➡ Reset State
1	0	1	0	➡ Set State
1	1	0	0	➡ Invalid

(b) Truth table

An SR Latch built with NAND gates

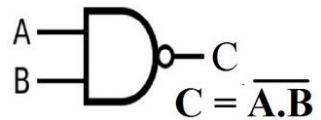
The next state Q can be written as either $Q(n+1)$ or $Q(t+1)$ or $Q+$
 The next state Q' can be written as either $Q'(n+1)$ or $Q'(t+1)$ or $Q'+$

} different authors follow different notations



S	R	S'	R'	Q(t+1)	Q' (t+1)	Remarks
0	0	1	1	Q(t)	Q'(t)	Same Previous value
0	1	1	0	0	1	Reset State
1	0	0	1	1	0	Set State
1	1	0	0	1	1	Not Allowed

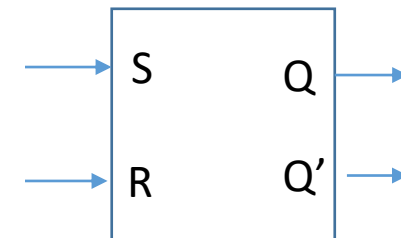
NAND GATE



Truth Table

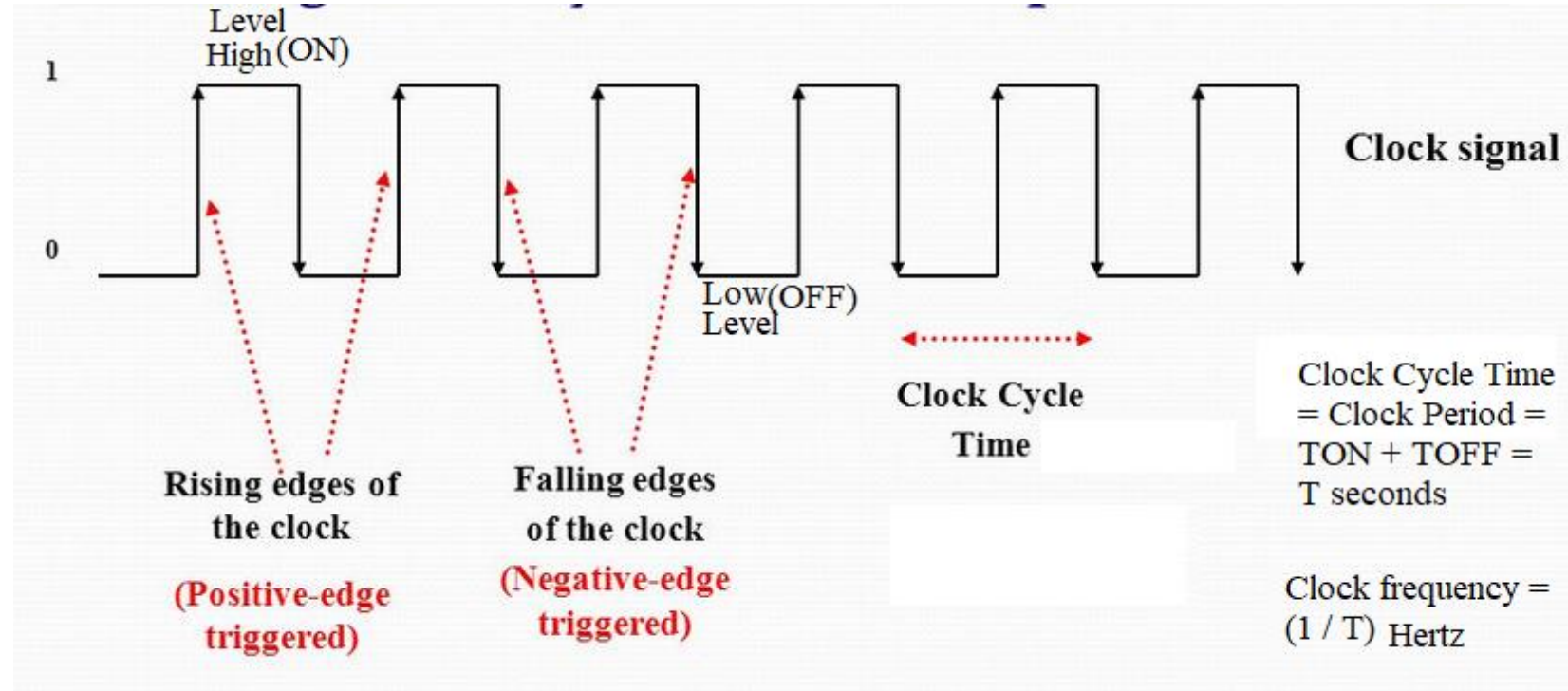
INPUT		OUTPUT
A	B	A NAND B
0	0	1
0	1	1
1	0	1
1	1	0

Symbol of SR Latch

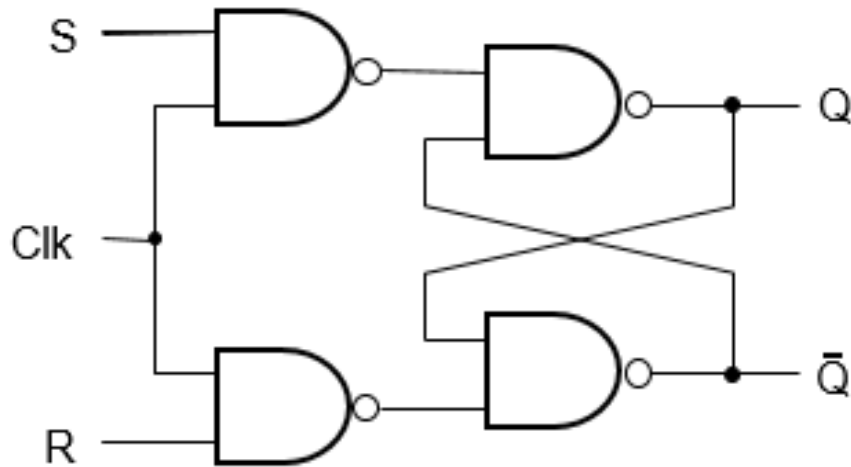


Clock

- A clock is that which continuously alternates between 0 and 1. i.e. it is continuous train of pulses
- The time it takes the clock to change from 1 to 0 and back to 1 (or vice versa) is called the clock period or clock cycle time.
- i.e. Cycle time of a clock is the sum of its high duration and its low duration. The length of the time the clock is high before changing states is its high duration and the length of the time the clock is low before changing states is its low duration
- The clock frequency is the inverse of the clock period. The unit of measurement for frequency is the Hertz
- Clocks are often used to synchronize circuits

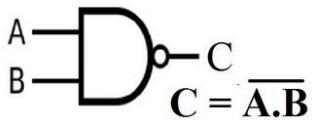


Gated SR Latch with NAND gates



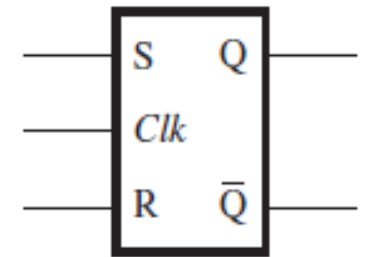
Clk	S	R	Q(t+1)	Q'(t+1)	Remarks
0	x	x	Q(t)	Q'(t)	No change. Same as that of the previous state
1	0	0	Q(t)	Q'(t)	No change. Same as that of the previous state
1	0	1	0	1	Reset state
1	1	0	1	0	Set state
1	1	1	1	1	Invalid

NAND GATE



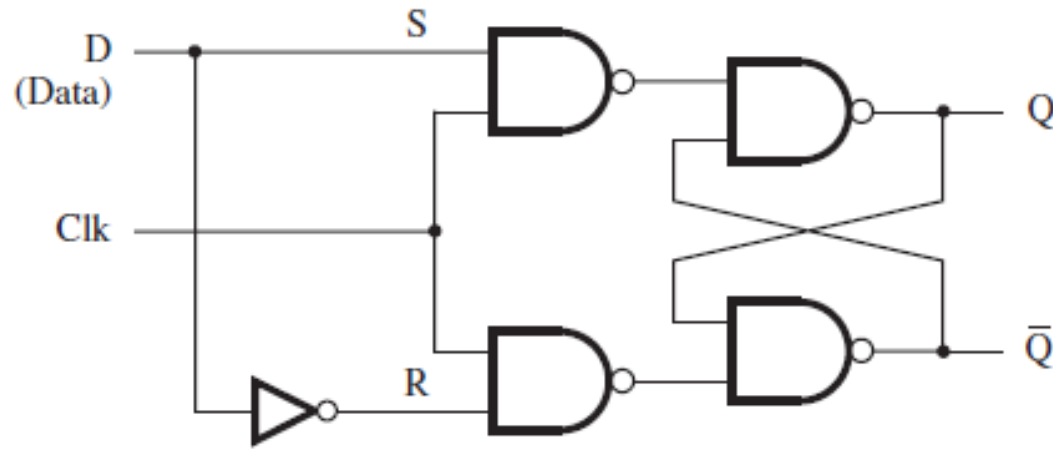
Truth Table

INPUT		OUTPUT
A	B	A NAND B
0	0	1
0	1	1
1	0	1
1	1	0



(d) Graphical symbol

Gated D (Data) Latch

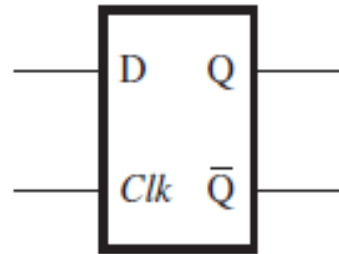


(a) Circuit

Clk	D	Q	Q'	Remarks
0	x	Q	Q'	No change. Same as that of the previous state
1	0	0	1	Q will be same as that of D input
1	1	1	0	Q will be same as that of D input

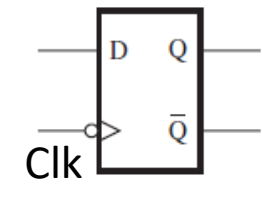
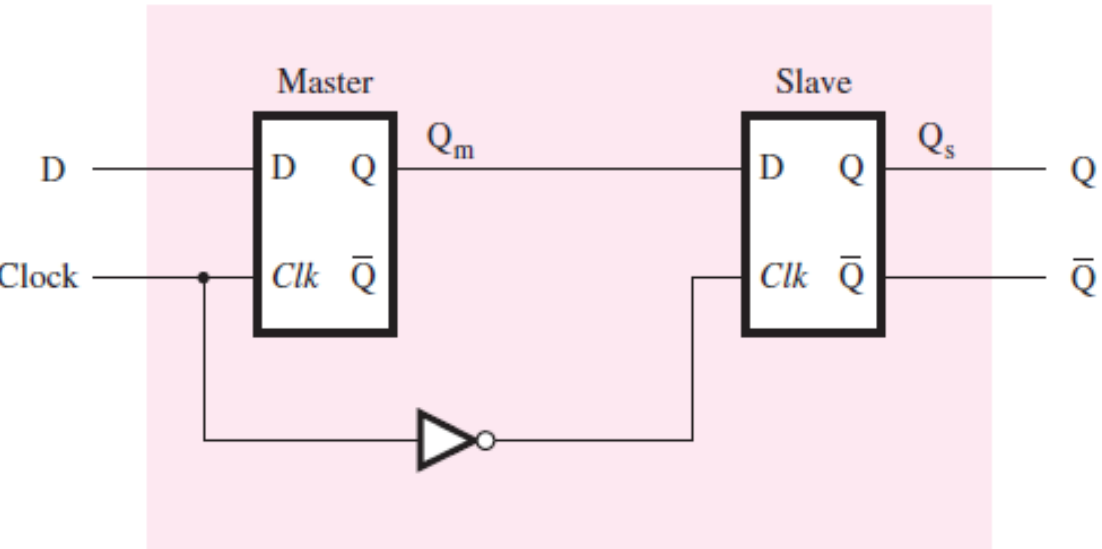
Clk	D	Q(t + 1)
0	x	Q(t)
1	0	0
1	1	1

(b) Truth table



(c) Graphical symbol

Master Slave D Flip Flop (Negative Edge Triggered D Flip Flop)

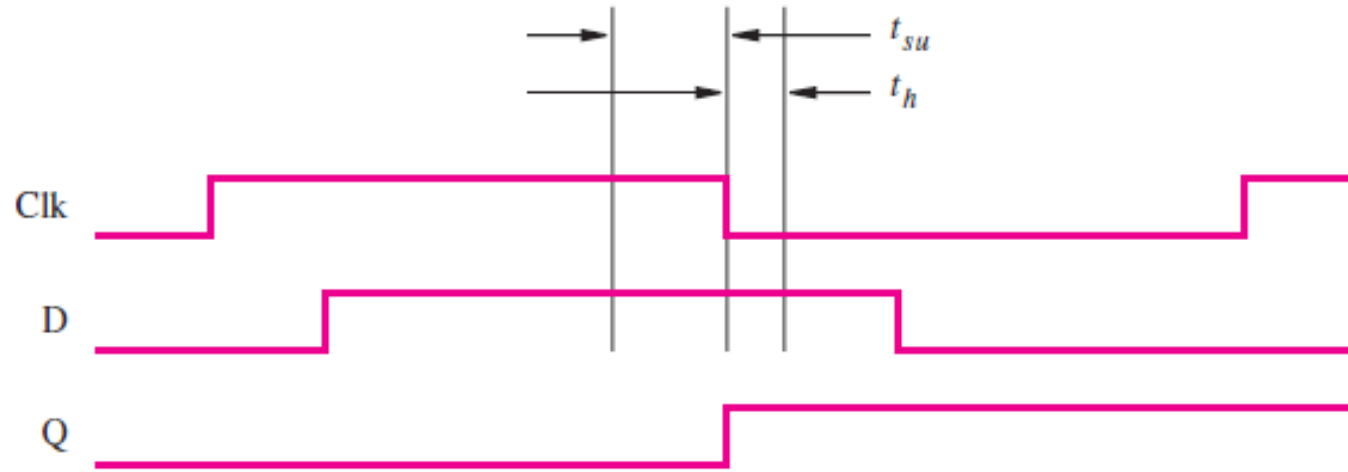


(c) Graphical symbol

Truth Table in short		
Clk	D (Data Input)	Q
	D	Q = D
Any other value on clock	x	Q = same as that of Previous value

Truth Table in a descriptive manner					
Clk		D	Qm	Qs = Q	Remarks
1	-ve edge on Clk	1	1	Previous output = Q	At negative edge, whatever is value of D data input the same value will appear on Q
0		x	1	1	
1	-ve edge on Clk	0	0	Previous output = Q	At negative edge, whatever is value of D data input the same value will appear on Q
0		x	0	0	
x		x	Qm	Q	At any other value other than negative edge trigger on the clock, the same previous value of Q will continue on Q

Set Up Time and Hold Time



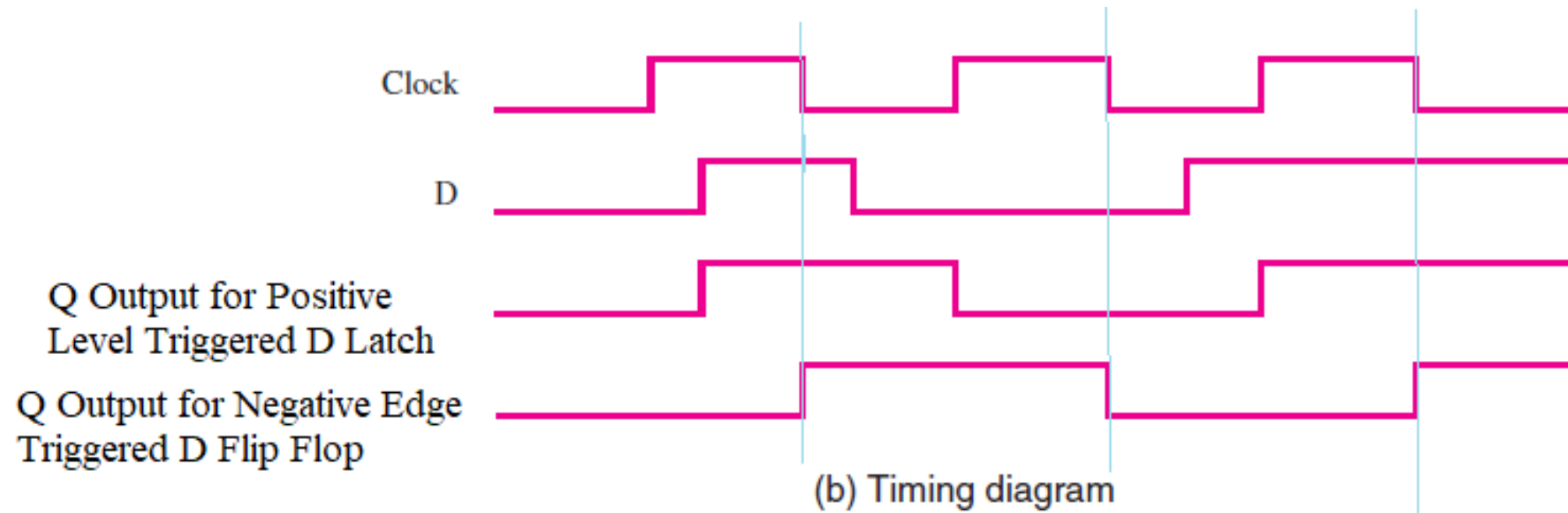
Set Up Time (t_{su}): The minimum time that the D signal must be stable prior to the negative edge (or positive edge based on what type of flipflop) of the Clk signal is called the *setup time*, t_{su} , of the flip flop.

Hold Time (t_h): The minimum time that the D signal must remain stable after the negative edge (or positive edge based on what type of flipflop) of the Clk signal is called the *hold time*, t_h , of the flip flop.

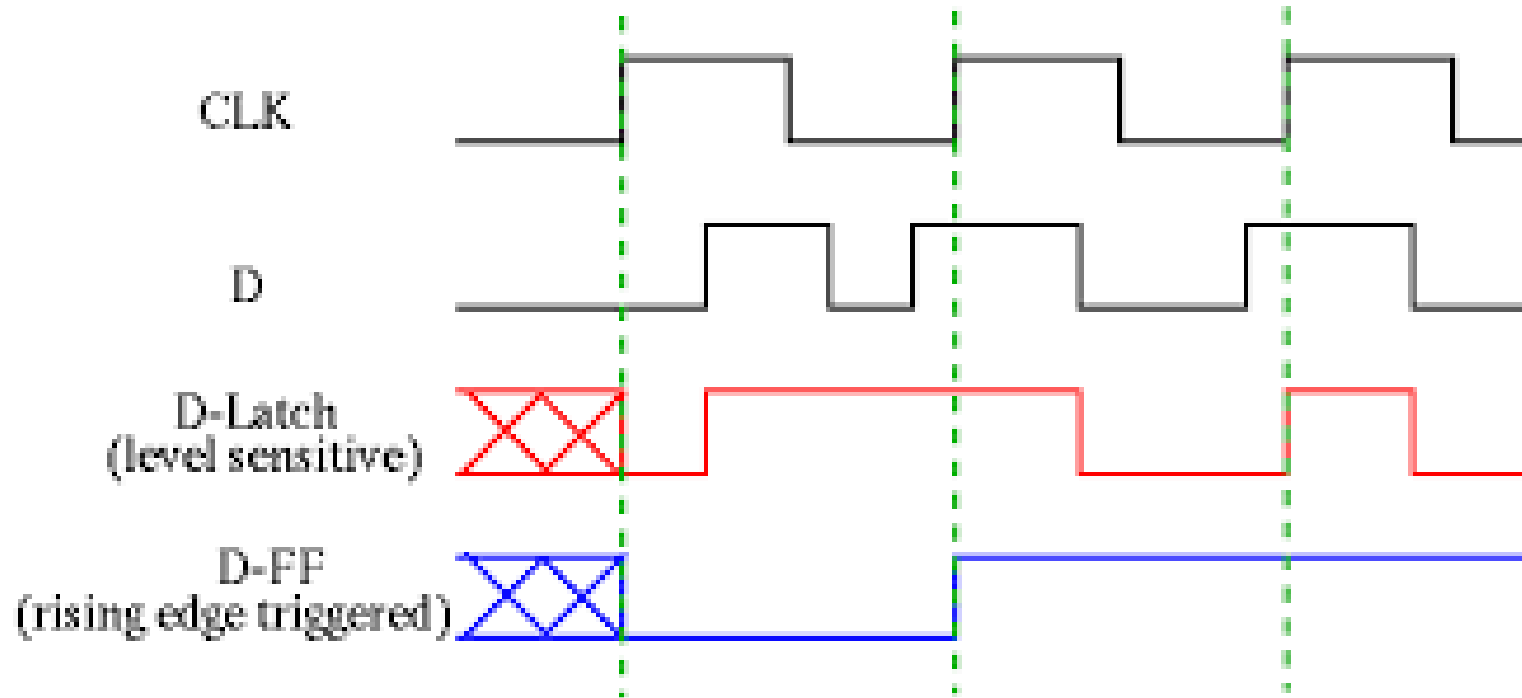
Typical values for $t_{su} = 3$ ns and $t_h = 2$ ns.

If Set up time and Hold time are not adhered then the circuit will enter in Metastable state.

Timing Diagram Showing Difference between D Latch and D Flip Flop Working

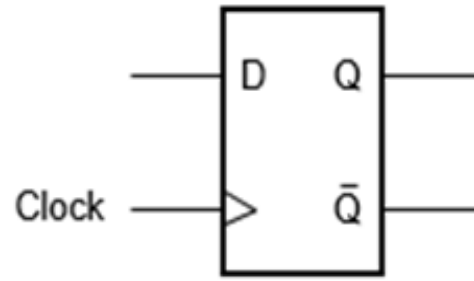


Analyze this Waveform: Exercise

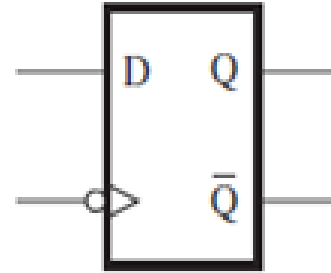


Graphic symbols

Positive Edge Triggered D Flip Flop

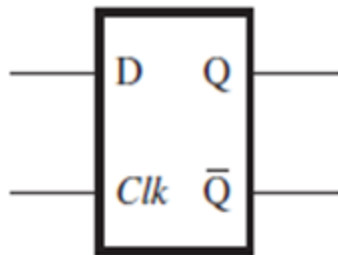


Negative Edge Triggered D Flip Flop

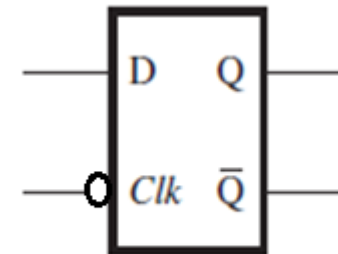


Flip flop is a edge triggered one bit storage device.

Positive Level Triggered D Latch



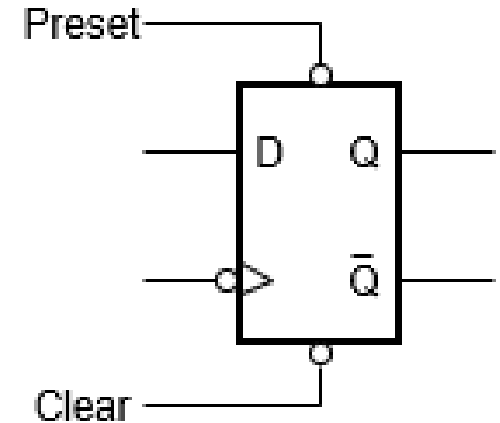
Negative Level Triggered D Latch



Latch is a level triggered one bit storage device.

Master-Slave D flip-flop with Clear and Preset

Preset	Clear	D Input	Q Output	Remarks
0	0	Don't Care	Don't Care	Invalid
0	1	Don't Care	1	Preset Condition
1	0	Don't Care	0	Clear or Reset Condition
1	1	Whatever Valid value on D Input	Q = D Valid Output on Q	Regular or Normal Functionality of D Flip flop begins

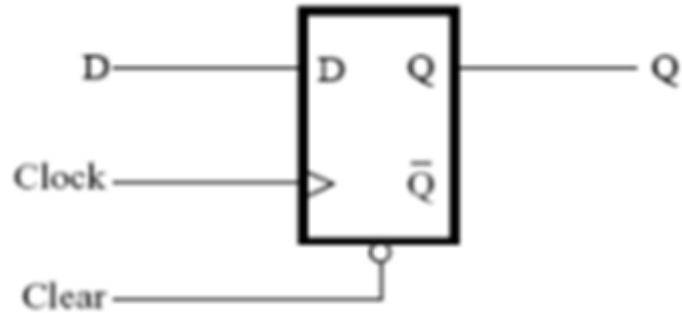


(b) Graphical symbol

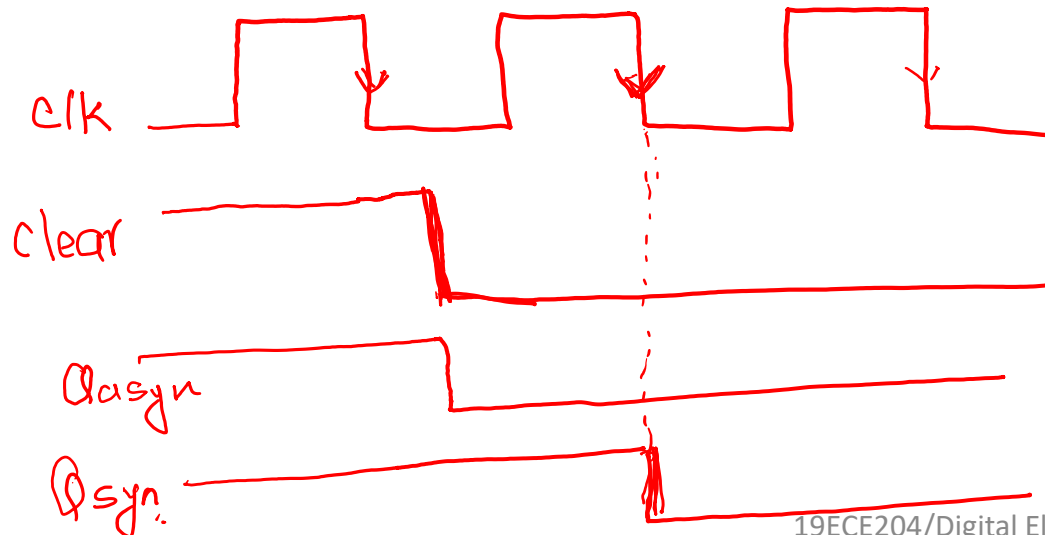
We have assumed Low Level Active Preset and Clear here.

* Practically, these may be Low level or High level active

Asynchronous Clear

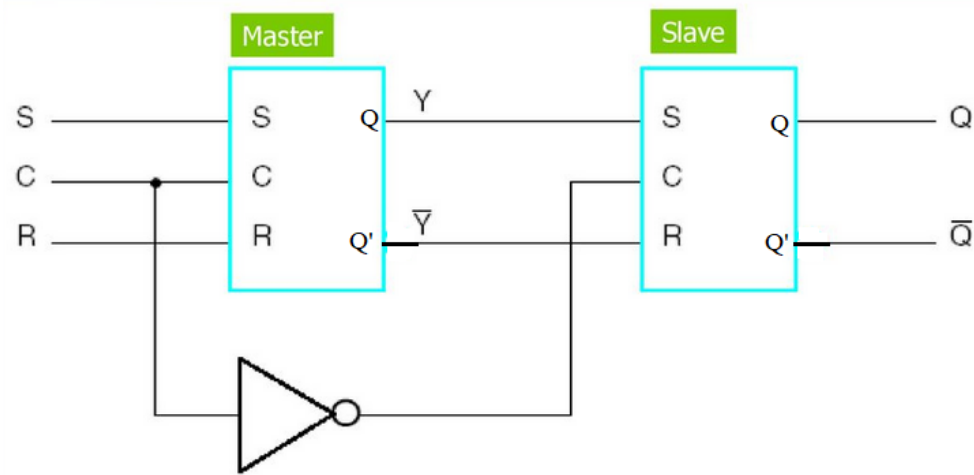


(a) D flip-flop with asynchronous clear

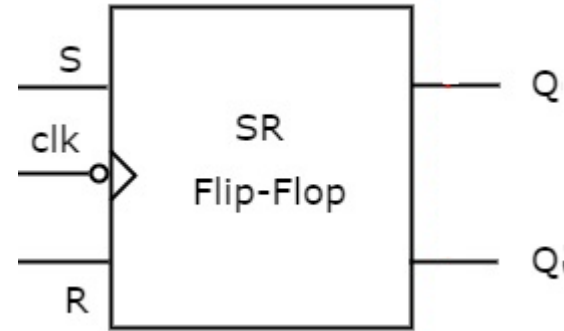


- If the Clear Signal is considered to be Asynchronous signal, then that means the Clear signal will not wait for clock (positive edge as per the diagram here), and Q will be made zero (0) immediately without waiting for the clock's required edge. That is Clear signal is not synchronized with Clock signal.
- If the Clear Signal is considered to be Synchronous signal, then that means the Clear signal will wait for clock till immediate positive edge (positive edge as per the diagram here) occurs on the clock and then Q will be made zero (0). That is Clear signal is synchronized with Clock signal.

Master Slave SR Flip Flop



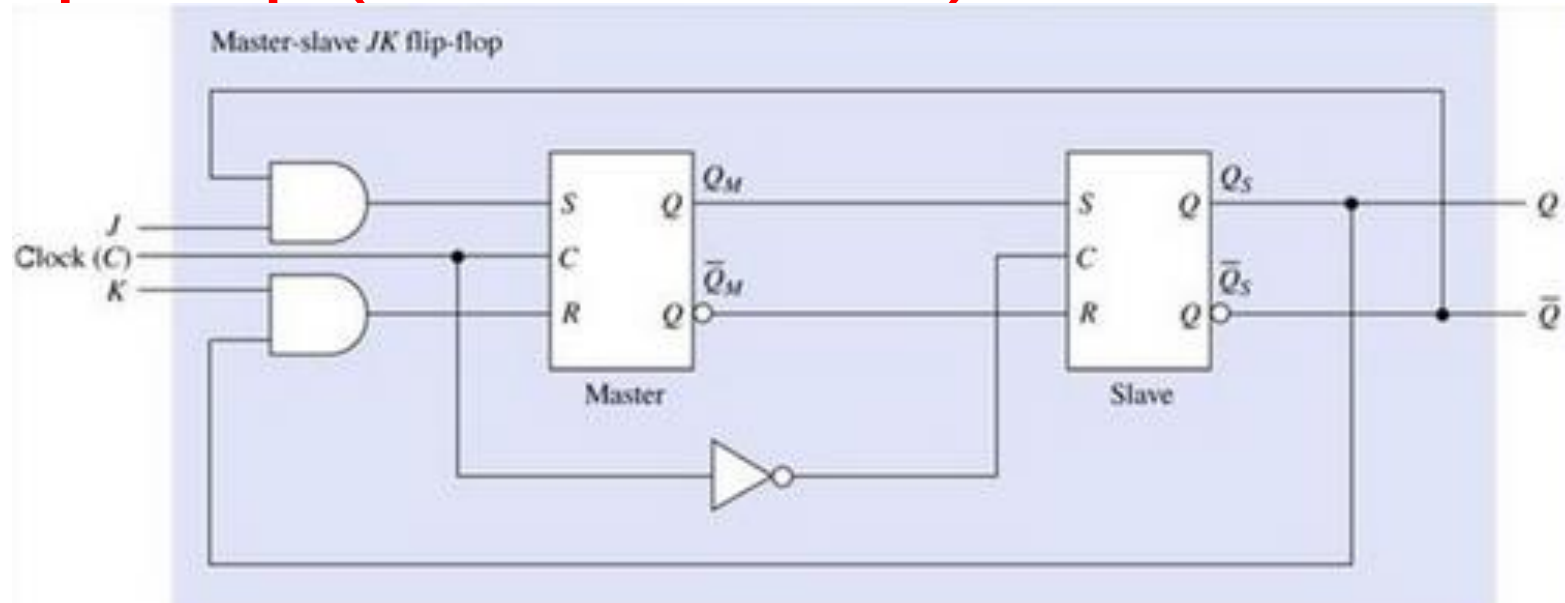
C = Clock = Clk



Clk	S	R	Q ⁺	Q' ⁺
0	x	x	Q	Q'
1	0	0	Q	Q
1	0	1	0	1
1	1	0	1	0
1	1	1	1	1

C = Clk = Clock	S	R	Q ⁺	Q' ⁺	Remarks
Negative edge on C	0	0	Q	Q'	Same as that of the previous value
Negative edge on C	0	1	0	1	Reset state
Negative edge on C	1	0	1	0	Set state
Negative edge on C	1	1	1	1	Invalid or Undefined
Any other value on C other than negative edge on the C	x	x	Q	Q'	Same as that of the previous value

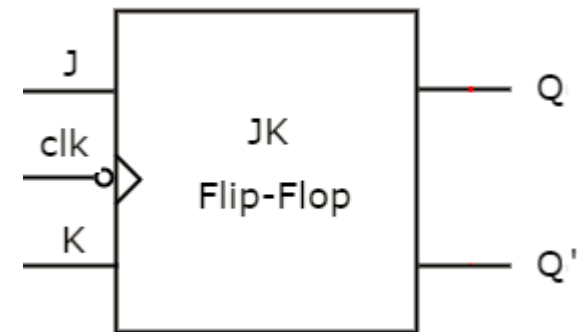
J K Flip-flop (Master Slave)



Clock = C = Clk

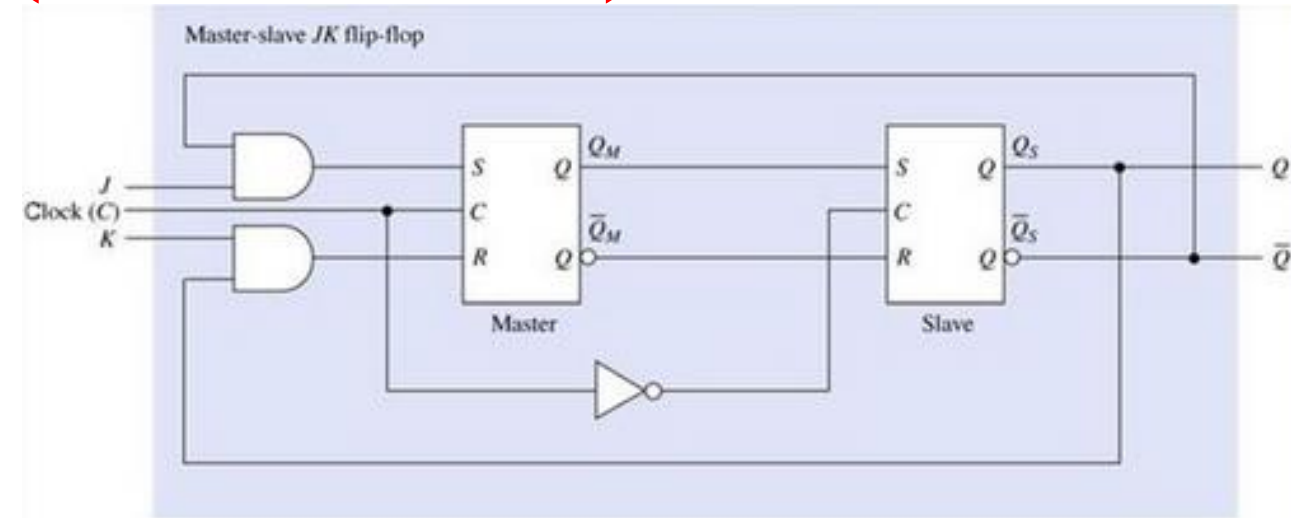
Clk	S	R	Q^+	Q'^+
0	x	x	Q	Q'
1	0	0	Q	Q
1	0	1	0	1
1	1	0	1	0
1	1	1	1	1

Truth Table of JK Flip Flop					
Clk	J	K	Q^+	Q'^+	Remarks
Negative edge on Clk	0	0	Q	Q'	Same as that of the previous value
Negative edge on Clk	0	1	0	1	
Negative edge on Clk	1	0	1	0	
Negative edge on Clk	1	1	Q'	Q	Invert of the previous value
Any other value on clock other than negative edge on clk	x	x	Q	Q'	Same as that of the previous value



J K Flip-flop (Master Slave)

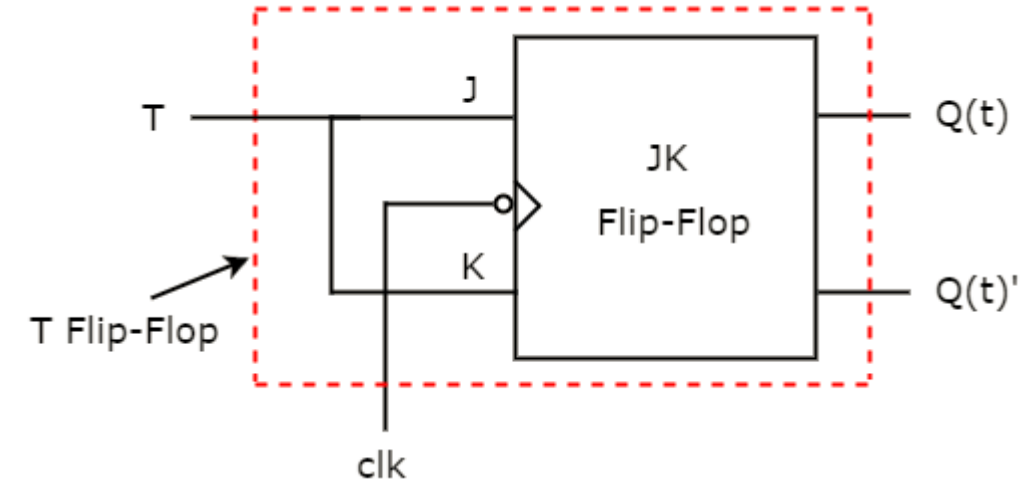
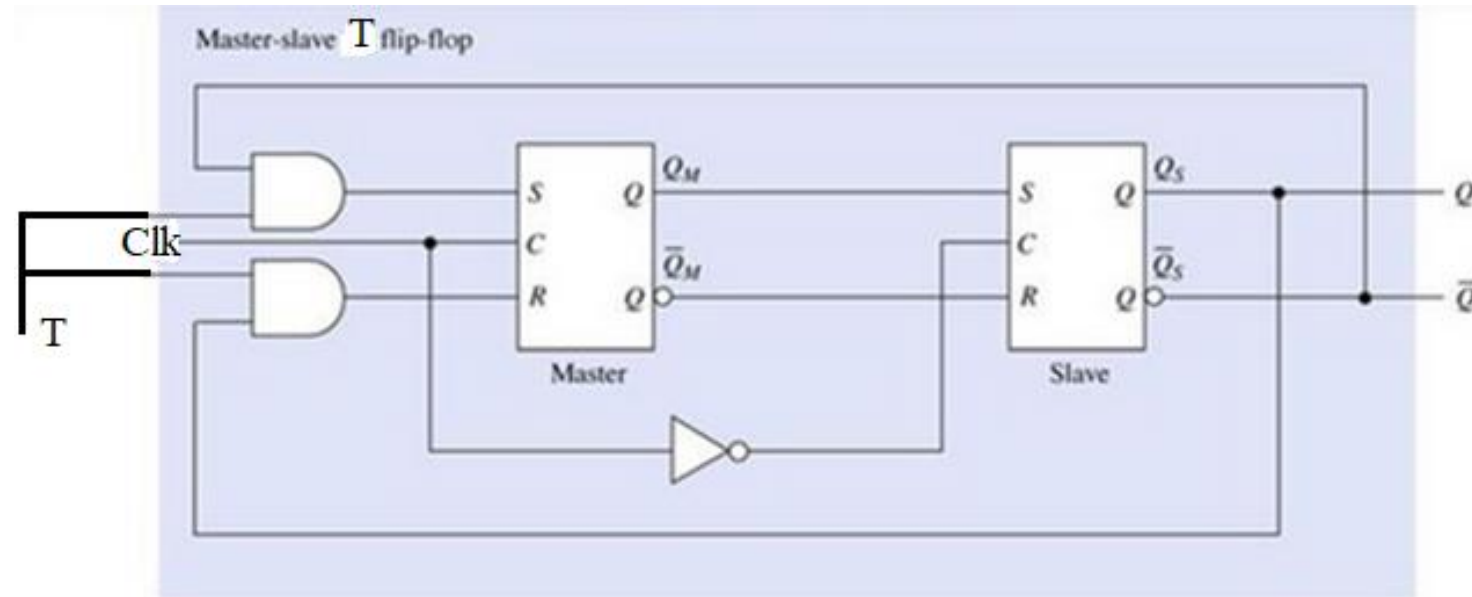
- When $\text{clk} = 1$ Master Latch is active and when $\text{clk} = 0$ Slave latch is active.
- a) **When $J = 0$ and $K = 0$** , Master Latch input $S = R = 0$ therefore Q_M and Q_M' will be same as that of the previous value. Therefore, based on previous value on $Q_M = 0/1$ and $Q_M' = 1/0$, the output $Q_S = 0/1$ and $Q_S' = 1/0$.
- b) **When $J = 0$ and $K = 1$** , Master Latch input $S = 0$ and R can not be defined hence we assume
 - Case i) If we assume at this time $Q = 0$ and $Q' = 1$, then $R = 0$ hence Q_M and Q_M' will have the same previous value $Q_M = 0$ and $Q_M' = 1$. This causes the Q_S and Q_S' to be 0 and 1 respectively (same as previous).
 - Case ii) If we assume at this time $Q = 1$ and $Q' = 0$, then $R = 1$ hence Q_M and Q_M' will have the value as $Q_M = 0$ and $Q_M' = 1$. This causes the Q_S and Q_S' to be 0 and 1 respectively.
 - Hence, we conclude that when $J = 0$ and $K = 1$, then $Q = 0$ and $Q' = 1$
- c) **When $J = 1$ and $K = 0$** , Master Latch input $R = 0$ and S can not be defined hence we assume
 - Case i) If we assume at this time $Q = 0$ and $Q' = 1$, then $S = 1$ hence $Q_M = 1$ and $Q_M' = 0$. This causes the Q_S and Q_S' to be 1 and 0 respectively.
 - Case ii) If we assume at this time $Q = 1$ and $Q' = 0$, then $S = 0$ hence Q_M and Q_M' will have the same previous value $Q_M = 1$ and $Q_M' = 0$. This causes the Q_S and Q_S' to be 1 and 0 respectively (same as previous).
 - Hence, we conclude that when $J = 1$ and $K = 0$, then $Q = 1$ and $Q' = 0$
- d) **When $J = 1$ and $K = 1$** , Master Latch input S and R inputs can not be defined hence we assume
 - Case i) If we assume at this time $Q = 0$ and $Q' = 1$, then $S = 1$ and $R = 0$, hence $Q_M = 1$ and $Q_M' = 0$. This causes the Q_S and Q_S' to be 1 and 0 respectively.
 - Case ii) If we assume at this time $Q = 1$ and $Q' = 0$, then $S = 0$ and $R = 1$, hence $Q_M = 0$ and $Q_M' = 1$. This causes the Q_S and Q_S' to be 0 and 1 respectively.
 - Hence, we conclude that when $J = 1$ and $K = 1$, then Q and Q' toggles with respect to the previous value and hence it is a valid output and removes the drawback of the Master Slave SR flipflop where for $S=R=1$, the output is invalid as discussed before.



Clk	S	R	Q^+	Q'^+
0	x	x	Q	Q'
1	0	0	Q	Q
1	0	1	0	1
1	1	0	1	0
1	1	1	1	1

Truth Table of JK Flip Flop				
Clk	J	K	Q^+	Q'^+
Negative edge on Clk	0	0	Q	Q'
Negative edge on Clk	0	1	0	1
Negative edge on Clk	1	0	1	0
Negative edge on Clk	1	1	Q'	Q
Any other value on clock other than negative edge on clk	x	x	Q	Q'

T (Toggle) Flip Flop



C = Clk = Clock	T	Q^+	Q'^+	Remarks
Negative edge on C	0	Q	Q'	Same as that of the previous value
Negative edge on C	1	Q'	Q	Toggles i. e. Invert of the previous value
Any other value on C other than negative edge on the C	x	Q	Q'	Same as that of the previous value

- Summary of Truth Table of SR, JK, D and T flip flops
 - Characteristic Table
 - Characteristic Equation
 - Excitation Table
- Characteristic Table = State Table
Function Table = Truth Table

Master-Slave JK Flip-Flop

Q_n	J_n	K_n	Q_{n+1}
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

State Table



J_n	K_n	Q_{n+1}
0	0	Q_n
0	1	0
1	0	1
1	1	$\overline{Q_n}$

Characteristic Table

$J_n K_n$	00	01	11	10
Q_n				
0	0	0	1	1
1	1	0	0	1

K-Map

$$Q_{n+1} = J_n \cdot \overline{Q_n} + \overline{K_n} \cdot Q_n$$

Characteristic Equation

Q_n	Q_{n+1}	J_n	K_n
0	0	0	d
0	1	1	d
1	0	d	1
1	1	d	0

Excitation Table

The d indicates a "don't care" term. That is, the signal value can be either a 0 or a 1 as it doesn't affect the next state.

D Flip-flop

Q_n	D_n	Q_{n+1}
0	0	0
0	1	1
1	0	0
1	1	1

State Table

D_n	Q_{n+1}
0	0
1	1

Characteristic Table

	D_n	
	0	1
Q_n	0	1
	0	1
	1	1

K-Map

Q_n	Q_{n+1}	D_n
0	0	0
0	1	1
1	0	0
1	1	1

Excitation Table

$$Q_{n+1} = D_n$$

Characteristic Equation

- As an exercise, derive the state table, K-map, and excitation table for an SR flip-flop from the characteristic table.

SR		S	R	Q	Q_{next}
		0	0	0	0
		0	0	1	1
		0	1	0	0
		0	1	1	0
		1	0	0	1
		1	0	1	1
		1	1	0	×
		1	1	1	×

	SR = 00	SR = 01	SR = 11	SR = 10
Q = 0	0	0	x	1
Q = 1	1	0	x	1

Characteristic Equation for SR ff = $Q_{next} = Q^+ = S + R'Q$

JK		J	K	Q	Q_{next}
		0	0	0	0
		0	0	1	1
		0	1	0	0
		0	1	1	0
		1	0	0	1
		1	0	1	1
		1	1	0	1
		1	1	1	0

	JK = 00	JK = 01	JK = 11	JK = 10
Q = 0	0	0	1	1
Q = 1	1	0	0	1

Characteristic Equation for JK ff = $Q_{next} = Q^+ = JQ' + K'Q$

D		D	Q	Q_{next}
		0	×	0
		1	×	1

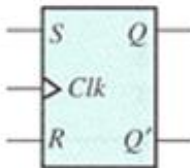
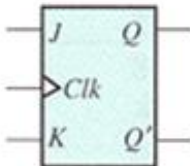
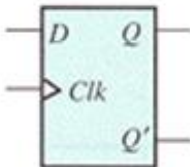
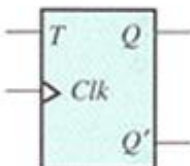
	D = 0	D = 1
Q = 0	0	1
Q = 1	0	1

Characteristic Equation for D ff = $Q_{next} = Q^+ = D$

T		T	Q	Q_{next}
		0	0	0
		0	1	1
		1	0	1
		1	1	0

	T = 0	T = 1
Q = 0	0	1
Q = 1	1	0

Characteristic Equation for T ff = $Q_{next} = Q^+ = T'Q + TQ' = T \oplus Q$

FLIP-FLOP NAME	FLIP-FLOP SYMBOL	CHARACTERISTIC TABLE	CHARACTERISTIC EQUATION	EXCITATION TABLE																																			
SR		<table><tr><th>S</th><th>R</th><th>Q(next)</th></tr><tr><td>0</td><td>0</td><td>Q</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>NA</td></tr></table>	S	R	Q(next)	0	0	Q	0	1	0	1	0	1	1	1	NA	$Q(next) = S + R'Q$	<table><tr><th>Q</th><th>Q(next)</th><th>S</th><th>R</th></tr><tr><td>0</td><td>0</td><td>0</td><td>X</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>X</td><td>0</td></tr></table>	Q	Q(next)	S	R	0	0	0	X	0	1	1	0	1	0	0	1	1	1	X	0
S	R	Q(next)																																					
0	0	Q																																					
0	1	0																																					
1	0	1																																					
1	1	NA																																					
Q	Q(next)	S	R																																				
0	0	0	X																																				
0	1	1	0																																				
1	0	0	1																																				
1	1	X	0																																				
JK		<table><tr><th>J</th><th>K</th><th>Q(next)</th></tr><tr><td>0</td><td>0</td><td>Q</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>Q'</td></tr></table>	J	K	Q(next)	0	0	Q	0	1	0	1	0	1	1	1	Q'	$Q(next) = JQ' + K'Q$	<table><tr><th>Q</th><th>Q(next)</th><th>J</th><th>K</th></tr><tr><td>0</td><td>0</td><td>0</td><td>X</td></tr><tr><td>0</td><td>1</td><td>1</td><td>X</td></tr><tr><td>1</td><td>0</td><td>X</td><td>1</td></tr><tr><td>1</td><td>1</td><td>X</td><td>0</td></tr></table>	Q	Q(next)	J	K	0	0	0	X	0	1	1	X	1	0	X	1	1	1	X	0
J	K	Q(next)																																					
0	0	Q																																					
0	1	0																																					
1	0	1																																					
1	1	Q'																																					
Q	Q(next)	J	K																																				
0	0	0	X																																				
0	1	1	X																																				
1	0	X	1																																				
1	1	X	0																																				
D		<table><tr><th>D</th><th>Q(next)</th></tr><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	D	Q(next)	0	0	1	1	$Q(next) = D$	<table><tr><th>Q</th><th>Q(next)</th><th>D</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	Q	Q(next)	D	0	0	0	0	1	1	1	0	0	1	1	1														
D	Q(next)																																						
0	0																																						
1	1																																						
Q	Q(next)	D																																					
0	0	0																																					
0	1	1																																					
1	0	0																																					
1	1	1																																					
T		<table><tr><th>T</th><th>Q(next)</th></tr><tr><td>0</td><td>Q</td></tr><tr><td>1</td><td>Q'</td></tr></table>	T	Q(next)	0	Q	1	Q'	$Q(next) = TQ' + T'Q$	<table><tr><th>Q</th><th>Q(next)</th><th>T</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	Q	Q(next)	T	0	0	0	0	1	1	1	0	1	1	1	0														
T	Q(next)																																						
0	Q																																						
1	Q'																																						
Q	Q(next)	T																																					
0	0	0																																					
0	1	1																																					
1	0	1																																					
1	1	0																																					

- Characteristic Table = State Table
Function Table = Truth Table

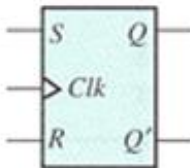
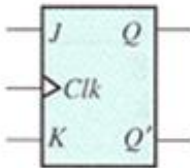
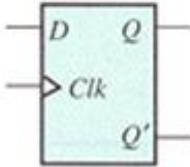
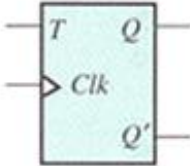
Digital Electronics & Systems

19ECE204

Lecture 51-54

Flip-Flop Conversion & Shift Registers

Ms. Bhavana V
Assistant Professor (Sr.Gr.)
Dept. of ECE
ASE, Bengaluru

FLIP-FLOP NAME	FLIP-FLOP SYMBOL	CHARACTERISTIC TABLE	CHARACTERISTIC EQUATION	EXCITATION TABLE																																			
SR		<table><tr><th>S</th><th>R</th><th>Q(next)</th></tr><tr><td>0</td><td>0</td><td>Q</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>NA</td></tr></table>	S	R	Q(next)	0	0	Q	0	1	0	1	0	1	1	1	NA	$Q(next) = S + R'Q$	<table><tr><th>Q</th><th>Q(next)</th><th>S</th><th>R</th></tr><tr><td>0</td><td>0</td><td>0</td><td>X</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>X</td><td>0</td></tr></table>	Q	Q(next)	S	R	0	0	0	X	0	1	1	0	1	0	0	1	1	1	X	0
S	R	Q(next)																																					
0	0	Q																																					
0	1	0																																					
1	0	1																																					
1	1	NA																																					
Q	Q(next)	S	R																																				
0	0	0	X																																				
0	1	1	0																																				
1	0	0	1																																				
1	1	X	0																																				
JK		<table><tr><th>J</th><th>K</th><th>Q(next)</th></tr><tr><td>0</td><td>0</td><td>Q</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>Q'</td></tr></table>	J	K	Q(next)	0	0	Q	0	1	0	1	0	1	1	1	Q'	$Q(next) = JQ' + K'Q$	<table><tr><th>Q</th><th>Q(next)</th><th>J</th><th>K</th></tr><tr><td>0</td><td>0</td><td>0</td><td>X</td></tr><tr><td>0</td><td>1</td><td>1</td><td>X</td></tr><tr><td>1</td><td>0</td><td>X</td><td>1</td></tr><tr><td>1</td><td>1</td><td>X</td><td>0</td></tr></table>	Q	Q(next)	J	K	0	0	0	X	0	1	1	X	1	0	X	1	1	1	X	0
J	K	Q(next)																																					
0	0	Q																																					
0	1	0																																					
1	0	1																																					
1	1	Q'																																					
Q	Q(next)	J	K																																				
0	0	0	X																																				
0	1	1	X																																				
1	0	X	1																																				
1	1	X	0																																				
D		<table><tr><th>D</th><th>Q(next)</th></tr><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	D	Q(next)	0	0	1	1	$Q(next) = D$	<table><tr><th>Q</th><th>Q(next)</th><th>D</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	Q	Q(next)	D	0	0	0	0	1	1	1	0	0	1	1	1														
D	Q(next)																																						
0	0																																						
1	1																																						
Q	Q(next)	D																																					
0	0	0																																					
0	1	1																																					
1	0	0																																					
1	1	1																																					
T		<table><tr><th>T</th><th>Q(next)</th></tr><tr><td>0</td><td>Q</td></tr><tr><td>1</td><td>Q'</td></tr></table>	T	Q(next)	0	Q	1	Q'	$Q(next) = TQ' + T'Q$	<table><tr><th>Q</th><th>Q(next)</th><th>T</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	Q	Q(next)	T	0	0	0	0	1	1	1	0	1	1	1	0														
T	Q(next)																																						
0	Q																																						
1	Q'																																						
Q	Q(next)	T																																					
0	0	0																																					
0	1	1																																					
1	0	1																																					
1	1	0																																					

- Characteristic Table = State Table
Function Table = Truth Table

Flip-Flop Conversion

Steps for Flip-Flop Conversions:

1. Identify available and required flip flop.
2. Write characteristic table for required flip flop.
3. Write excitation table for available flip flop.
4. Make conversion table and using K-map write Boolean expression for available flip flop.
5. Draw the circuit.

Conversion of a JK-to-D Flip-Flop

- 1. Available Flip Flop : JK Flip Flop
Required Flip Flop : D Flip Flop
- 2. Characteristic Table of Required FF: D Flip Flop

Inputs	Present Output	Next Output
D	Q_n	Q_{n+1}
0	0	0
0	1	0
1	0	1
1	1	1

- 3. Excitation Table of Available FF: JK FF

Q	Q^+	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

Excitation of J-K flip-flop

- 4. Conversion Table and Boolean Expression

D Input	Outputs		JK Inputs	
	Present State	Next State	J	K
D	Q_n	Q_{n+1}	J	K
0	0	0	0	X
0	1	0	X	1
1	0	1	1	X
1	1	1	X	0

Quarna

0 1

0 0 1

1 1 X

J = D

Quarna

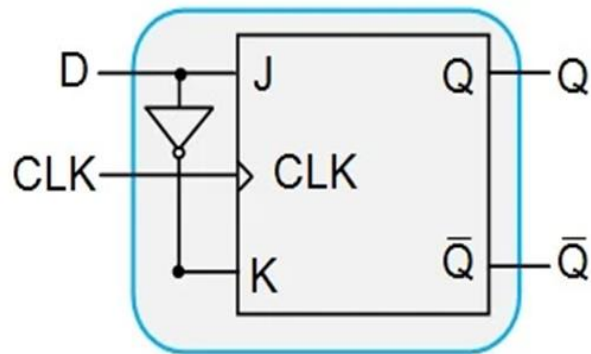
0 1

0 X 1

1 X 0

K = D

- 5. Circuit Diagram



Conversion of D to JK Flip-Flop

Truth Table of JK Flip-flop

Inputs		Outputs	
J	K	Present State Q_n	Next State Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

JK Inputs		Outputs		D Input
J	K	Present State Q_n	Next State Q_{n+1}	
0	0	0	0	0
0	0	1	1	1
0	1	0	0	0
0	1	1	0	0
1	0	0	1	1
1	0	1	1	1
1	1	0	1	1
1	1	1	0	0

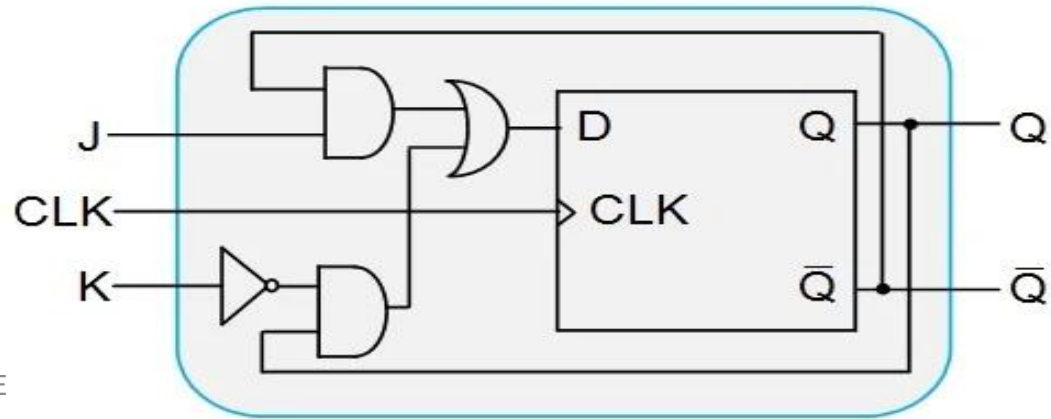
Excitation Table of D Flip-flop

Outputs		Input
Present State Q_n	Next State Q_{n+1}	
0	0	0
0	1	1
1	0	0
1	1	1

D to JK Conversion Table

J \ K Q_n	00	01	11	10
0	0 ⁰	1 ¹	0 ³	0 ²
1	1 ⁴	1 ⁵	0 ⁷	1 ⁶

$$D = J\bar{Q}_n + \bar{K}Q_n$$



Try Yourself:

1. SR FF to JK FF
2. D FF to T FF
3. T FF to D FF
4. JK FF to T FF

Example - Convert a positive edge triggered JK flip-flop to a positive edge triggered XY flip flop as defined by function table (characteristics table):

X	Y	Q_{n+1}
0	0	0
0	1	Q'
1	0	Q
1	1	1

Solution- The application table (excitation Table) for the JK flip-flop and function table for the XY flip-flop are:

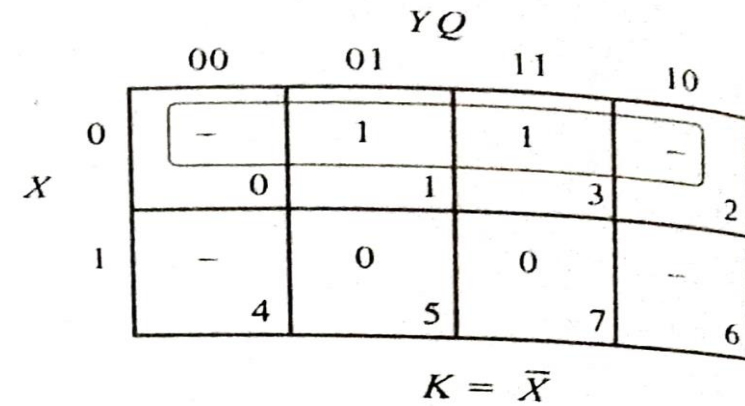
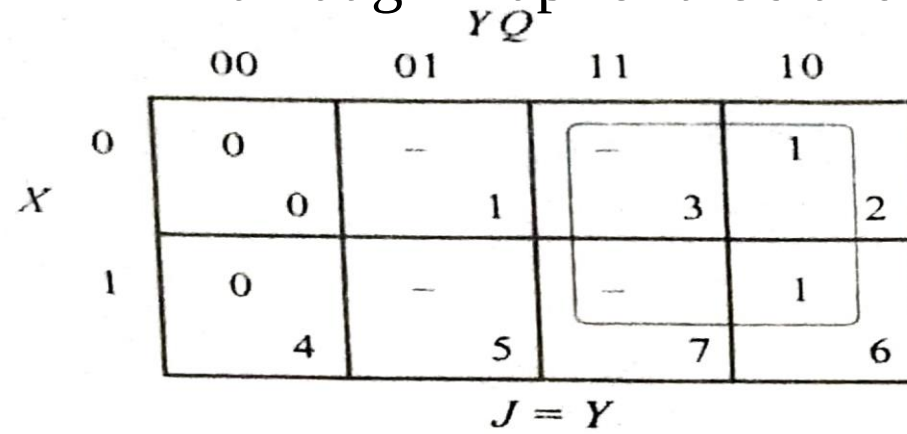
$Q \rightarrow$	Q_{n+1}	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

X	Y	Q_{n+1}
0	0	0
0	1	Q'
1	0	Q
1	1	1

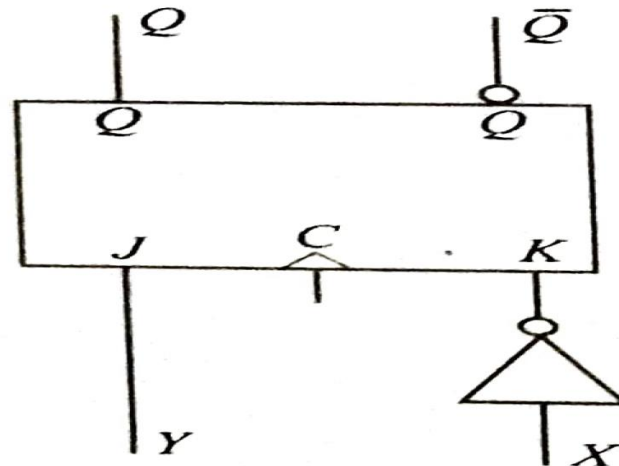
Excitation Table for conversion:

Cell No.	X	Y	Q	Q _{n+1}	J	K
0	0	0	0	0	0	X
1	0	0	1	0	X	1
2	0	1	0	1	1	X
3	0	1	1	0	X	1
4	1	0	0	0	0	X
5	1	0	1	1	X	0
6	1	1	0	1	1	X
7	1	1	1	1	X	0

Karnaugh Map for the J and K inputs



Converted Circuit:



Example- Convert a positive edge triggered D flip-flop into the positive edge triggered XY flip flop defined by characteristics table shown below:

X	Y	Q_{n+1}
0	0	0
0	1	Q'
1	0	Q
1	1	1

Solution: The application table for the D flip-flop and the function table for the XY flip-flop are:

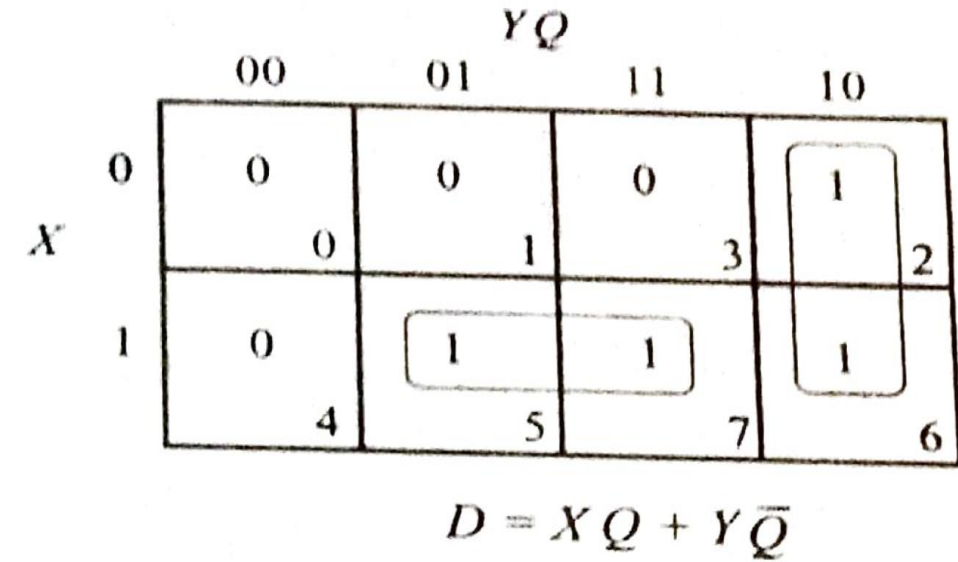
$Q \rightarrow$	Q_{n+1}	D
0	0	0
0	1	1
1	0	0
1	1	1

X	Y	Q_{n+1}
0	0	0
0	1	Q'
1	0	Q
1	1	1

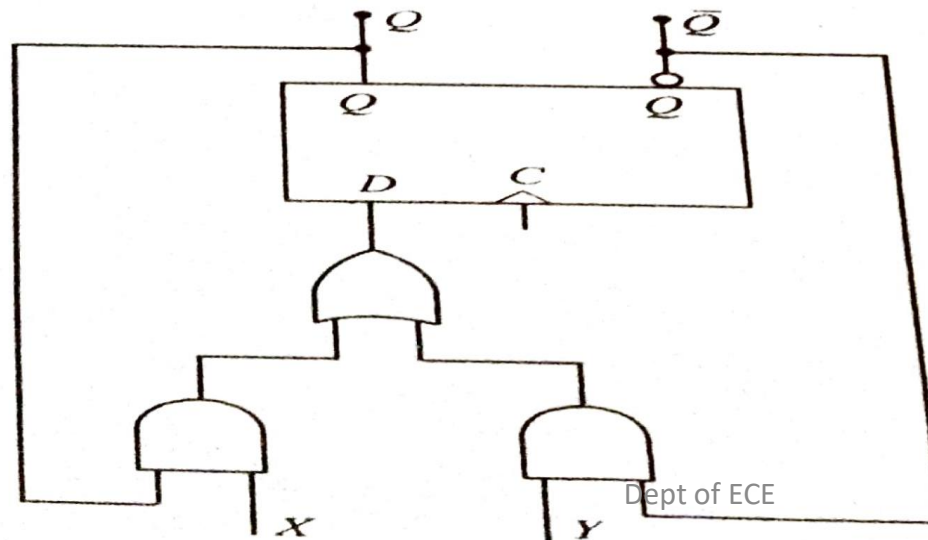
Excitation Table:

Cell no.	X	Y	$Q \rightarrow Q'$	D
0	0	0	0	0
1	0	0	1	0
2	0	1	0	1
3	0	1	1	0
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	1

Karnaugh Map for D input:



Converted Circuit:



Excitation Table:

Q(t)	Q(t+1)	P	N
0	0	0	X
0	1	1	X
1	0	X	0
1	1	X	1

PN flip-flop can be converted to a D flip-flop, By connecting P and N together.

$$Q(t+1) = DQ' + DQ = D$$

Example -Convert a positive edge triggered XY flip-flop into the positive edge triggered MN flip-flop defined by below function tables:

X	Y	Q _{n+1}
0	0	0
0	1	Q'
1	0	Q
1	1	1

M	N	Q _{n+1}
0	0	1
0	1	Q
1	0	Q'
1	1	0

Solution-

1. Available Flip Flop : XY Flip Flop
Required Flip Flop : MN Flip Flop

2. Excitation Table of Available FF: XY FF

Q	Q _{n+1}	X	Y
0	0	X	0
0	1	X	1
1	0	0	X
1	1	1	X

3. Characteristic Table of Required FF: MN Flip Flop

Inputs		Present Output	Next Output
M	N	Q _n	Q _{n+1}
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	0

4. Conversion Table and Boolean Expression:

M	N	Q _n	Q _{n+1}	X	Y
0	0	0	1	X	1
0	0	1	1	1	X
0	1	0	0	X	0
0	1	1	1	1	X
1	0	0	1	X	1
1	0	1	0	0	X
1	1	0	0	X	0
1	1	1	0	0	X

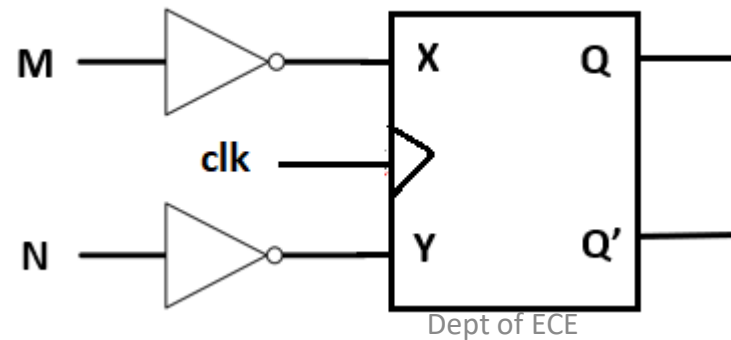
M\NQ _n	00	01	11	10
0	X	1	1	X
1	X	0	0	X

$$X = M'$$

M\NQ _n	00	01	11	10
0	1	X	X	0
1	1	X	X	0

$$Y = N'$$

5. Converted Circuit:



Registers

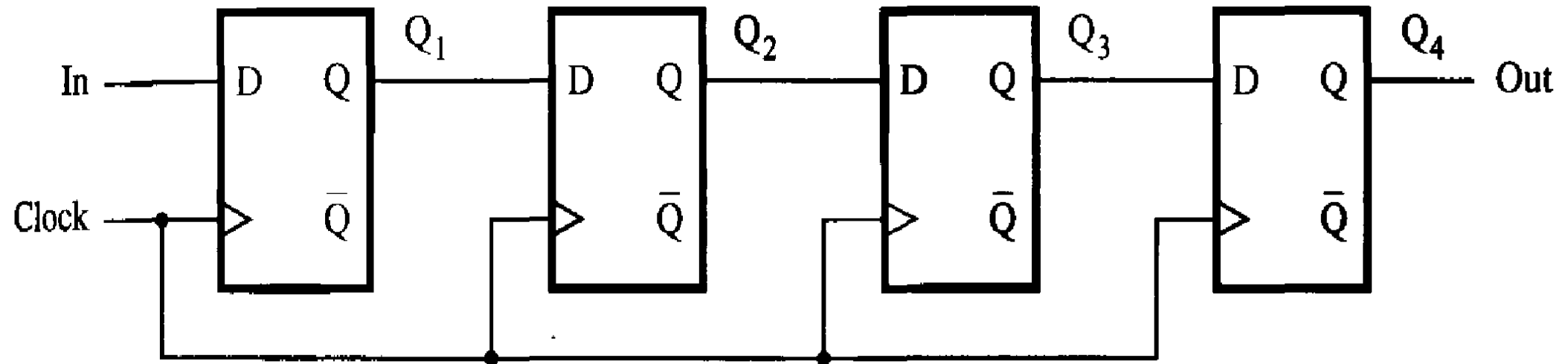
- Flip-flop stores one bit of information. When a set of n flip-flops is used to store n bits of information, such as an n -bit number, we refer to these flip-flops as a *register*.
- A common clock is used for each flip-flop in a register.
- The term register is merely a convenience for referring to n -bit structures consisting of flip-flops.
- Register consists of a group of flip-flops and gates that effect their transition.
- The flip-flops hold the binary information and the gates control, when and how information is transferred to the register.
- The simplest register is that which only contains flip-flops, with no external gates.

Shift Registers

- We know that a given number is multiplied by 2 if its bits are shifted one bit position to the left and a 0 is inserted as the new least-significant bit.
- Similarly, the number is divided by 2 if the bits are shifted one bit-position to the right.
- A register that provides the ability to shift its contents is called a *shift register*.
- Four types of Shift registers-
 1. Serial-in-serial out (SISO) register
 2. Serial-in-parallel-out (SIPO) register
 3. Parallel-in-parallel-out (PIPO) register
 4. Parallel-in-serial-out register (PISO) register

Shift Registers

- Figure shows a four-bit shift register that is used to shift its contents one bit-position to the right.
- The data bits are loaded into the shift register in a serial fashion using the In input.
- The contents of each flip-flop are transferred to the next flip-flop at each positive edge of the clock.



Shift Registers

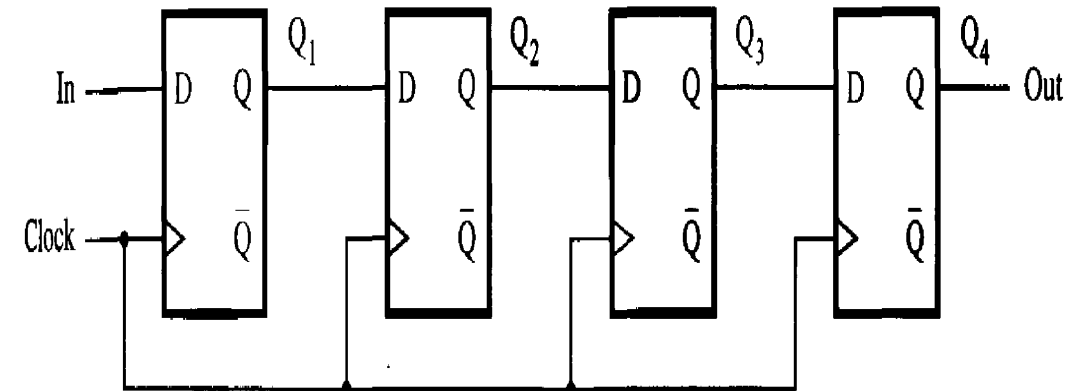
- An illustration of the transfer is shown below, which shows what happens when the signal values at In during eight consecutive clock cycles are 1,0,1,1,1,0,0, and 0, assuming that the initial state of all flip-flops is 0.

	In	Q ₁	Q ₂	Q ₃	Q ₄ = Out
t ₀	1	0	0	0	0
t ₁	0	1	0	0	0
t ₂	1	0	1	0	0
t ₃	1	1	0	1	0
t ₄	1	1	1	0	1
t ₅	0	1	1	1	0
t ₆	0	0	1	1	1
t ₇	0	0	0	1	1

SISO

SIPO

Take Data = 1011



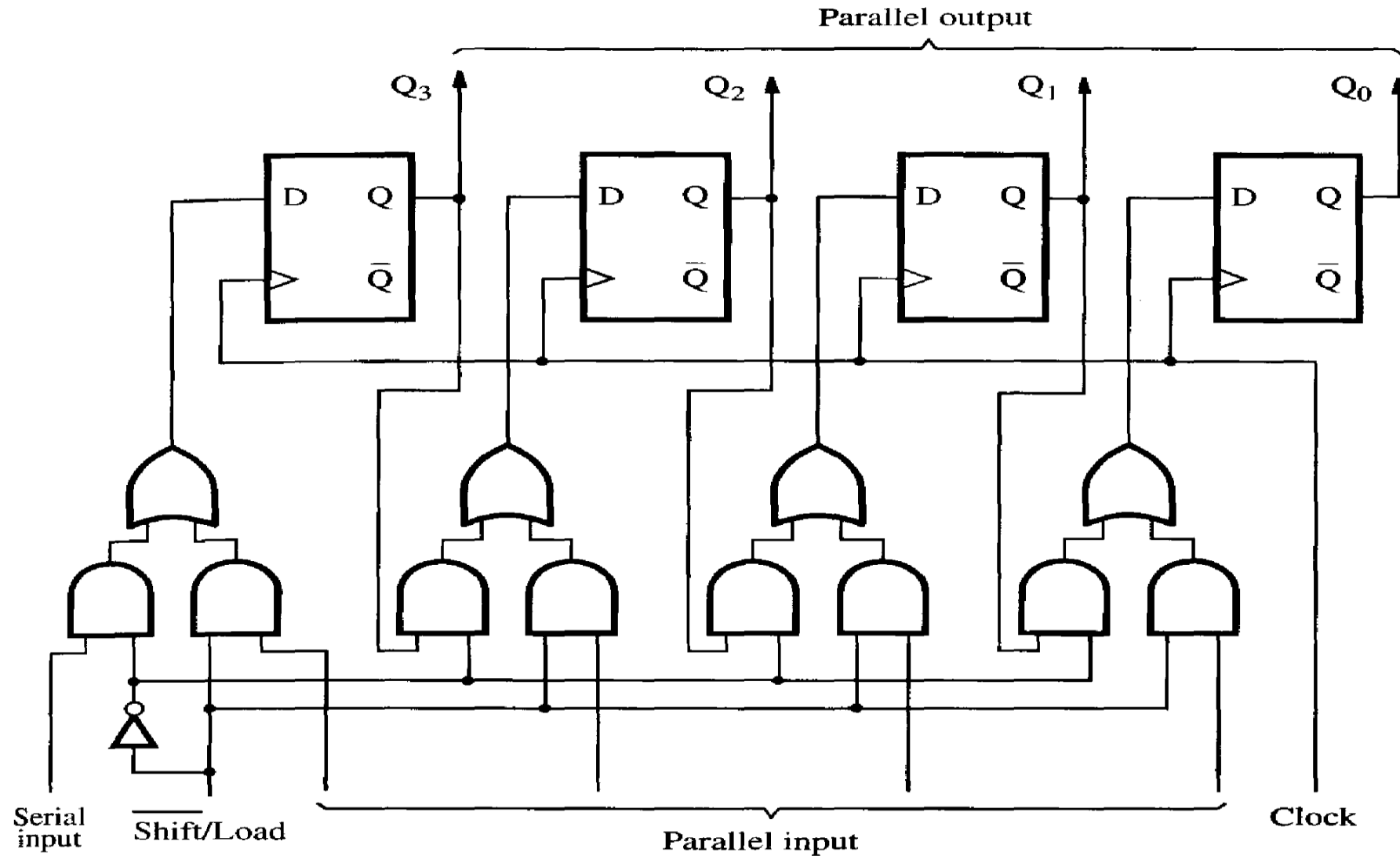
Shift Registers

- To implement a shift register, it is necessary to use either edge-triggered or master-slave flip-flops.
- The level-sensitive gated latches are not suitable, because a change in the value of In would propagate through more than one latch during the time when the clock is equal to 1.

Parallel-access Shift Registers

- In computer systems it is often necessary to transfer n -bit data items.
- This may be done by transmitting all bits at once using n separate wires, in which case we say that the transfer is performed in *parallel*.
- But it is also possible to transfer all bits using a single wire, by performing the transfer one bit at a time, in n consecutive clock cycles. We refer to this scheme as *serial* transfer.
- To transfer an n -bit data item serially, we can use a shift register that can be loaded with all n bits in parallel (in one clock cycle).
- Then during the next n clock cycles, the contents of the register can be shifted out for serial transfer.
- The reverse operation is also needed. If bits are received serially, then after n clock cycles the contents of the register can be accessed in parallel as an n -bit item.

Parallel-access Shift Registers



Parallel-access Shift Registers

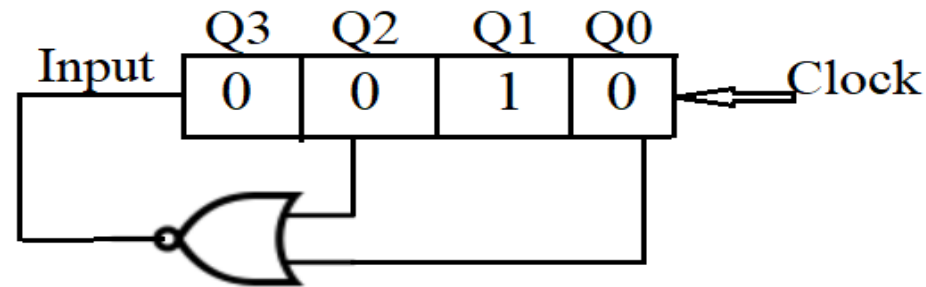
- Figure shows a four-bit shift register that allows the parallel access.
- Instead of using the normal shift register connection, the D input of each flip-flop is connected to two different sources.
 - One source is the preceding flip-flop, which is needed for the shift register operation.
 - The other source is the external input that corresponds to the bit that is to be loaded into the flip-flop as a part of the parallel-load operation.
- The control signal $\overline{Shift/Load}$ is used to select the mode of operation. If $\overline{Shift/Load} = 0$, then the circuit operates as a shift register. If $\overline{Shift/Load} = 1$, then the parallel input data are loaded into the register.
- In both cases the action takes place on the positive edge of the clock.
- In Figure we have chosen to label the flip-flops outputs as $Q_3, \dots, Q_0$ because shift registers are often used to hold binary numbers.

Parallel-access Shift Registers

- The contents of the register can be accessed in parallel by observing the outputs of all flip-flops.
- The flip-flops can also be accessed serially, by observing the values of Q0 during consecutive clock cycles while the contents are being shifted.
- A circuit in which data can be loaded in series and then accessed in parallel is called a series-to-parallel converter.
- Similarly, the opposite type of circuit is a parallel-to-series converter.
- The circuit shown in Figure can perform both of these functions.

Registers

Question 1- A 4-bit shift register performing right shift operation is shown in figure. Initially the output of shift register $Q_3Q_2Q_1Q_0 = 0010$. After three clock pulses appear, what will be the output $Q_3Q_2Q_1Q_0$?



Answer-

	Q3	Q2	Q1	Q0
Initially	0	0	1	0
1st Clock	1	0	0	1
2nd Clock	0	1	0	0
3rd Clock	0	0	1	0

Registers

Question 2- A 4-bit serial-in-parallel-output shift register performing right shift operation. The input bit stream applied to shift register is 11001010 (Assume MSB bit is first input bit). Write the output $Q_3Q_2Q_1Q_0$ of shift register for 6 clock cycles. Initially the content of shift register is 0100.

Answer-

Clock	Input	Q3	Q2	Q1	Q0
Initially	1	0	1	0	0
1 st Clock	1	1	0	1	0
2 nd Clock	0	1	1	0	1
3 rd Clock	0	0	1	1	0
4 th Clock	1	0	0	1	1
5 th clock	0	1	0	0	1
6 th Clock	1	0	1	0	0

Digital Electronics & Systems
19ECE204
Lecture 55-56
Counters

Ms. Bhavana V
Assistant Professor (Sr.Gr.)
Dept. of ECE
ASE, Bengaluru

Overview

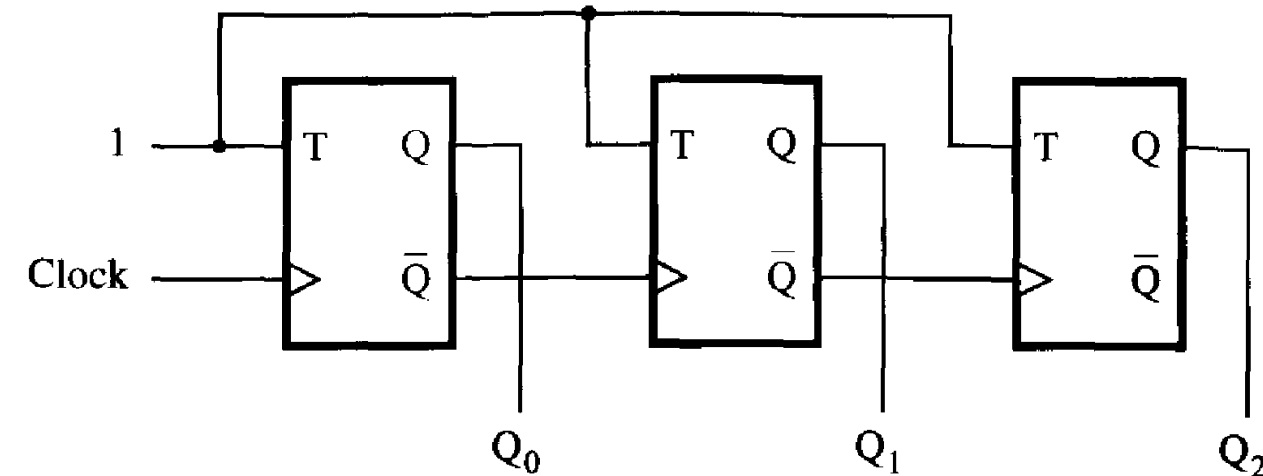
- Counters
 - Asynchronous Counter
 - Synchronous Counter
- Other Types of Counters
 - Ring Counter
 - Johnson Counter

Counters

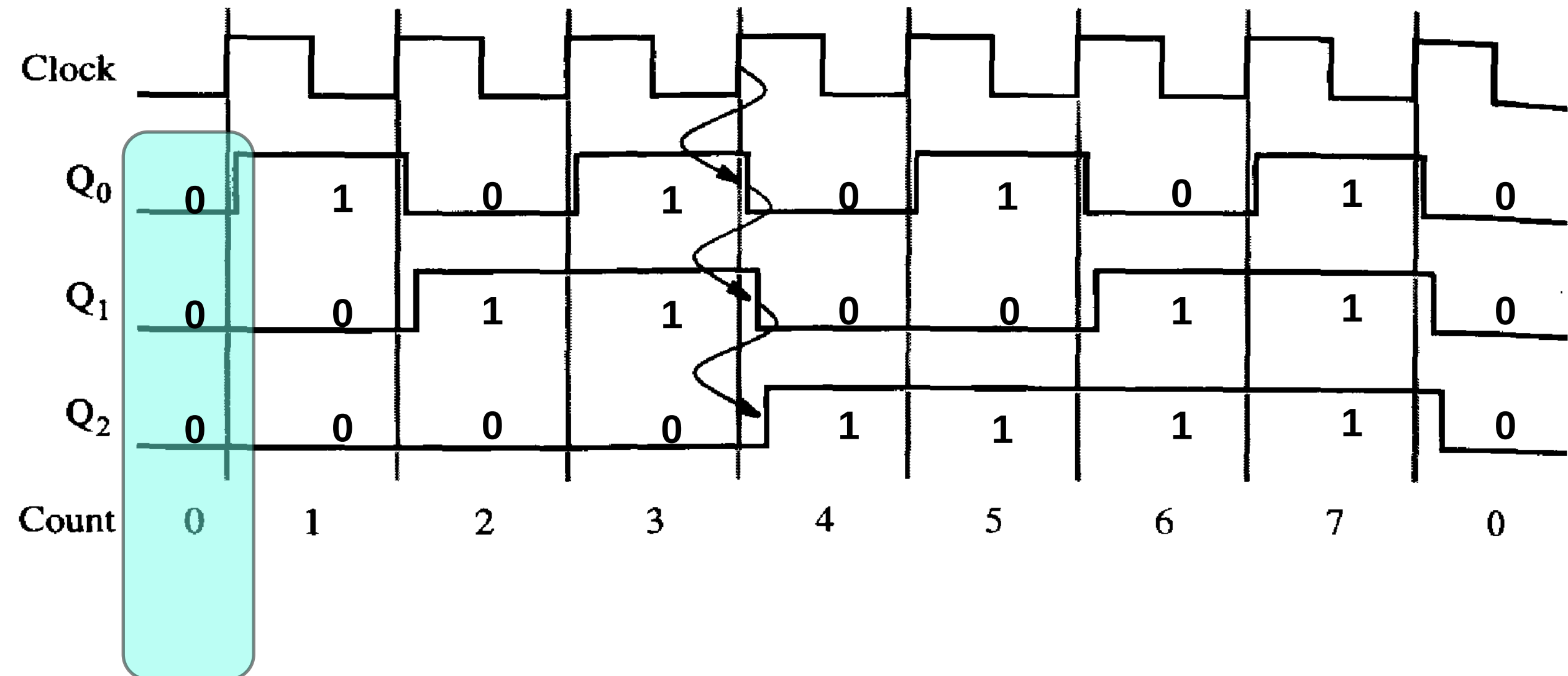
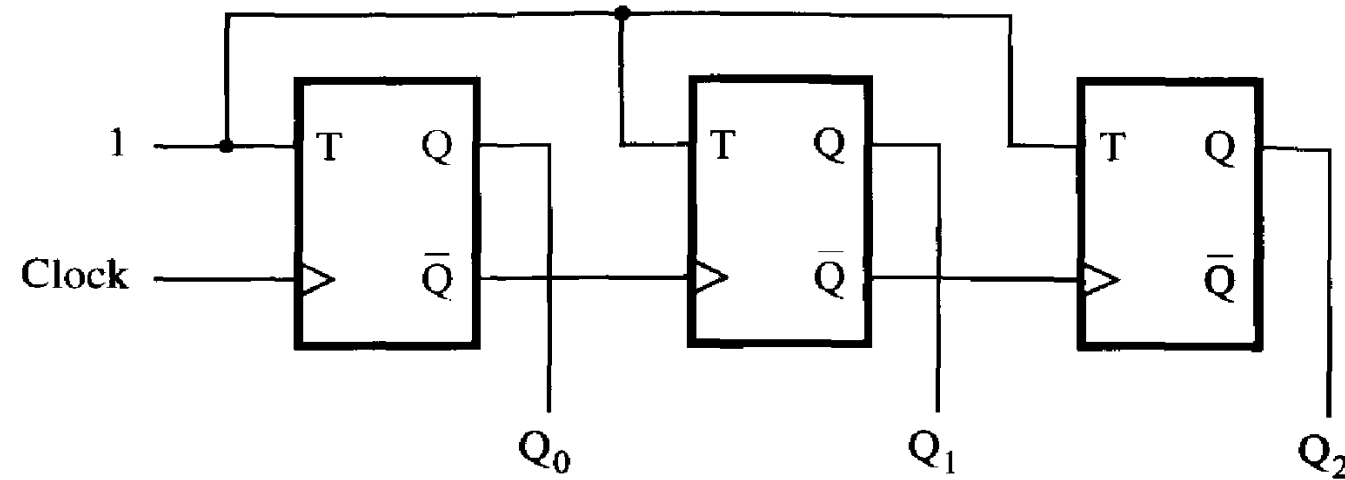
- Counter is a special purpose **adder/subtractor** which increments or decrement a **count by 1** for Up counter or Down counter respectively
- Counter circuits are used in digital systems for many purposes. They may count the number of occurrences of certain events, generate timing intervals for control of various tasks in a system, keep track of time elapsed between specific events, and so on.
- Counters can be implemented using the adder/subtractor circuits and the registers. However, since we only need to change the Contents of a counter by 1, it is not necessary to use such elaborate circuits.
- Instead, we can use much simpler circuits that have a significantly lower cost. Counter circuits can be designed using T and D flip-flops.

Asynchronous Counter - Up Counter

- The simplest counter circuits can be built using T flip-flops because the toggle feature is naturally suited for the implementation of the counting operation.
- Shown figure is a three-bit counter capable of counting from 0 to 7.
- The clock inputs of the three flip-flops are connected in cascade.
- The T input of each flip-flop is connected to a constant 1, which means that the state of the flip-flop will be reversed (toggled) at each positive edge of its clock.



Asynchronous Counter - Up Counter

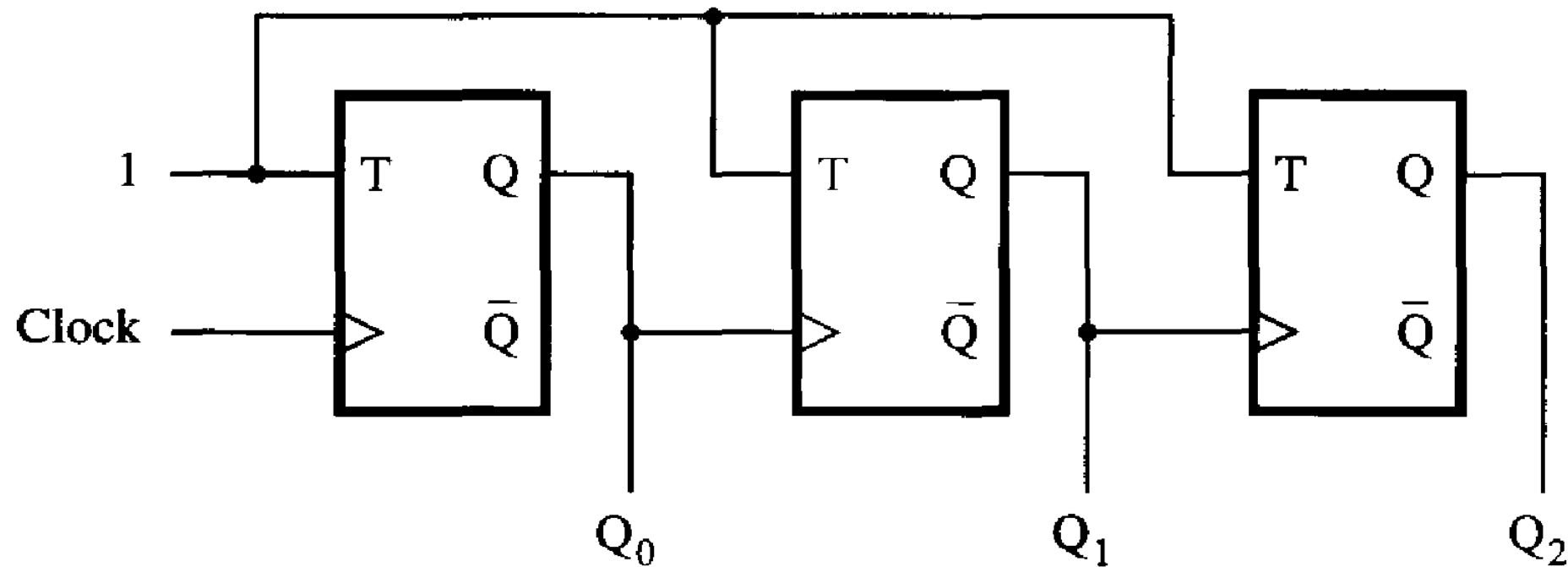


Asynchronous Counter - Up Counter

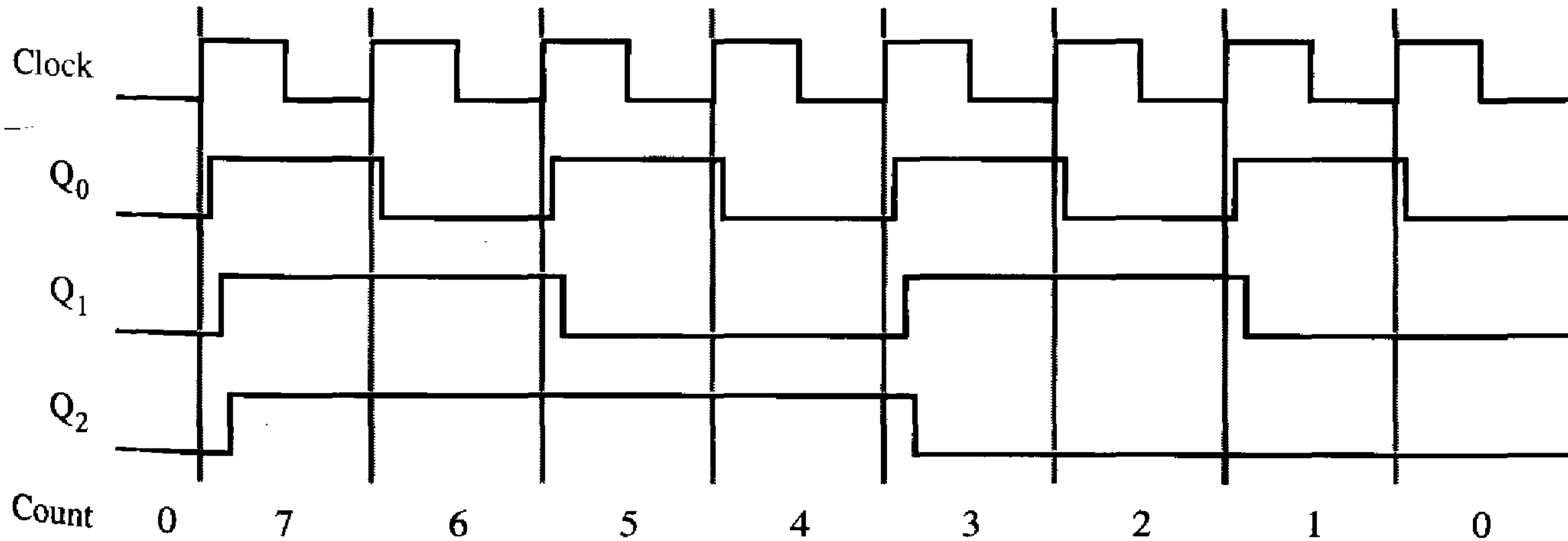
- This circuit is a modulo-8 counter. Because it counts in the upward direction, we call it an up-counter.
- This counter has three stages, each comprising a single flip-flop. Only the first stage responds directly to the Clock signal; we say that this stage is synchronized to the clock.
- The other two stages respond after an additional delay. The change in Q1 appears after a second propagation, the change in Q2 occurs after a third delay
- This behavior is similar to the rippling of carries in the ripple-carry adder circuit, hence it is called as asynchronous counter, or a ripple counter

Asynchronous Counter - Down Counter

- The only difference is that in the clock inputs of the second and third flip-flops are driven by the Q outputs of the preceding stages, rather than by the Q' outputs.



Asynchronous Counter - Down Counter

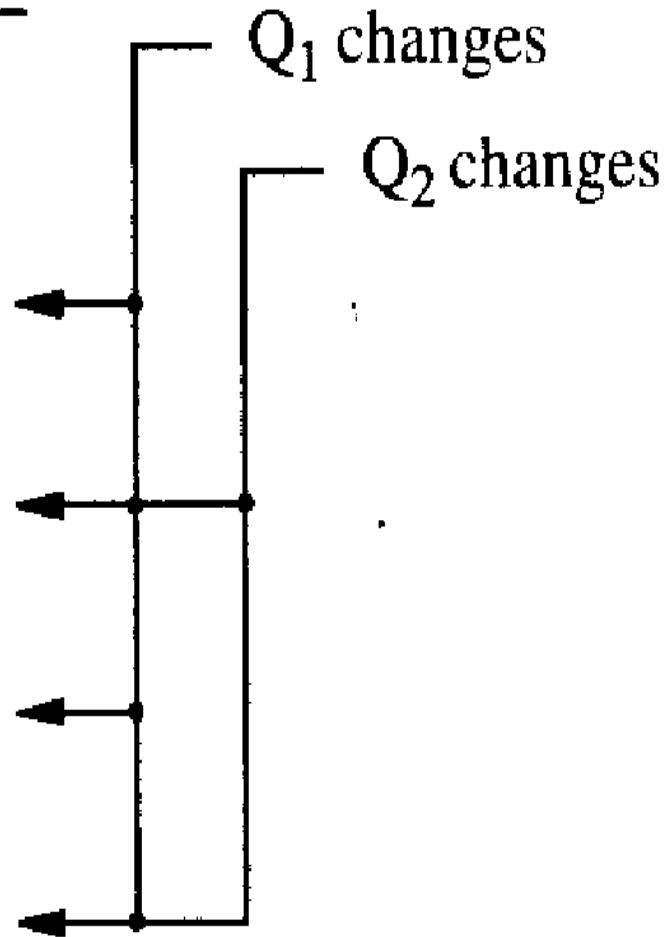


Synchronous Counter

- The asynchronous counters are simple, but not very fast
- If a counter with a larger number of bits is constructed in this manner, then the delays caused by the cascaded clocking scheme may become too long to meet the desired performance requirements.
- We can build a faster counter by clocking all flip-flops at the same time. This kind of counter is called as synchronous counter.

Synchronous Counter

Clock cycle	Q ₂	Q ₁	Q ₀
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1
8	0	0	0



- Q₀ changes on each clock cycle
- Q₁ changes only when Q₀ = 1
- Q₂ changes only when both Q₁ and Q₀ are equal to 1

$$T_0 = 1$$

$$T_1 = Q_0$$

$$T_2 = Q_0 Q_1$$

$$T_3 = Q_0 Q_1 Q_2$$

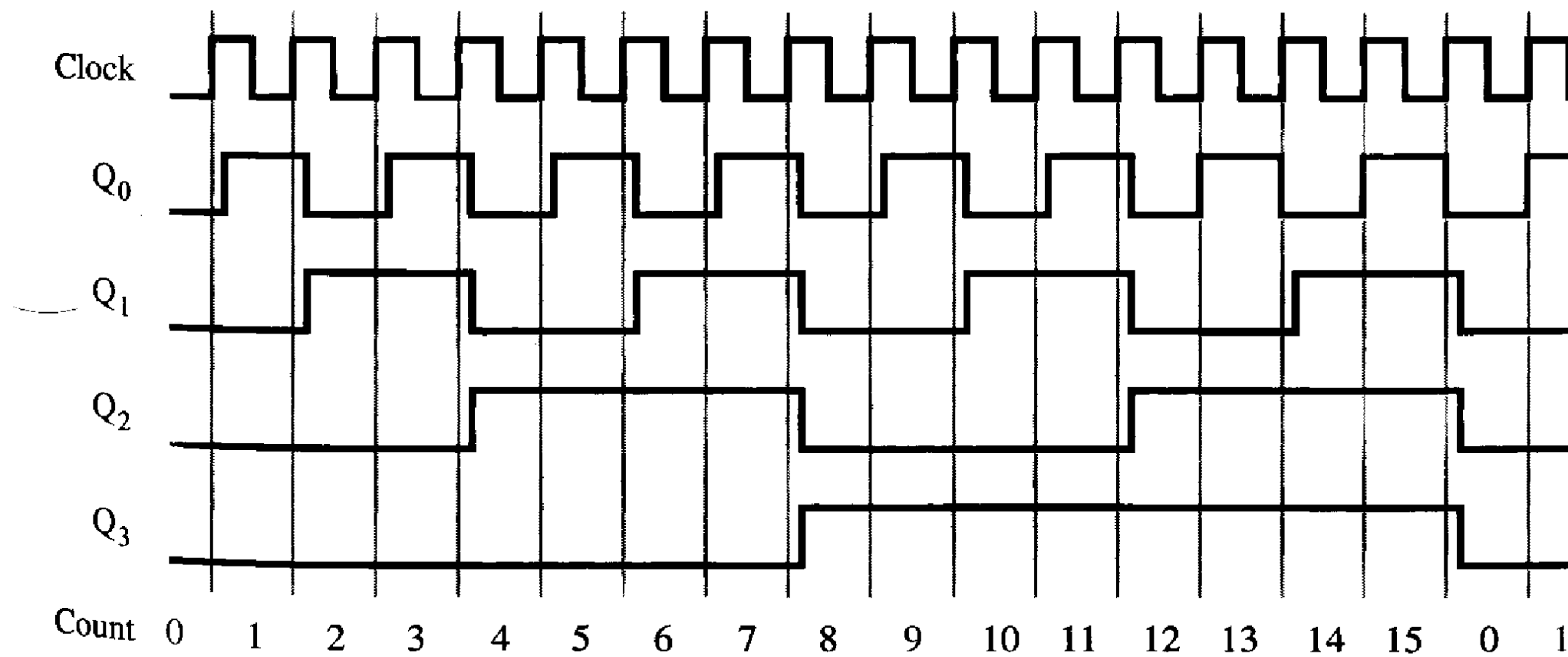
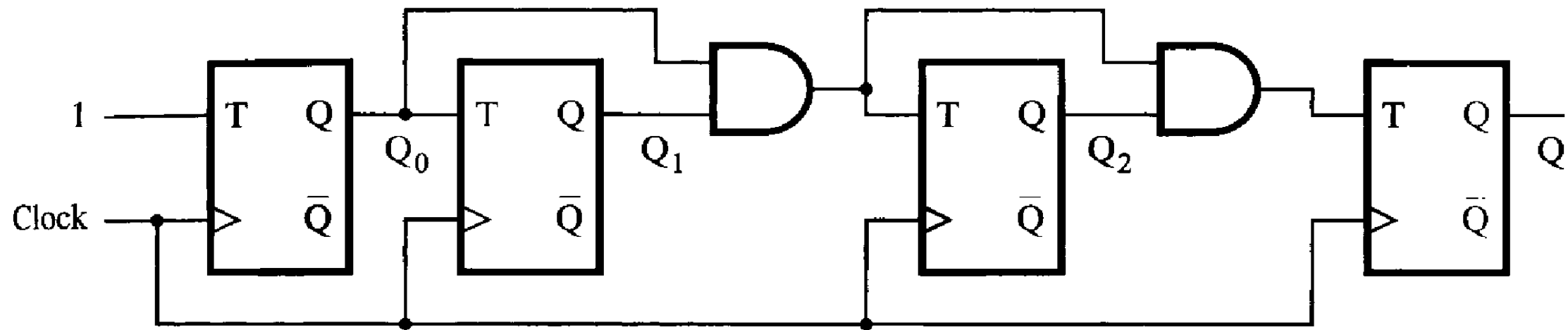
.

.

.

$$T_n = Q_0 Q_1 \cdots Q_{n-1}$$

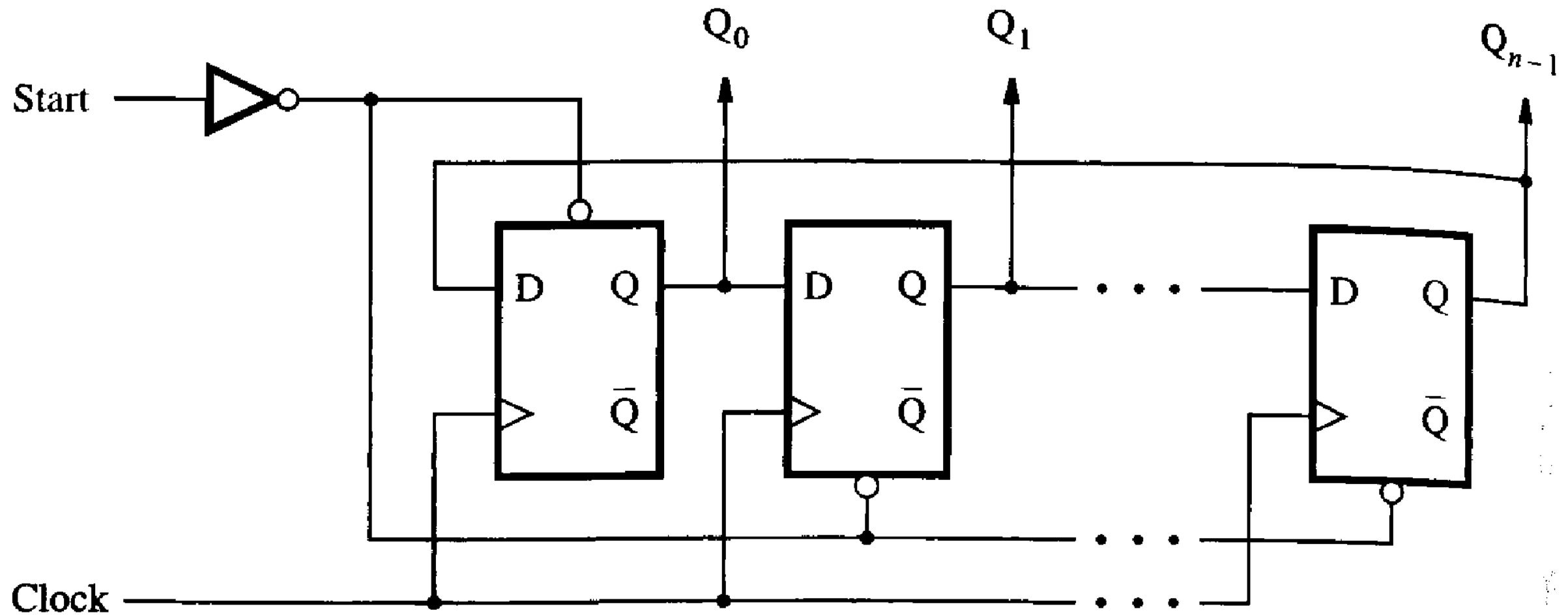
Synchronous Counter



Ring Counter

- It is possible to devise a counter like circuit in which each flip-flop reaches the state $Q_i = 1$ for exactly one count, while for all other counts $Q_i = 0$.
- Then Q_i indicates directly an occurrence of the corresponding count.
- Since this does not represent binary numbers, it is better to say that the outputs of the flips-flops represent a code.
- Such a circuit can be constructed from a simple shift register.

Ring Counter



	Q0	Q1	Q2	Q3
Initially	0	0	0	0
	1	0	0	0
	0	1	0	0
	0	0	1	0
	0	0	0	1
	1	0	0	0

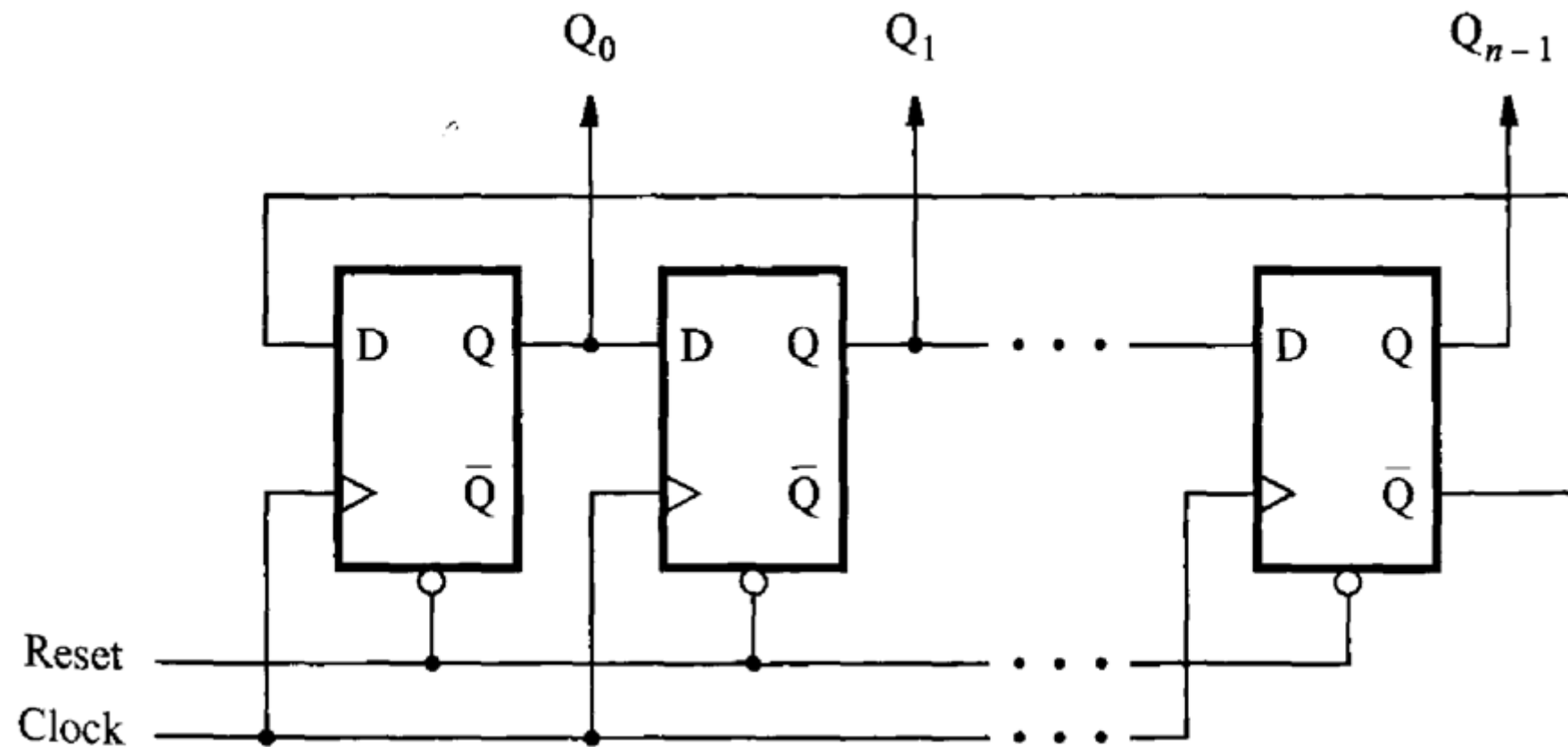
n different states

Ring Counter

- The Q output of the last stage in the shift register is fed back as the input to the first stage, which creates a ring like structure.
- **To start a ring counter the preset signal of first flip-flop and clear signal of all other flip-flops should be activated.**
- If a single 1 is injected into the ring, this 1 will be shifted through the ring at successive clock cycles.
- For example, in a four-bit structure, the possible codes $Q_0Q_1Q_2Q_3$ will be 1000, 0100, 0010, and 0001. **So n-bit Ring counter has n different states.**
- Such encoding, where there is a single 1 and the rest of the code variables are 0, is called a one-hot code.

Johnson Counter

- An interesting variation of the ring counter is obtained if, instead of the Q output, we take the Q output of the last stage and feed it back to the first stage. Johnson counter is also known as twisted Ring counter.



	Q0	Q1	Q2	Q3
Initially	0	0	0	0
	1	0	0	0
	1	1	0	0
	1	1	1	0
2n states	1	1	1	1
	0	1	1	1
	0	0	1	1
	0	0	0	1
	0	0	0	0

Johnson Counter

- An n -bit counter of this type generates a counting sequence of length $2n$.
- To start a Johnson Counter the clear signal of all the flip-flops should be activated.
- For example, a four-bit counter produces the sequence 0000, 1000, 1100, 1110, 1111, 0111, 0011, 0001, 0000, and so on.
- Note that in this sequence, only a single bit has a different value for two consecutive codes.

Digital Electronics & Systems
19ECE204
Lecture 57-58
Counter Design

Ms. Bhavana V
Assistant Professor (Sr.Gr.)
Dept. of ECE
ASE, Bengaluru

Steps to Design a Synchronous Counter

For synchronous counters, all the flip-flops are using the same CLOCK signal. Thus, the output would change synchronously.

Procedure to design synchronous counter are as follows:-

STEP 1: Obtain the **State Diagram**.

STEP 2: Obtain the **Excitation Table** using state transition table for any particular FF (JK or D). Determine number of FF used.

STEP 3: Obtain and simplify the function of each FF input using **K-Map**.

STEP 4: Draw the **circuit**.

Excitation Tables

During the design of circuits, we need a table that lists the required input combinations for a given change of state. Such table is called a flip flop *excitation table*.

		RS	
Q_t	Q_{t+1}	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

		JK	
Q_t	Q_{t+1}	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

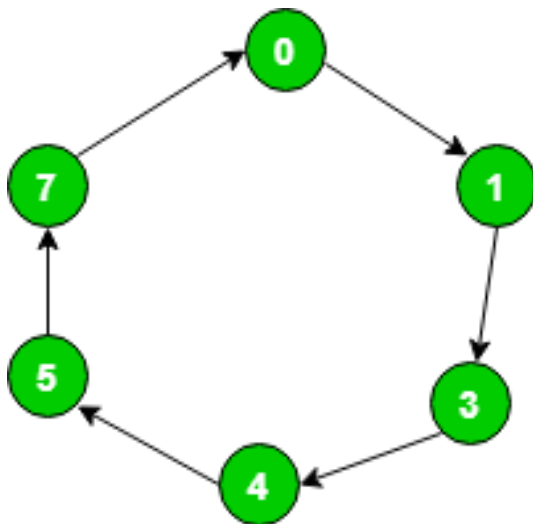
		D
Q_t	Q_{t+1}	D
0	0	0
0	1	1
1	0	0
1	1	1

		T
Q_t	Q_{t+1}	T
0	0	0
0	1	1
1	0	1
1	1	0

Example 1- Design synchronous counter for sequence: $0 \rightarrow 1 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 7 \rightarrow 0$, using T flip-flop.

Solution- For given sequence, state transition diagram as following below

State Diagram



State transition table logic:

PRESENT STATE	NEXT STATE
0	1
1	3
3	4
4	5
5	7
7	0

PRESENT STATE			NEXT STATE		
Q_3	Q_2	Q_1	$Q_3(t+1)$	$Q_2(t+1)$	$Q_1(t+1)$
0	0	0	0	0	1
0	0	1	0	1	1
0	1	1	1	0	0
1	0	0	1	0	1
1	0	1	1	1	1
1	1	1	0	0	0

INPUT TABLE OF FLIP-FLOPS		
T_3	T_2	T_1
0	0	1
0	1	0
1	1	1
0	0	1
0	1	0
1	1	1

T flip-flop – If value of Q changes either from 0 to 1 or from 1 to 0 then input for T flip-flop is 1 else input value is 0.

Q_T	Q_{T+1}	T
0	0	0
0	1	1
1	0	1
1	1	0

Compare Q_n with $Q_n(t+1)$ and use excitation table of the flip-flop to be used and write the excitation table for the counter to be designed

Find value of T_3 , T_2 , T_1 in terms of Q_3 , Q_2 , Q_1 using K-Map:

T3 $Q_3 \backslash Q_2 Q_1$	00	01	11	10
0	0	0	1	X
1	0	0	1	X

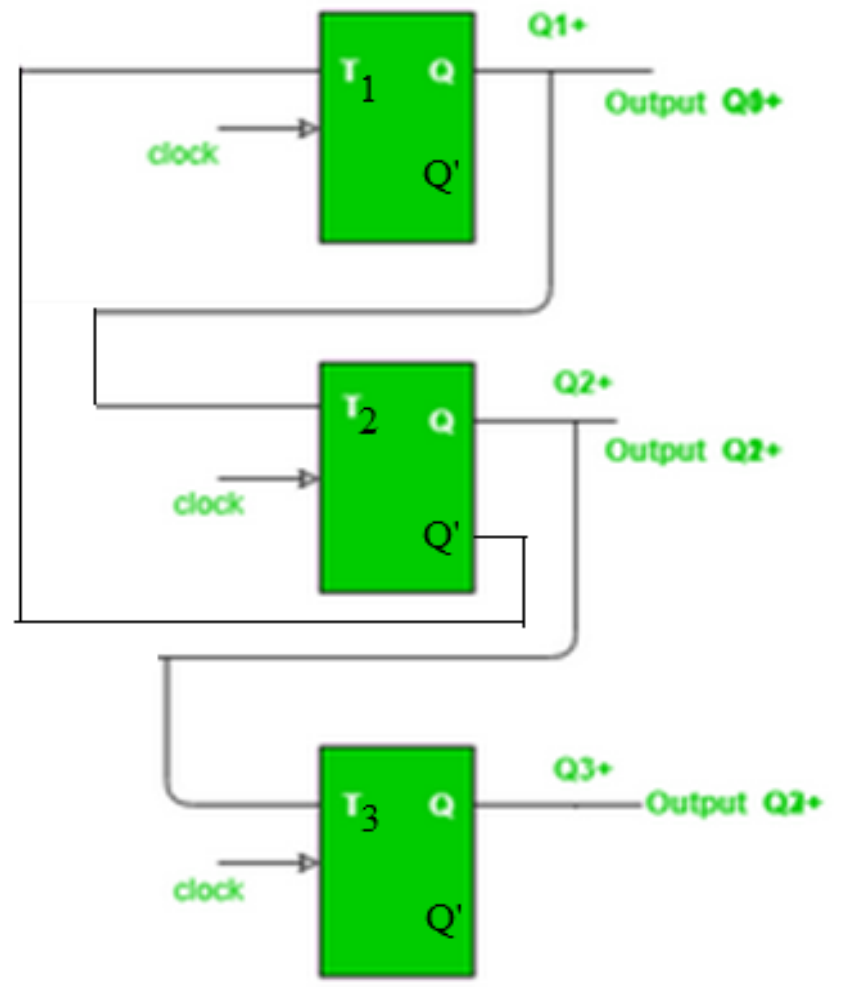
Therefore,
 $T_3 = Q_2$

T2 $Q_3 \backslash Q_2 Q_1$	00	01	11	10
0	0	1	1	X
1	0	1	1	X

Therefore,
 $T_2 = Q_1$

T1 $Q_3 \backslash Q_2 Q_1$	00	01	11	10
0	1	1	0	X
1	1	1	0	X

Therefore,
 $T_1 = Q_2'$



Example 2-

Design a Synchronous Counter to Count 4,7,3,0 and 2 respectively using JKFlip Flop negative triggered by showing:

- i. Flip Flop Used
- ii. State Transition Diagram
- iii. Excitation Table / Present state, next State
- iv. Karnaugh Map & perform Simplified Function
- v. The Synchronous Counter

Solution:

Step 1 : Flip Flop Used

Find Modulo, $N = 2^n$

$$M = 7, M = 2^n - 1 = 7$$

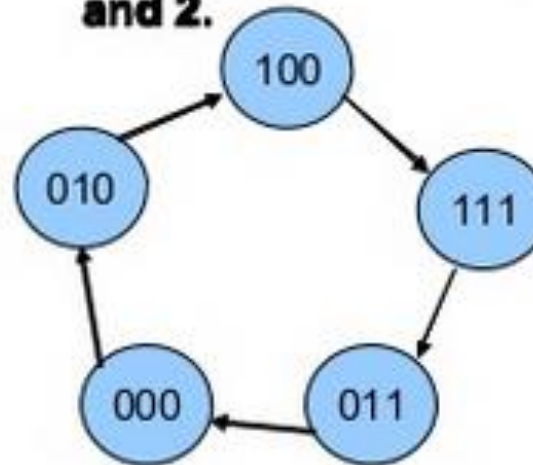
$$2^n = N, \text{ so, } N = 7 + 1 = 8, \text{ MOD } 8$$

$$2^n = 8, n = \log 8 / \log 2$$

$n = 3 \text{ bit} = 3 \text{ Flip Flop.}$

Solution:

Step 2 : State Transation Diagram, to count 4, 7, 3, 0 and 2.



Excitation Truth Table For Counter using JK FlipFlop

Present Q_n	Next State Q_{n+1}	J	K
0	0	0	x
0	1	1	x
1	0	x	1
1	1	x	0

Step 3 : Exitation Table /present State, Next State

Deci mal	Present State			Next State			JC	KC	JB	KB	JA	KA
	QC	QB	QA	QC	QB	QA						
4	1	0	0	1	1	1	x	0	1	x	1	x
7	1	1	1	0	1	1	x	1	x	0	x	0
3	0	1	1	0	0	0	0	x	x	1	x	1
0	0	0	0	0	1	0	0	x	1	x	0	x
2	0	1	0	1	0	0	1	x	x	1	0	x

K-Map For
JA = QC

A \ CB	00	01	11	10
0	0	0	X	1
1	X	X	X	X

K-Map For
KA = QC

A \ CB	00	01	11	10
0	X	X	X	X
1	X	1	0	X

To get the minimized cost, the unused state can be assumed to go to don't care state. So the states 1,5,6 will be act as don't care terms.

Step 3 : Exitation Table /present State, Next State

Deci mal	Present State			Next State			JC	KC	JB	KB	JA	KA
	QC	QB	QA	QC	QB	QA						
4	1	0	0	1	1	1	x	0	1	x	1	x
7	1	1	1	0	1	1	x	1	x	0	x	0
3	0	1	1	0	0	0	0	x	x	1	x	1
0	0	0	0	0	1	0	0	x	1	x	0	x
2	0	1	0	1	0	0	1	x	x	1	0	x

K-Map For
JB = 1

CB \ A	00	01	11	10
0	1	x	x	1
1	x	x	x	x

K-Map For
KB = QC

CB \ A	00	01	11	10
0	x	1	x	x
1	x	1	0	x

K-Map For
JC = QA' QB

CB \ A	00	01	11	10
0	0	1	x	x
1	x	0	x	x

K-Map For
KC = QB

CB \ A	00	01	11	10
0	x	x	x	0
1	x	x	1	x

Example 3- Design a synchronous counter with count sequence 1,3,6,5,1 using D flip-flop.

Solution- Number of flip-flops required to design given counter = 3

The d flip-flop is used to design the counter.

Excitation Table for
D flip-flop

Q_n	Q_{n+1}	D
0	0	0
0	1	1
1	0	0
1	1	1

To get the minimized cost the unused state can be assumed to go to don't care state. The unused states are 0,2,4,7

Excitation Table for Counter design

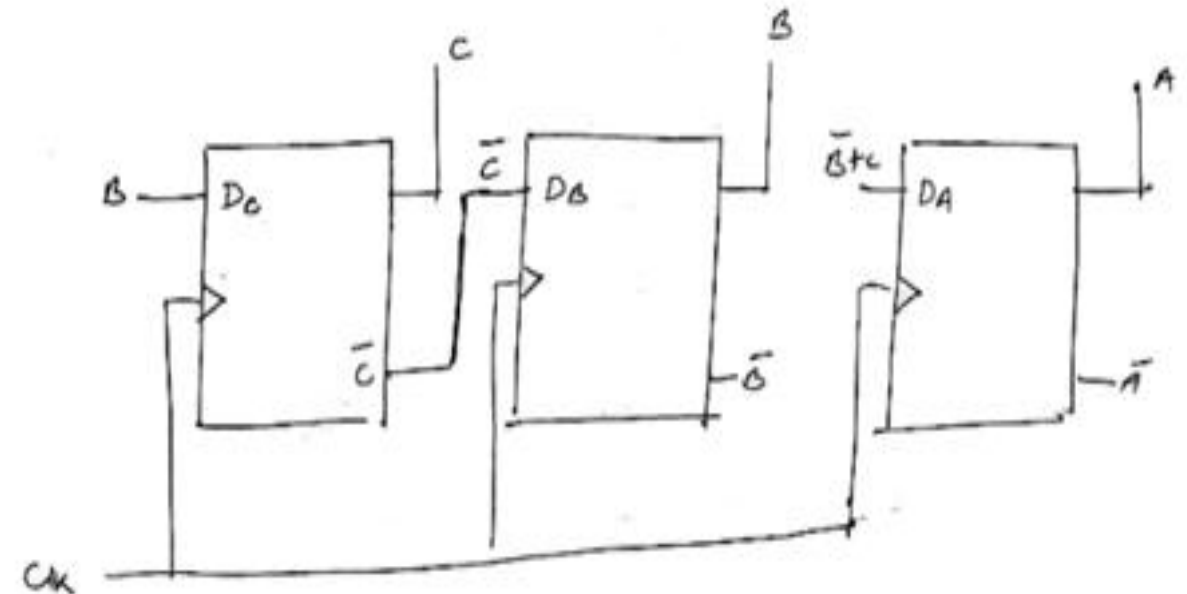
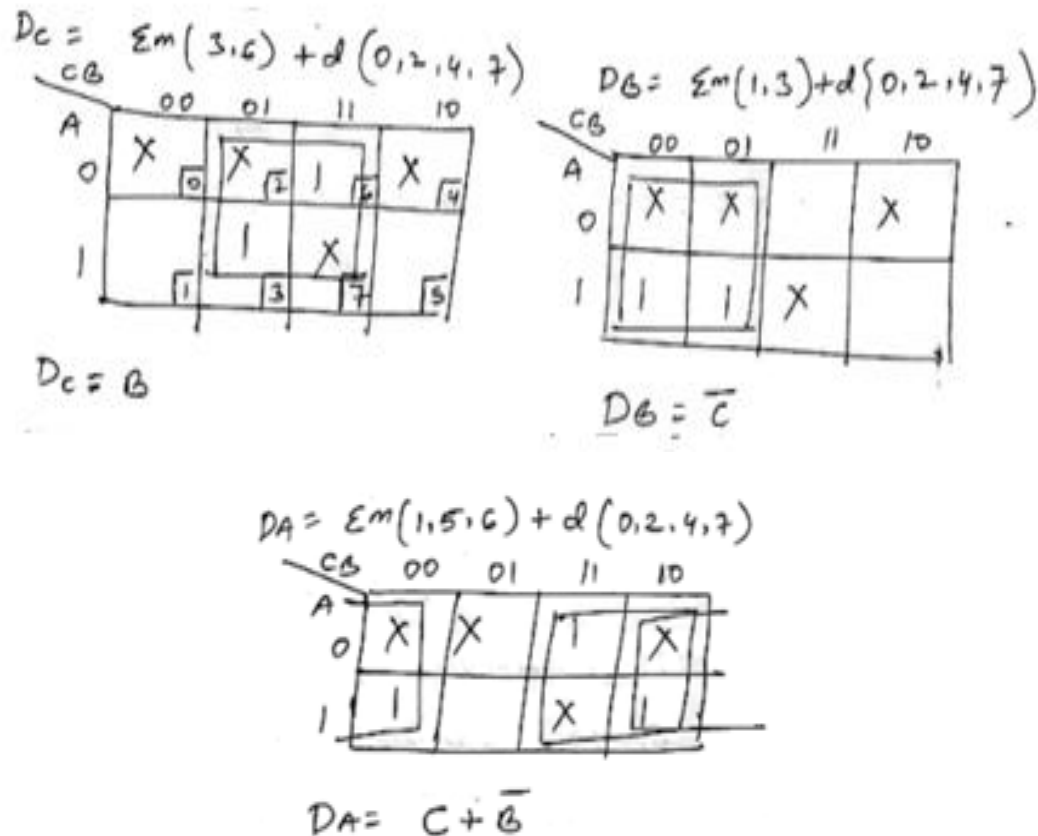
Current state			Next state			D _C	D _B	D _A
C	B	A	C [*]	B [*]	A [*]			
0	0	0	X	X	X	X	X	X
0	0	1	0	1	1	0	1	1
0	1	0	X	X	X	X	X	X
0	1	1	1	1	0	1	1	0
1	0	0	X	X	X	X	X	X
1	0	1	0	0	1	0	0	1
1	1	0	1	0	1	1	0	1
1	1	1	X	X	X	X	X	X

Example 3- Design a synchronous counter with count sequence 1,3,6,5,1.

Solution Continued....

K-Map Simplification

Logic Circuit



Use D flip-flop

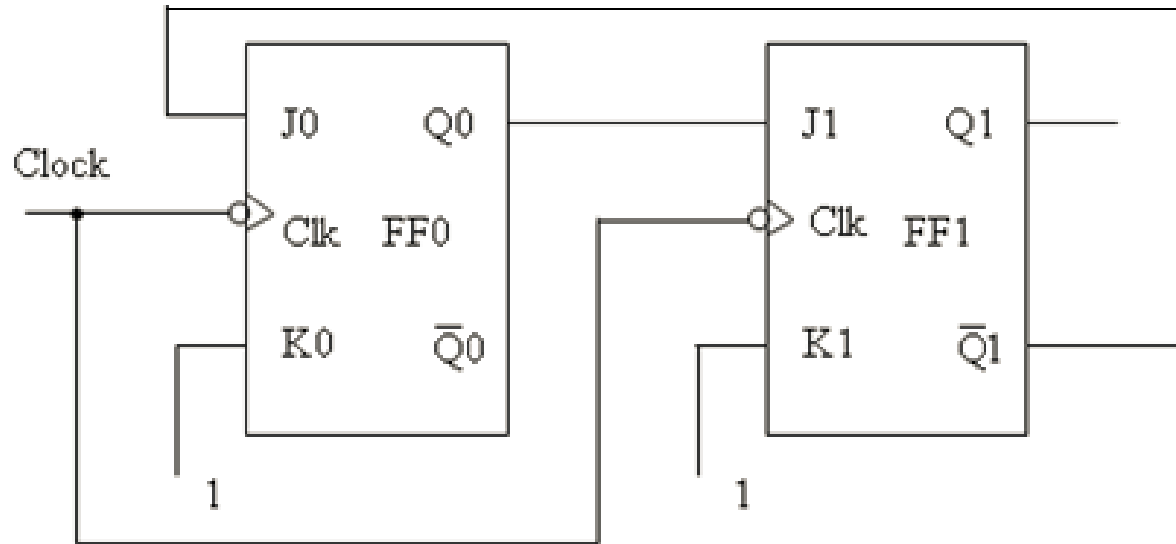
Exercises:

- **Design a counter to count in the following sequence: 6, 4, 2, 3, 1.**
- **Design a counter to count in the following sequence: 15,9,11,5,2,13,1.**





Home Work - Identify the output of the following Circuit

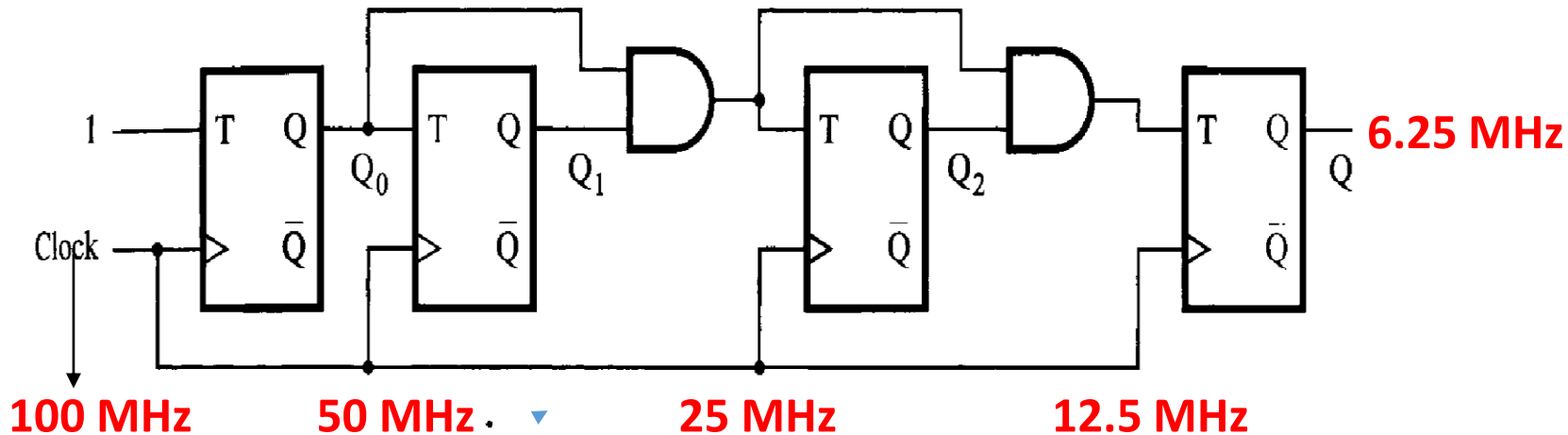
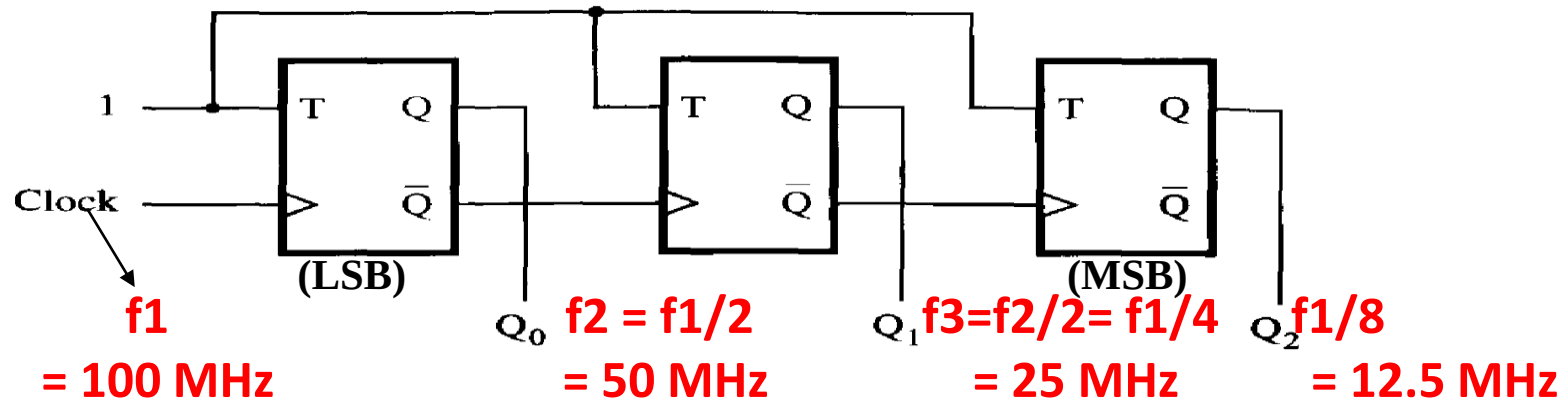


Clock	Q1	Q0
	0	0
1 st	0	1
2 nd	1	0
3 rd	0	0
4 th	0	1

Assume initial value on Q1 Q0 = 00

Counter as Frequency Divider

- 3-bit Counter



Clock cycle	Q ₂	Q ₁	Q ₀
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1
8	0	0	0

Counter

How many flip-flops are required to design following counters?

1. Mode 16 counter

Ans. Mode 16 – 0 to 15, Max count = 15 so 4 bits

or $2^n = 16$ so $n = 4$

So 4 flip-flops are required

2. Mode 17 counter

Ans. Mode 17 – 0 to 16, Max count = 16 so 5 bits

So 5 flip-flops are required

3. Counter with count sequence 5,9,11,19,12,3,0,5.....

Ans. Max count = 19 so 5 bits

So 5 flip-flops are required

4. Modulo 1024 counter

Ans. 10 flip-flops are required

Digital Electronics & Systems
19ECE204
Lecture 59-60
Synchronous Sequential
Circuits

Ms. Bhavana V
Assistant Professor (Sr.Gr.)
Dept. of ECE
ASE, Bengaluru

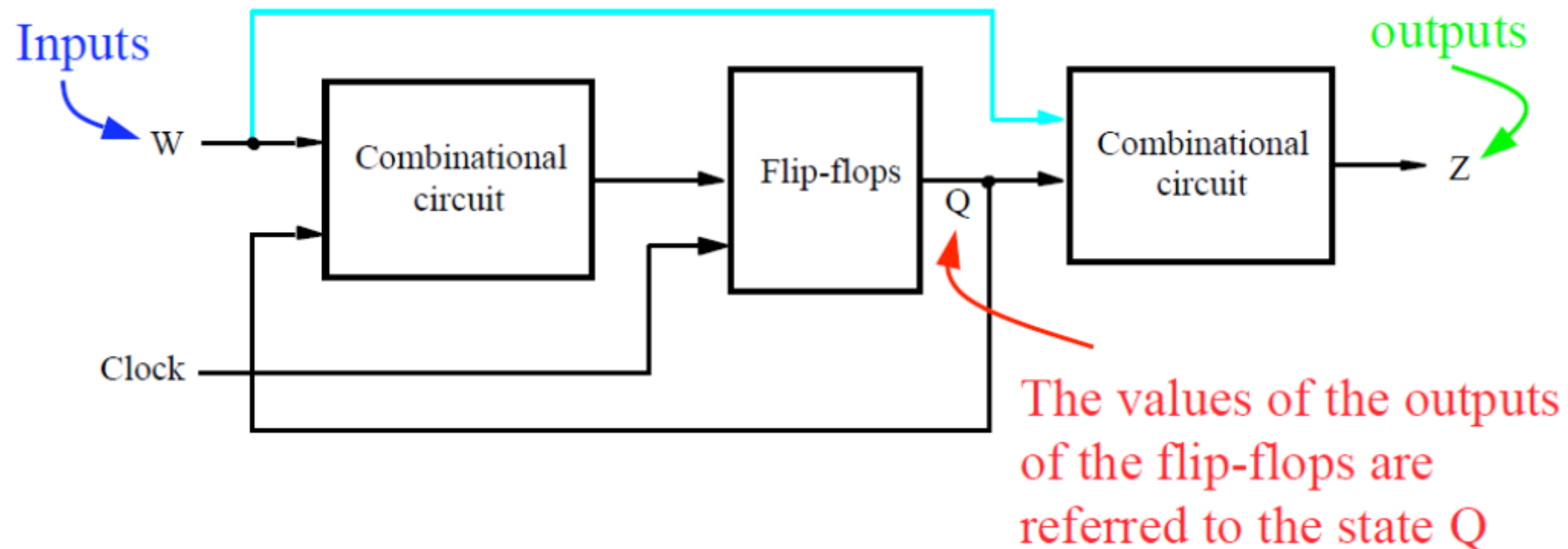
Sequential Circuits

- We considered combinational logic circuits in which outputs are fully determined by the present values of the inputs.
- We also considered simple storage elements that can be implemented in the form of flip-flops for which the output depends on their states rather than the value of the inputs; the inputs cause changes in the state.
- In this lecture we deal with a general class of circuits in which the output depend on the past behavior of the circuit, as well on the present values of the inputs. They are called *sequential circuits*.

Synchronous vs Asynchronous

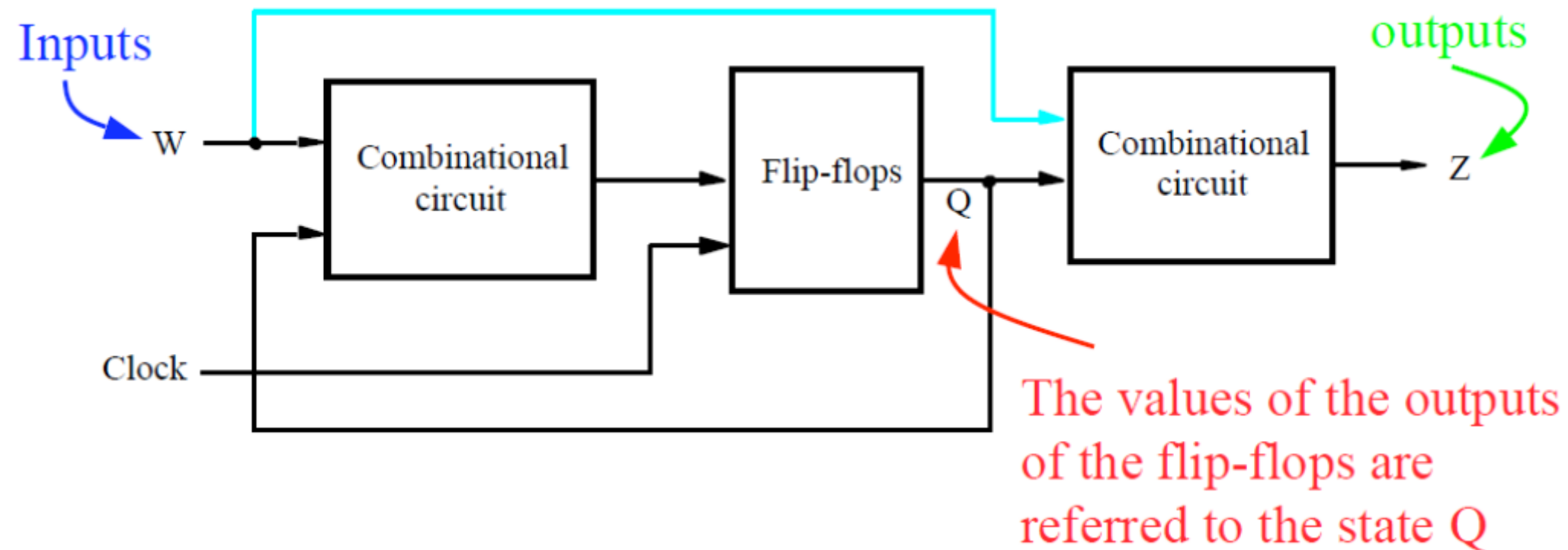
- In most cases a clock signal is used to control the operation of a sequential circuit. Such circuit is called *synchronous*.
- The alternative, in which clock signal is not used, is called *asynchronous sequential circuits*.
- Synchronous circuits are easier to design and are used in the vast majority of practical applications.

Synchronous Sequential Circuits



- This circuit are realized with combinational logic and one or more flip-flops.
- Under the control of clock signal, the flip-flop outputs change their state as determined by the combinational logic that feeds the inputs of the flip-flops. Therefore the circuit moves from one state to the next: $Q_t \rightarrow Q_{t+1}$.
- The combinational logic that provide the input signals of the flip-flops derives its inputs from two sources: the primary inputs W, and the current outputs of the flip-flops Q. thus changes in the state depend on both present state and the values of the primary inputs.
- The output of the circuit are generated by another combinational circuit, such that the outputs are a function of the present state and the primary inputs.

Synchronous Sequential Circuits



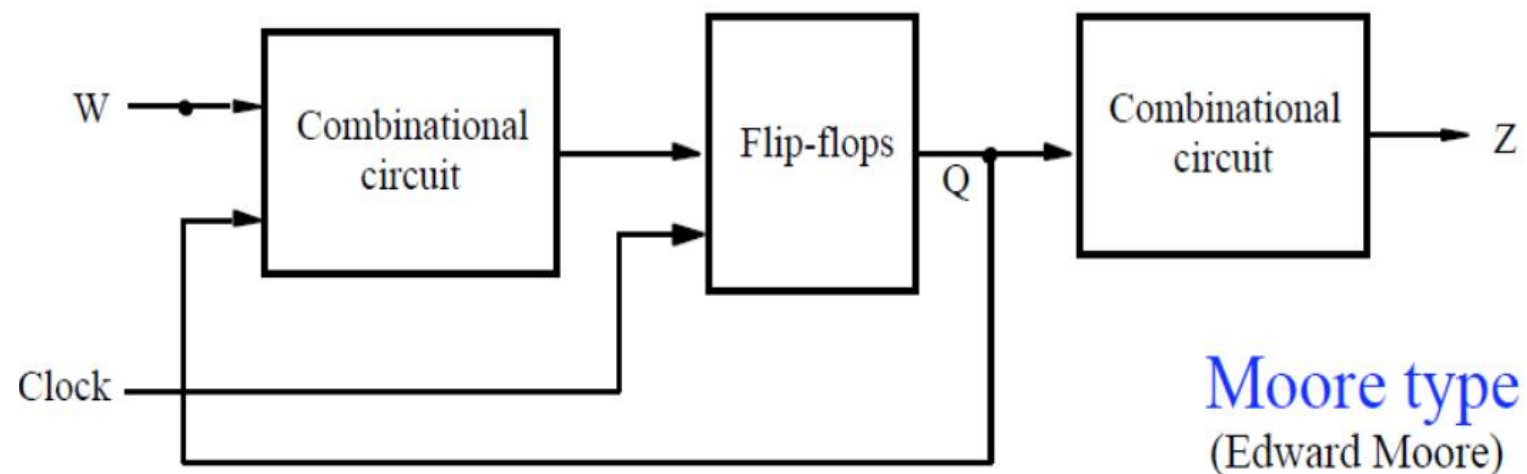
- Although the outputs always depend on present state, but they don't have to depend directly on primary inputs. Thus the connection in the blue may not exist.

Finite State Machine (FSM)

- Sequential circuits are also called *finite state machines (FSMs)*, which is a more formal name that is often found in technical literature.
- The name derives from the fact that the functional behavior of these circuits can be represented using a finite number of states.

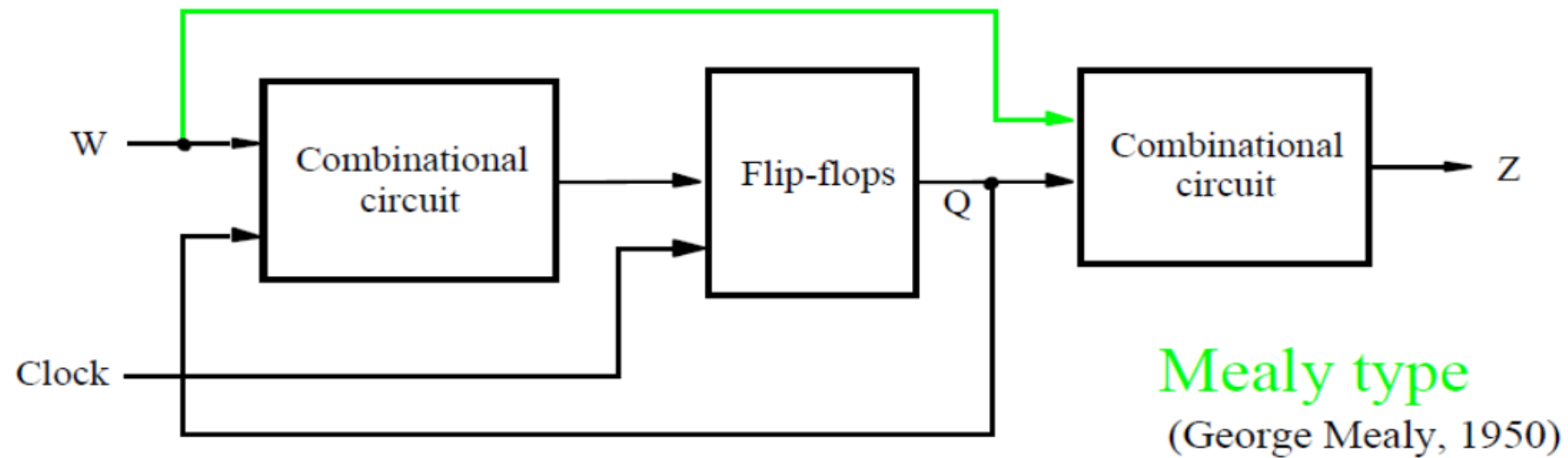
Two types of FSM-

- Moore FSM- The sequential circuits whose outputs depend only on the state of the circuit are of *Moore* type.



Finite State Machine (FSM)

- Mealy FSM - those whose outputs depend on both the state and the primary inputs are of *Mealy* type.



Finite State Machine (FSM)

Sequence detector



- A **sequence detector** is a sequential state machine which takes an input string of bits and generates an output 1 whenever the target sequence has been detected.
- Sequence detector is of two types:
 1. Overlapping
 2. Non-Overlapping
- In an overlapping sequence detector, the last bit of one sequence becomes the first bit of next sequence. However, in non-overlapping sequence detector the last bit of one sequence does not become the first bit of next sequence.

Finite State Machine (FSM)

Sequence detector

Design a sequence detector to detect two consecutive 1's i.e “11”.

Basic Design Steps-

We wish to design a circuit that meets the following specification:

1. The circuit has one input, w , and one output, z .
2. All changes in the circuit occur on the positive edge of a clock signal.
3. The output z is equal to 1 if during two immediately preceding clock cycles the input w was equal to 1. Otherwise, the value of z is equal to 0.

Finite State Machine (FSM)

Sequence detector

Design a sequence detector to detect two consecutive 1's i.e “11”.

➤ Moore FSM

Sequences of input and output signals.

Clock cycle:	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
w :	0	1	0	1	1	0	1	1	1	0	1
z :	0	0	0	0	0	1	0	0	1	1	0

➤ Mealy FSM

Sequences of input and output signals.

Clock cycle:	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
w :	0	1	0	1	1	0	1	1	1	0	1
z :	0	0	0	0	1	0	0	1	1	0	0

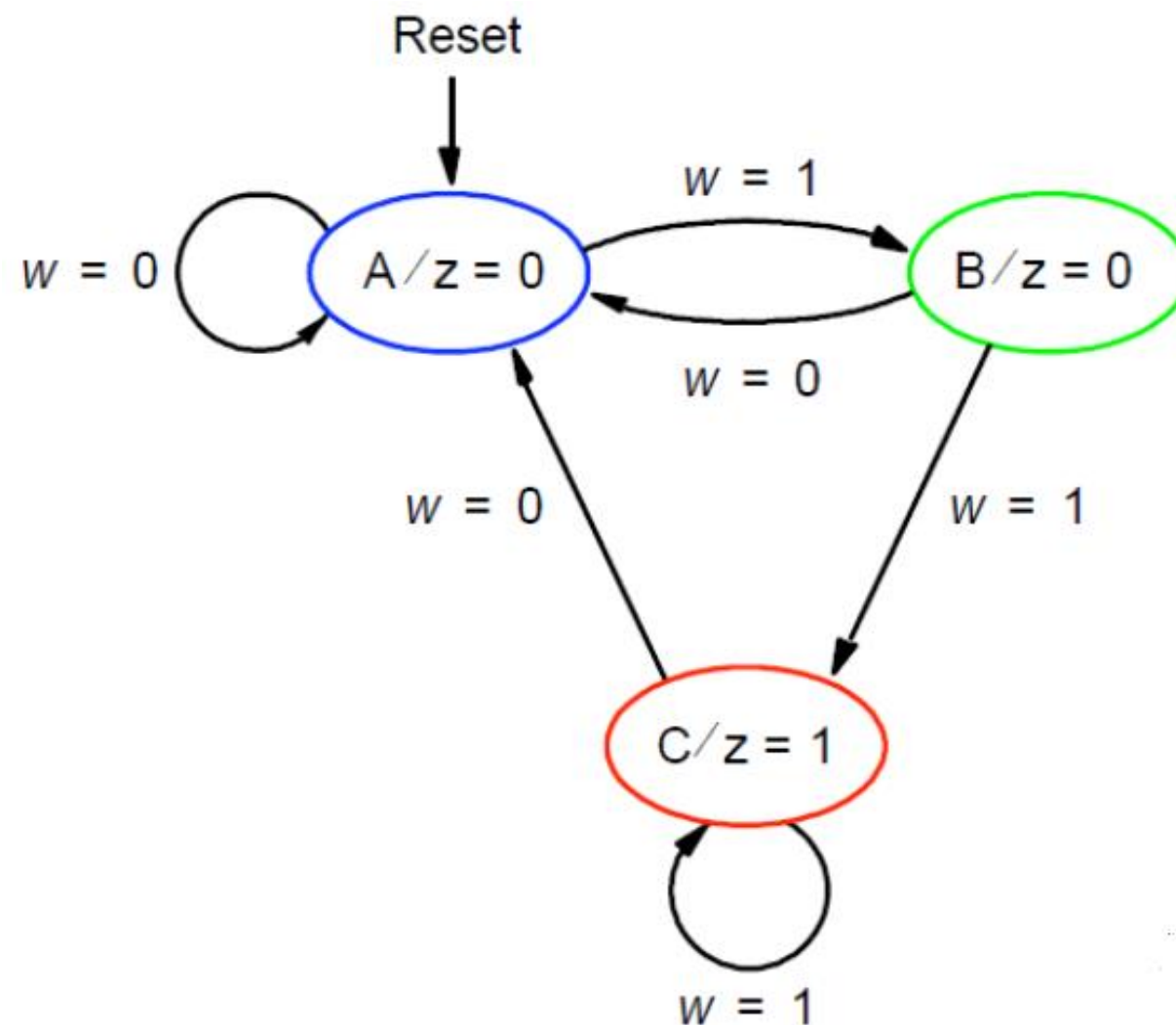
Finite State Machine (FSM)

Sequence detector- Moore FSM

Design a sequence detector to detect two consecutive 1's i.e “11”.

Moore FSM - State Diagram:

Behavior of a sequential circuit can be described in several different ways. The conceptually simplest method is to use a pictorial representation in the form of a *state diagram*, which is a graph that depicts states of the circuit as nodes (circles) and transitions between states as directed arcs.

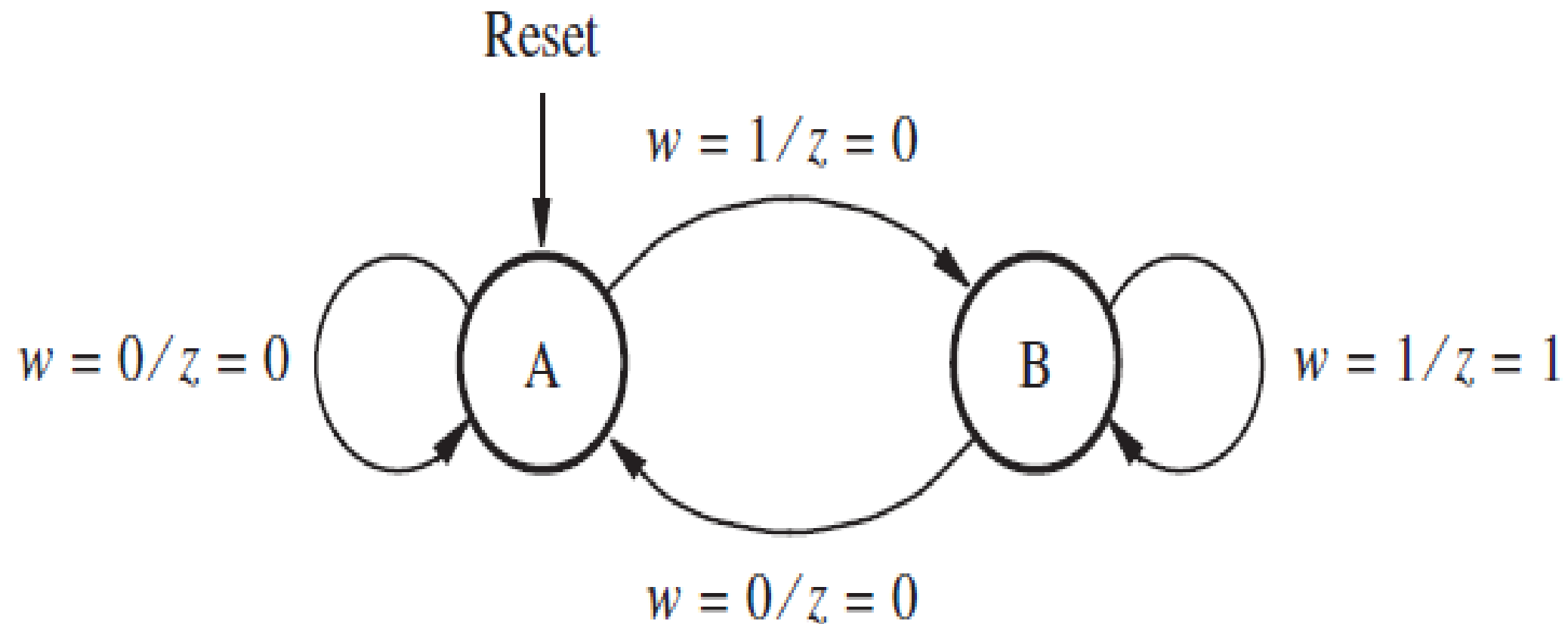


Finite State Machine (FSM)

Sequence detector- Mealy FSM

Design a sequence detector to detect two consecutive 1's i.e “11”.

Mealy FSM - State Diagram:

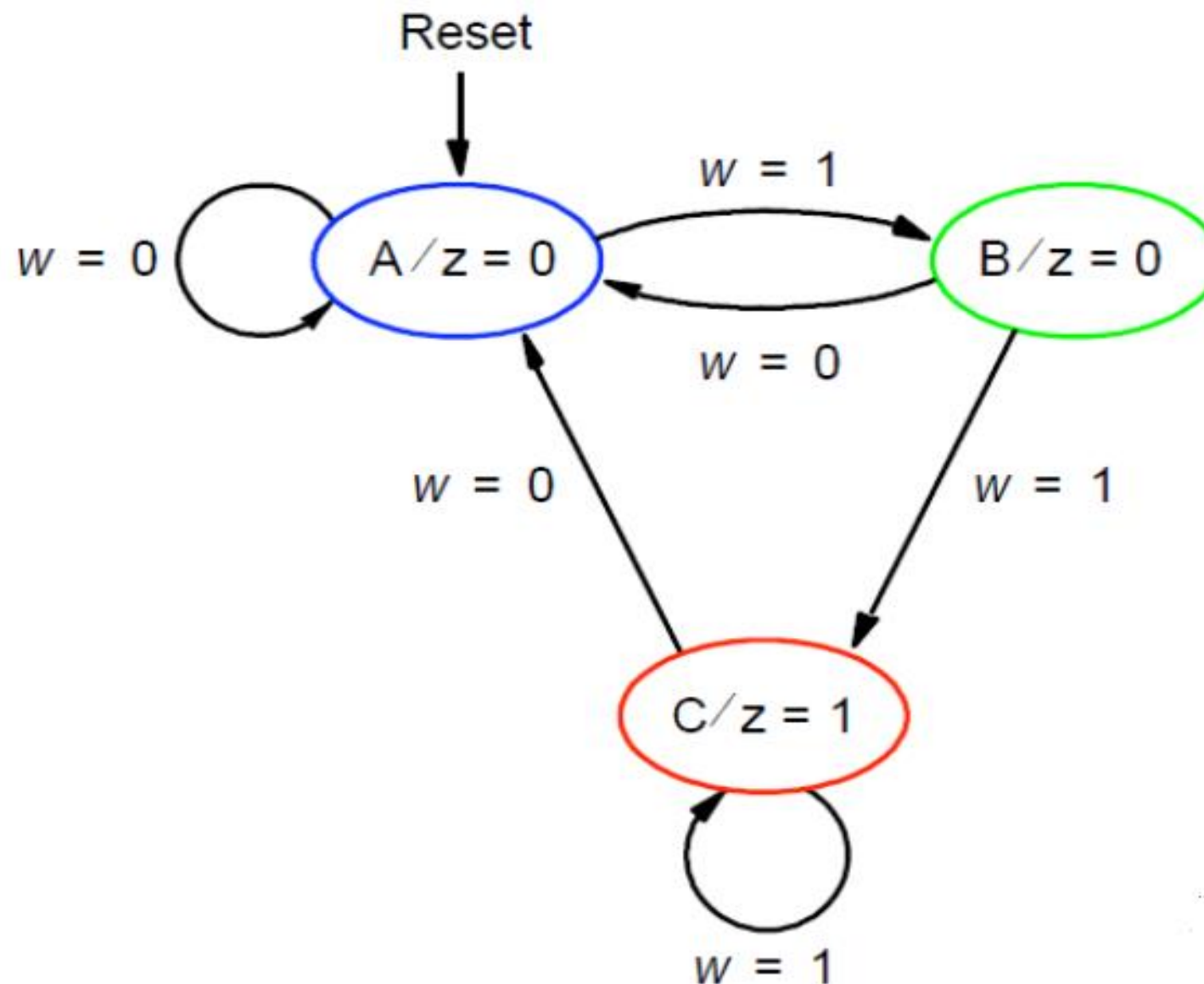


Finite State Machine (FSM)

Sequence detector- Moore FSM

Design a sequence detector to detect two consecutive 1's i.e “11”.

Step 1- State Diagram:



Finite State Machine (FSM)

Sequence detector- Moore FSM

Design a sequence detector to detect two consecutive 1's i.e “11”.

Step 2- State Table:

- State diagrams provide a description of the behavior of a sequential circuit that is easy to understand.
- To proceed with the implementation of the circuit it is convenient to translate the information contained in the diagram in tabular form.

Finite State Machine (FSM)

Sequence detector- Moore FSM

Design a sequence detector to detect two consecutive 1's i.e “11”.

Step 2- State Table:

Present state	Next state		Output z
	$w = 0$	$w = 1$	
A	A	B	0
B	A	C	0
C	A	C	1

- The table indicate all transitions from each present state to the next state for different values of the input signal.
- The output signal is defined with respect to the present state.
- We did not include the Reset input signal; instead we made the implicit assumption that the first state in the table is the starting state.

Finite State Machine (FSM)

Sequence detector- Moore FSM

Design a sequence detector to detect two consecutive 1's i.e “11”.

Step 3- State Assignment:

Present state	Next state		Output z
	$w = 0$	$w = 1$	
A	A	B	0
B	A	C	0
C	A	C	1

- When implemented as a logic circuit, each state is represented by the valuation (combination of values) of *state variables*.
- Each state may be implemented in the form of a flip-flop.
- Since three states have to be realized, it is sufficient to use two state variables.

Finite State Machine (FSM)

Sequence detector- Moore FSM

Design a sequence detector to detect two consecutive 1's i.e “11”.

Step 3- State Assignment:

Gray code is an efficient way to assign the states.

	Present state y_2y_1	Next state		Output z
		$w = 0$	$w = 1$	
		Y_2Y_1	Y_2Y_1	
A	00	00	01	0
B	01	00	11	0
C	11	00	11	1
	10	dd	dd	d

Finite State Machine (FSM)

Sequence detector- Moore FSM

Design a sequence detector to detect two consecutive 1's i.e “11”.

Step 4- Selecting Flip flops:

Using D flip flops to design the detector.

	Present state y_2y_1	Next state		Output z
		$w = 0$	$w = 1$	
		Y_2Y_1	Y_2Y_1	
A	00	00	01	0
B	01	00	11	0
C	11	00	11	1
	10	dd	dd	d

In D flip flop D inputs are same as next state

Finite State Machine (FSM)

Sequence detector- Moore FSM

Design a sequence detector to detect two consecutive 1's i.e “11”.

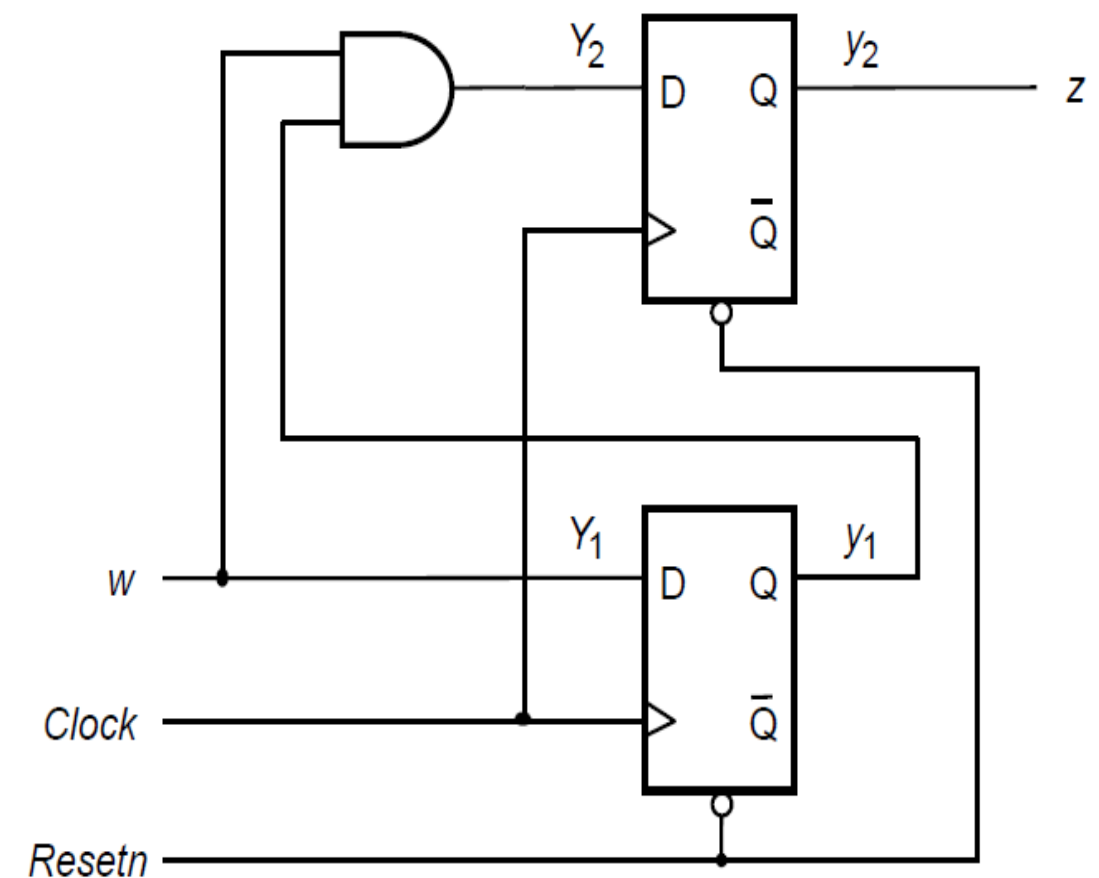
Step 5- Logic circuit Design:

	Present state	Next state		Output z
		$w = 0$	$w = 1$	
	y_2y_1	Y_2Y_1	Y_2Y_1	
A	00	00	01	0
B	01	00	11	0
C	11	00	11	1
	10	dd	dd	d

$$Y_1 = D_1 = w$$

$$Y_2 = D_2 = wy_1$$

$$z = y_2$$

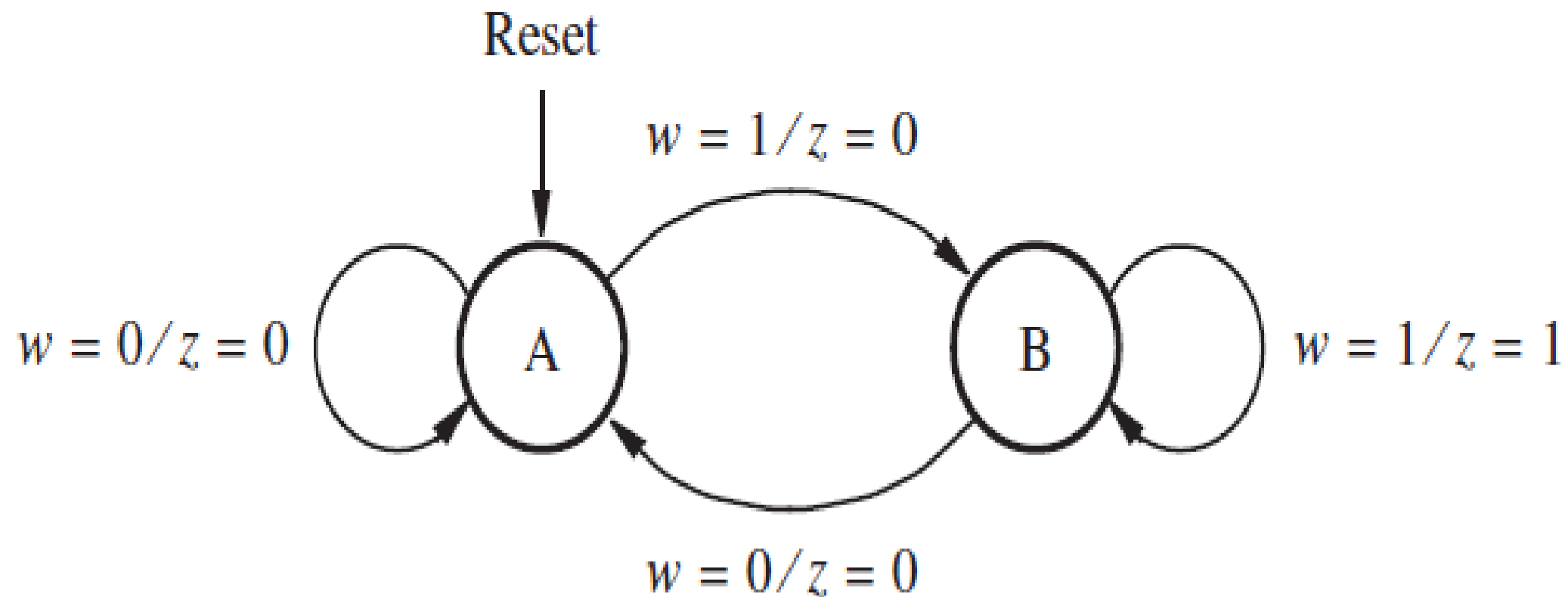


Finite State Machine (FSM)

Sequence detector- Mealy FSM

Design a sequence detector to detect two consecutive 1's i.e. "11".

Step 1- State Diagram:



Finite State Machine (FSM)

Sequence detector- Mealy FSM

Design a sequence detector to detect two consecutive 1's i.e “11”.

Step 2- State Table:

Present state	Next state		Output z	
	$w = 0$	$w = 1$	$w = 0$	$w = 1$
A	A	B	0	0
B	A	B	0	1

Step 3- State Assignment:

	Present state	Next state		Output	
		$w = 0$	$w = 1$	$w = 0$	$w = 1$
	y	Y	Y	z	z
A	0	0	1	0	0
B	1	0	1	0	1

Finite State Machine (FSM)

Sequence detector- Mealy FSM

Design a sequence detector to detect two consecutive 1's i.e “11”.

Step 4- Logic Circuit:

$$D = Y = w$$

$$z = yw$$

